Machine Learning

Supervised Learning, Neural Networks & Ensemble Methods

Prof. Dr. M. A. H. Akhand ? Dept. of CSE, KUET

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1 01 - Introduction & Research Achieve... p. 3

2 02 - Machine Learning Fundamentals &... p. 21

3 03 - Biological Neuron to Artificial... p. 58

4 04 - ML Classification Model Evaluat... p. 138

5 05 - Backpropagation Learning Detail... p. 154

6 06 - BP Learning Modes, Momentum & B... p. 168

7 07 - Convergence Criteria in Backpro... p. 180

8 08 - K-Means Clustering & Radial Bas... p. 186

9 09 - Ensembles of Diverse Neural Net... p. 203

10 10 - Classification with Decision Tr... p. 239

11 11 - Ensemble Learning with Decision... p. 278

12 12 - Comparative Study of Decision T... p. 293

13 13 - Deep Learning Aspects & Modern ... p. 296

14 14 - Convolutional Neural Networks (... p. 334

Research Foundation, Timeline, Outcome and Achievement

Prof. Dr. Md. Aminul Haque Akhand
Dept. of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET), Bangladesh
www.kuet.ac.bd/cse/akhand

Outline of the Presentation

1. **Research Foundation**
2. **Foreign Collaboration**
3. **Research Outcomes and Achievements**

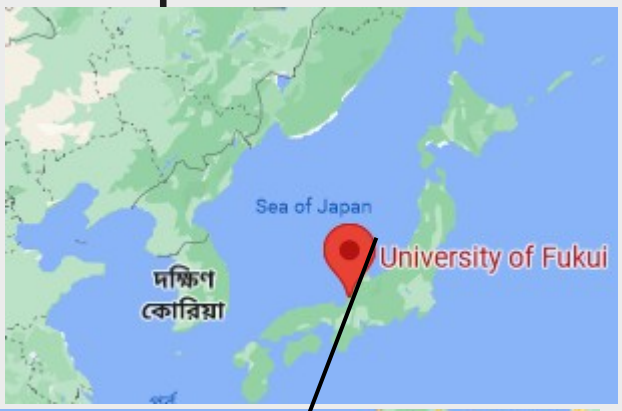
Research Foundation : Graduation from KUET

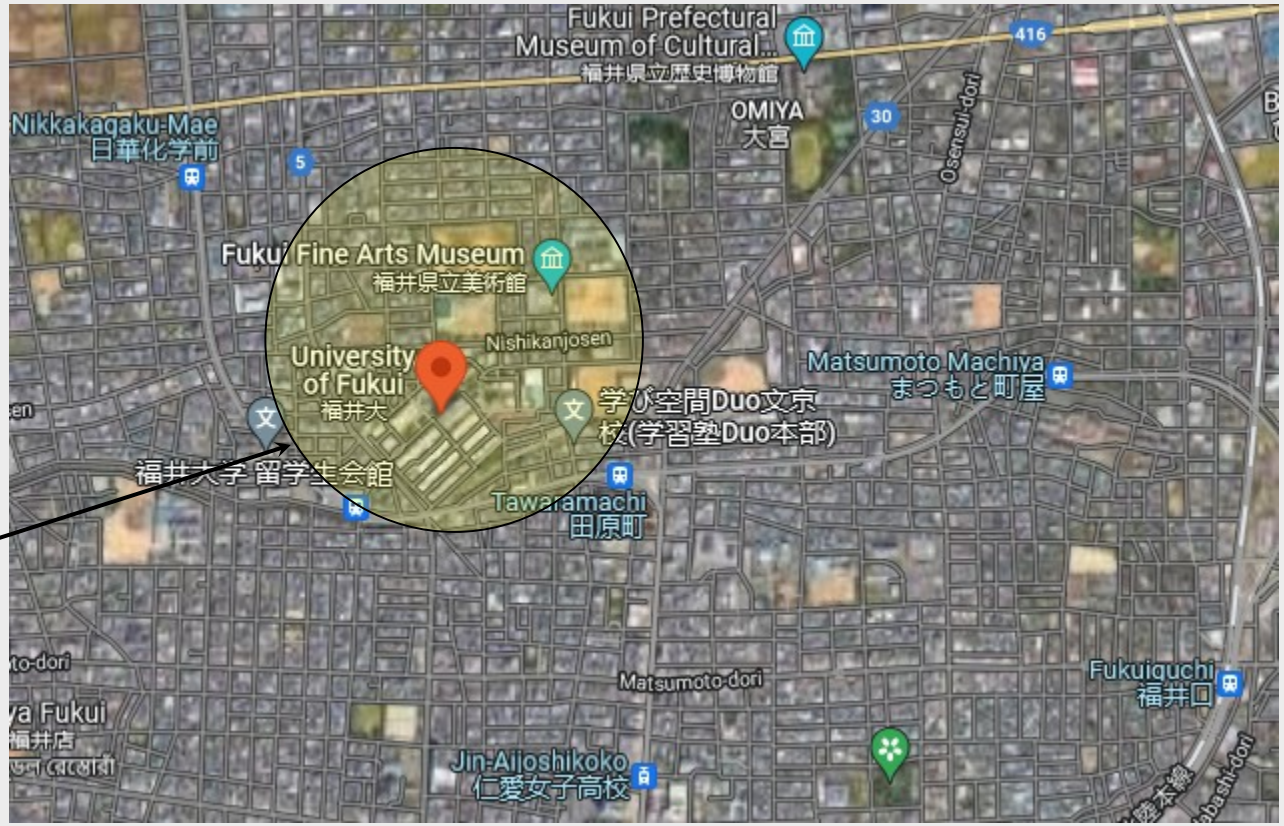
Khulna University of Engineering & Technology (KUET)

A top ranked engineering university in Bangladesh

Research Foundation : Graduation from KUET

Research Foundation: MSc and PhD from University of Fukui, Japan





University of Fukui, Fukui, Japan

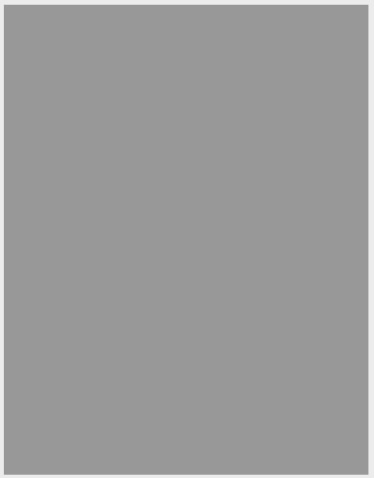
A top ranked university in Japan and Worldwide

Research Foundation: MSc and PhD from University of Fukui, Japan

Research Foundation: Founder

Prof. Dr. Kazuyuki Murase
University of Fukui

Prof. Dr. Md. Monirul Islam
BUET, Bangladesh



Late Prof. Dr. H. Ikeda
University of Fukui

Prof. Dr. Md. Shahjahan
KUET, Bangladesh

Dr. Md. Faijul Amin
Intel Corporation, USA

Dr. Fady S.K. Alnajjar
UAE University, UAE

Foreign Research Collaboration

Dr. Md. Abdus Samad KAMAL
Graduate School of Science and Technology
Gunma University, Japan
Web Link: https://www.mst.st.gunma-u.ac.jp/kamal/

Dr Nazmul Siddique
School of Computing, Engineering & Intel. Systems
Ulster University, UK
Web Link: https://pure.ulster.ac.uk/en/persons/nazmul-siddique

Prof. Dr. Tetsuya Shimamura
Graduate School of Science and Engineering
Saitama University, Japan
http://s-read.saitama-u.ac.jp/researchers/pages/researcher_en/BuBfsmBx

+ Some Other Universities in KSA, USA, Australia

Research Outcomes in Google Scholar

Dr. Md. Aminul Haque Akhand

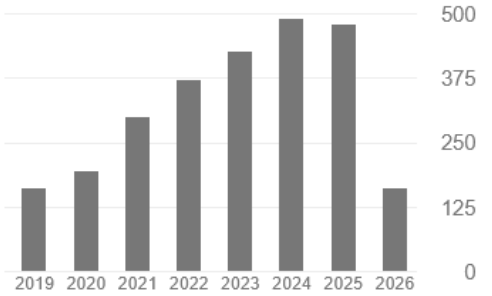
FOLLOW

Professor, Dept. of Computer Sceince and Engineering, Khulna University of Engineering & Technology
Verified email at cse.kuet.ac.bd - Homepage

Machine Learning Deep Learning Optimization Swarm Intelligence

TITLE	CITED BY	YEAR
Discrete Spider Monkey Optimization for Travelling Salesman Problem MAH Akhand, SI Ayon, SA Shahriyar, N Siddique, H Adeli Applied Soft Computing 86, 105887	277	2020
Optimization of university course scheduling problem using particle swarm optimization with selective search SI Hossain, MAH Akhand, MIR Shuvo, N Siddique, H Adeli Expert Systems with Applications 127, 9-24	199	2019
Bangla handwritten character recognition using convolutional neural network MM Rahman, MAH Akhand, S Islam, PC Shill, MH Rahman International Journal of Image, Graphics and Signal Processing 7 (8), 42-49	176	2015
Genetic algorithm based fuzzy decision support system for the diagnosis of heart disease AK Paul, PC Shill, MRI Rabin, MAH Akhand 2016 5th International Conference on Informatics, Electronics and Vision ...	122	2016
Continuous heart rate and body temperature monitoring system using Arduino UNO and Android device MA Miah, MH Kabir, MSR Tanveer, MAH Akhand 2015 2nd International Conference on Electrical Information and ...	91	2015

Cited by	All	Since 2021
Citations	3088	2228
h-index	26	20
i10-index	71	49



Co-authors EDIT

- Nazmul Siddique, Ulster University
- Kazuyuki Murase, International Professional Univer...
- M M Hafizur Rahman, Associate Professor, Dept. of CN...
- Md Abdus Samad Kamal, Graduate School of Science and ...
- Md. Monirul Islam

https://scholar.google.com/citations?hl=en&user=NKBjhR8AAAAJ

Position in AD Scientific Index 2023

AD Scientific Index — Rankings for Scientist
University, Subject, Country, Region, World

World Scientist and University Rankings 2023

Khulna University of Engineering & Technology

Md Aminul Haque Akhand

	Scores	In Khulna University of Engineering & Technology (210)	In Bangladesh (9156)	In Asia (402872)	World (1349705)
		Rankings			
Total H	21	#4	#375	#69608	#382772
Last 6 year H	18	#4	#345	#61956	#313746
Last 6 year H / total H	0.857				
Total i10	45	#4	#266	#51874	#277918
Last 6 years i10	36	#4	#247	#48263	#240270
Last 6 years i10 / Total i10	0.800				
Total Citation	1634	#9	#425	#77480	#440812
Last 6 years Citation	1295	#4	#339	#63085	#331528
Last 6 years Citation / Total Citation	0.793				
Engineering & Technology *		#4 (90) *	#108 (2132) *	#20668 (78442) *	#72126 (209087) *
Computer Science *		#3 (36) *	#28 (761) *	#4542 (22216) *	#17084 (58952) *

www.adscientificindex.com

https://www.adscientificindex.com/scientist/md-aminul-haque-akhand/293902

Position in AD Scientific Index 2025

AD Scientific Index — Rankings for Scientist

More Than a Ranking

World Scientist and University Ranking 2025

Khulna University of Engineering & Technology

AD Scientific Index ID: 293902 — Md Aminul Haque Akhand

	Scores	In Khulna University of Engineering & Technology (454)	In Bangladesh (16,559)	In Asia (682,363)	World (2,384,773)
		Rankings			
H-Index (Total)	24	#5	#440	#69,954	#364,264
H-Index (Last 6 years)	17	#6	#582	#78,082	#364,110
H-Index Last 6 years / Total Ratio	0.708				
i10 Index (Total)	57	#4	#289	#48,375	#245,624
i10 Index (Last 6 years)	40	#5	#313	#49,320	#228,552
i10 Index Last 6 years / Total Ratio	0.702				
Citation (Total)	2,358	#6	#442	#71,332	#397,165
Citation (Last 6 years)	1,709	#6	#382	#56,443	#280,286
Citation Last 6 years / Total Ratio	0.725				
Engineering & Technology *		#4 (225) *	#141 (4,528) *	#25,052 (163,285) *	#87,069 (427,234) *
Computer Science *		#3 (82) *	#35 (1,866) *	#5,588 (52,526) *	#21,476 (134,119) *

www.adscientificindex.com

https://www.adscientificindex.com/scientist/md-aminul-haque-akhand/293902

KUET Personal Website

Home | Education | Experience | Scholarship and Award | Research | Teaching and Supervision | Student Corner | Publications

Dr. Muhammad Aminul Haque Akhand

Professor

akhand@cse.kuet.ac.bd

Research Interest: Ongoing Researches Research Area: Machine Learning, Bio-Inspired Computing Techniques and Pattern Recognition

Print Profile

Download CV

Bio

Welcome to my personal site. I am servicing as a Professor in the Dept. of CSE, KUET. I have been graduated from this university in 1999 and then started service as a programmer in Software Company. I started research work while doing Master of Engineering in University of Fukui, Japan. After getting M. Eng. in Human and Artificial Intelligent Systems, I enrolled doctoral program in the same university and awarded Doctor of Engineering in System Design Engineering on Intelligent Information Systems in March, 2009. Now I have devoted myself to teaching and research in KUET. In this site you may find short description about my education, professional experience, research, etc.

I was **Associate Director (Academic and Research Wing)** Institute of Information and Communication Technology (IICT) . I was the manager of **Integrated Automation of KUET (IAoKUET), HEQEP Project** funded by World Bank and Govt. of Bangladesh. I have completed two years tenure period on December 31, 2019 as **Head of the CSE Department** .

I was in Japan (Oct-Dec, 2017) as a **JSPS fellow with honor of Visiting Professor at University of Fukui**. Research knowledge exchange and establishment of collaboration are the great achievements during the fellowship while visiting different laboratories of University of Fukui, Toyama Prefectural University, Tokyo Metropolitan University, Saitama University and Tohoku University.

About research, I am interested in Artificial Intelligence specifically (i) Artificial Neural Networks (ANN) and Neural Networks Ensemble (NNE), (ii) Evolutionary Computation and other Bio-inspired Computing Techniques, (iii) Swarm Intelligence and Optimization (iv) Biomedical Signal Processing and, (v) Pattern Recognition. About my research outcome, I have touched **100 publications by the end of 2017**. By 2022, I have reached **150 Publications** including 03 books, 50+ journal articles. I received **Best Paper Awards** in several international conferences. Among the books, *â€˜Deep Learning Fundamentals- A Practical Approach to Understanding Deep Learning Methodsâ€™* is published by University Grants Commission (UGC), Bangladesh in 2021. Contract signing of the book was came as **News Report:** Kalerkantho , Samakal , Alokito Bangladesh , Bangla Tribune

I regularly review articles from prominent journals of IEEE, Elsevier Science, Springer and other international publishers. I am also **Editorial Board Member** of several international journals including **Journal of Computer Science (JCS)**, a Scopus Index Open Access Journal of Science Publications, USA and Australia. I have also research collaboration with several researchers from universities from Japan, UK, USA. I expect better collaborative research in coming days.

In COVID-19 period, I was involved in design and develop a Continuous Positive Airway Pressure (CPAP) for respiratory support with Mâ€™s Engineering Solution Ltd. Dhaka . The news reports on the device: Samakal , à¦•à§‡à¦Ÿà§‡ à¦Ÿà¦²à¦¿à¦¯à¦¼à¦¾à¦¨à¦¾, à¦œà§€à¦¬à¦¨à¦•à¦¾à¦²à¦¿à¦¯à¦¼à¦¾

On November 19, 2023, I joined as the Project Director (PD) of **Improving Computer and Software Engineering Tertiary Education Project (ICSETEP)**. ICSETEP is an ADB funded (as 100 Million USD loan) national project of 1219.80 Core Taka and University Grants Commission of Bangladesh is the implementing authority. BUET, Dhaka University, and JUST are the major stakeholders in the project through new academic building constructions, modern lab developments, and curriculum updates of CSE and modern ICT fields. Research, innovations, and some other components are open for all the universities to enhance CSE/ICT tertiary education in Bangladesh.
I praised to Almighty Allah for giving me scope to work for our beloved country Bangladesh. I am also grateful to KUET authorities, faculty members, officers, and staff for their continuous support in reaching such a position. Finally, I want support and DUA from everybody to lead ICSETEP successfully so that ADB Loan becomes most fruitful for the country.

Website: https://www.kuet.ac.bd/cse/akhand/

Improving Computer and Software Engineering Tertiary Education Project (ICSETEP)

Project Director (Nov 19, 2023 to July 03,2025)

বাংলাদেশ
বিশ্ববিদ্যালয় মঞ্জুরী কমিশন

Improving Computer and Software Engineering Tertiary Education Project

University Grants Commission of Bangladesh
Secondary & Higher Education Division, Ministry of Education

ICSETEP

About ICSETEP | Management | Notice | Workshop/Training | Report & Publications | Grants | Contact Us

Message from Project Director

Education is the backbone of a nation. Modern Computer and Software Engineering (CSE) education is an element to lead a nation towards its quick prosperity through Information Technology (IT) based smart society development. The noble aim of ICSETEP is to improve socio-economic condition of Bangladesh by establishing a more relevant Higher Education in CSE/IT Programs in terms of quality, efficiency, competent graduates & cutting edge research capacity to meet the challenges of Fourth Industrial Revolution (4IR) and develop technology entrepreneurships to lead of the country in IT sectors.

ICSETEP is a unique national project, first time in Bangladesh, to construct new CSE academic buildings, develop modern labs, and update curriculums in BUET, DU and JUST. Research, innovations, and some other components are open for all the universities to enhance CSE/ICT tertiary education in Bangladesh. I hope ICSETEP will be a milestone for our beloved country to its prosperity in different sectors.

I sincerely thank UGC and SHED, Education Ministry authorities for selecting me for the Project Director (PD), ICSETEP as deputation from my KUET Professor (CSE) position. I believe proper utilization of the ICSETEP grant (including 100 Million USD ADB loan) is the key issue to reach the goal, and I will be sincere on this issue. In this regard, supports and suggestions from SHED, UGC and PIUs (of BUET, DU & JUST) are also equally necessary. Our combined efforts might make ICSETEP most fruitful through smooth operation and finish it timely.

Finally, I appreciate any suggestions for the betterment of ICSETEP.

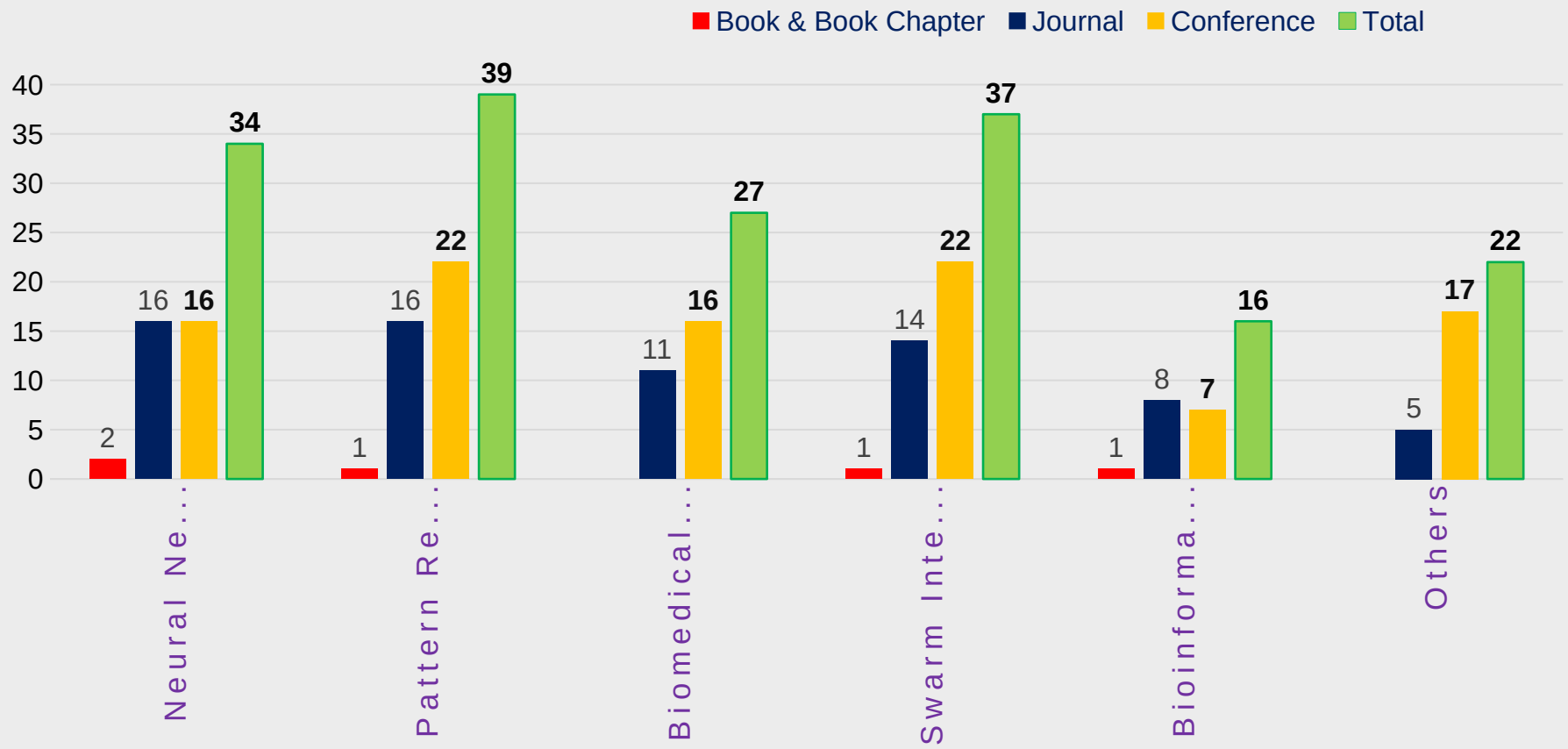
Dr. Muhammad Aminul Haque Akhand
Project Director

Dr. Muhammad Aminul Haque Akhand
Professor
Dept. of Computer Science and Engineering(CSE)
Khulna University of Engineering & Technology (KUET)

Email: pd-icsetep@ugc.gov.bd, akhand@cse.kuet.ac.bd
For PD's teaching, research and project management experiences please visit www.kuet.ac.bd/cse/akhand

Website: https://icsetep.ugc.gov.bd/

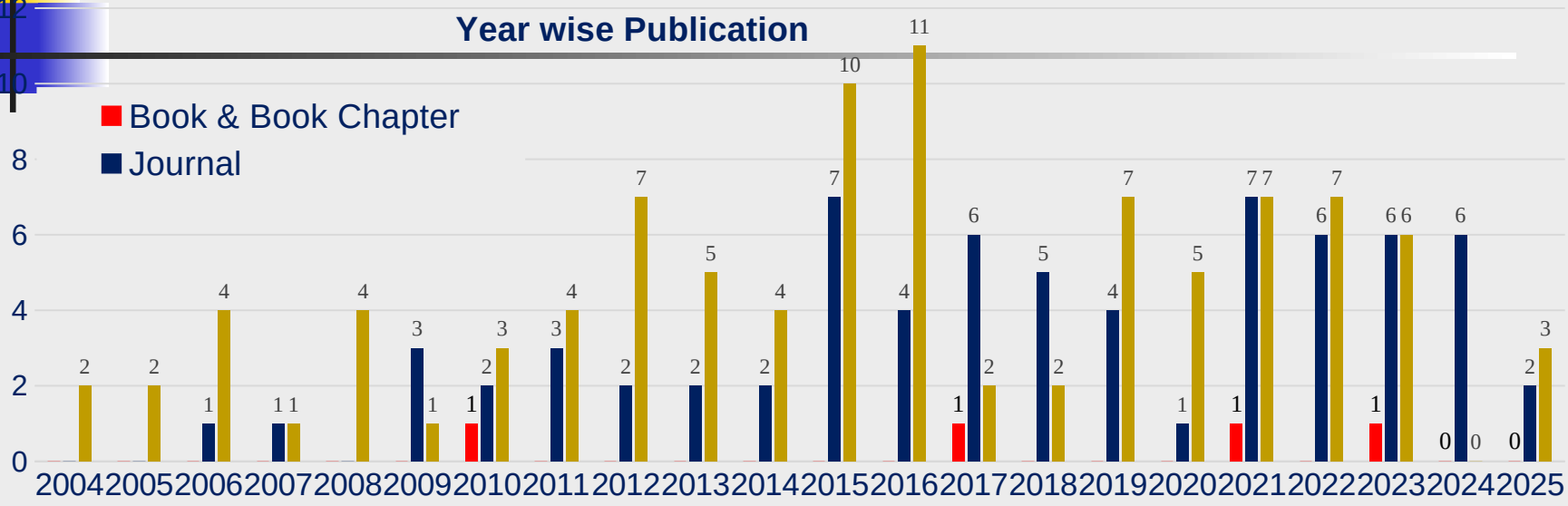
Research Outcomes: Field wise Summary



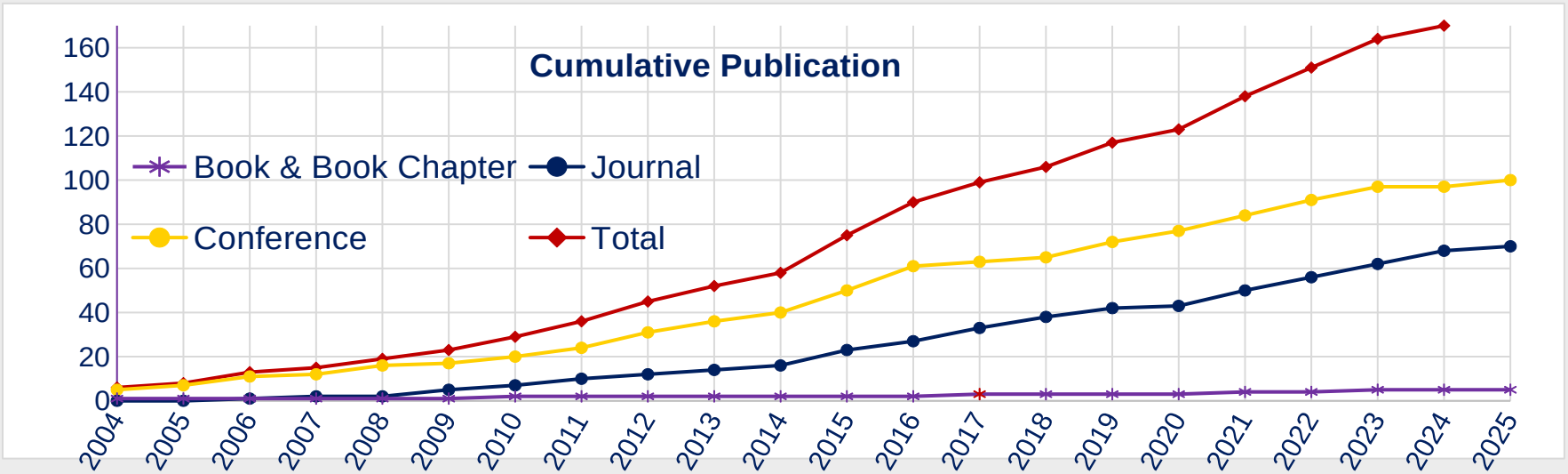
MSc and PhD
Univ. Fukui

KUET Service

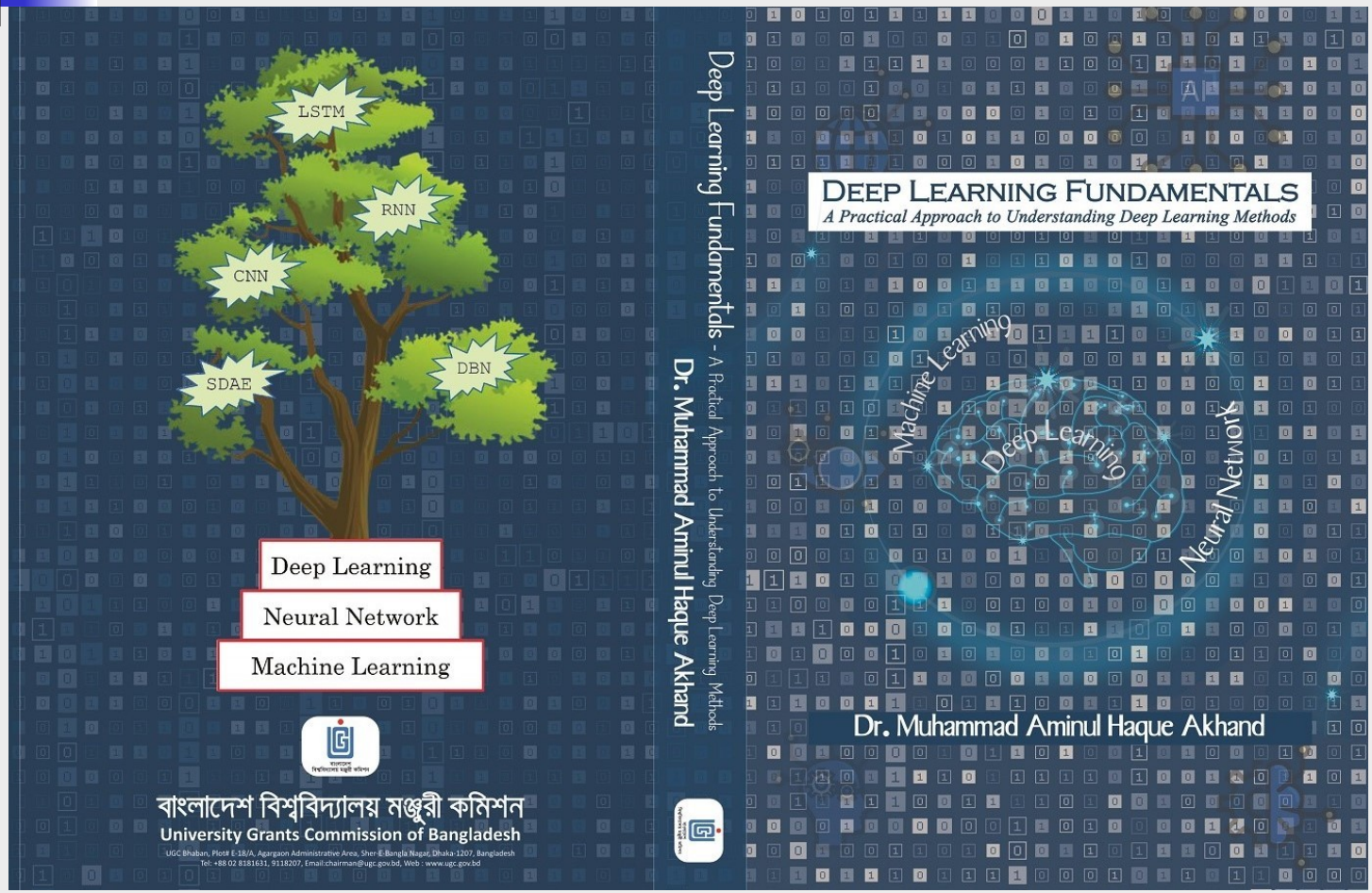
Research Outcomes: Year wise Summary



< MSc, Fukui > <PhD, Fukui> < Service in KUET >



Major Contribution: Deep Learning Book

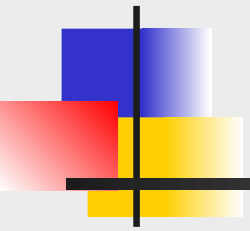


Research Achievements : Best Paper Awards

Conference and Year	Organizer / Place	Paper Title
ICIEV 2023	London, UK	Investigation of Emotional Effects on Brain Network Stimulation through EEG Signals
ICECE 2022	BUET	Emotion Recognition from EEG with Normalized Mutual Information and Convolutional Neural Network
MIET 2022	NSTU	Autism Spectrum Disorder Detection from EEG through Hjorth Parameters and Classification using Neural Network
EICT 2021	KUET	A Comprehensive Emotion Recognition from Microblogs Using Deep Learning
IJCCI 2019	ULAB	Handwritten Numeral Superposition to Printed Form Using Convolutional Auto-encoder and Recognition Using Convolutional Neural Network
ICCIT 2015	MIST	A Context Likelihood of Relatedness with Maximal Information Coefficient for Gene Regulatory Network Inference
SCIS&ISIS 2006	Tokyo, Japan	A New Algorithm to Design Multiple Hidden Layered Artificial Neural Networks

+ Some Other Awards

Thanks for Attention



Machine, Learning, Machine Learning, Deep Learning

Prof. Dr. Md. Aminul Haque Akhand
Dept. of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET)
www.kuet.ac.bd/cse/akhand

Machine

A machine is a physical system using power to apply forces and control movement to perform an action. … Machines can be driven by animals and people, by natural forces such as wind and water, and by chemical, thermal, or electrical power ...They can also include computers and sensors that monitor performance and plan movement, often called mechanical systems. **https://en.wikipedia.org/wiki/Machine**

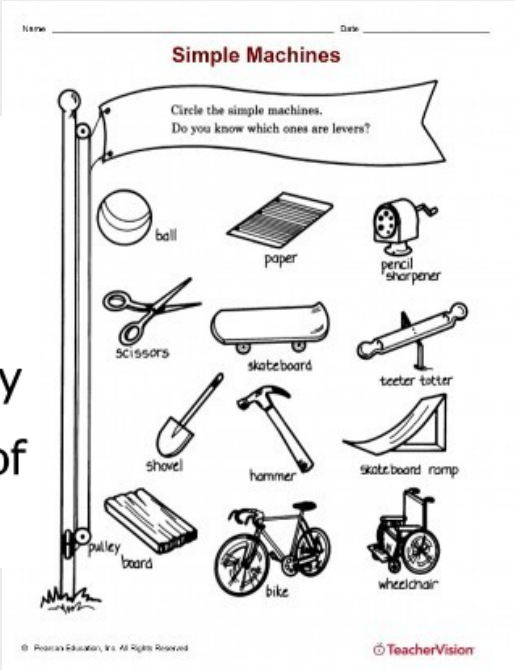
An apparatus using or applying mechanical power and having several parts, each with a definite function and together performing a particular task. *'a fax machine'*

Oxford Dictionary

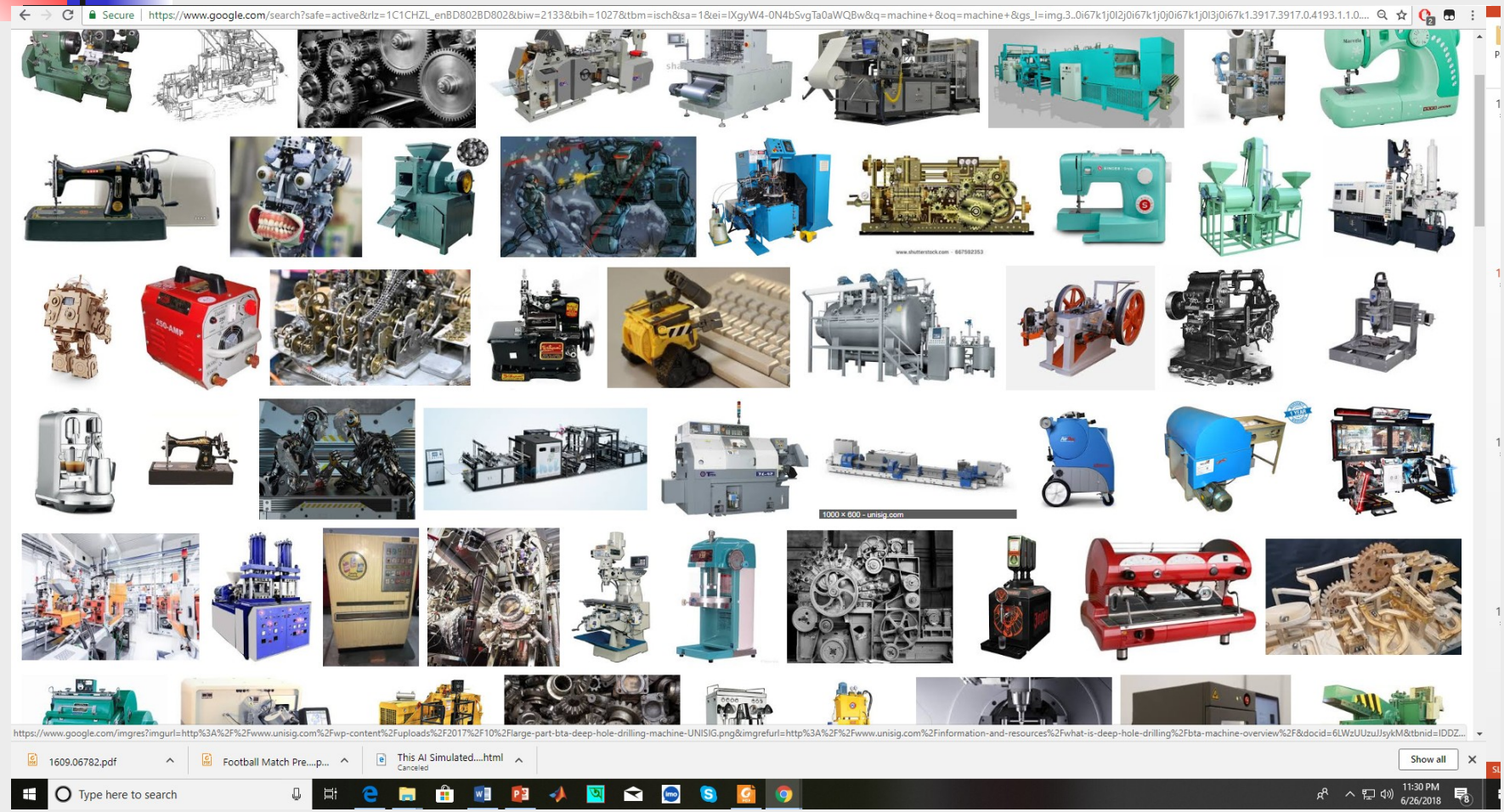
1.General: Semi or fully automated device that magnifies human physical and/or mental capabilities in performing one or more operations.

2.Mechanics: Device that makes mechanical work easier by overcoming a resistance (load) at one end by application of effort (force) at the other end.

BusinessDictionary.com



Machine



Machines include a system of mechanisms that shape the actuator input to achieve a specific application of output forces and movement.

How make decision to perform task(s)?

Learning

Learning is the process of acquiring new or existing modifying knowledge, behaviors, skills, values, or preferences

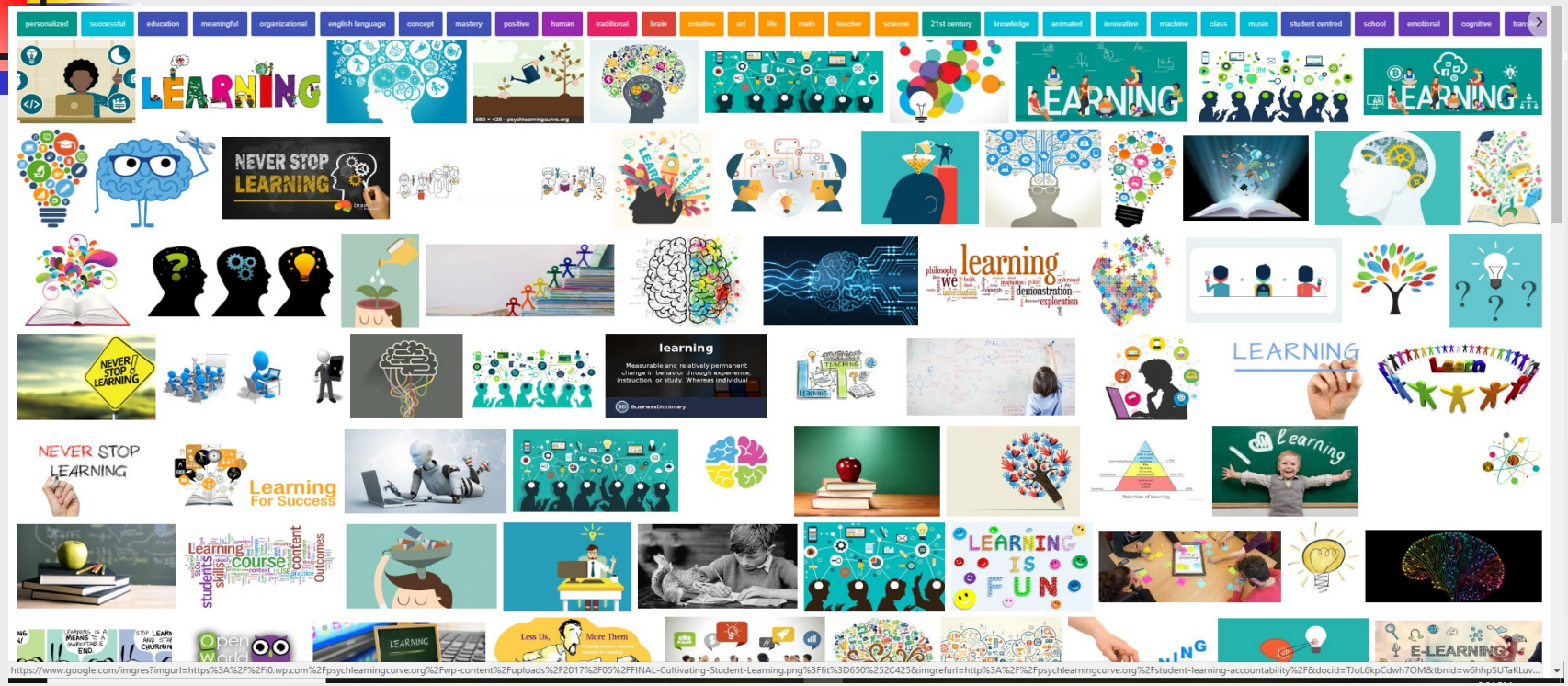
https://en.wikipedia.org/wiki/Learning

Learning is “a process that leads to change, which occurs as a result of experience and increases the potential for improved performance and future learning”
---(Ambrose et al, 2010, p.3).

“A change in human disposition or capability that persists over a period of time and is not simply ascribable to processes of growth.”

— From The Conditions of Learning by Robert Gagne

Learning



The change in the learner may happen at the level of knowledge, attitude, or behaviour. As a result of learning, learners come to see concepts, ideas, and/or the world differently.

Learning is not something done to students, but rather something students themselves do. It is the direct result of how students interpret and respond to their experiences.

https://www.teachwithmrst.com/post/what-is-learning

Human brain is the main element of learning

Machine Learning

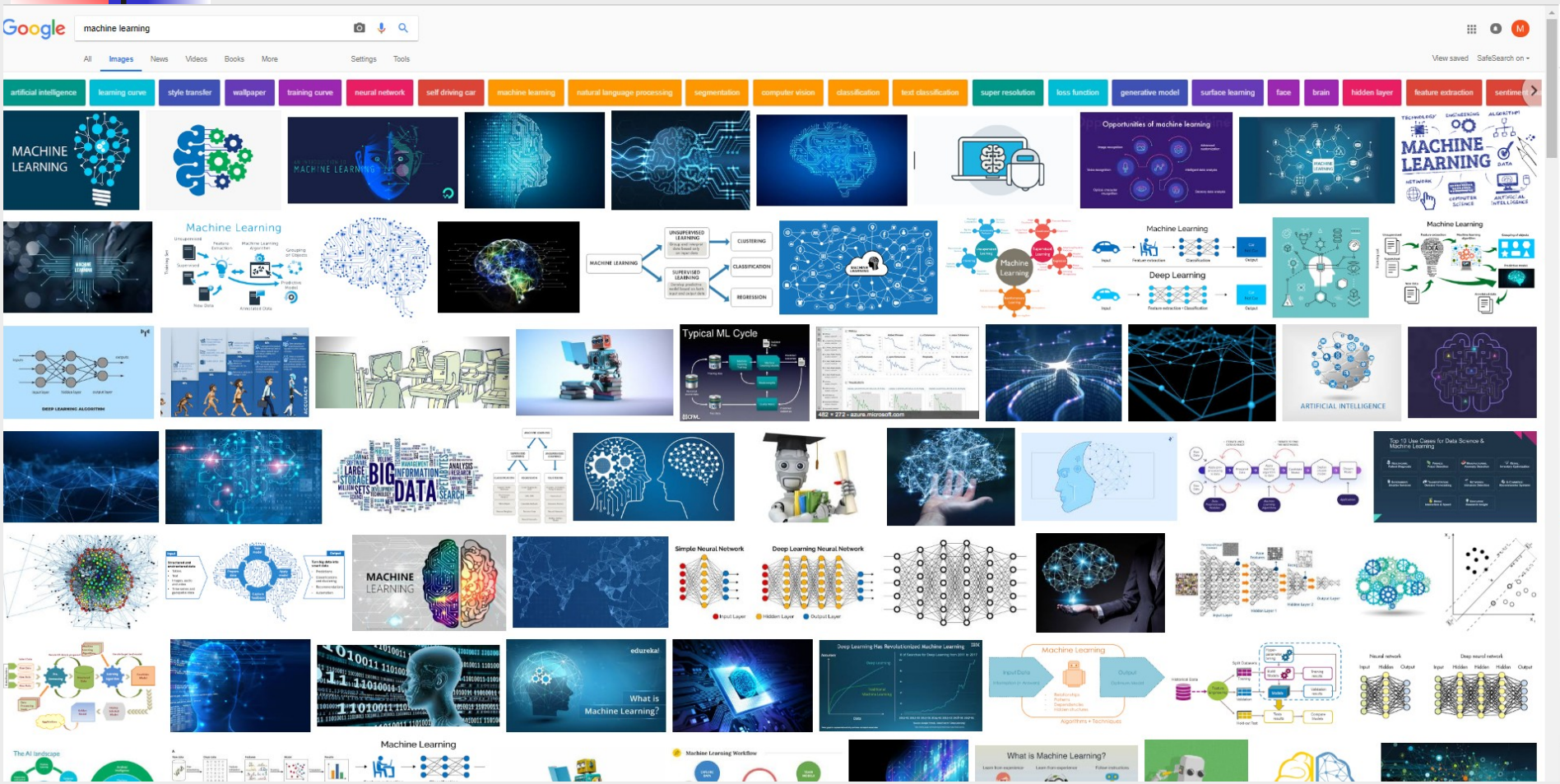
In 1959, Arthur Samuel, a pioneer in the field of machine learning (ML) defined it as the "field of study that gives computers the ability to learn without being explicitly programmed".

https://theconversation.com/what-is-machine-learning-76759

Machine learning (ML) is the study of computer algorithms that can improve automatically through experience and by the use of data.[1] It is seen as a part of artificial intelligence. Machine learning algorithms build a model based on sample data, known as training data, in order to make predictions or decisions without being explicitly programmed to do so.[2] Machine learning algorithms are used in a wide variety of applications, such as in medicine, email filtering, speech recognition, and computer vision, where it is difficult or unfeasible to develop conventional algorithms to perform the needed tasks.[3]

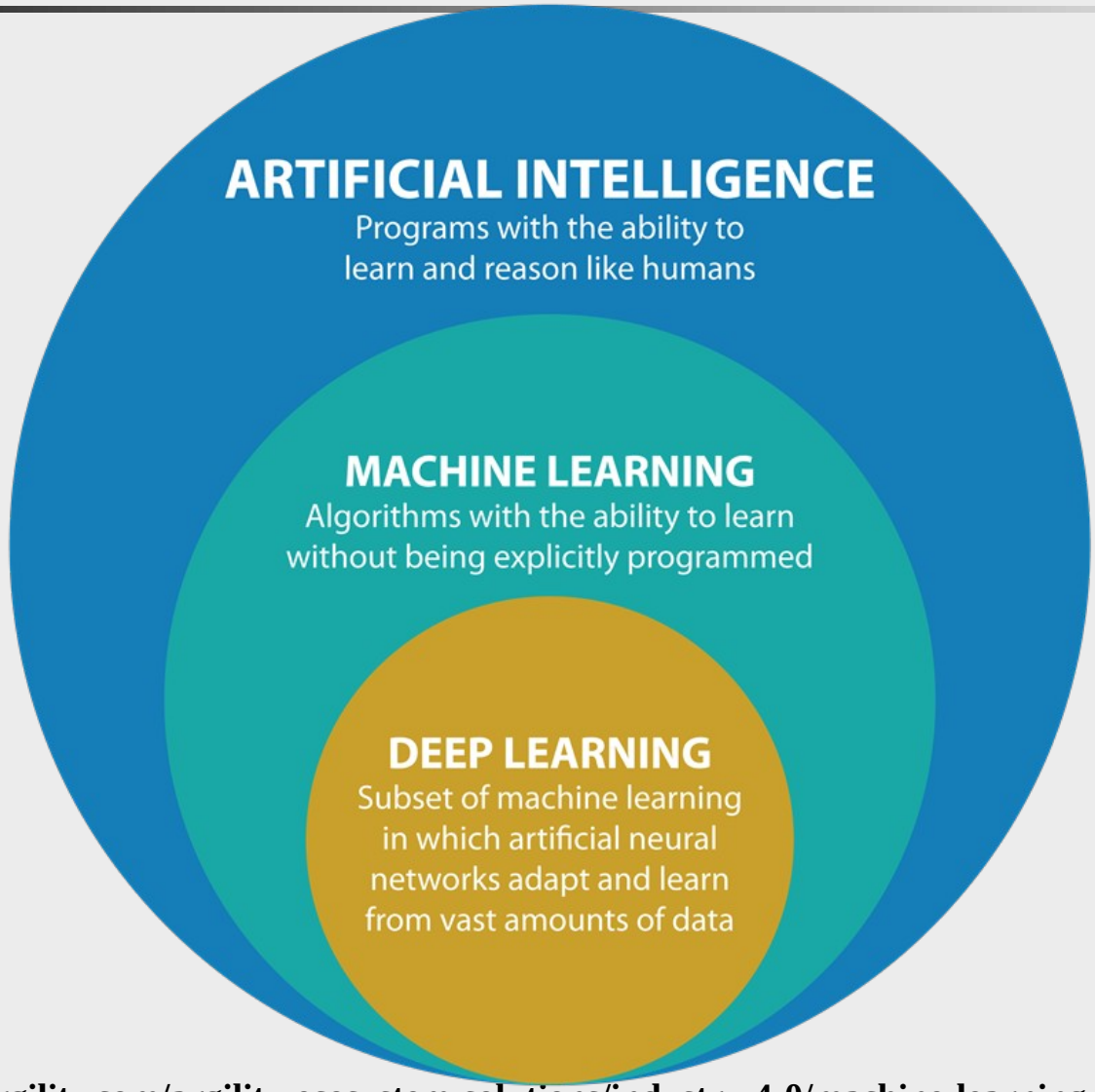
https://en.wikipedia.org/wiki/Machine_learning

Machine Learning



Technique to give computer brain like learning ability through progressively update with data, without being explicitly programmed.

AI ->Machine Learning->Deep Learning



https://www.argility.com/argility-ecosystem-solutions/industry-4-0/machine-learning-deep-learning/

AI ->Machine Learning->Deep Learning

Artificial Intelligence (AI)

AI is the broadest term, applying to any technique that enables computers to mimic human intelligence, using logic, if-then rules, decision trees, and machine learning (including deep learning).

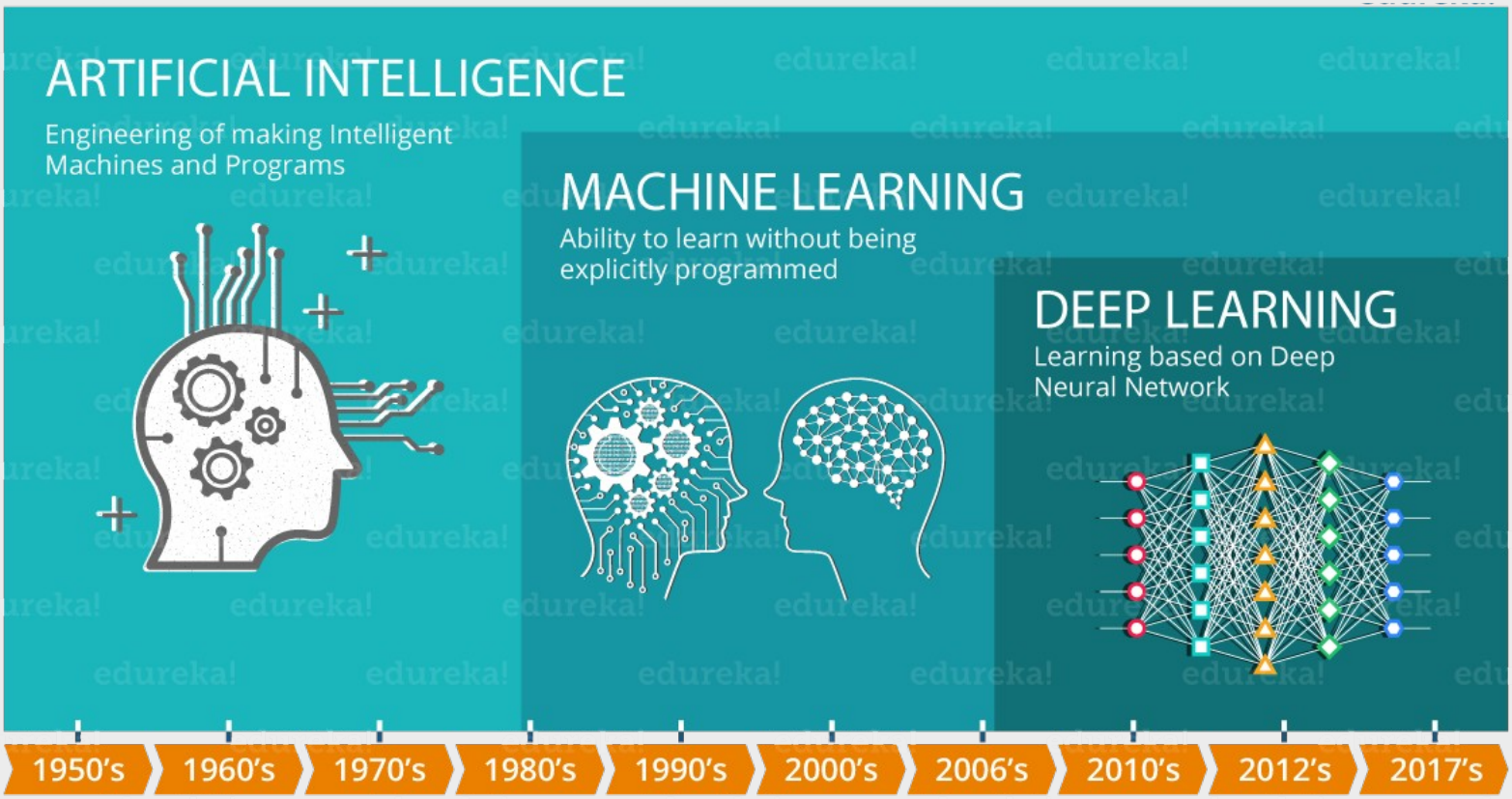
Machine Learning

The subset of AI that includes that enable machines to improve at tasks with experience. The category includes deep learning.

Deep Learning

The subset of machine learning composed of algorithms to train itself to perform tasks, like speech and image recognition, by exposing multilayered neural networks to vast amounts of data.

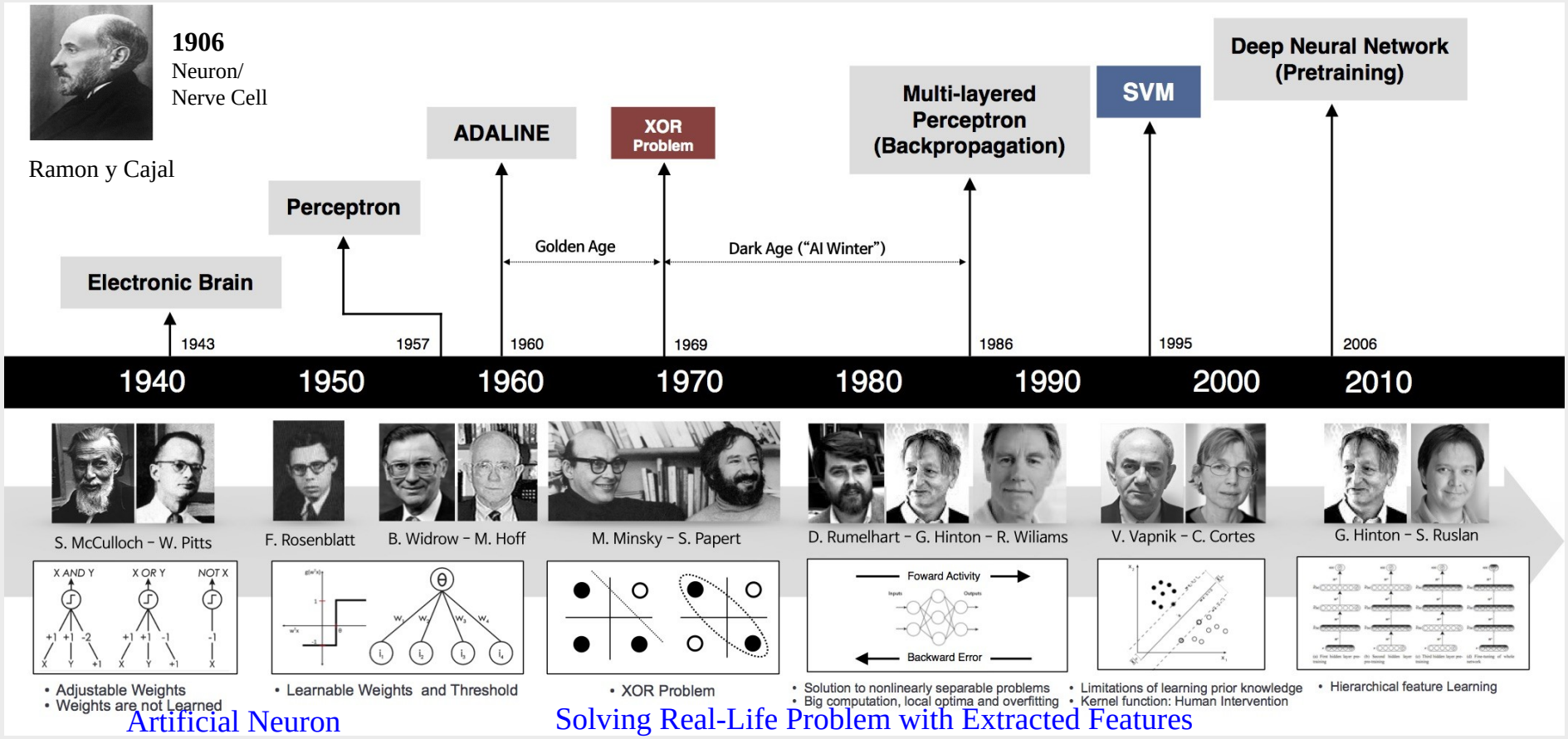
AI ->Machine Learning->Deep Learning



History of ML and Deep Learning

1906
Neuron/
Nerve Cell

Ramon y Cajal



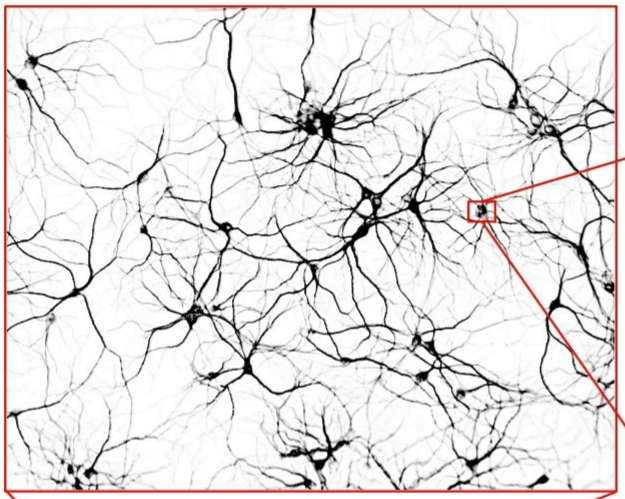
Development of Traditional Machine Learning

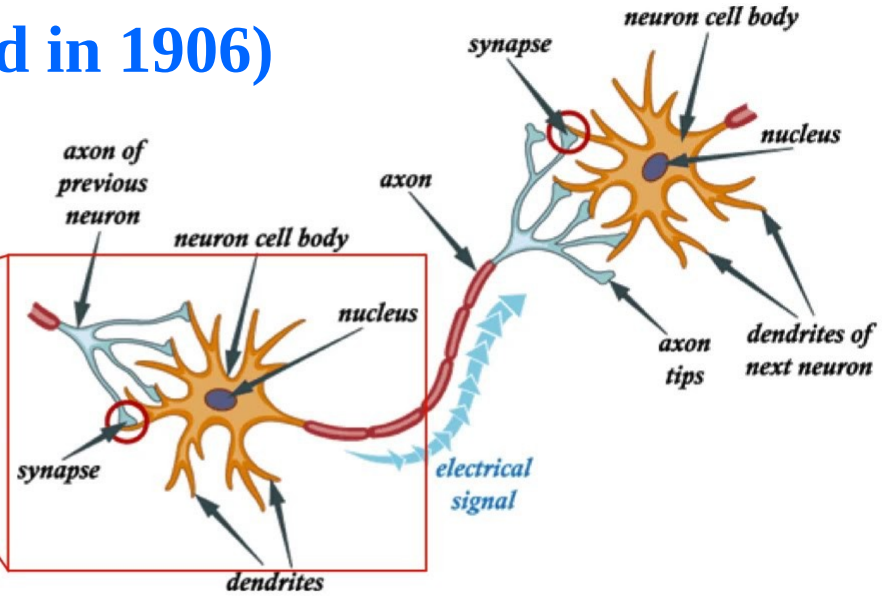
Deep Leaning

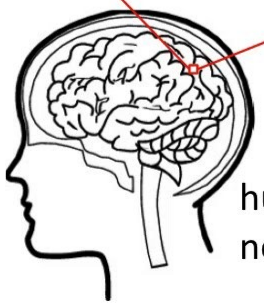
https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html

Neuron : Learning Element in Brain

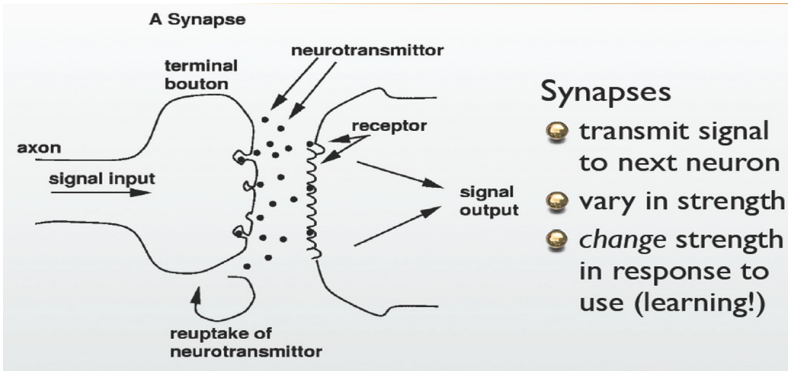
(Discovered in 1906)







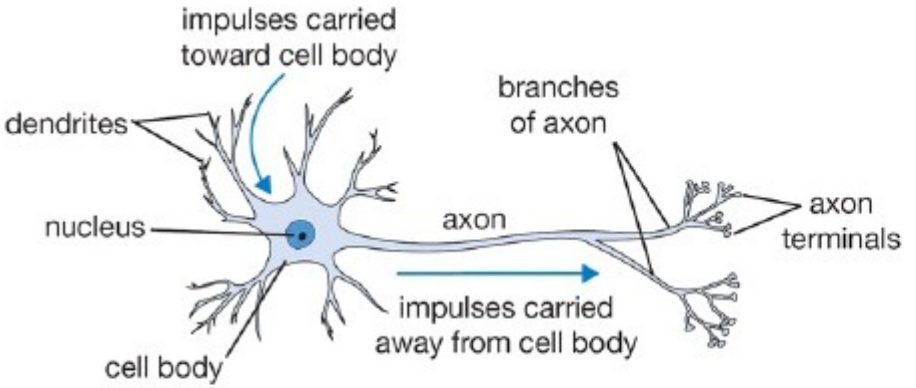
humans don't need features

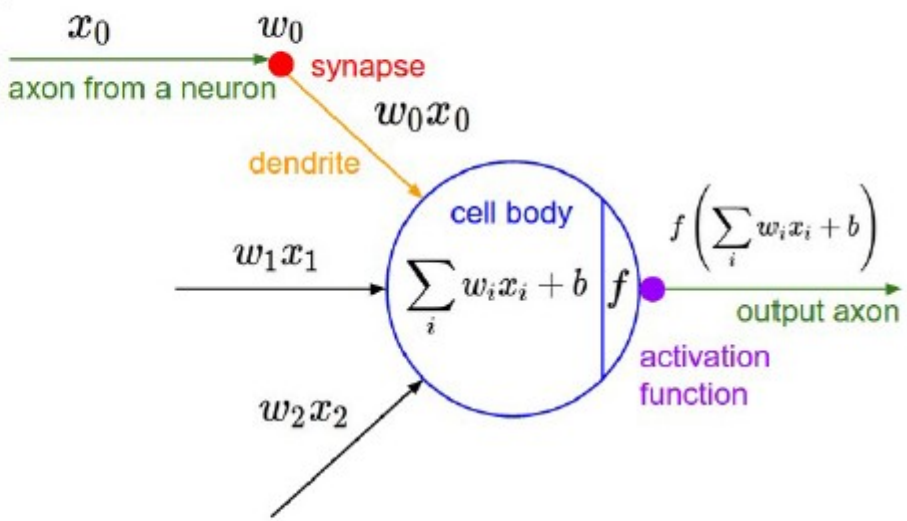


Copyright © 2014 Victor Lavrenko

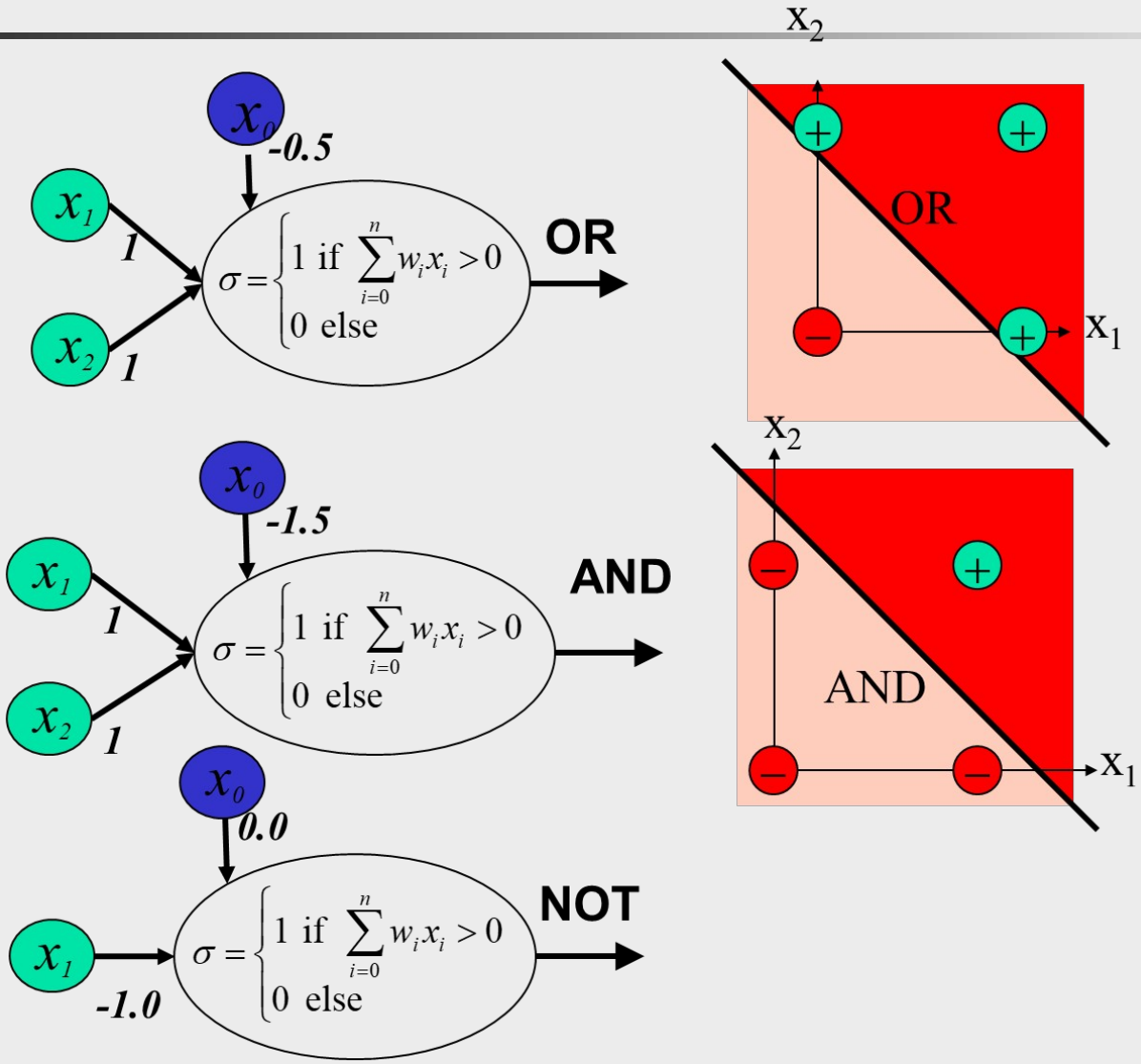
McCulloch-Pitts model of a Neuron

(Proposed in 1943)

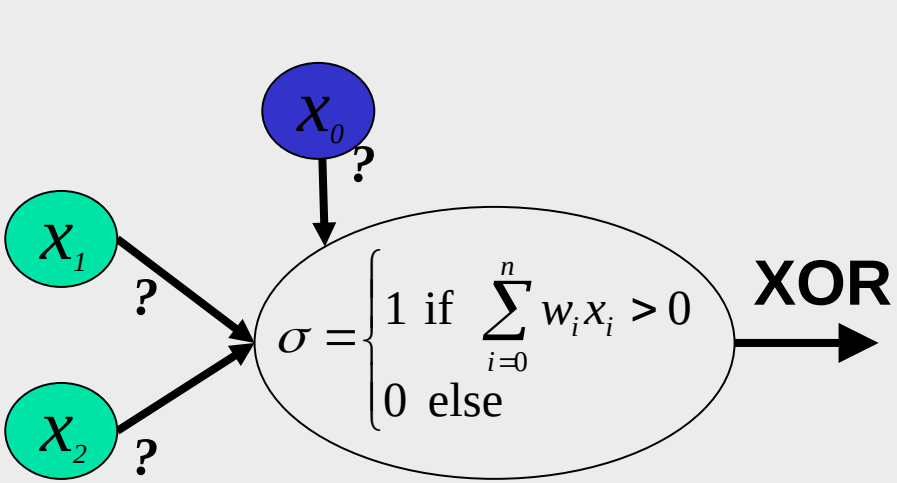


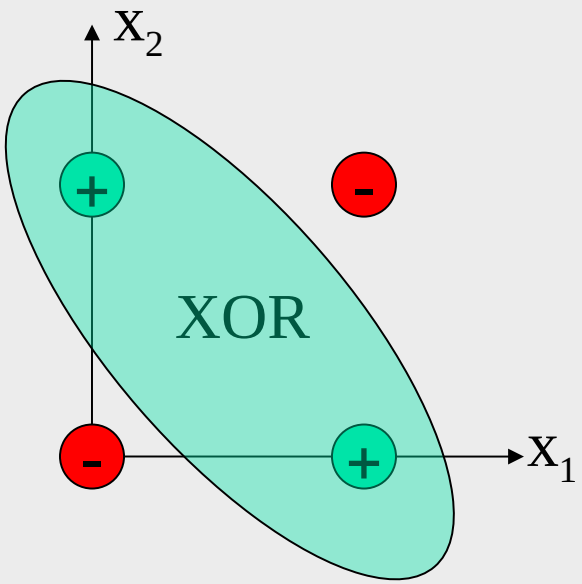


Ability of Single Neuron : Solving Linear Problem



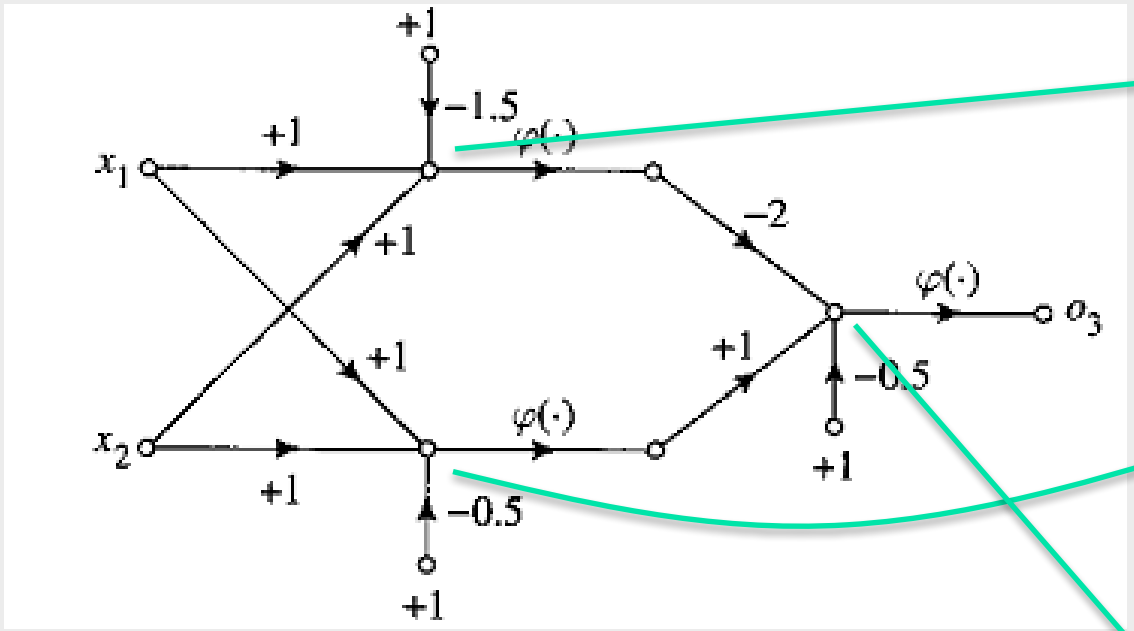
Ability of Single Neuron: Unable to Solve Nonlinear Problem

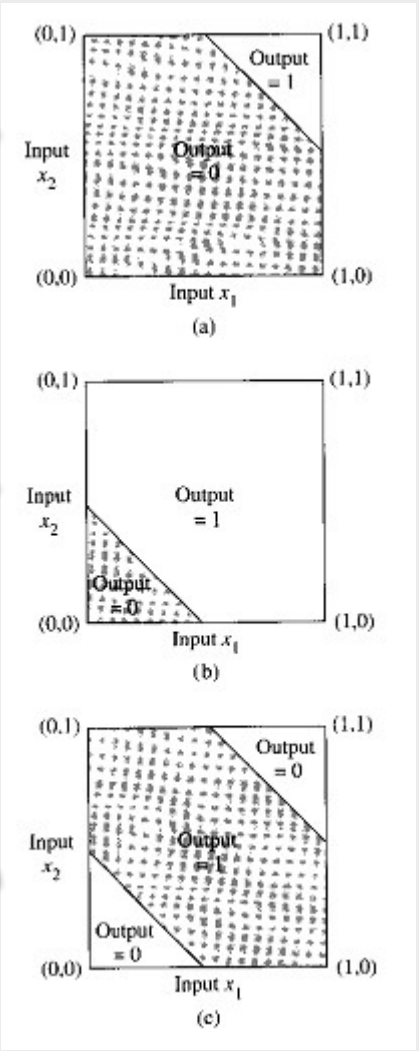




No arrangement work, not linearly separable. Require two boundary lines.

XOR solution with Hidden Neuron





AND

OR

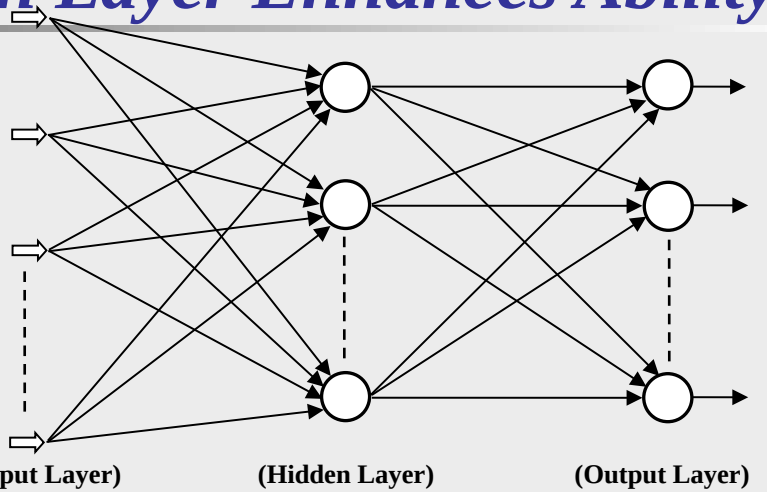
XOR

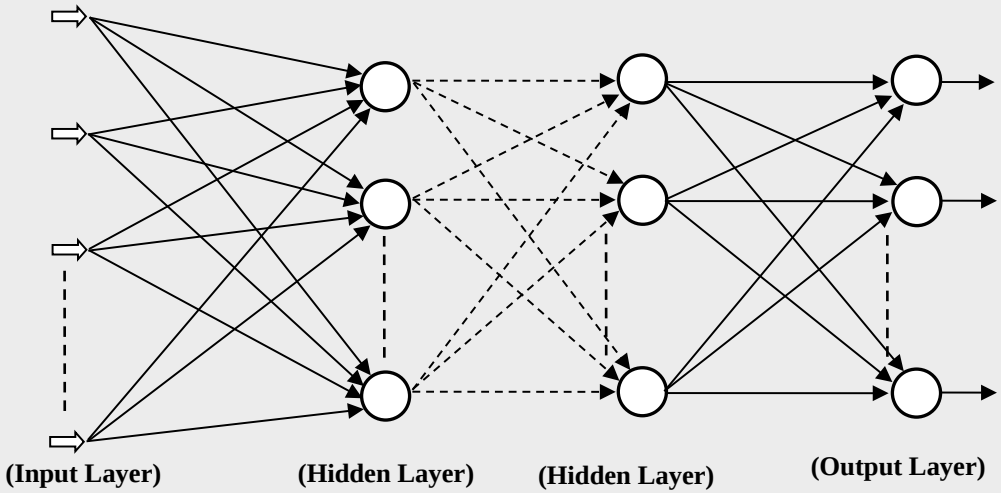
XOR logic through manipulation by hidden neurons

Open doors to Solve Real life Nonlinear Problem

Neural Network: Hidden Layer Enhances Ability

Hidden Layer(s) Enhances Ability to Solve Real Life Nonlinear Problems



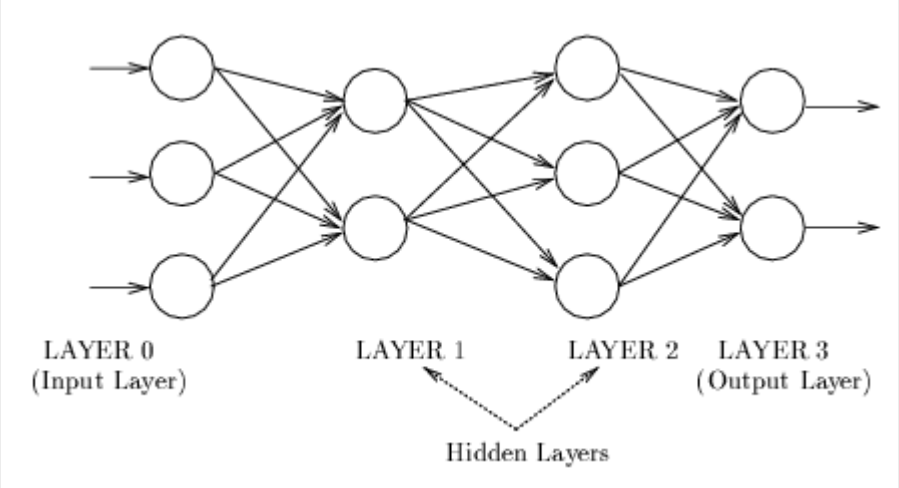


How to Select Appropriate Weight ?
Learning?

How to get appropriate weight set?

Back-propagation is the famous algorithm for training feed-forward networks

(Proposed in 1986)



http://en.wikipedia.org/wiki/Backpropagation

Backpropagation, or propagation of error, is a common method of teaching artificial neural networks how to perform a given task.

It was first described by Paul Werbos in 1974, but it wasn't until 1986, through the work of David E. Rumelhart, Geoffrey E. Hinton and Ronald J. Williams, that it gained recognition, and it led to a **"renaissance"** in the field of artificial neural network research.

nature publishing group

nature.com | about npg | news@nature.com | naturejobs | natureevents | help | site index

nature

my account | e-alerts | subscribe | register

SEARCH JOURNAL Go

Tuesday 06 October 2009

Journal Home
Current Issue
AOP
Archive

THIS ARTICLE

Download PDF
References

Export citation
Export references

Send to a friend

More articles like this

Table of Contents
< Previous | Next >

letters to nature

Nature **323**, 533 - 536 (09 October 1986); doi:10.1038/323533a0

Learning representations by back-propagating errors

DAVID E. RUMELHART*, GEOFFREY E. HINTON† & RONALD J. WILLIAMS*

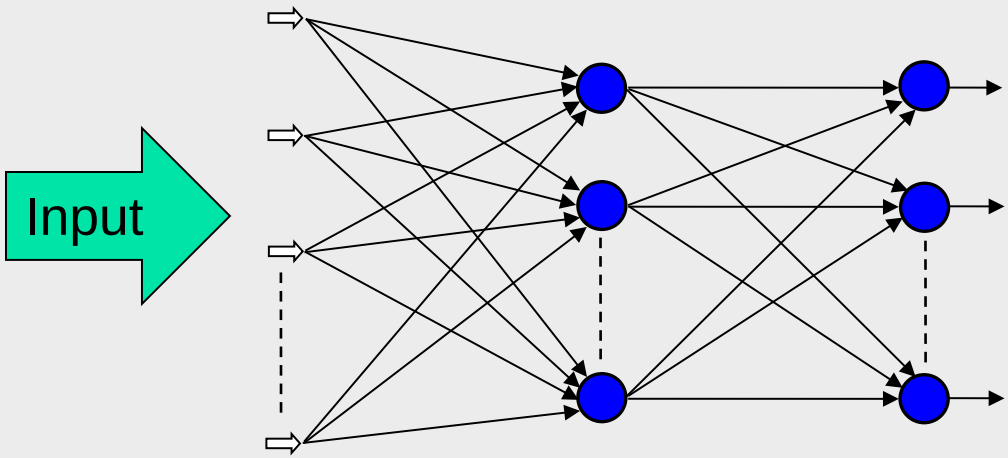
*Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
†Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA
†To whom correspondence should be addressed.

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

References

1. Rosenblatt, F. *Principles of Neurodynamics* (Spartan, Washington, DC, 1961).
2. Minsky, M. L. & Papert, S. *Perceptrons* (MIT, Cambridge, 1969).
3. Le Cun, Y. *Proc. Cognitiva* **85**, 599–604 (1985).

Back-propagation(BP)

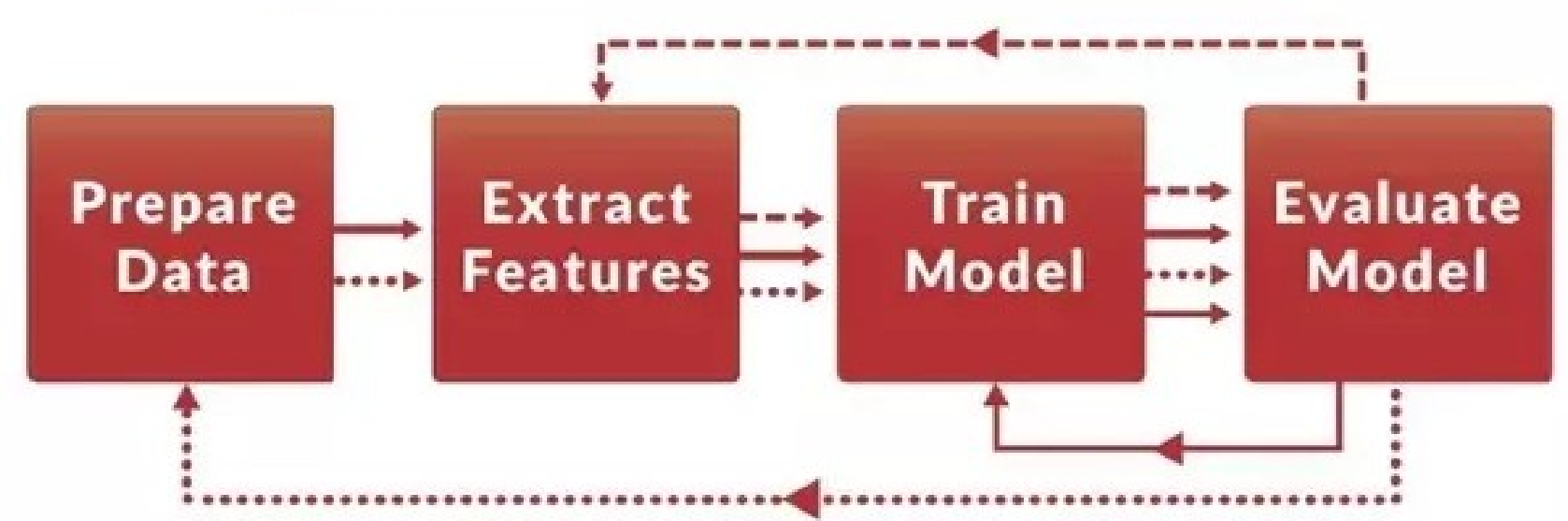


Error =
Desired Output
- Actual Output

Forward Pass: Error is calculated based on desired and actual output.

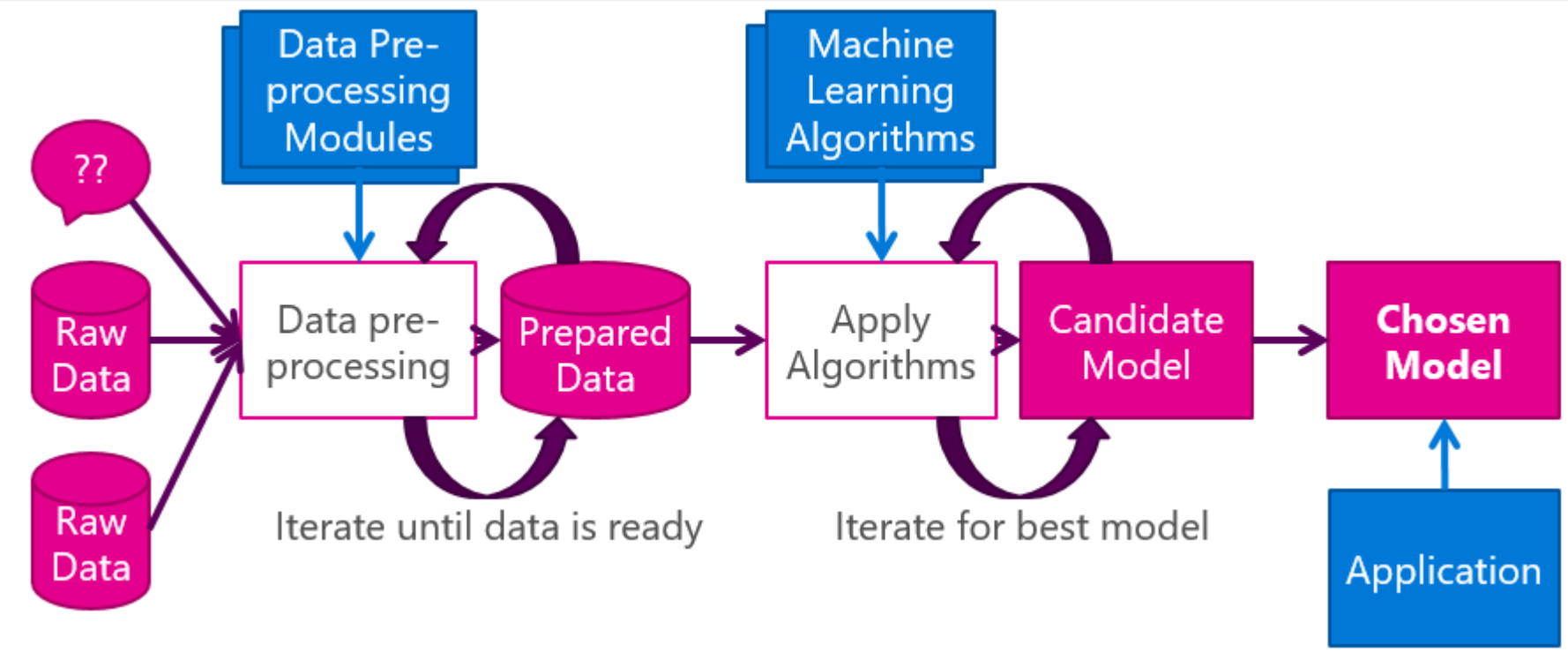
Backward Pass: Synaptic weights are adjusted based on calculated error.

Machine Learning Process



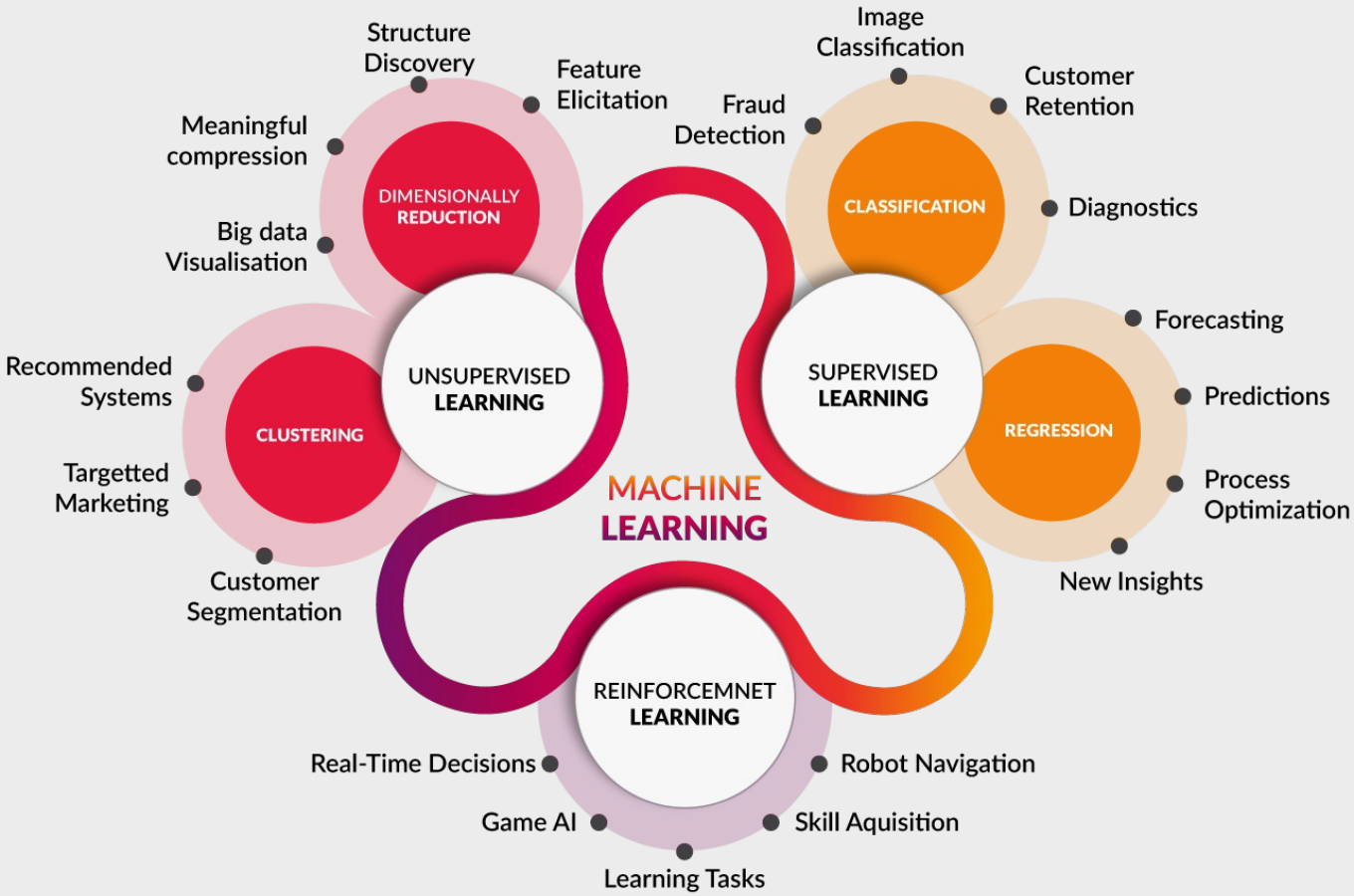
Performance depends on feature extraction which requires heuristics and, most notably, a great deal of time

Machine Learning Process (Different View)



Performance depends on data pre-processing (i.e., feature extraction) which requires human expertize

Applications of Machine Learning



UCI Machine Learning Repository - Windows Internet Explorer

http://archive.ics.uci.edu/ml/

UCI Machine Learning Repository

About Citation Policy Donate a Data Set Contact

Search

Repository Web

View ALL Data Sets

Machine Learning Repository

Center for Machine Learning and Intelligent Systems

Welcome to the UC Irvine Machine Learning Repository!

We currently maintain 174 data sets as a service to the machine learning community. You may **view all data sets** through our searchable interface. Our old web site is still available, for those who prefer the old format. For a general overview of the Repository, please visit our About page. For information about citing data sets in publications, please read our citation policy. If you wish to donate a data set, please consult our donation policy. For any other questions, feel free to contact the Repository librarians. We have also set up a mirror site for the Repository.

Supported By: In Collaboration With: Rexa.info

Latest News:

07-23-2008: Repository mirror has been set up.
03-24-2008: New data sets have been added!
06-25-2007: Two new data sets have been added: UJI Pen Characters, MAGIC Gamma Telescope
04-13-2007: Research papers that cite the repository have been associated to specific data sets.
04-09-2007: Three new data sets have been added: Poker Hand, Callt2 Building People Counts, Dodgers Loop Sensor.
09-08-2006: The Beta site has been launched.
09-01-2006: SPECTF.test has been modified by the donor.

Featured Data Set: Statlog (Shuttle)

Task: Classification
Data Type: Multivariate
 # Attributes: 9
 # Instances: 58000

The shuttle dataset contains 9 attributes all of which are numerical. Approximately 80% of the data belongs to class 1

Newest Data Sets:

06-26-2008: Parkinsons
04-21-2008: Ozone Level Detection
04-03-2008: Abscisic Acid Signaling Network
03-20-2008: Hill-Valley
03-12-2008: Bag of Words
03-08-2008: Reuters Transcribed Subset
02-29-2008: Gisette
02-29-2008: Dorothea
02-29-2008: Madelon

Most Popular Data Sets (hits since 2007):

39351: Iris
31585: Adult
26458: Wine
23553: Breast Cancer Wisconsin (Diagnostic)
18449: Abalone
18321: Poker Hand
13471: Yeast
12996: Internet Advertisements
11793: SPECT Heart

UCI Machine Learning Repository - Windows Internet Explorer

Thesis | idea of neural ne... | UCI Machine Lea... | FTP directory: /p... | Ensemble of Div... | H-Bibliography.d... | Desktop | 4:39 PM

Benchmark Problems

Problems Related to Human Life

Problem	Task
Breast Cancer Wisconsin	Predicts whether a tumor is benign (not dangerous to health) or malignant (dangerous) based on a sample tissue taken from a patient's breast.
BUPA Liver Disorder	Identify lever disorders based on blood tests along with other related information such as alcohol consumption.
Diabetes	Investigate whether the patient shows or not the signs of diabetes.
Heart Disease Cleveland	Predicting whether at least one of four major heart vessels is reduced in diameter by more than 50%.
Hepatitis	Anticipate status (i.e., live or die) of hepatitis patient.
Lymphography	Predict the situation of lymph nodes and lymphatic vessels.
Lungcancer	Identify types of pathological lung cancers.
Postoperative	Determine place to send patients for postoperative recovery.

Input Features of Diabetes

1. Number of times pregnant
2. Plasma glucose concentration
3. Diastolic blood pressure
4. Triceps skin fold thickness (mm)
5. 2-Hour serum insulin (mu U/ml)
6. Body mass index
7. Diabetes pedigree function
8. Age

Benchmark Problems

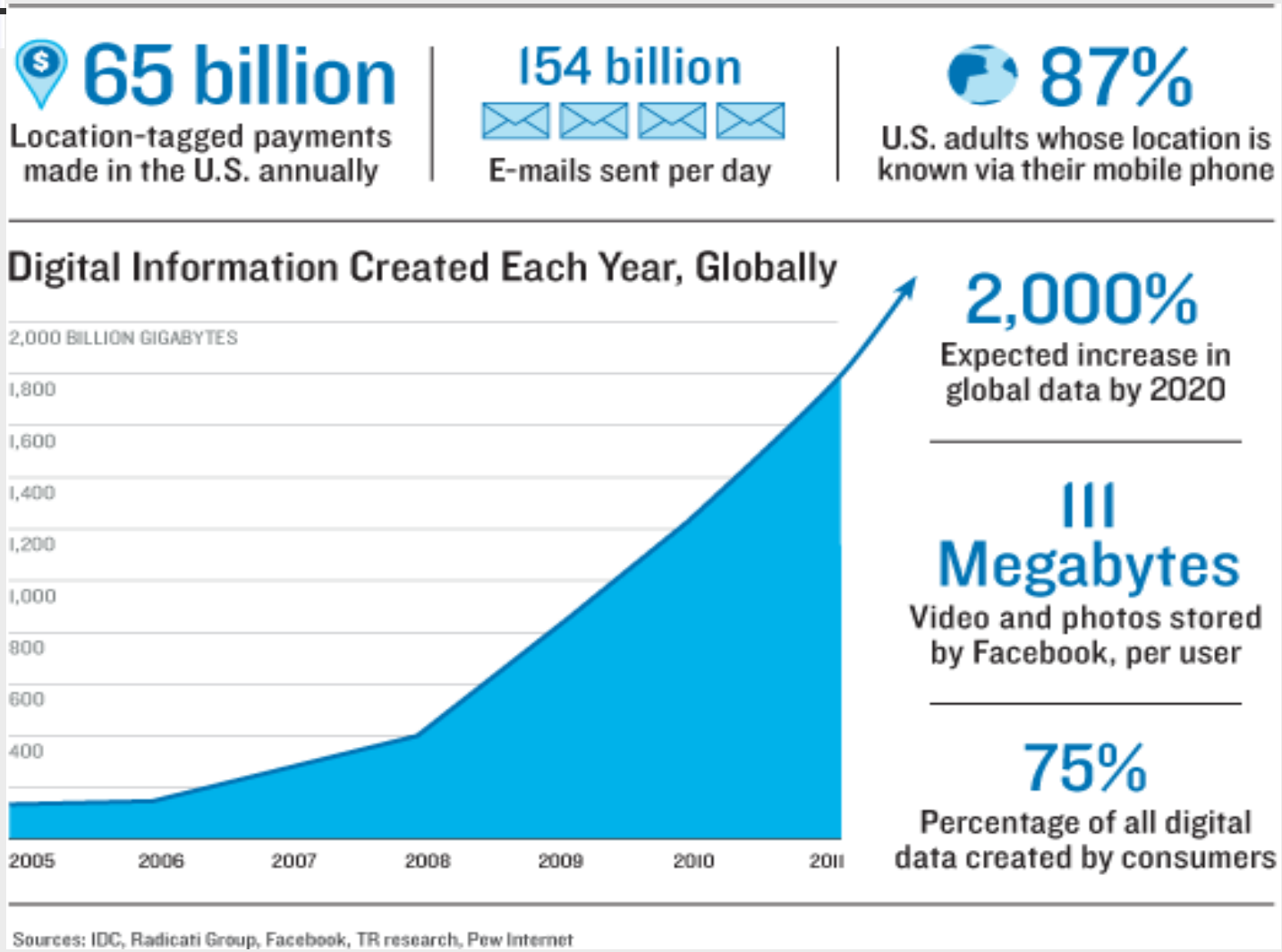
Problems Related to Finance

Problem	Task
Australian Credit Card	Classify people as good or bad credit risks depend on applicants' particulars.
Car	Evaluate cars based on price and facilities.
Labor Negotiations	Identify a worker as good or bad i.e., contract with him beneficial or not.
German Credit Card	Like Australian Card, this problem also concerns to predict the approval or non-approval of a credit card to a customer.

Problems Related to Plants

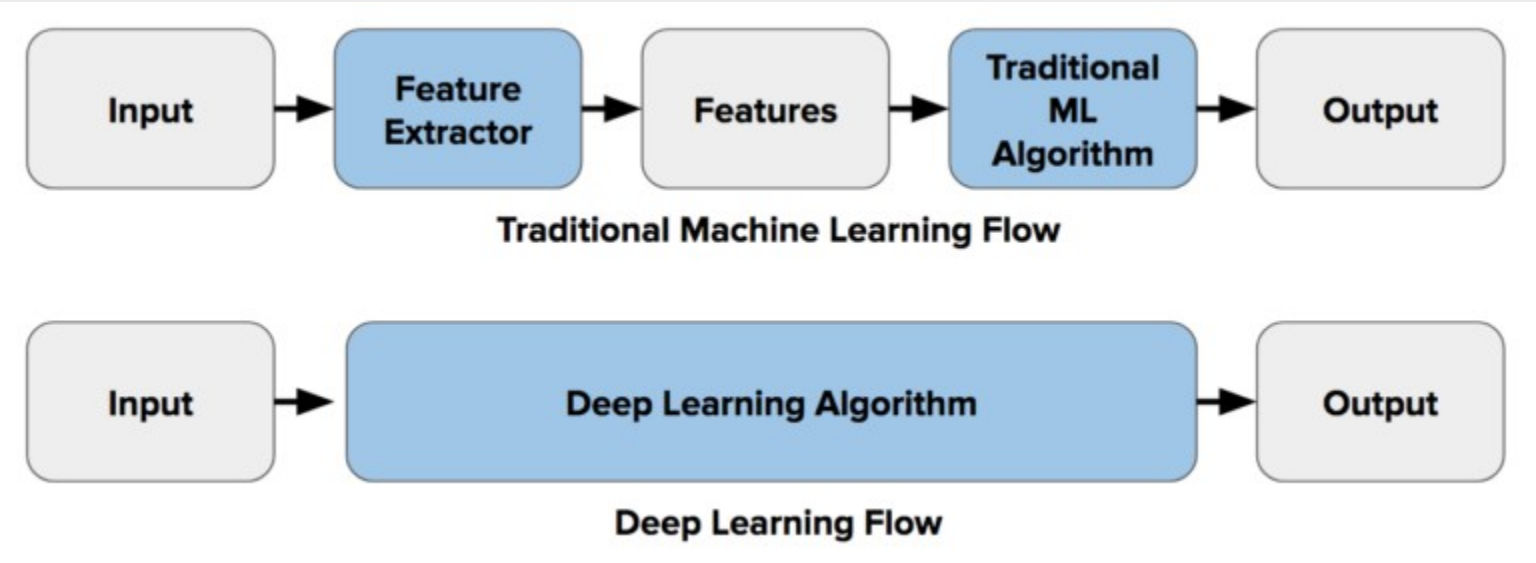
Problem	Task
Iris Plants	Classify iris plant types.
Mushroom	Identify whether a mushroom is edible or not based on a description of the mushroom's shape, color, odor, and habitat.
Soybean	Recognize 19 different diseases of soybeans.

Why Deep Architecture?

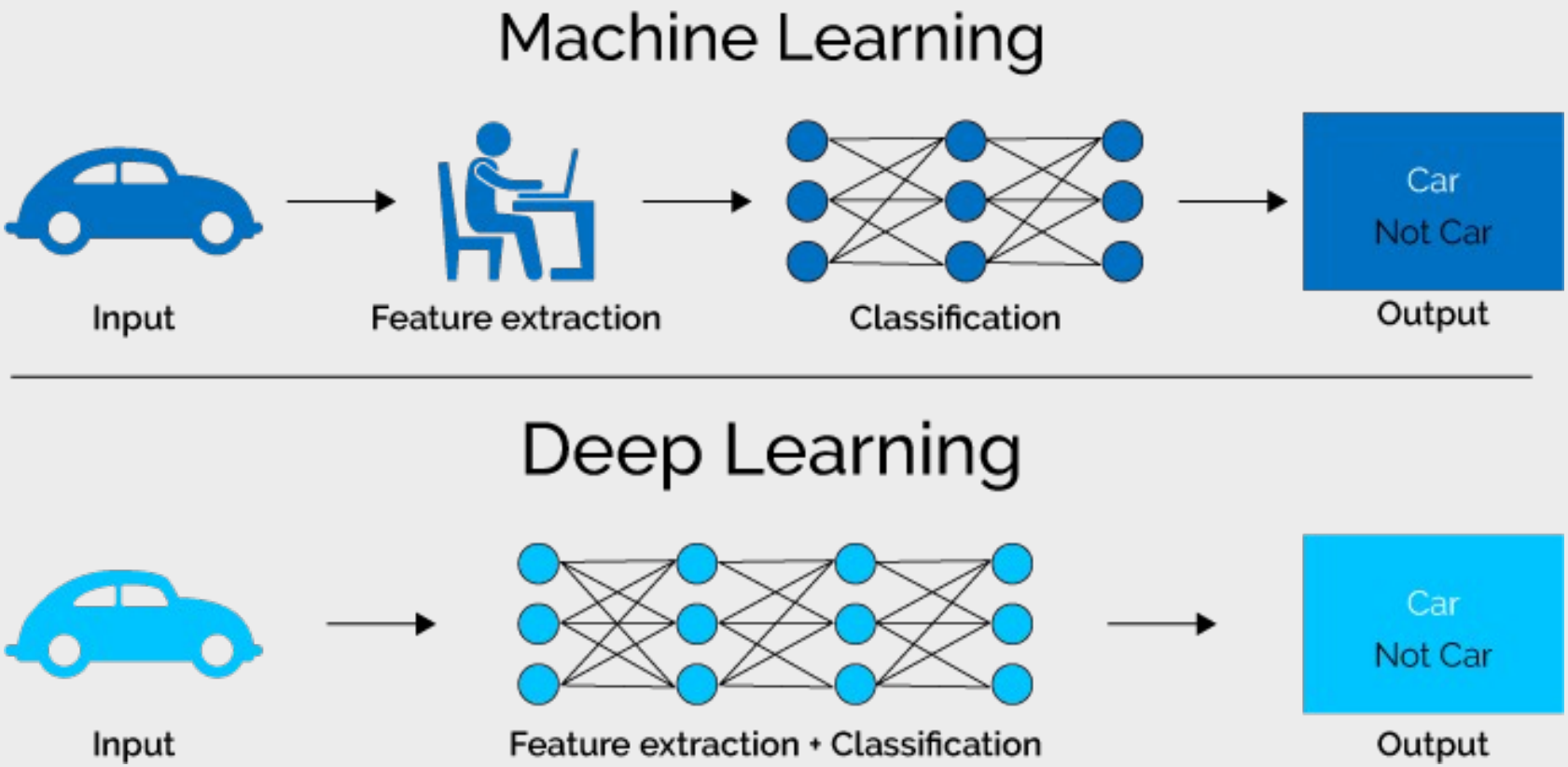


For huge heterogeneous data feature extraction is challenging and training time unacceptable

Motivation of Deep Learning



Machine Learning vs Deep Learning



Understanding Deep Learning

1. what exactly is deep learning ? And,

2. why is it generally better than other methods on image, speech and certain other types of data?

The short answers

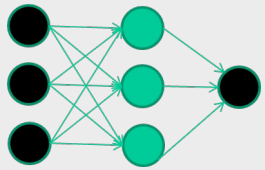
1. ***'Deep Learning' means using a neural network with several layers of nodes between input and output***

2. ***the series of layers between input & output do feature identification and processing in a series of stages, just as our brains seem to.***

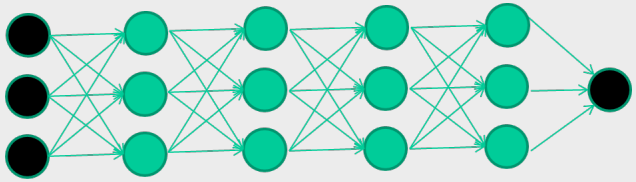
Understanding Deep Learning

3. multilayer neural networks have been around for 25 years. What's actually new?

we have always had good algorithms for learning the weights in networks with 1 hidden layer



but these algorithms are not good at learning the weights for networks with more hidden layers



what's new is: ***<u>algorithms for training many-layer networks</u>***

When to use Deep Learning (DL)?

- **DL** outperforms other techniques if the ***data size is large***.
- When there is ***lack of domain understanding for feature introspection***, **DL** techniques outshines others as you have to worry less about feature engineering.
- **DL** really shines when it comes to ***complex problems such as image classification, natural language processing, and speech recognition***.

60+ STARTUPS USING DEEP LEARNING

CORE AI: COMPUTER VISION

clarifai · Tractable · netra · HYPERVERGE · captricity · deepomatic · sentient · Sighthound · terraloupe · imagenii · pilot.ai

CORE AI: OTHER

TERADEEP · affectiva · Wave Computing · DigitalGenius · LEAPMIND · Arya · twentybn

BI, SALES & CRM

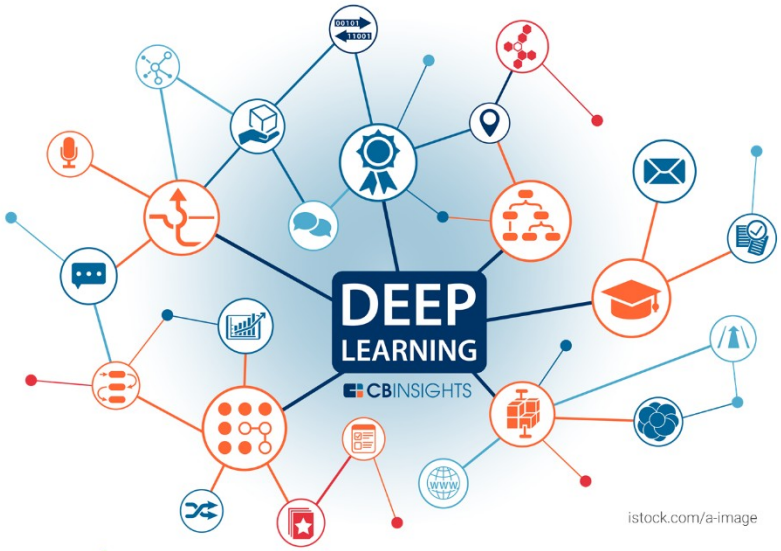
SKYMIND · TalkIQ · intranetum · ripjar · FRACTAL INDUSTRIES · MARIANA · DEEPGRAM · True AI · KOVID · ditto · Gridspace

CORE AI: VOICE INTERFACE

VIV · Mobvoi 出门问问 · MaluubA · OBEN

ROBOTICS & AUTO

netradyne · AdasWorks · drive.ai · Rokid · 图灵机器人 TURING ROBOT · comma.ai



E-COMMERCE

Reflektion · CORTEXICA · ViSENZE · STAQU

ACQUIRED

(2014-2016YTD)

Etsy Blackbird · a Orbeus · IBM AlchemyAPI · intel Movidius · nervana SYSTEMS · Perceptio · vocaliq · turi · DEEPMIND · DNNresearch · Dark Blue Labs · Vision Factory · MAGIC PONY TECHNOLOGY · MADBITS · salesforce MetaMind

HEALTHCARE

imagia · Mindshare MEDICAL · BAYLABS · BUTTERFLY Network, Inc. · deep genomics · Atomwise · zebra MEDICAL VISION · PATHWAY GENOMICS · AVALON AI · Behold.ai · MedyMatch · ADVENIO · enlitic · Lunit · SIGTUPLE

SECURITY

deepinstinct · umbo CV · signalsense

OTHER

Alpaca · indico · Iris Automation

CBINSIGHTS

Deep Learning Review in Nature

rnal/v521/n7553/full/nature14539.html

nature International weekly journal of science

Search Advanced search

Home | News & Comment | Research | Careers & Jobs | Current Issue | Archive | Audio & Video | For Authors

Archive > Volume 521 > Issue 7553 > Insights > Reviews > Article

NATURE | INSIGHT | REVIEW

Deep learning

Yann LeCun, Yoshua Bengio & Geoffrey Hinton

Affiliations | Corresponding author

Nature **521**, 436–444 (28 May 2015) | doi:10.1038/nature14539

Received 25 February 2015 | Accepted 01 May 2015 | Published online 27 May 2015

PDF | Citation | Rights & permissions | Article metrics

Abstract

Abstract · Supervised learning · Backpropagation to train multilayer architectures · Convolutional neural networks · Image understanding with deep convolutional networks · Distributed representations and language processing · Recurrent neural networks · The future of deep learning · References · Acknowledgements · Author information

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

Subject terms: Mathematics and computing · Computer science

At a glance

nl#insight

TOP TEN
Business Intelligence
TRENDS FOR 2017
GET THE WHITEPAPER
+ableau

Editors' pick

Image credit: Gabriela Hasbun for Nature

The rise and fall and rise again of 23andMe

Science jobs | Science events

naturejobs.com

Professor and Faculty Positions at the Academy of Medical Sciences (AMS), Zhengzhou University
The Academy of Medical Sciences of Zhengzhou University

Associate Editor / Senior Editor roles, Nature Research - Talent Pool 2017
Springer Nature

Postdocs, Key Lab for Neuroinformation, University of Electronic Sciences and

Prominent Deep Learning Methods

- Stacked Denoising Autoencoder (SDAE)
- Deep Belief Network (DBN)

Final Architecture same as NN but preparation differs

- Convolutional Neural Network (CNN)

Efficient for 2D data e.g., Image

- Long Short-Term Memory (LSTM)

Developed for Sequential Data

2024 Nobel Prize in Physics

NEWS | 08 October 2024

Physics Nobel scooped by machine-learning pioneers

John Hopfield and Geoffrey Hinton pioneered computational methods that enabled the development of neural networks.

By Elizabeth Gibney & Davide Castelvecchi

Physics Nobel scooped by machine-learning pioneers

The winners were announced by the Royal Swedish Academy of Sciences in Stockholm. Credit: Jonathan Nackstrand/AFP via Getty

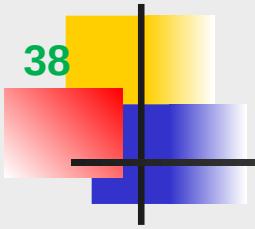
Two researchers who developed tools for understanding the neural networks that underpin today's boom in artificial intelligence (AI) have won the 2024 Nobel Prize in Physics.

John Hopfield at Princeton University in New Jersey, and Geoffrey Hinton at the University of Toronto, Canada, share the 11-million Swedish kronor (US$1 million) prize, announced by the Royal Swedish Academy of Sciences in Stockholm on 8 October.

Nature 634, 523-524 (2024)

doi: https://doi.org/10.1038/d41586-024-03213-8

https://www.nature.com/articles/d41586-024-03213-8



Thanks for your attention

Question and Answer

Biological Neuron to Artificial Neural Network

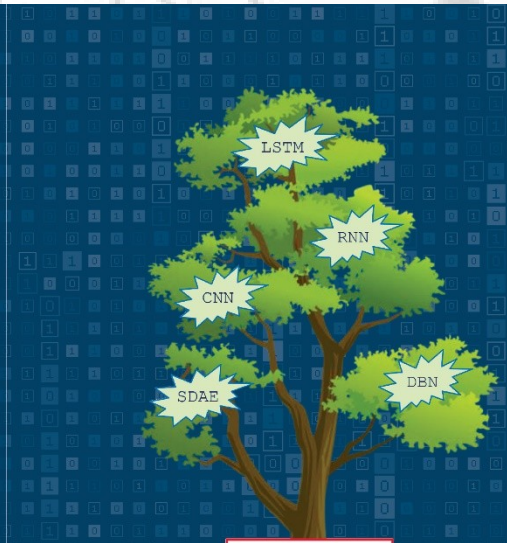
Ref.: Chapter 1: Preliminaries, Deep Learning Fundamentals - A Practical Approach to Understanding Deep Learning Methods ISBN:978-984-35-0812-6, University Grants Commission (UGC), Bangladesh, 2021

Prof. Dr. Md. Aminul Haque Akhand
Dept. of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET)
www.kuet.ac.bd/cse/akhand

Deep Learning Fundamentals
- A Practical Approach to Understanding Deep Learning Methods

Dr. Muhammad Aminul Haque Akhand
Professor
Department of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET)
Website: www.kuet.ac.bd/cse/akhand

Dr. M. A. H. Akhand is currently a Professor of CSE department at KUET, Bangladesh. He is also head of the Computational Intelligence Research Group of this department. His primary research interests are in machine learning, artificial neural network, deep learning, swarm intelligence, optimization, and pattern recognition. He received his B.Sc. degree in Electrical and Electronic Engineering from KUET in 1999, the M.E. degree in Human and Artificial Intelligent Systems in 2006, and the Doctoral degree in System Design Engineering in 2009 from University of Fukui, Japan. He has more than 100 publications in International Journals and Conferences. Dr. Akhand received several best paper Prizes in International Conferences. He is also an editorial board member of several International Journals.

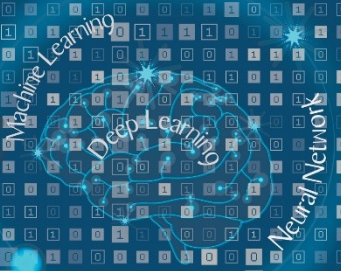


Deep Learning
Neural Network
Machine Learning

বাংলাদেশ বিশ্ববিদ্যালয় মঞ্জুরী কমিশন
University Grants Commission of Bangladesh

Deep Learning Fundamentals - A Practical Approach to Understanding Deep Learning Methods
Dr. Muhammad Aminul Haque Akhand

DEEP LEARNING FUNDAMENTALS
A Practical Approach to Understanding Deep Learning Methods



Dr. Muhammad Aminul Haque Akhand

About the Book

Machine learning (ML) is the virtue of modern science and its outcomes are intelligent systems that became integral parts of daily lives. Artificial neural network (NN) is the most prominent ML method and deep learning (DL) is its latest form mimicking the learning mechanism of human brain. At present, DL is the most prominent research area and its different methods are employed in different research studies and development of significant systems. This book contains a comprehensive study of prominent DL methods from NN foundation. The book is written based on the experience of many years following pedagogical features with illustrations, step-by-step algorithms, worked examples and MATLAB code for real-world problems. This book might be a textbook for DL related courses in PG level of Computer Science, Engineering and related disciplines. This book will also be useful for young researchers to work with DL related methods.

Cover Concept & Design: N. K. Kaikobad Rana

https://kuet.ac.bd/cse/akhand/

Contents

1. Introduction

2. Biological Neural Networks

3. Artificial Neural Networks

4. Neural Networks for Classification Tasks

5. Performance Evaluation and Benchmark Problems

History of ML and Deep Learning

Nobel Prize Nobel Prize in Physiology or Medicine 1906

Nobel Prize in Physics 2024

1906
Neuron/
Nerve Cell

Ramon y Cajal

Electronic Brain (1943)
Perceptron (1957)
ADALINE (1960)
XOR Problem (1969)
Golden Age
Dark Age ("AI Winter")
Multi-layered Perceptron (Backpropagation) (1986)
SVM (1995)
Deep Neural Network (Pretraining) (2006)

1940 1950 1960 1970 1980 1990 2000 2010

S. McCulloch – W. Pitts

F. Rosenblatt

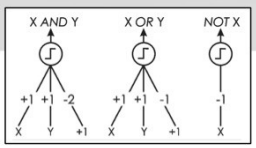
B. Widrow – M. Hoff

M. Minsky – S. Papert

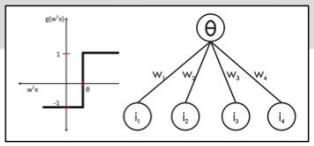
D. Rumelhart – G. Hinton – R. Wiliams

V. Vapnik – C. Cortes

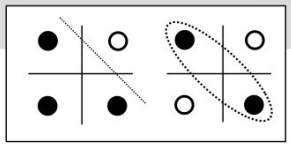
G. Hinton – S. Ruslan



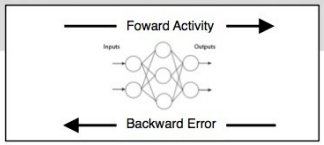
- Adjustable Weights
- Weights are not Learned



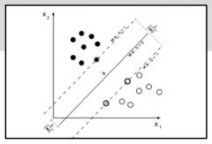
- Learnable Weights and Threshold



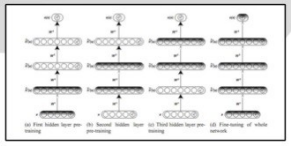
- XOR Problem



- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting



- Limitations of learning prior knowledge
- Kernel function: Human Intervention



- Hierarchical feature Learning

Artificial Neuron

Solving Real-Life Problem with Extracted Features

Development of Traditional Machine Learning

Deep Leaning

The Nobel Prize in Physics 2024 was awarded jointly to John J. Hopfield and Geoffrey Hinton "for foundational discoveries and inventions that enable machine learning with artificial neural networks"

Why Artificial Neural Networks?

Human brain has many desirable characteristics not presented in modern computers:

massive parallelism,
distributed representation and computation,
learning ability,
generalization ability,
adaptability,
inherent contextual information processing,
fault tolerance, and
low energy consumption.

It is hope that devices based on biological neural networks will process some of these characteristics.

Why Artificial Neural Networks?

- Modern digital computers outperform humans in the domain of numeric computation and the related symbolic manipulation.
- However, humans can solve complex perceptual problems effortlessly, and more efficiently than the world's fastest computer.
- Human performs these remarkable benefits due to the characteristics of his/her neural system whose (neural system) working procedure is completely different from a conventional computer system.

Comparison of Brains and Traditional Computers

- 200 billion neurons, 32 trillion synapses
- Element size: 10^{-6} m
- Energy use: 25W
- Processing speed: 100 Hz
- Parallel, Distributed
- Fault Tolerant
- Learns: Yes
- Intelligent/Conscious: Usually

- 1 billion bytes RAM but trillions of bytes on disk
- Element size: 10^{-9} m
- Energy watt: 30-90W (CPU)
- Processing speed: 10^{9} Hz
- Serial, Centralized
- Generally not Fault Tolerant
- Learns: Some
- Intelligent/Conscious: Generally No

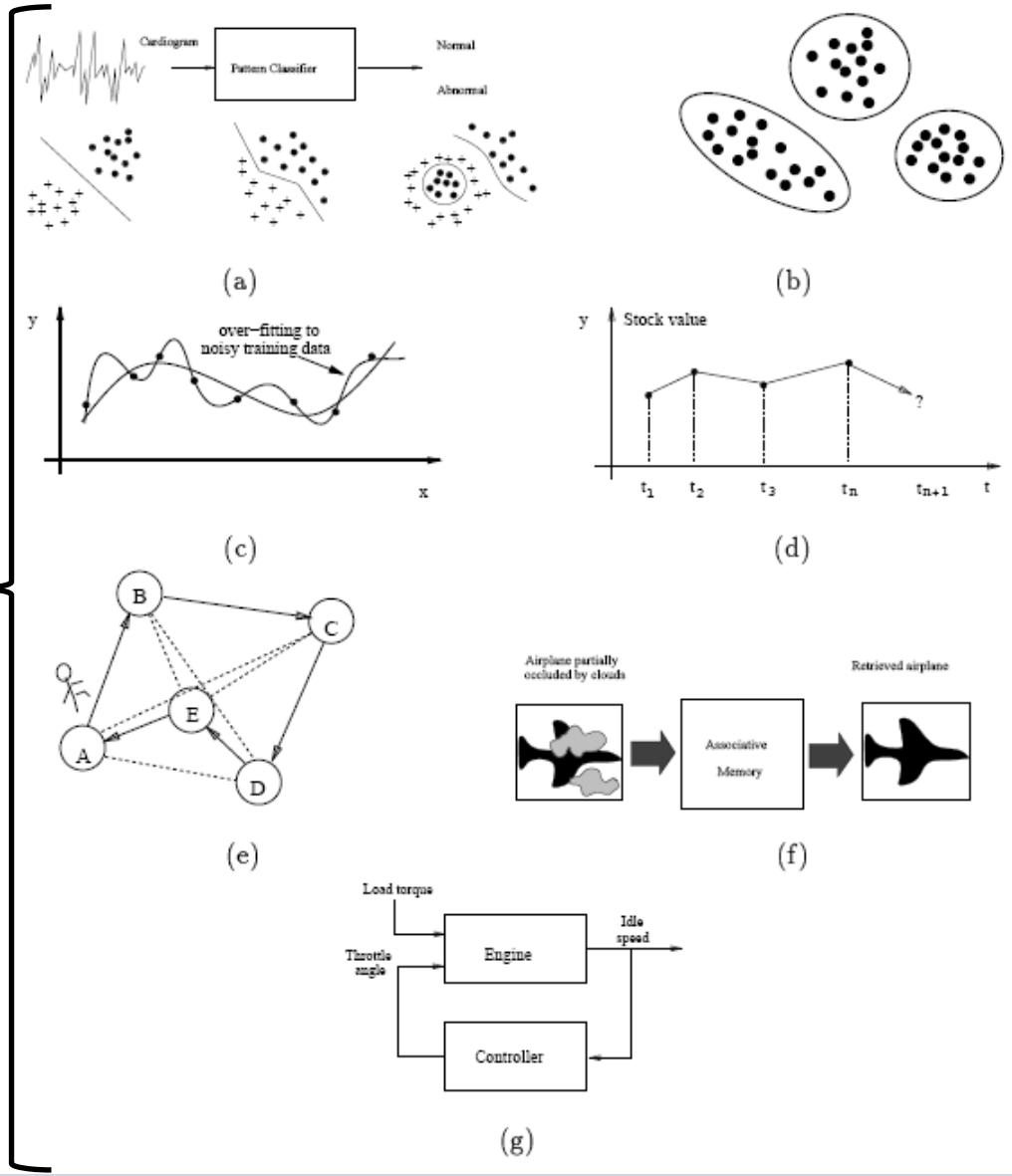
von Neumann Computer vs. Biological Neural System

	von Neumann Computer	Biological Neural System
processor	complex high-speed one (or a few)	simple low-speed a large number
memory	separate localized noncontent addressable	integrated in the neuron distributed content addressable
computing	centralized sequential strored-programs	distributed parallel self-learning
reliability	vulnerable	robust
expertise	numerical symbolic	perceptual problems manipulations
environment	well-defined	poorly defined unconstrained

The Goal

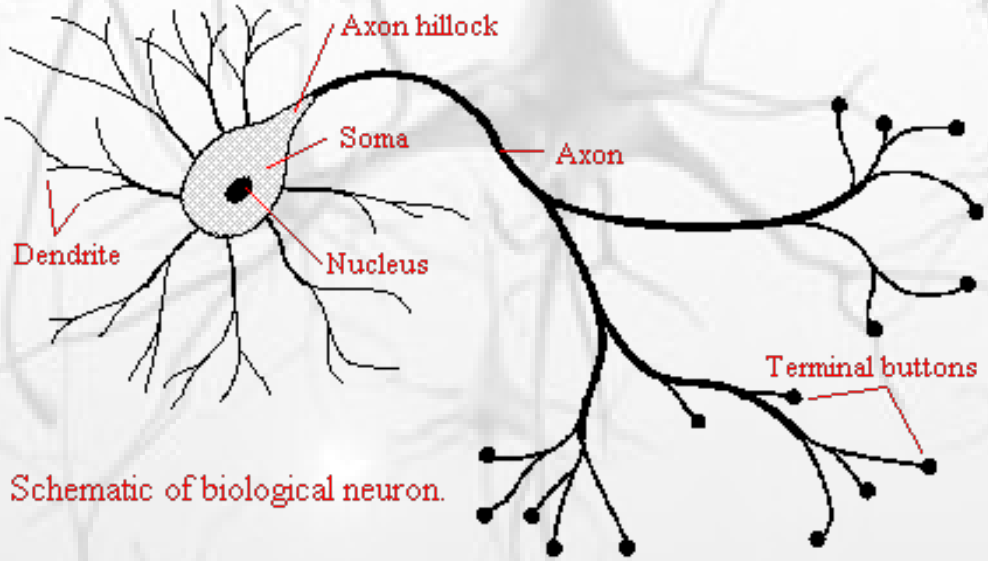
ANN is designed with the goal of building **intelligent machines to solve complex perceptual problems**, such as pattern recognition and optimization, by mimicking the networks of real neurons in the human brain.

Tasks handled by ANNs



Artificial Neural Networks(ANNs) Motivations

- ANNs are inspired by biological neural networks



Schematic of biological neuron.

A *neuron* (or nerve cell) is a special biological cell, the essence of life, with information processing ability. The introduction of neurons as basic structural constituents of the brain was credited to Ramon y Cajal who won the 1906 Nobel prize for physiology and medicine (shared with Camillo Golgi) for the crucial discovery of the extensive interconnections within the *cerebral cortex*, the portion of the brain where approximately 90% of the neurons in the human are located.

Biological Neuron

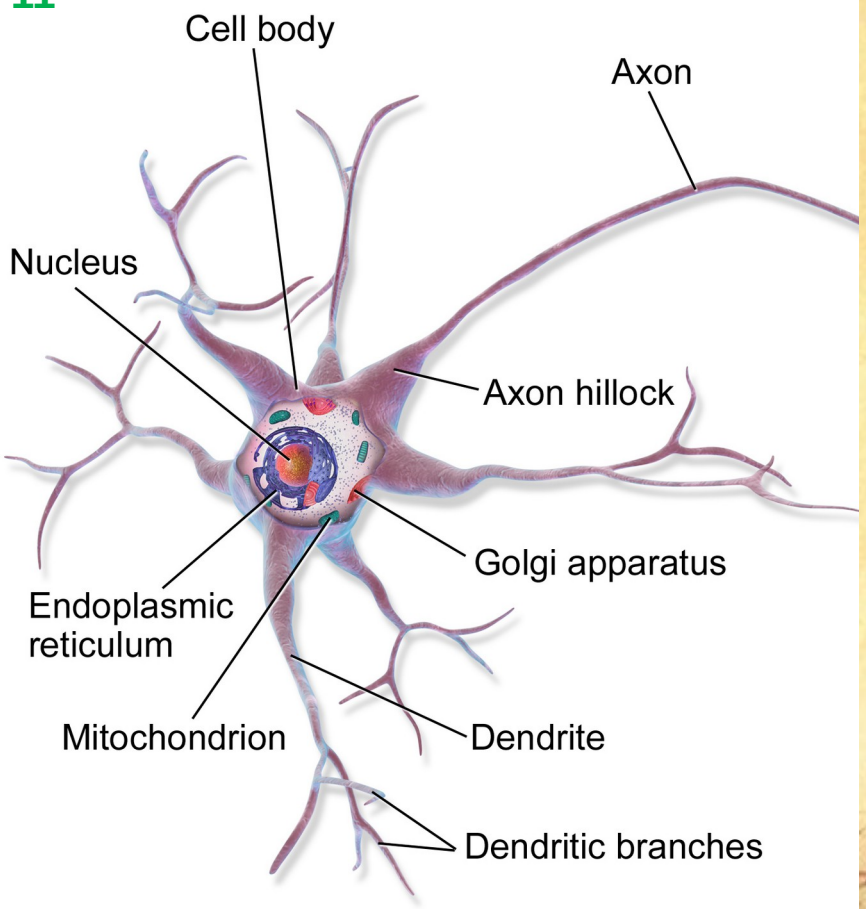
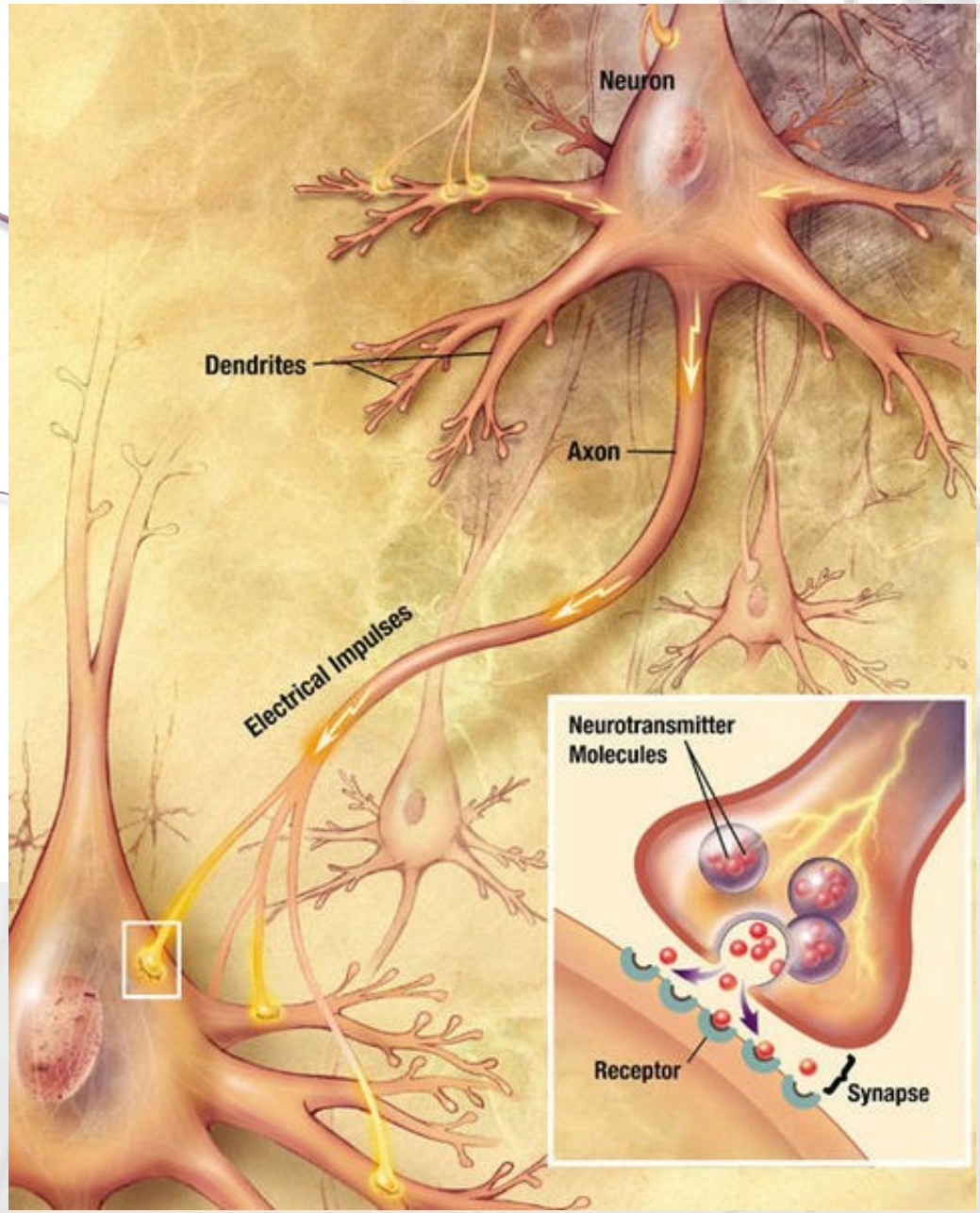
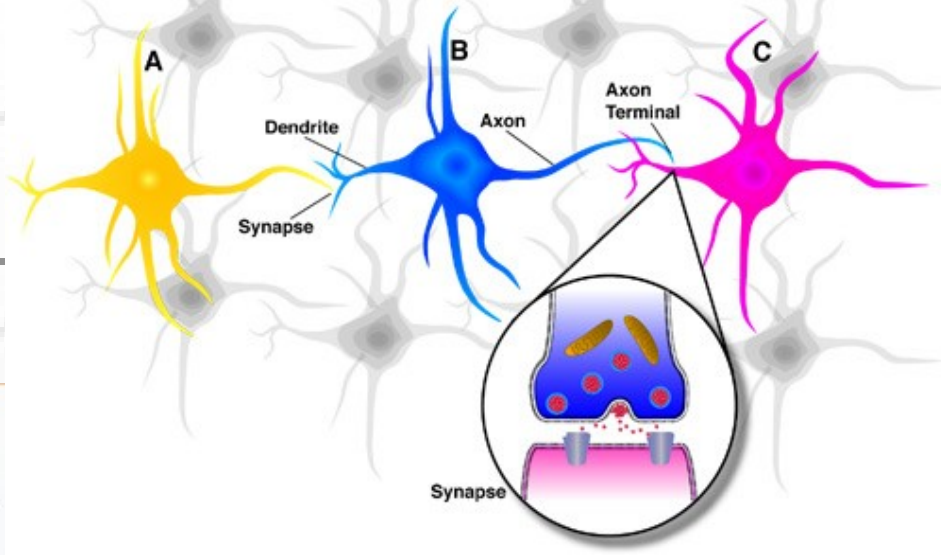
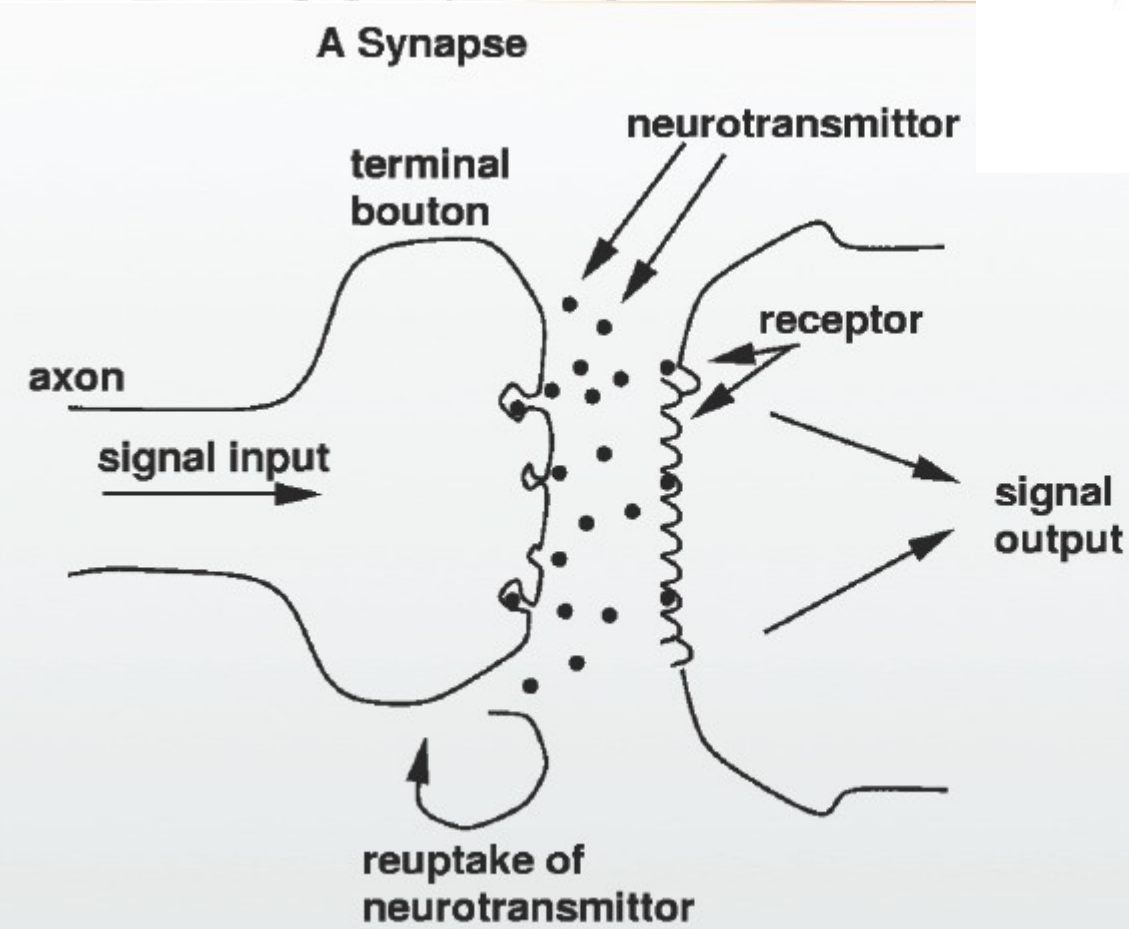


Figure 1.1: A sketch of a biological neuron.



Synapses



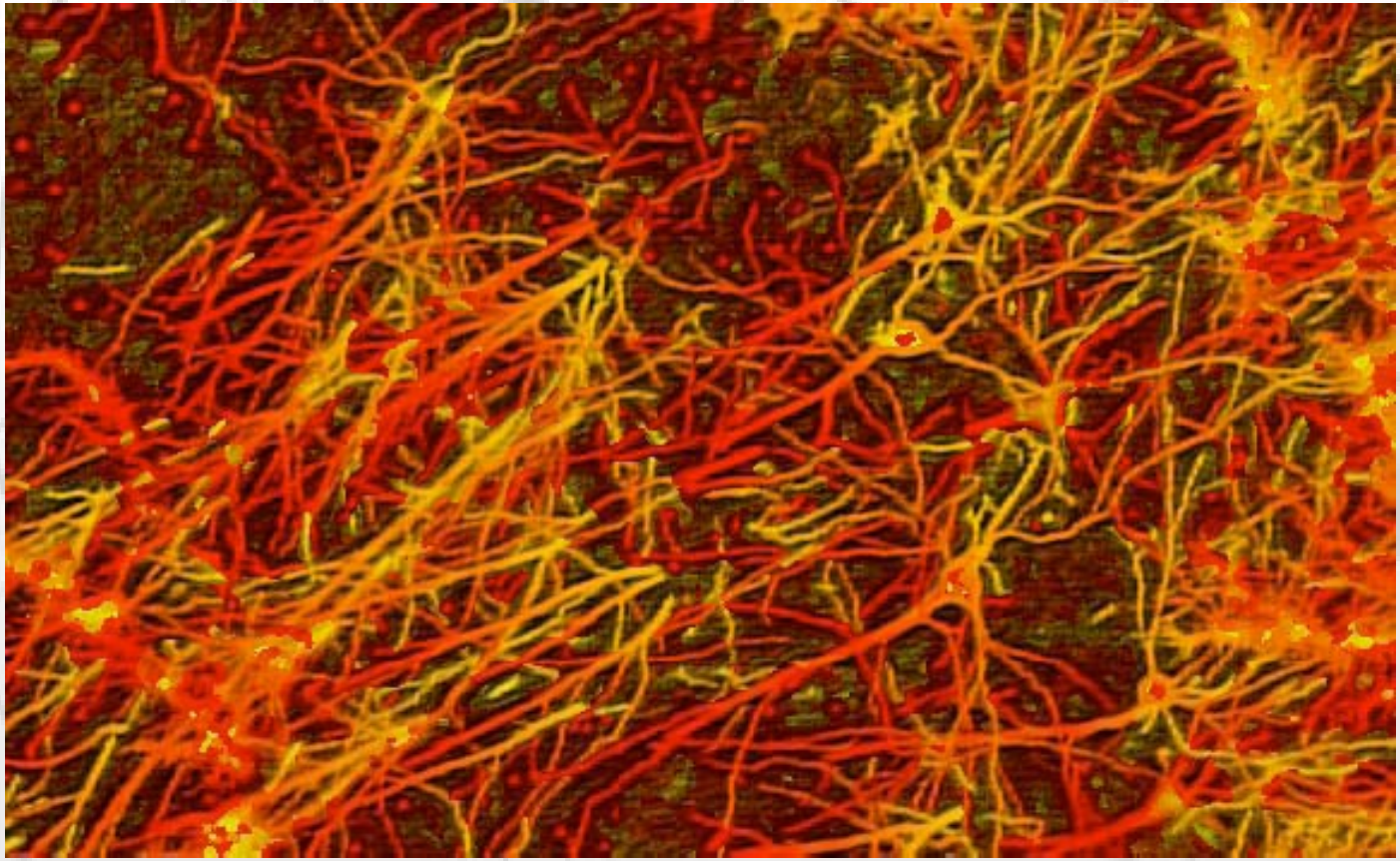


Synapses

- transmit signal to next neuron
- vary in strength
- *change* strength in response to use (learning!)

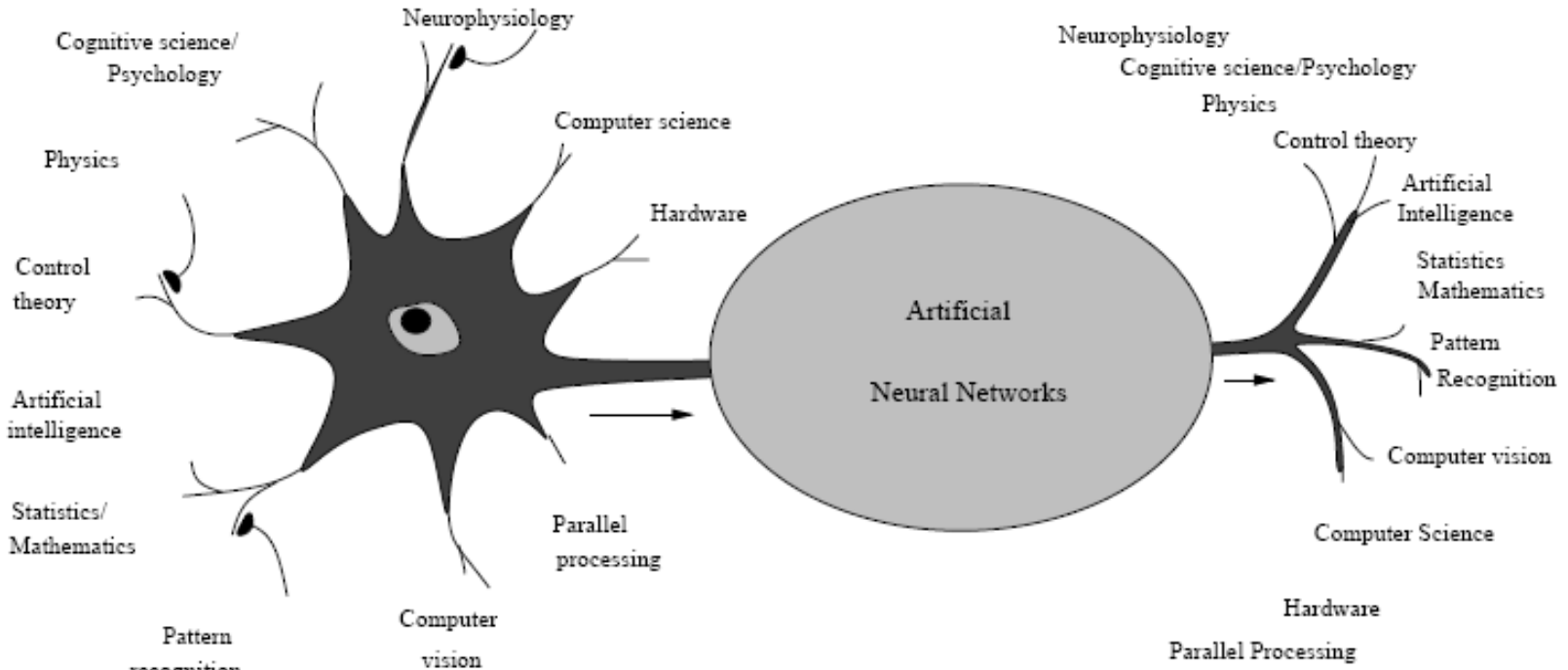
https://www.simplypsychology.org/synapse.html

A Real Neural Network



Neurons are massively connected, much more complex and denser than today's telephone networks. Each neuron is connected to $10^3 - 10^4$ other neurons. The number of interconnections depends on the location of the neuron in the brain and the type of the neuron. In total, the human brain contains approximately $10^{14} - 10^{15}$ interconnections.

ANN's Relationship with Other Disciplines



Computational Model

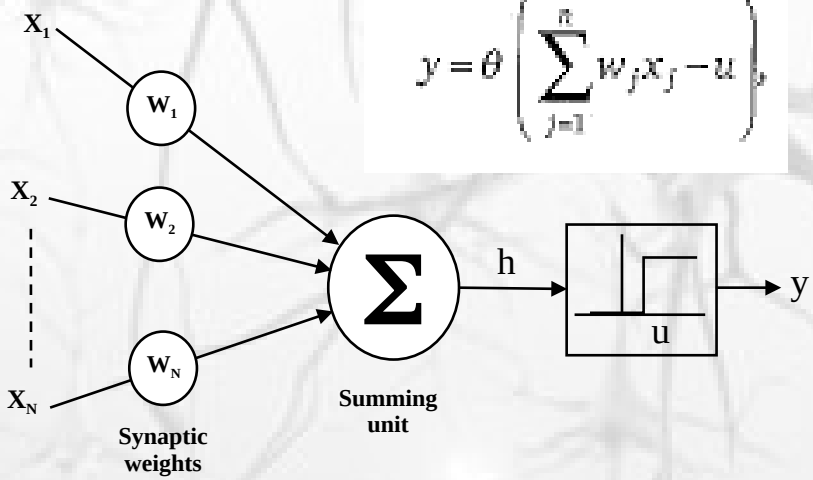


Figure 1.2: McCulloch-Pitts model of a neuron.

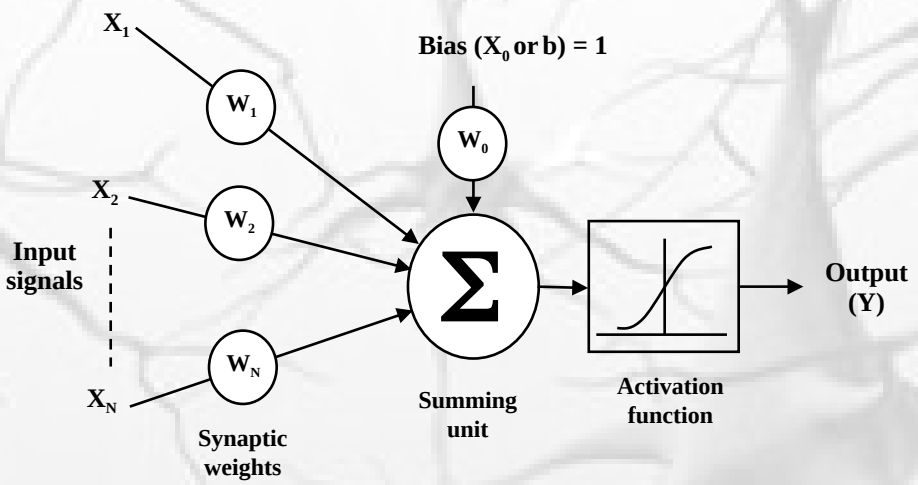


Figure 1.3: Generalized model of an artificial neuron.

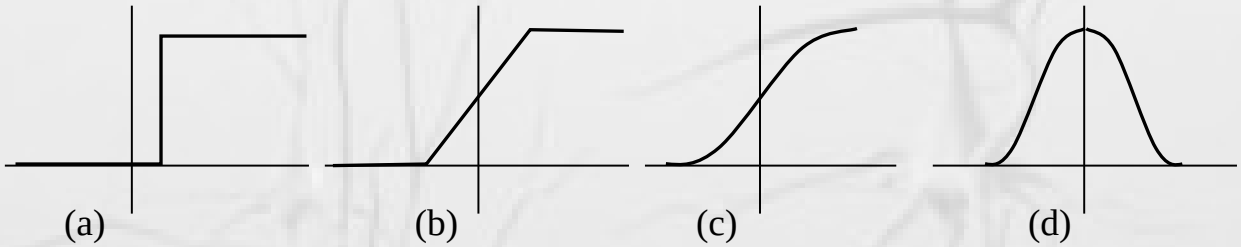


Figure 1.4: Different types of activation functions: (a) threshold, (b) piecewise linear, (c) sigmoid, and (d) Gaussian.

Sigmoid Activation Function

$$y=1/(1+exp(-ax))$$

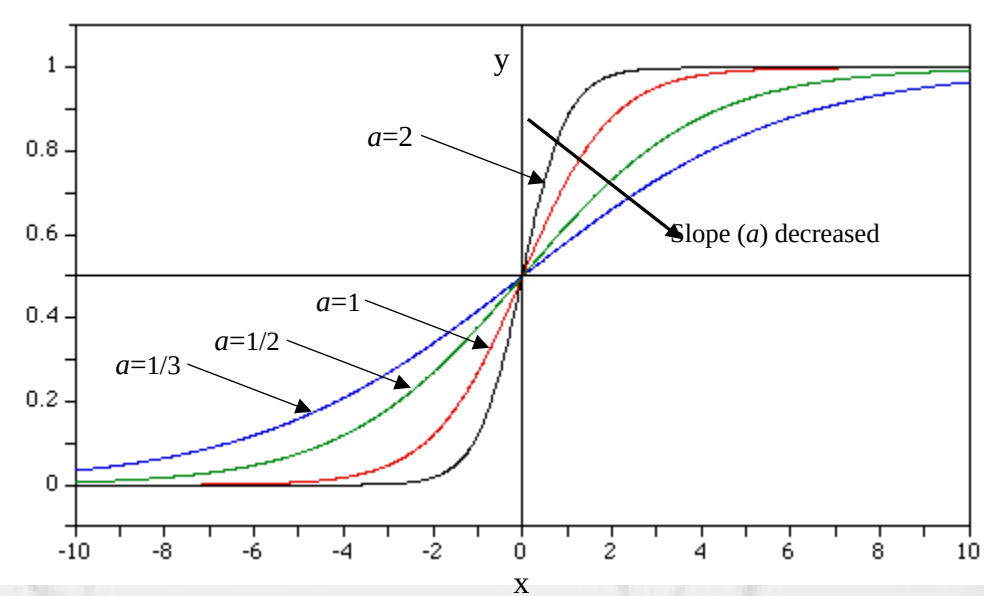
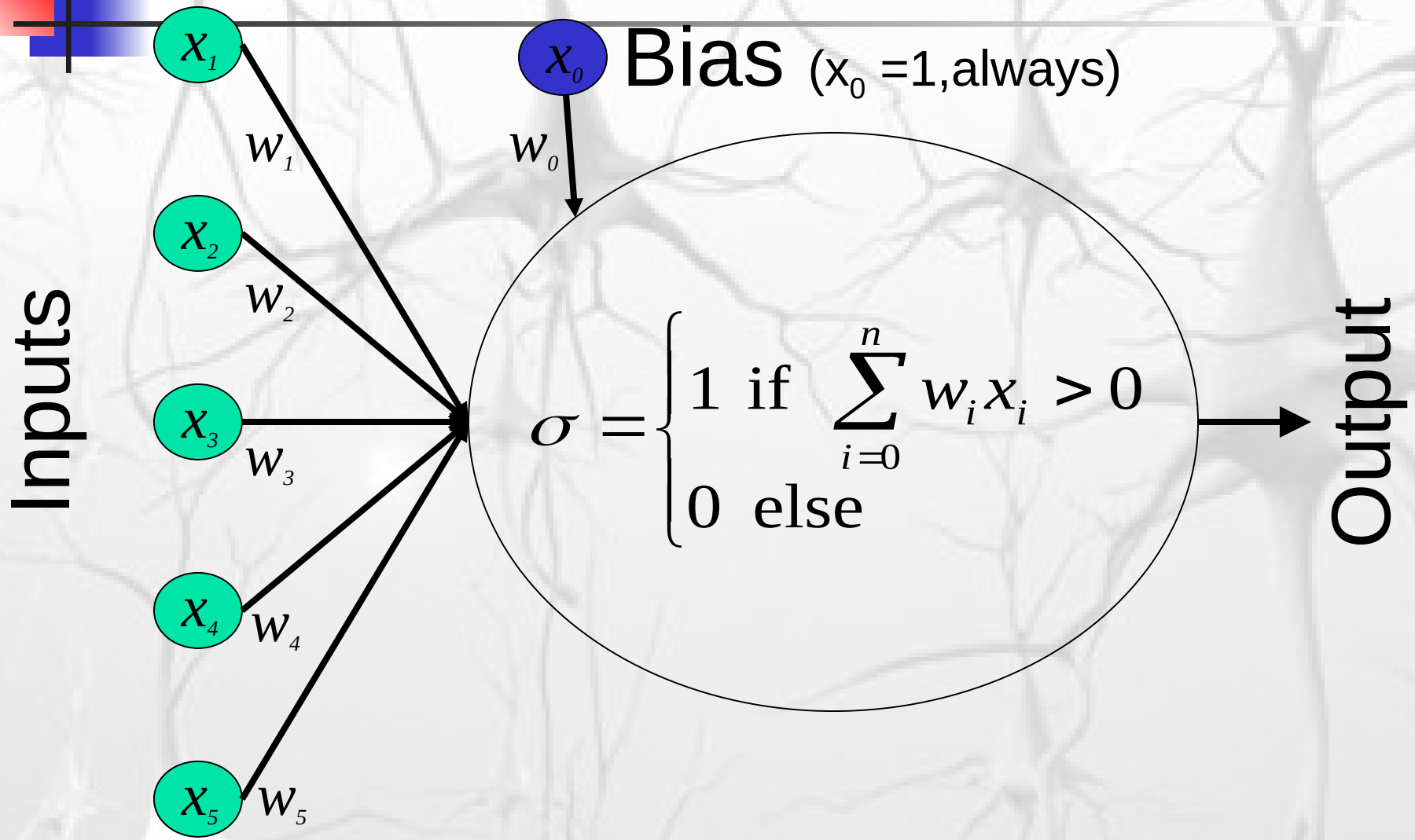
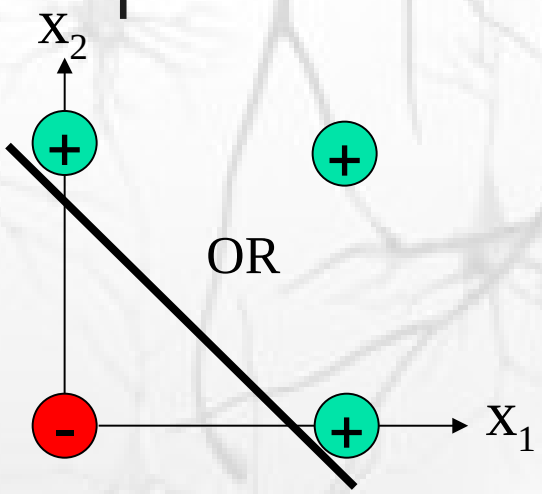


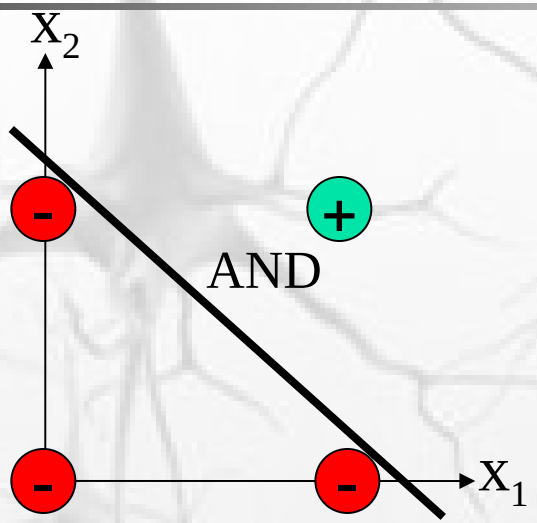
Figure 1.5: Sigmoid function with various slope (a) values.

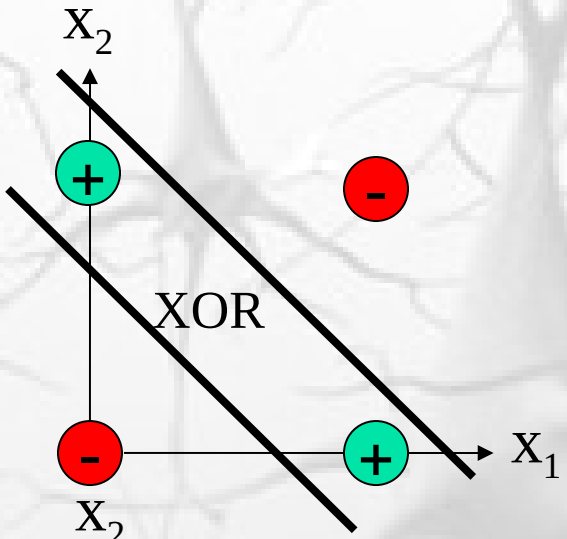
Ability of Single Neuron

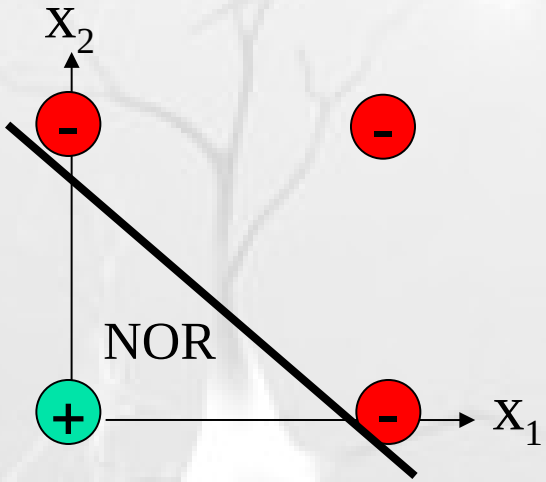


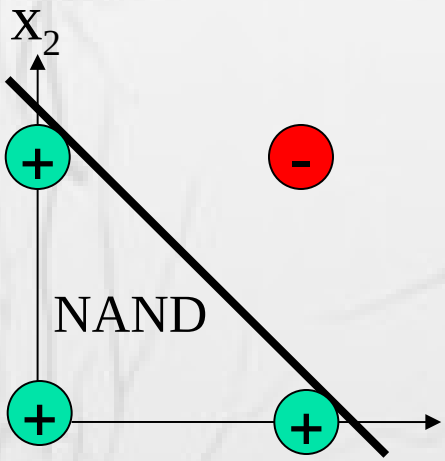
Logical Operators

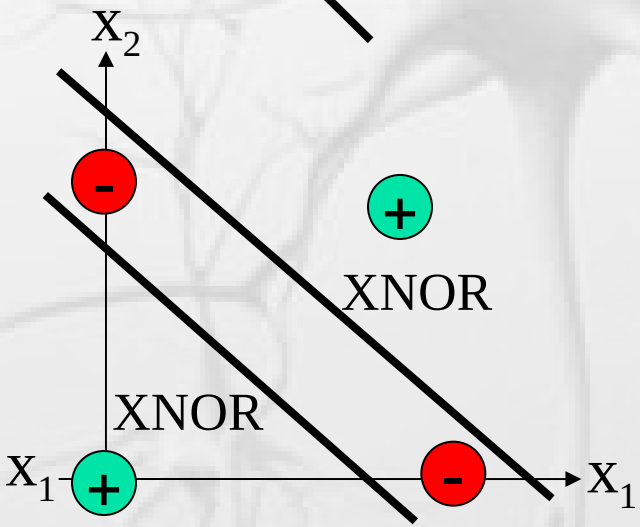




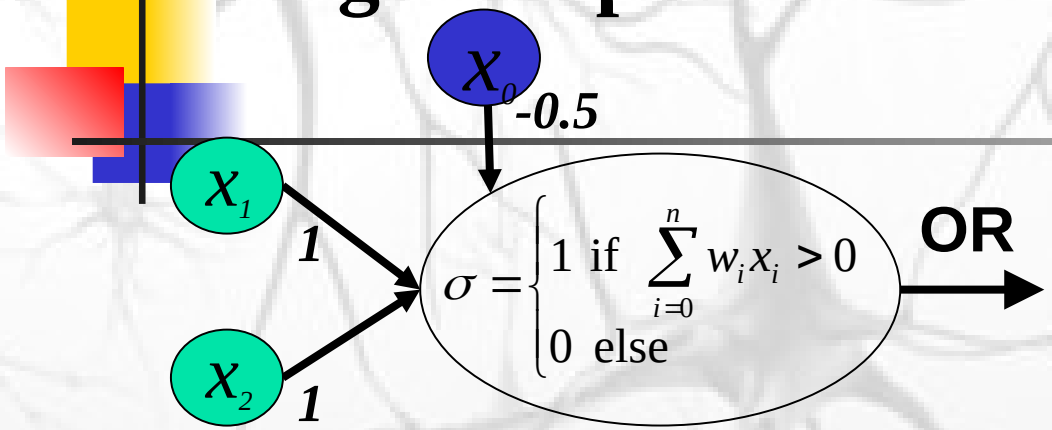


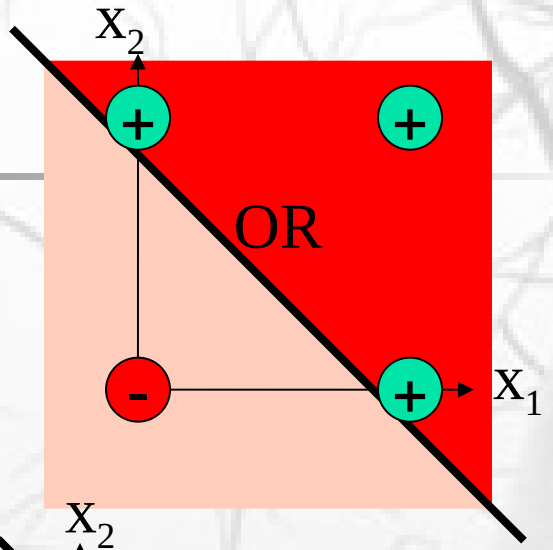


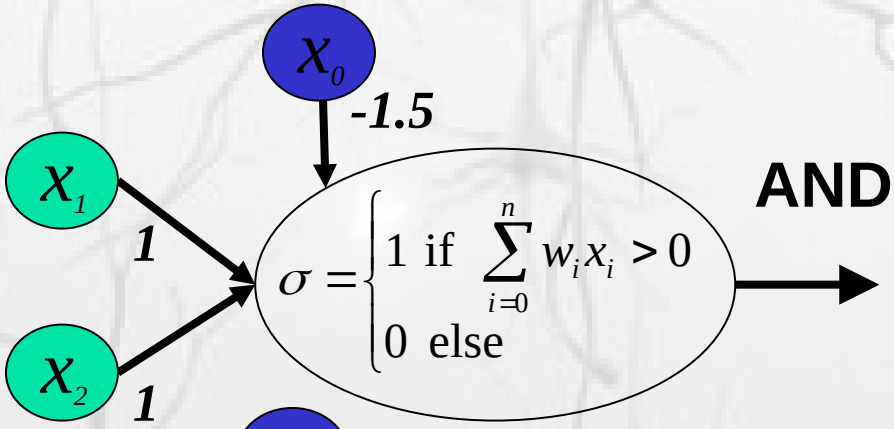


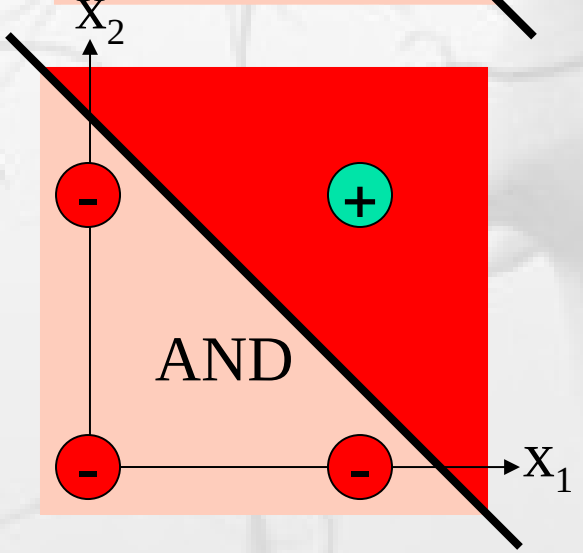


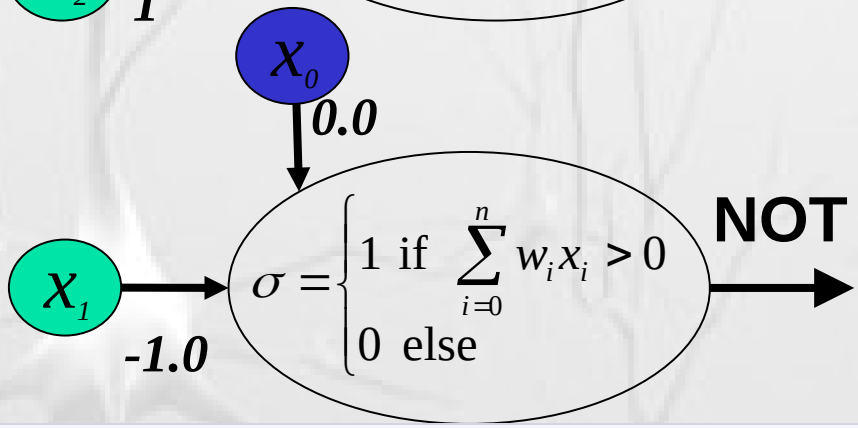
Logical Operators



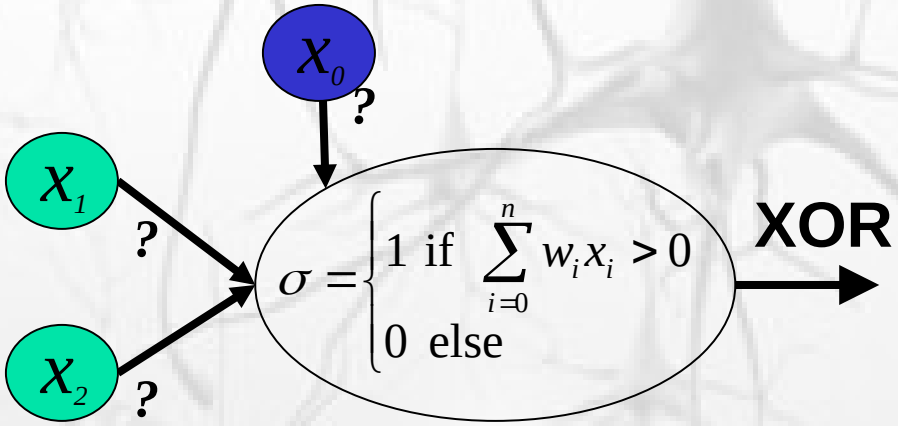


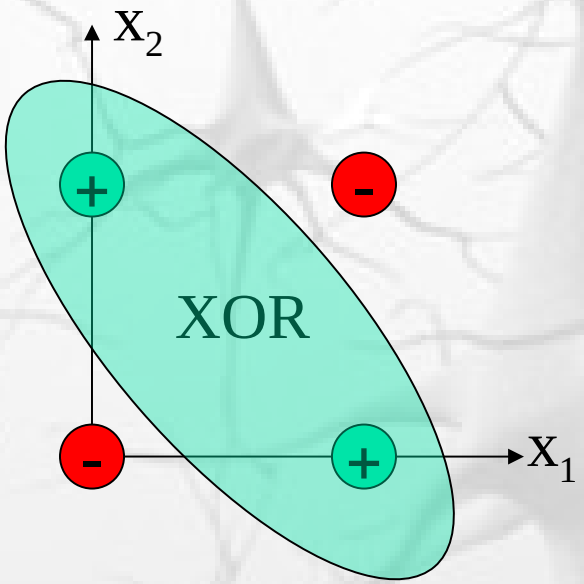






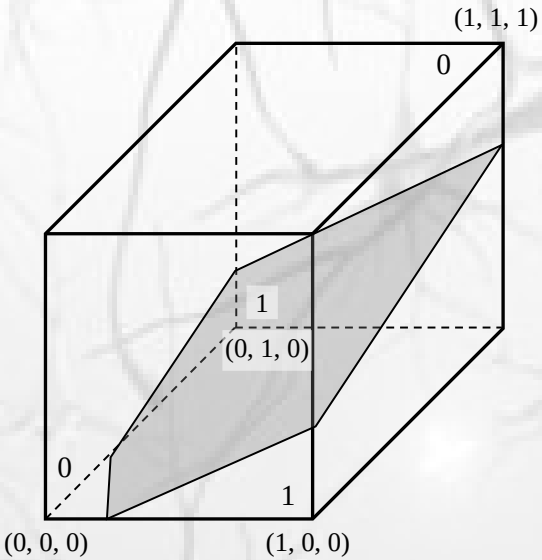
Nonlinear Problem



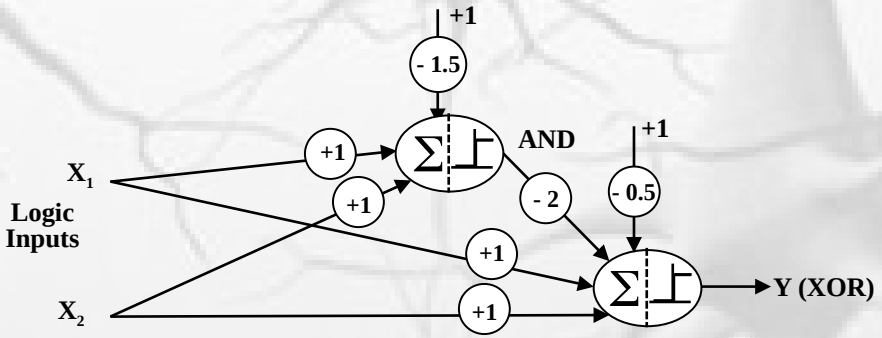


No arrangement work, not linearly separable. Require two boundary lines.

XOR solution in Higher Dimension



a) Third dimension uplift XOR (1, 1) as (1, 1, 1) in three dimension.



b) XOR solution with additional input from AND gate.

Figure 1.8: Solution of XOR in three dimension.

An additional dimension (input) converts the problem to a linearly separable.

XOR solution with HN

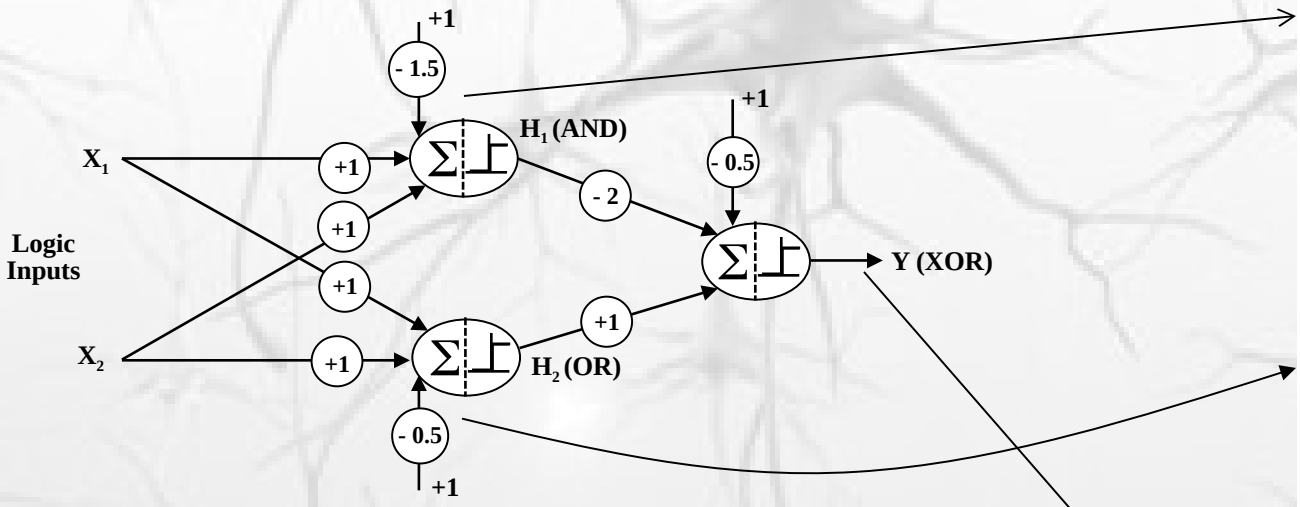
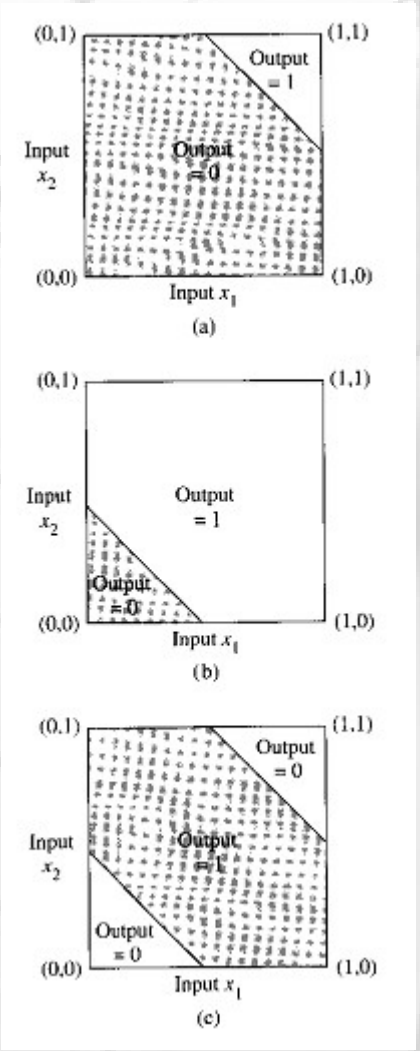


Figure 1.9: XOR logic combining AND and OR logic operations.



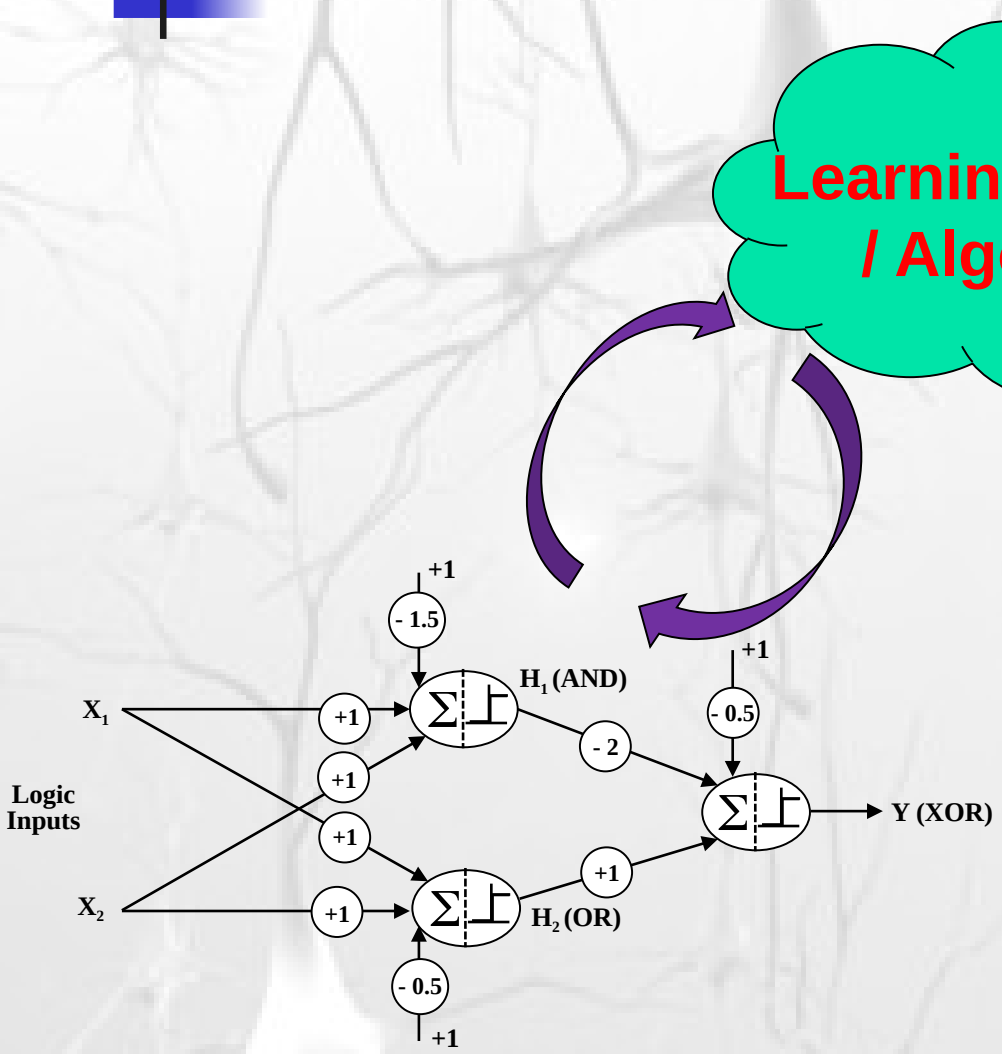
AND

OR

XOR

XOR logic through manipulation by hidden neurons

Learning



X1	X2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Learning

- The ability to learn is a fundamental trait (characteristic) of intelligence.
- ANN-> Updating NN architecture and connection weights to perform a specific task efficiently.
- ANN ability to automatically learning from examples makes them attractive.
- To understand or design a learning process need: a model of environment or information is available to NN and learning rules.
- A learning algorithm -> procedure for adjusting weights
- Three main learning paradigms-> Supervised (learning with a teacher), unsupervised and hybrid
- Reinforcement learning is variant of supervised learning receives critique on the correctness of network output instead of correct answer.

Learning

- Learning theory must address three fundamental and practical issues:

  1. Capacity – how many patterns can be stored , what functions and decision boundaries a NN can perform

  2. Sample Complexity – Number training patterns needed to train

  3. Computational Complexity – Time required for a learning algorithm to estimate a solution

- Four basic types of Learning rules: Error-Correction, Boltzman, Hebbian, and Competitive.

X1	X2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Network Architecture / Topology

Based on Connection pattern (architecture): feed forward and recurrent (feedback)

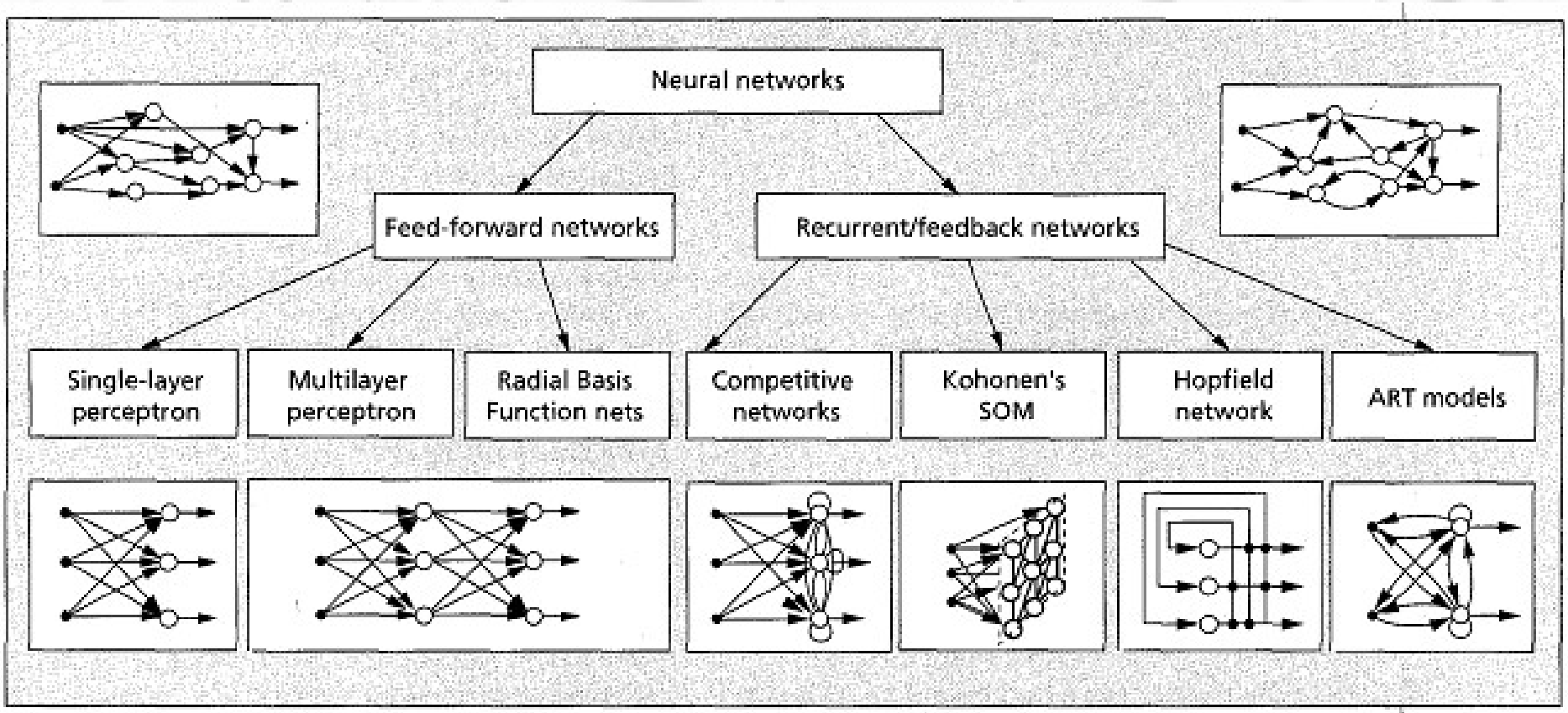


Figure 4. A taxonomy of feed-forward and recurrent/feedback network architectures.

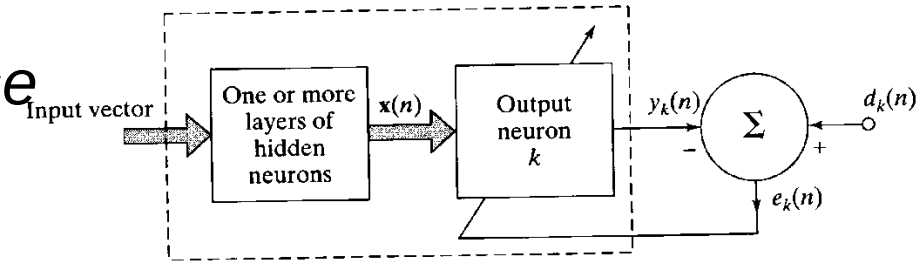
Different architectures require appropriate learning algorithms.

Well Known Learning Algorithms

Paradigm	Learning rule	Architecture	Learning algorithm	Task
Supervised	Error-correction	Single- or multilayer perceptron	Perceptron learning algorithms Back-propagation Adaline and Madaline	Pattern classification Function approximation Prediction, control
	Boltzmann	Recurrent	Boltzmann learning algorithm	Pattern classification
	Hebbian	Multilayer feed-forward	Linear discriminant analysis	Data analysis Pattern classification
	Competitive	Competitive	Learning vector quantization	Within-class categorization Data compression
		ART network	ARTMap	Pattern classification Within-class categorization
Unsupervised	Error-correction	Multilayer feed-forward	Sammon's projection	Data analysis
	Hebbian	Feed-forward or competitive	Principal component analysis	Data analysis Data compression
		Hopfield Network	Associative memory learning	Associative memory
	Competitive	Competitive	Vector quantization	Categorization Data compression
		Kohonen's SOM	Kohonen's SOM	Categorization Data analysis
		ART networks	ART1, ART2	Categorization
Hybrid	Error-correction and competitive	RBF network	RBF learning algorithm	Pattern classification Function approximation Prediction, control

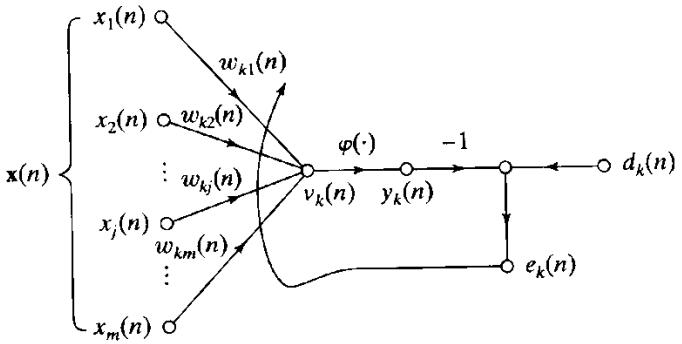
Learning Rule: Error-Correction

- *error signal = desired response – output signal*
- $e_k(n) = d_k(n) - y_k(n)$
- $e_k(n)$ *actuates a control mechanism to make the output signal* $y_k(n)$ *come closer to the desired response* $d_k(n)$ *in step by step manner*



Multilayer feedforward network

(a) Block diagram of a neural network, highlighting the only neuron in the output layer

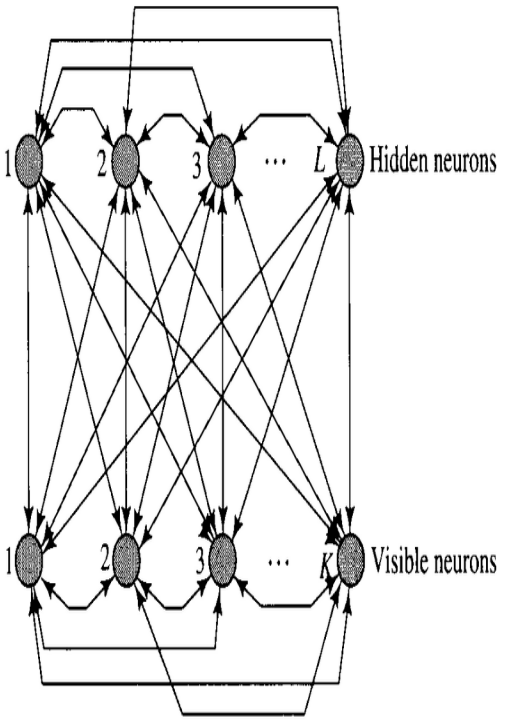


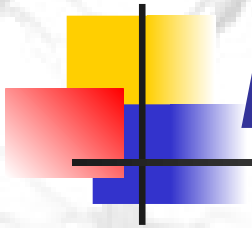
(b) Signal-flow graph of output neuron

FIGURE 2.1 Illustrating error-correction learning.

Learning Rule: Boltzmann

- *The primary goal of Boltzmann learning is to produce a neural network that correctly models input patterns according to a Boltzmann distribution.*
- *The Boltzmann machine consists of stochastic neurons. A stochastic neuron resides in one of two possible states (± 1) in a probabilistic manner.*
- *The use of symmetric synaptic connections between neurons.*
- *The stochastic neurons partition into two functional groups: visible and hidden.*
- *During the training phase of the network , the visible neurons are all clamped onto specific states determined by the environment.*
- *The hidden neurons always operate freely; they are used to explain underlying constraints contained in the environmental input vectors.*





Learning Rule: Hebbian

1. *If two neurons on either side of synapse (connection) are activated simultaneously, then the strength of that synapse is selectively increased.*

2. *If two neurons on either side of a synapse are activated asynchronously, then that synapse is selectively weakened or eliminated.*

A Hebbian synapse increases its strength with positively correlated presynaptic and postsynaptic signals, and decreases its strength when signals are either uncorrelated or negatively correlated.

Learning Rule: Competitive

- *The output neurons of a neural network compete among themselves to become active.*
- *- a set of neurons that are all the same (excepts for synaptic weights)*
- *- a limit imposed on the strength of each neuron*
- *- a mechanism that permits the neurons to compete -> a winner-takes-all*

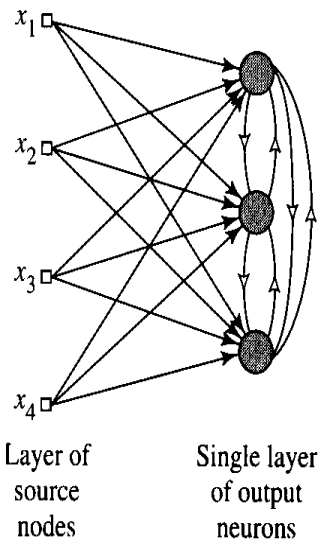


FIGURE 2.4 Architectural graph of a simple competitive learning network with feedforward (excitatory) connections from the source nodes to the neurons, and lateral (inhibitory) connections among the neurons; the lateral connections are signified by open arrows.

- *The standard competitive learning rule*
- $\Delta w_{kj} = \eta(x_j - w_{kj})$ *if neuron k wins the competition* =
  0 if neuron k loses the competition

Learning Paradigms : Learning with a Teacher

Learning with a Teacher (=supervised learning)

The teacher has knowledge of the environment

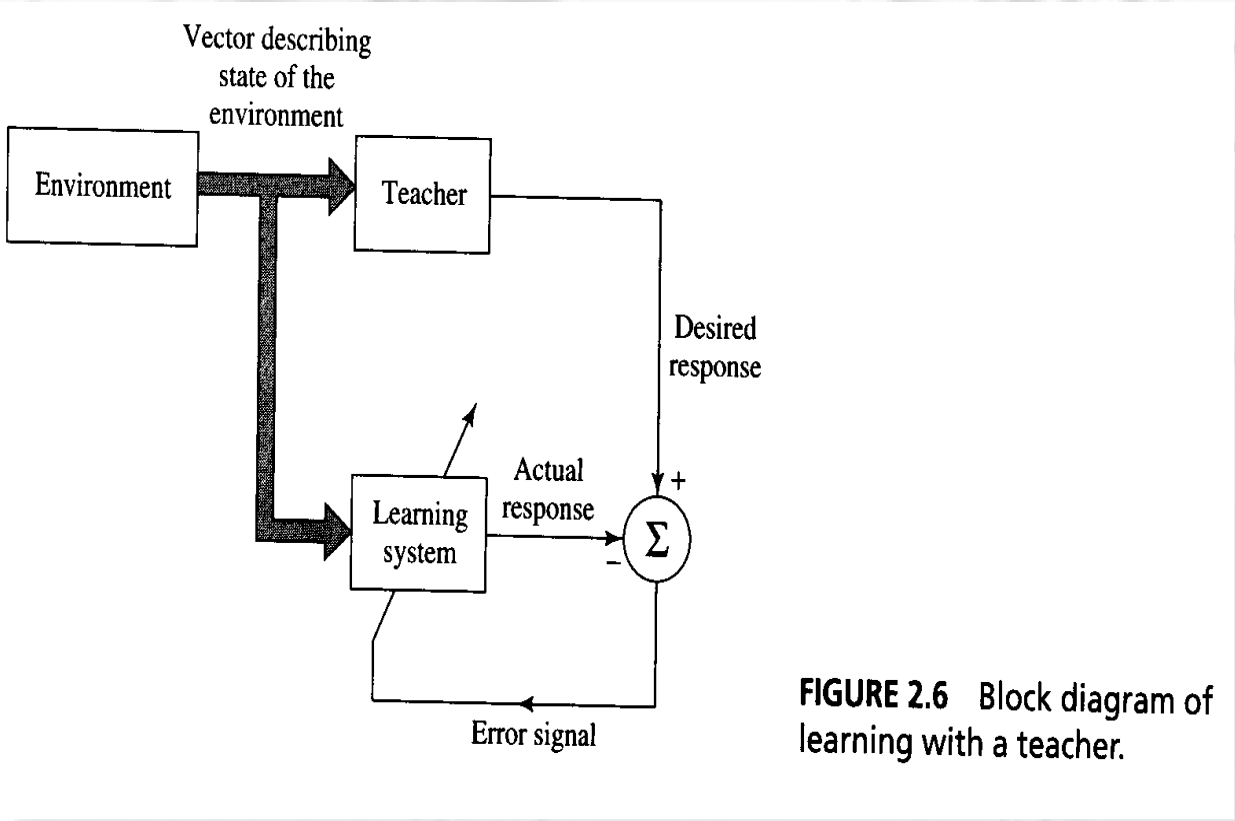
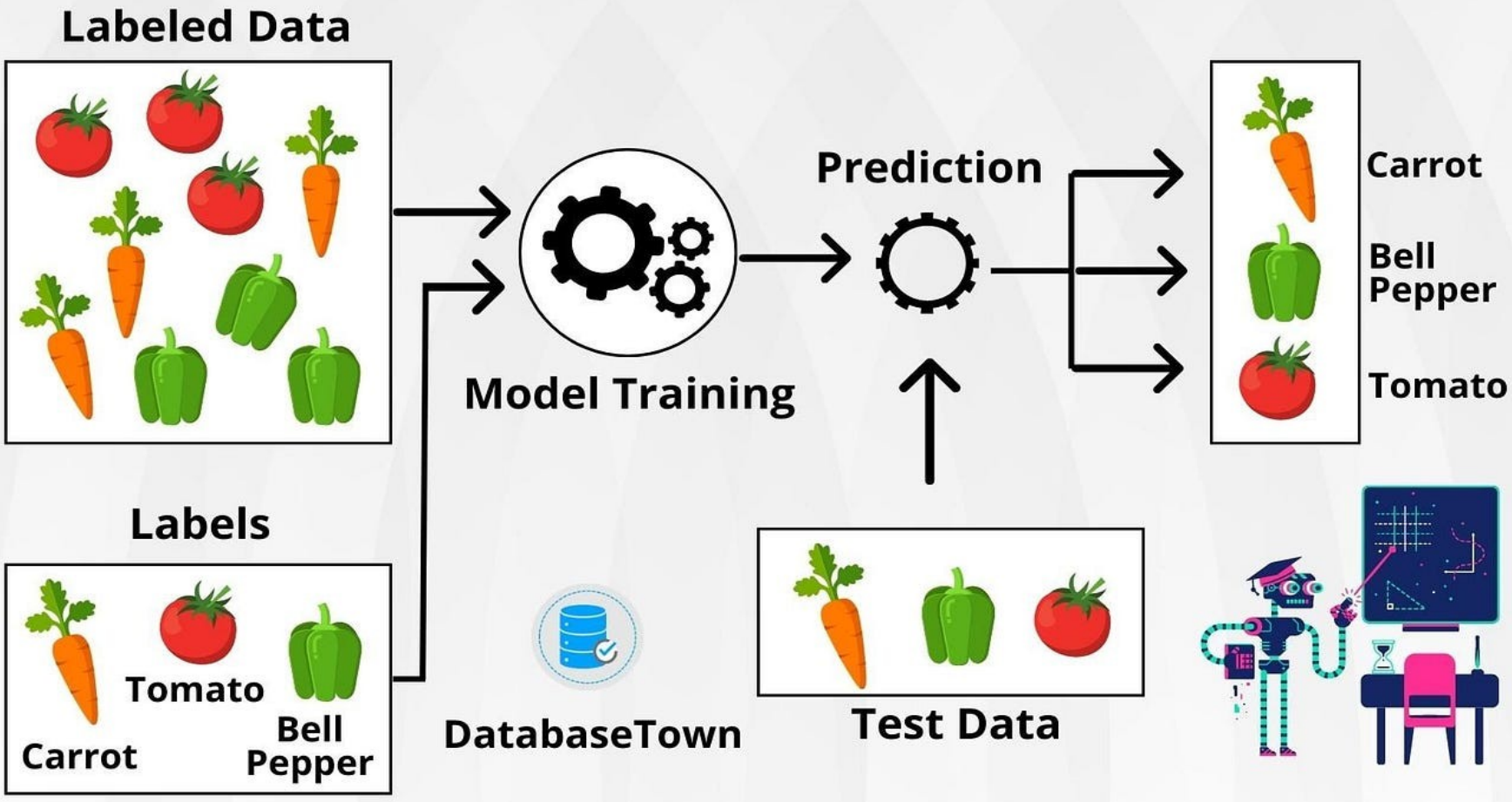


FIGURE 2.6 Block diagram of learning with a teacher.

Supervised Learning

Supervised machine learning is a branch of artificial intelligence that focuses on training models to make predictions or decisions based on labeled training data.



Learning Paradigms: Learning without a Teacher

Learning without a Teacher: no labeled examples available of the function to be learned.

1) Reinforcement learning

2) Unsupervised learning

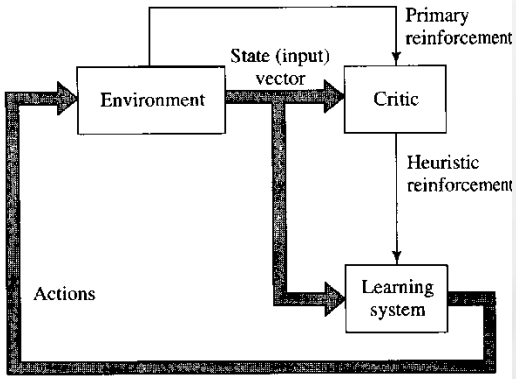


FIGURE 2.7 Block diagram of reinforcement learning.

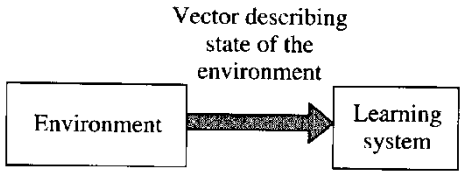


FIGURE 2.8 Block diagram of unsupervised learning.

Learning Paradigms: Reinforcement

Reinforcement learning:

The learning of input-output mapping is performed through continued interaction with the environment in oder to minimize a scalar index of performance.

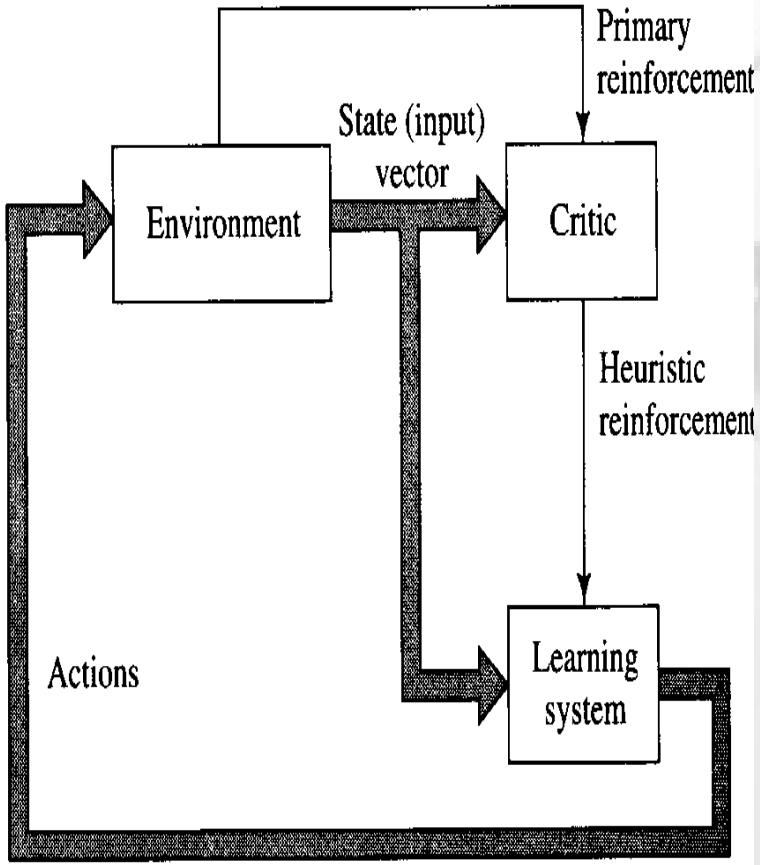


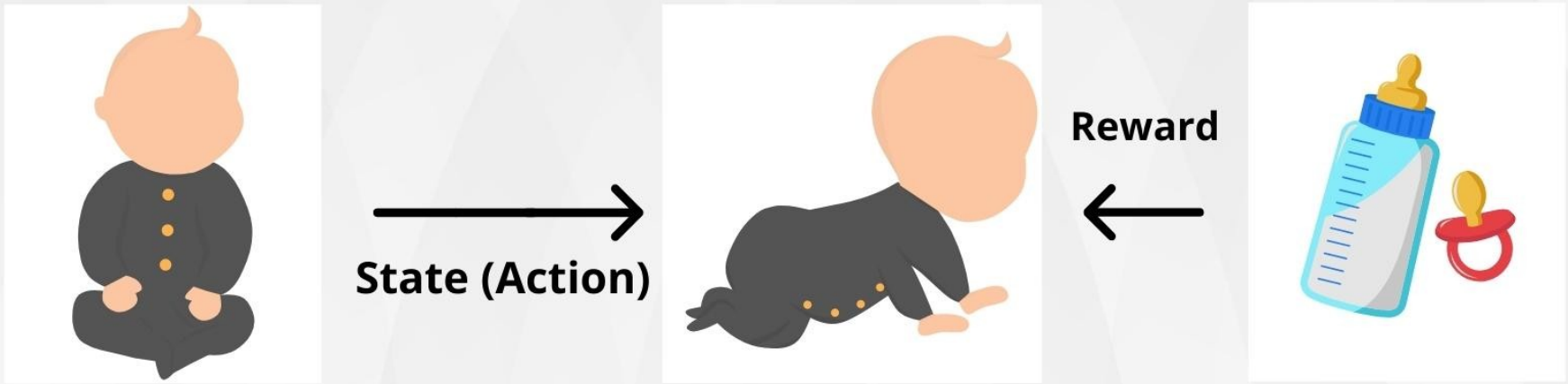
FIGURE 2.7 Block diagram of reinforcement learning.

Learning Paradigms: Reinforcement

Reinforcement learning is a machine learning paradigm that focuses on how agents learn to interact with an environment to maximize cumulative rewards.

DatabaseTown

Baby (Agent)



Algorithms and Approaches in Reinforcement Learning

- Q-learning
- Deep Q-networks (DQN)
- Policy Gradients Methods
- Proximal Policy Optimization (PPO)

Learning Paradigms: Unsupervised

- *Unsupervised Learning: There is no external teacher or critic to oversee the learning process.*

- *The provision is made for a task independent measure of the quality of representation that the network is required to learn.*

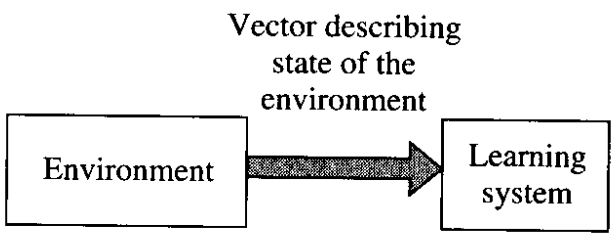
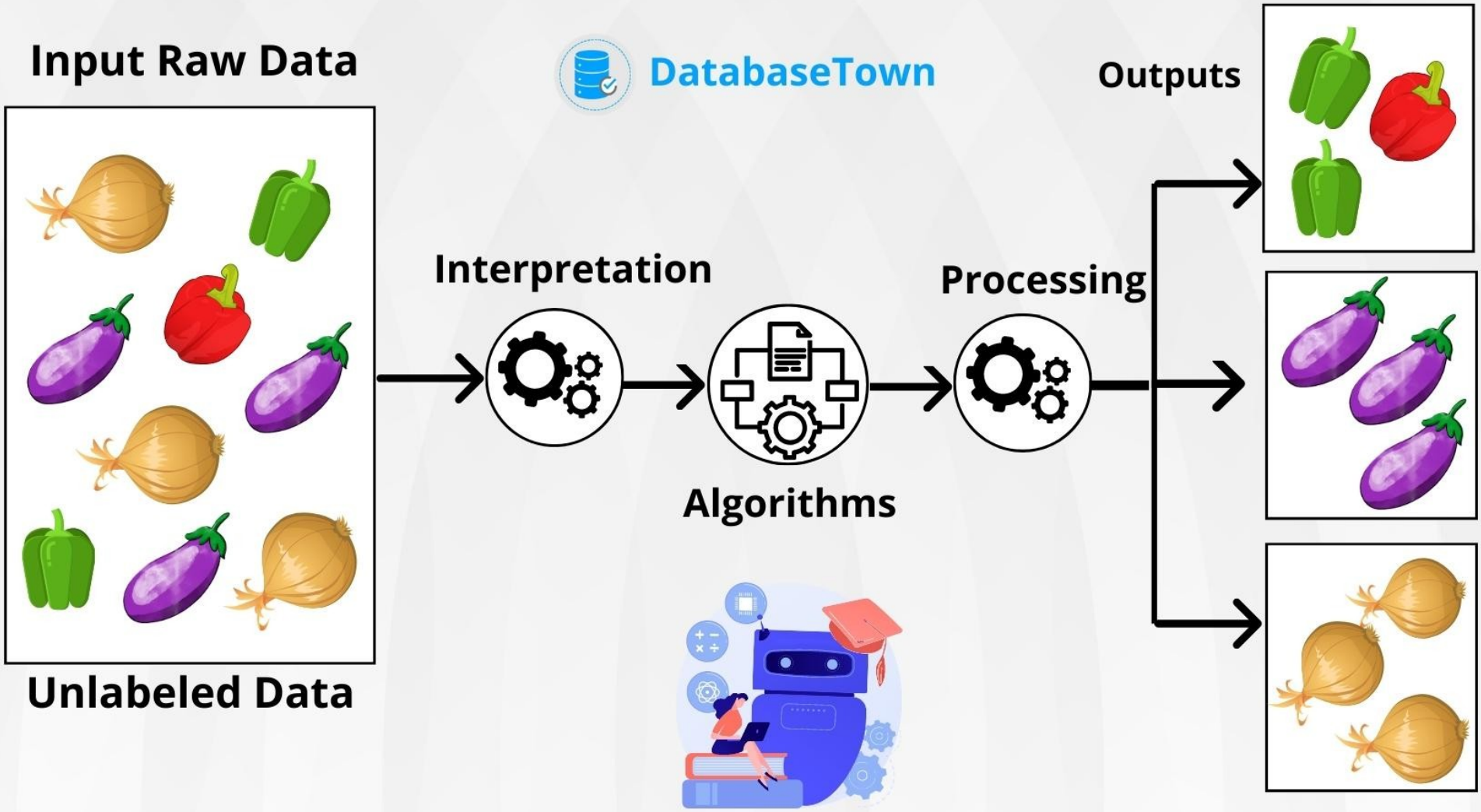


FIGURE 2.8 Block diagram of unsupervised learning.

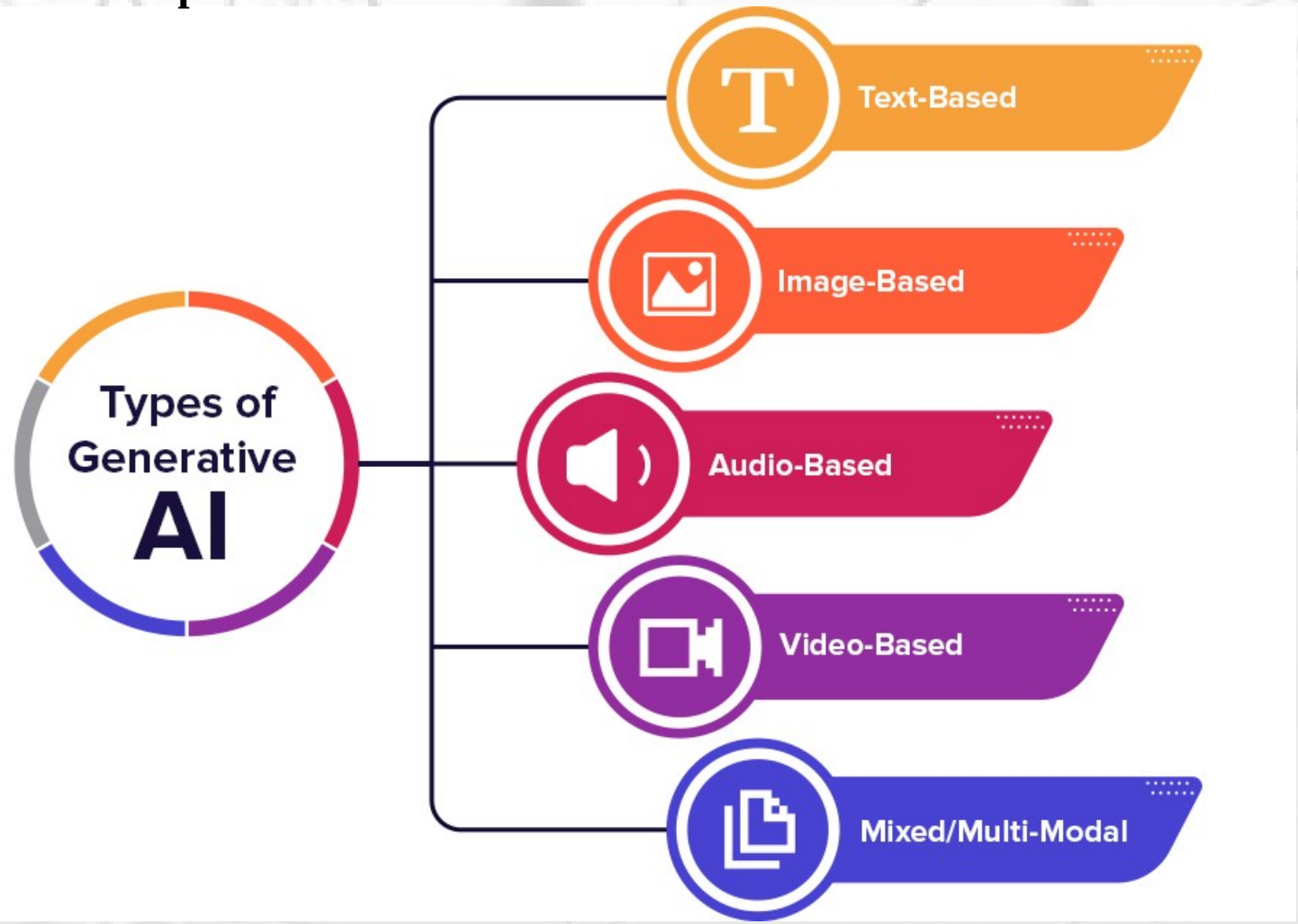
Learning Paradigms: Unsupervised

Unsupervised learning is a type of machine learning where the algorithm learns from unlabeled data without any predefined outputs or target variables.



Learning Paradigms: Generative AI

https://www.thinkebiz.net/understanding-generative-ai-and-the-emerging-landscape-how-businesses-need-to-embrace/



Supportive Tools and Techniques

DeepSeek

Parent organization: High-Flyer

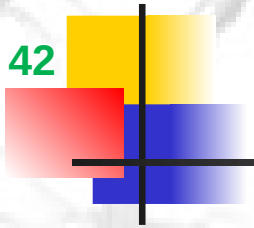
Founded: May 2023, Hangzhou, China

Feedback

ChatGPT

Software

ChatGPT is a generative artificial intelligence chatbot developed by OpenAI and launched in 2022. It is currently based on the GPT-4o large language model.
Wikipedia ›

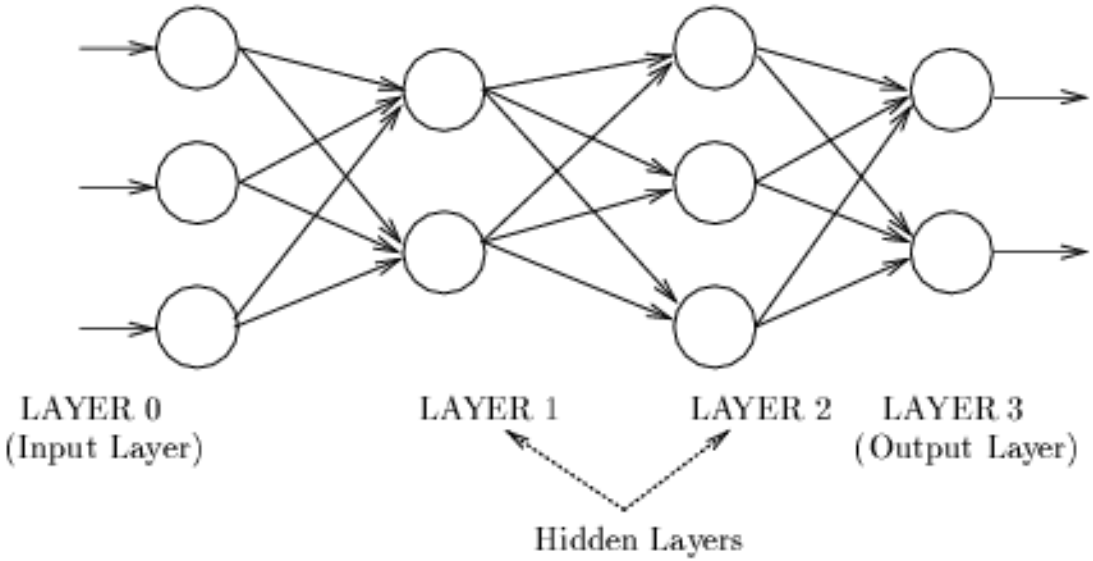


Training of Multilayer Feed-Forward Neural Networks : Back-propagtaion

Multilayer Feed-Forward Neural Networks

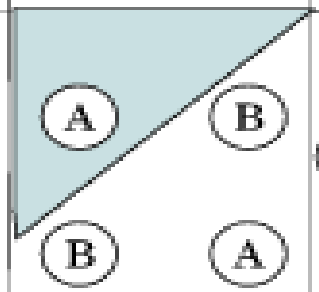
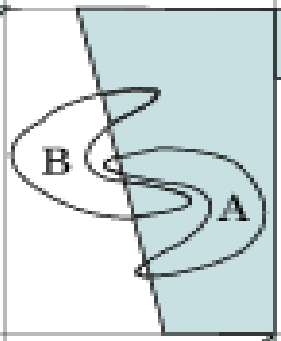
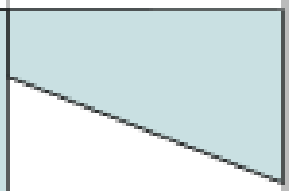
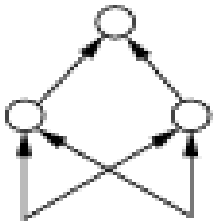
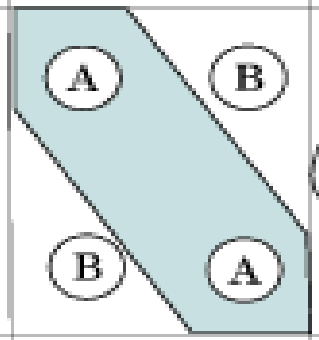
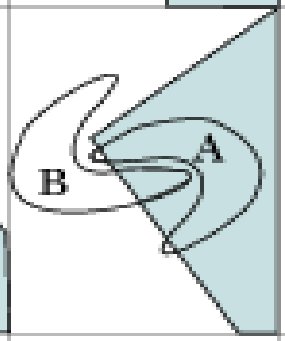
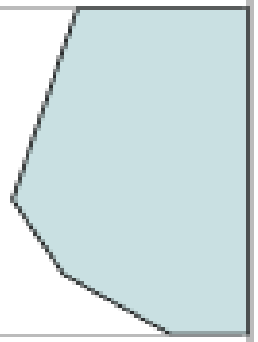
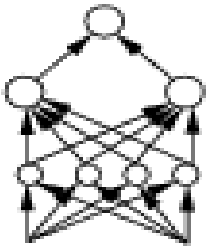
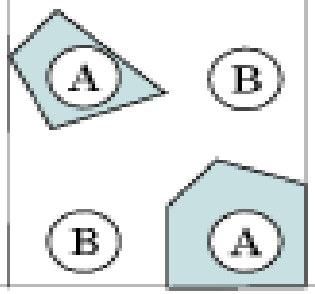
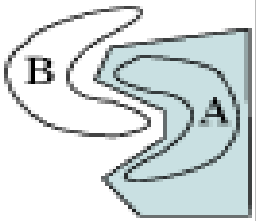
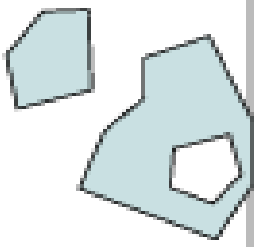
Feed -forward Networks

- ***A connection is allowed from a node in layer* i *only to nodes in layer* i + *1.***
- ***Most widely used architecture.***

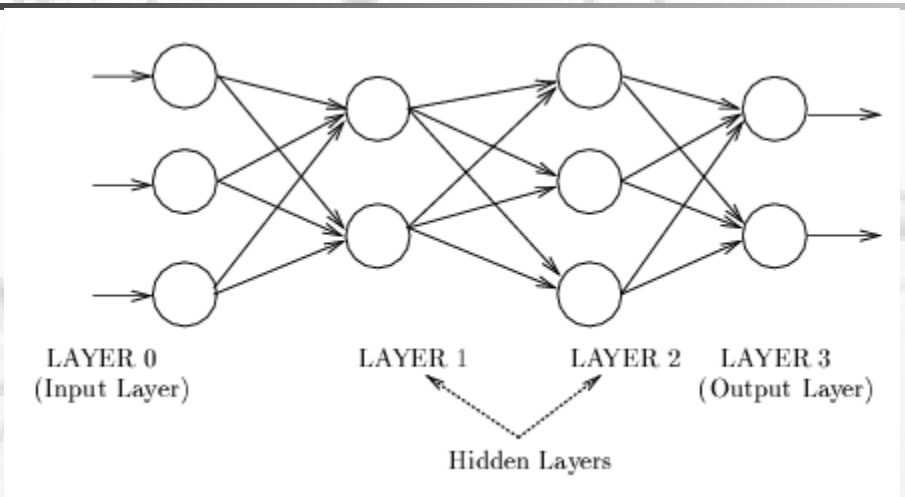


Conceptually, nodes at higher levels successively abstract features from preceding layers

Geometric interpolation of the role of HN

Structure	Types of decision regions	Exclusive OR problem	Classes with meshed regions	Most general region shapes
Single-layer	Half Plane (Bounded by hyperplane)			
Two-layer 	Convex (Open or closed regions)			
Three-layer 	Arbitrary (Complexity limited by number of neurons)			

How to get appropriate weight set?



Back-propagation is the famous algorithm for training feed-forward networks

http://en.wikipedia.org/wiki/Backpropagation

Backpropagation, or propagation of error, is a common method of teaching artificial neural networks how to perform a given task.
It was first described by Paul Werbos in 1974, but it wasn't until 1986, through the work of David E. Rumelhart, Geoffrey E. Hinton and Ronald J. Williams, that it gained recognition, and it led to a **"renaissance"** in the field of artificial neural network research.

nature publishing group

nature.com | about npg | news@nature.com | naturejobs | natureevents | help | site index

nature

my account | e-alerts | subscribe | register

SEARCH JOURNAL Go

Tuesday 06 October 2009

Journal Home
Current Issue
AOP
Archive

THIS ARTICLE

Download PDF
References

Export citation
Export references

Send to a friend

More articles like this

Table of Contents
< Previous | Next >

letters to nature

Nature **323**, 533 - 536 (09 October 1986); doi:10.1038/323533a0

Learning representations by back-propagating errors

DAVID E. RUMELHART*, GEOFFREY E. HINTON† & RONALD J. WILLIAMS*

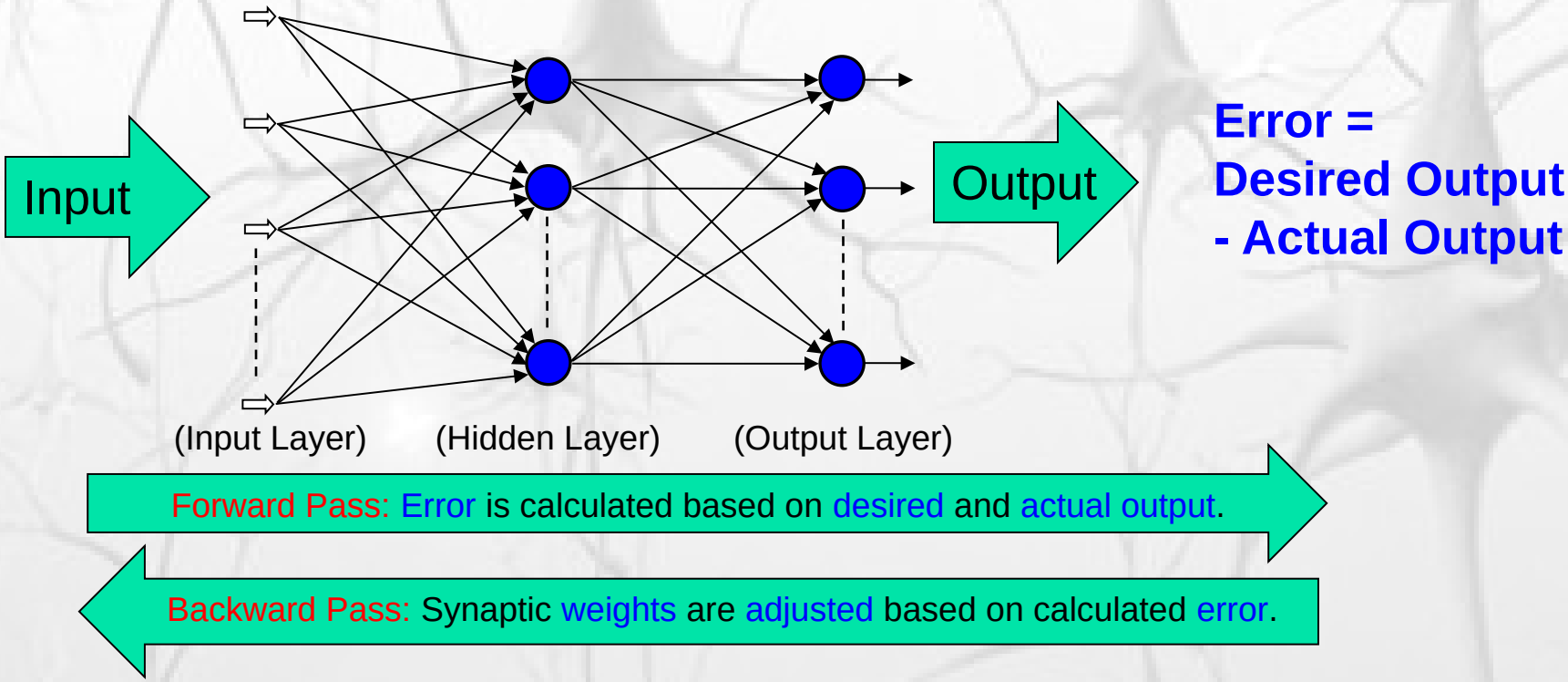
*Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
†Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA
†To whom correspondence should be addressed.

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

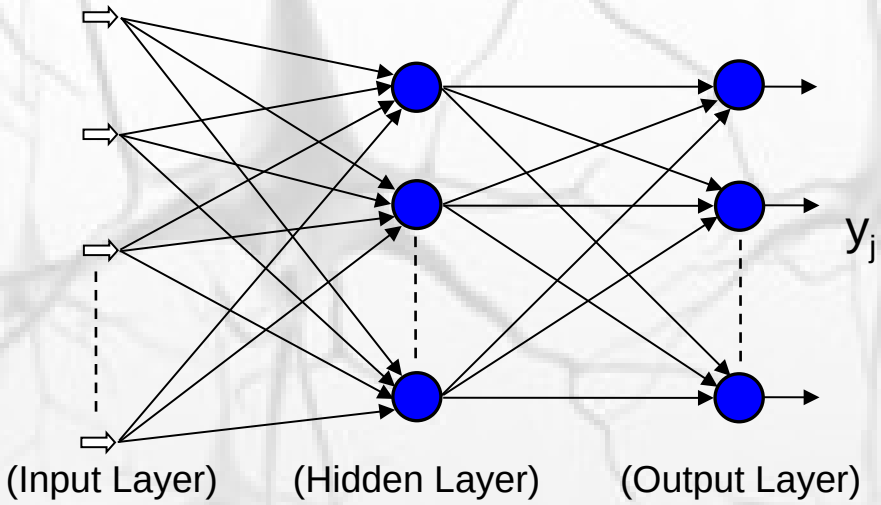
References

1. Rosenblatt, F. *Principles of Neurodynamics* (Spartan, Washington, DC, 1961).
2. Minsky, M. L. & Papert, S. *Perceptrons* (MIT, Cambridge, 1969).
3. Le Cun, Y. *Proc. Cognitiva* **85**, 599–604 (1985).

Back-propagation(BP)



Back-propagation



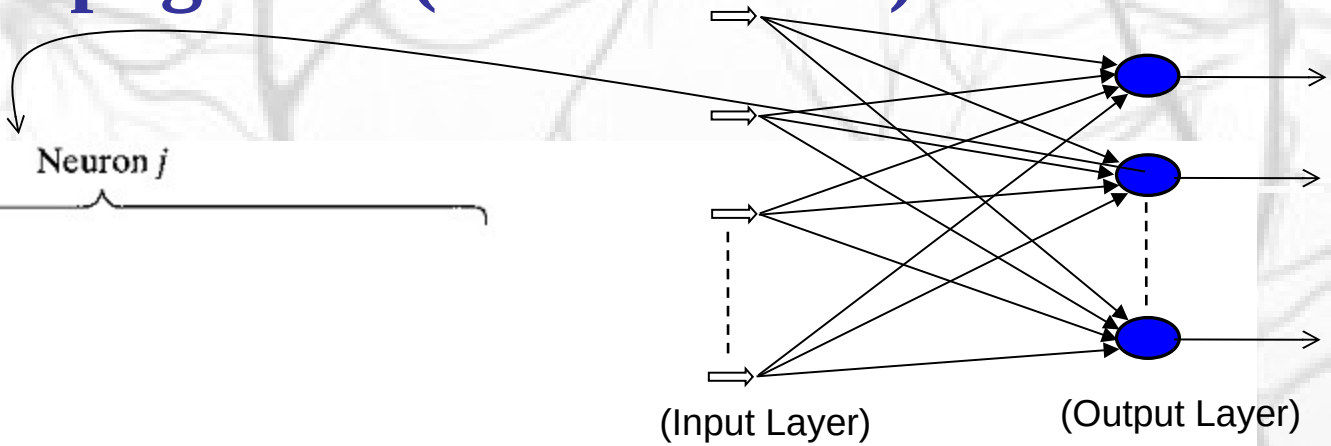
$$e_j(n) = d_j(n) - y_j(n), \qquad \text{neuron } j \text{ is an output node} \tag{4.1}$$

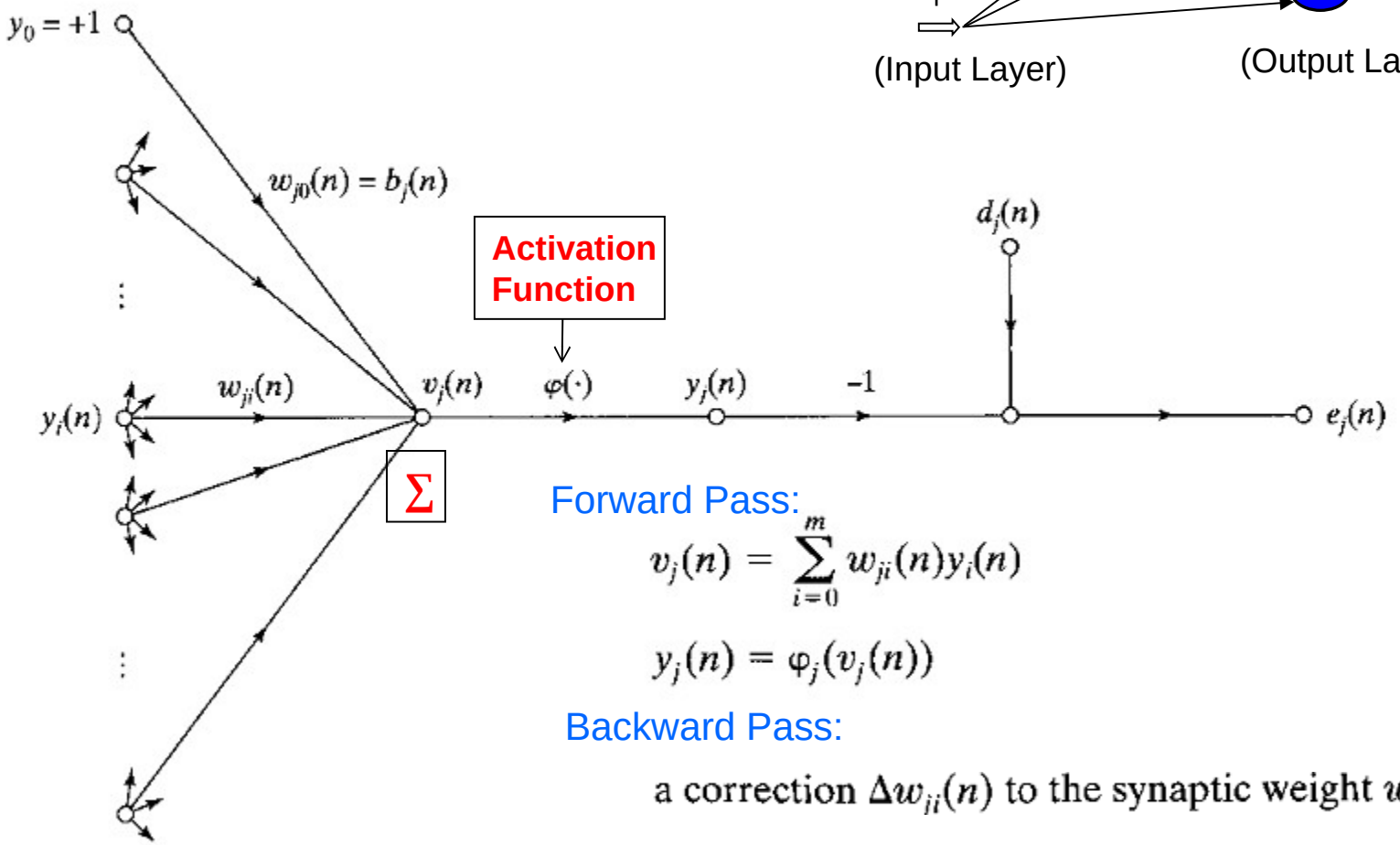
$$\mathscr{E}(n) = \frac{1}{2}\sum_{j \in C} e_j^2(n) \tag{4.2}$$

$$\mathscr{E}_{\text{av}} = \frac{1}{N}\sum_{n=1}^{N} \mathscr{E}(n) \tag{4.3}$$

performance. The objective of the learning process is to adjust the free parameters of the network to minimize $\mathscr{E}_{\text{av}}$. ~~To do this minimization, we use an approximation similar~~

Back-propagation (Without HN)





Forward Pass:

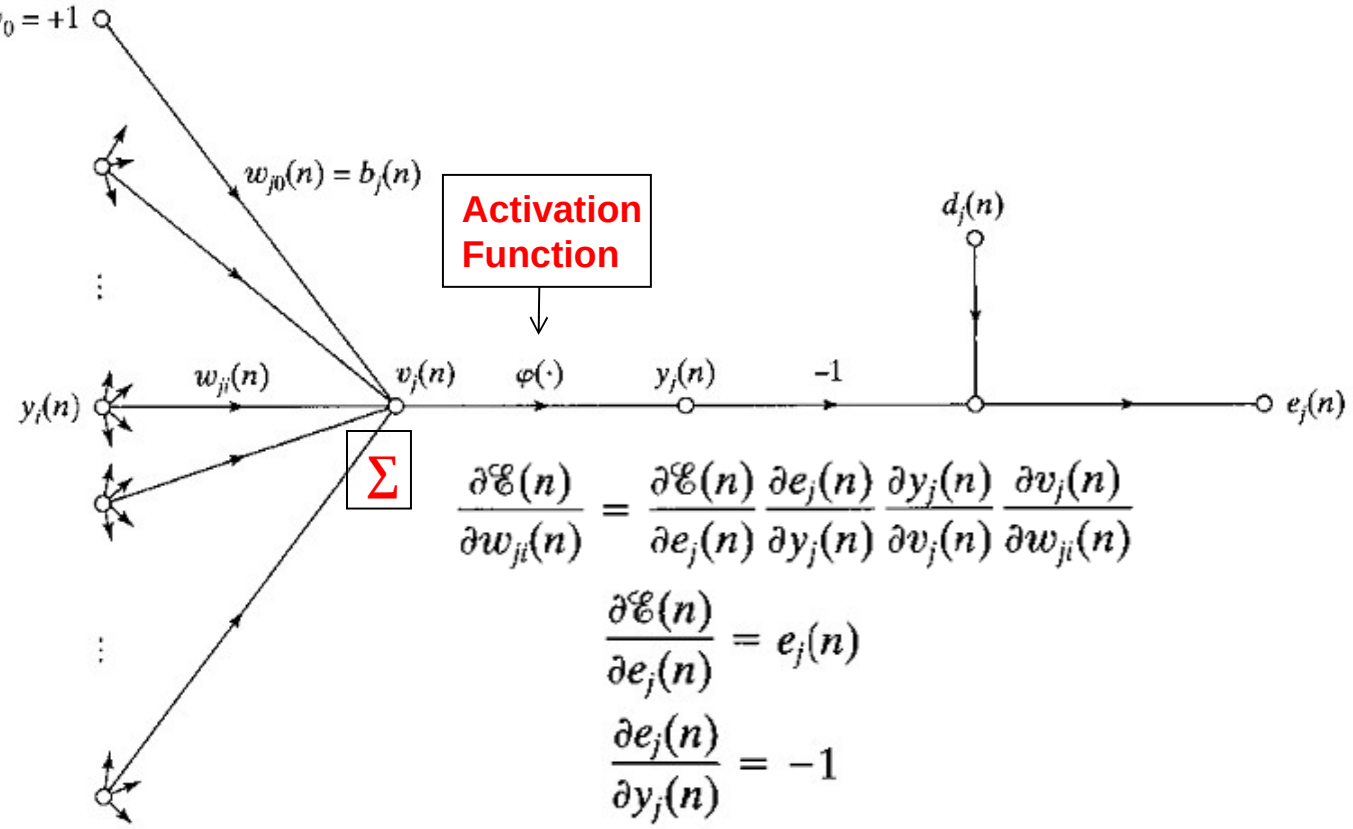
$$v_j(n) = \sum_{i=0}^{m} w_{ji}(n) y_i(n) \qquad (4.4)$$

$$y_j(n) = \varphi_j(v_j(n)) \qquad (4.5)$$

Backward Pass:

a correction $\Delta w_{ji}(n)$ to the synaptic weight $w_{ji}(n)$

FIGURE 4.3 Signal-flow graph highlighting the details of output neuron j.



$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)} \tag{4.6}$$

$$\frac{\partial \mathcal{E}(n)}{\partial e_j(n)} = e_j(n) \tag{4.7}$$

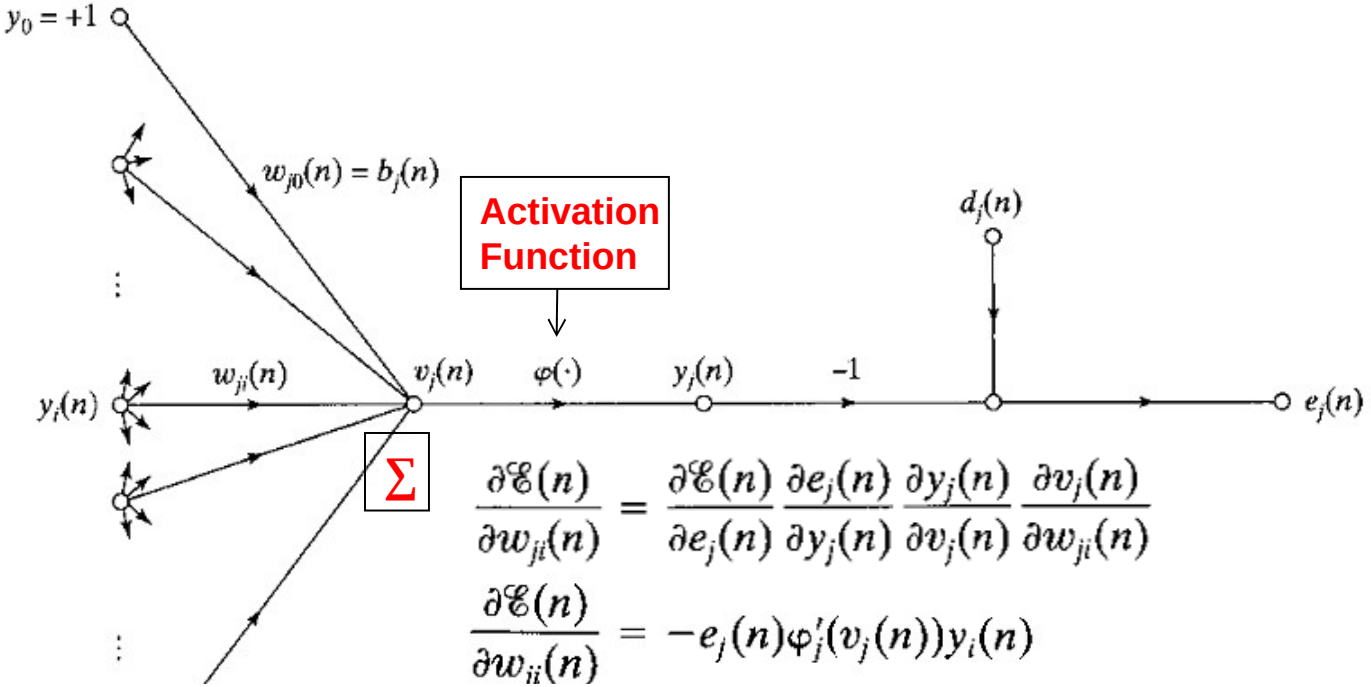
$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1 \tag{4.8}$$

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \varphi_j'(v_j(n)) \tag{4.9}$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n) \tag{4.10}$$

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = -e_j(n)\varphi_j'(v_j(n))y_i(n) \tag{4.11}$$

Depends on Activation Function



$$\frac{\partial \mathscr{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathscr{E}(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)} \tag{4.6}$$

$$\frac{\partial \mathscr{E}(n)}{\partial w_{ji}(n)} = -e_j(n)\varphi_j'(v_j(n))y_i(n) \tag{4.11}$$

The correction $\Delta w_{ji}(n)$ applied to $w_{ji}(n)$ is defined by the *delta rule*:

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathscr{E}(n)}{\partial w_{ji}(n)} \tag{4.12}$$

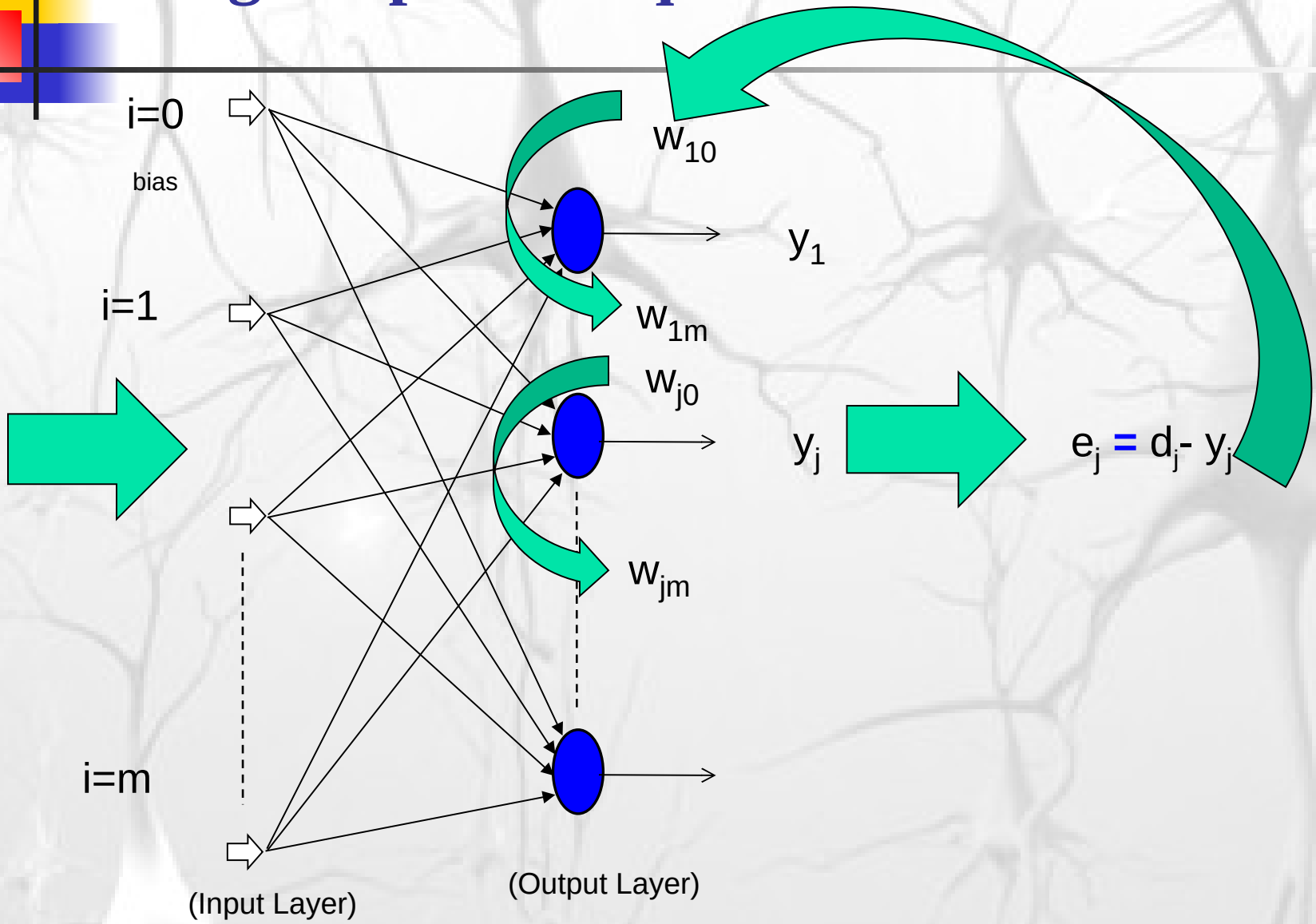
where η is the *learning-rate parameter* of the back-propagation algorithm. The use of the minus sign in Eq. (4.12) accounts for *gradient descent* in weight space (i.e., seeking a direction for weight change that reduces the value of $\mathscr{E}(n)$). Accordingly, the use of Eq. (4.11) in (4.12) yields

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n) \tag{4.13}$$

where the *local gradient* $\delta_j(n)$ is defined by

$$\delta_j(n) = -\frac{\partial \mathscr{E}(n)}{\partial v_j(n)} = -\frac{\partial \mathscr{E}(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} = e_j(n)\varphi_j'(v_j(n)) \tag{4.14}$$

Weight Update Sequence in BP



BP for Multilayer (With HN)

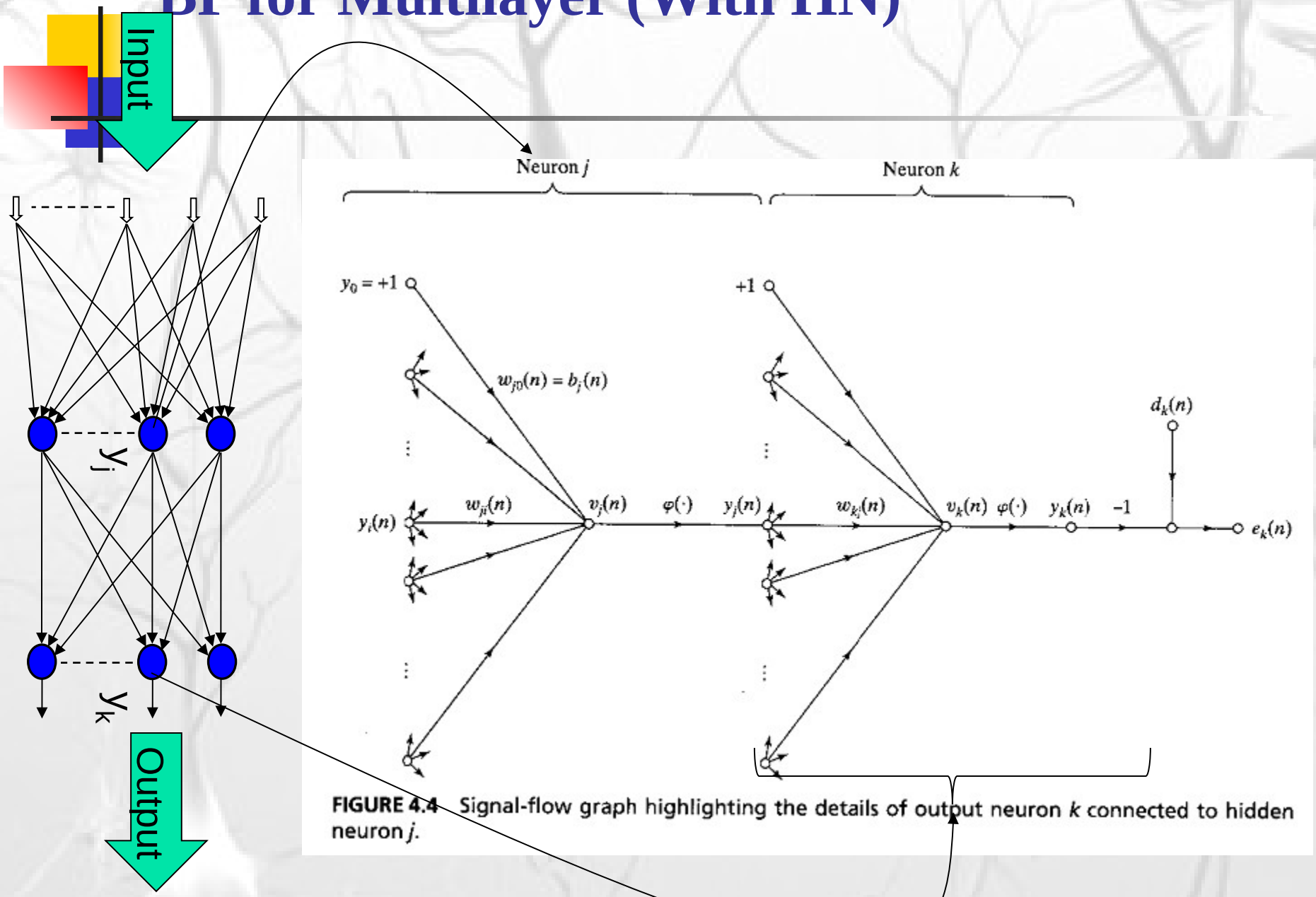
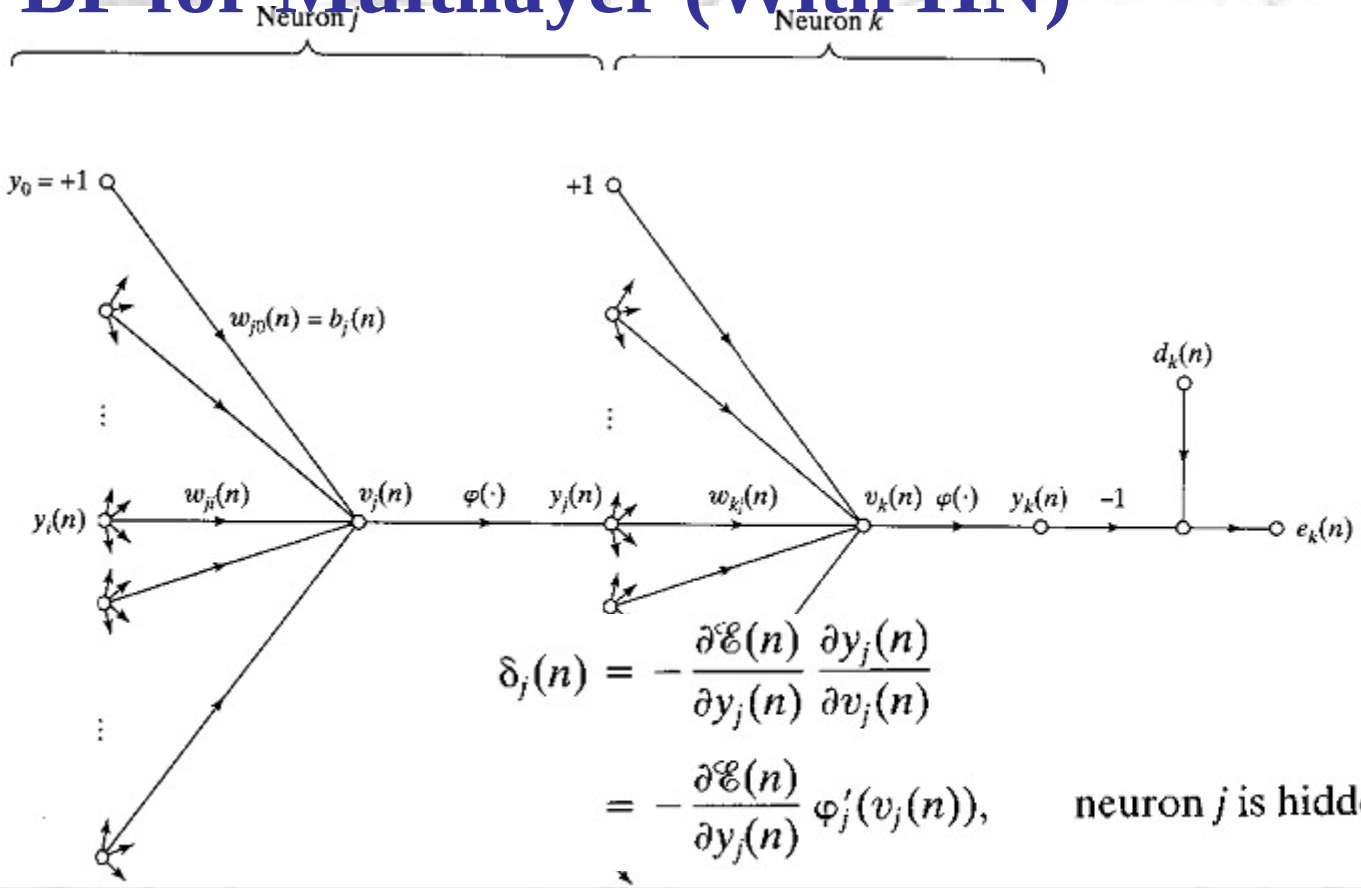


FIGURE 4.4 Signal-flow graph highlighting the details of output neuron k connected to hidden neuron j.

BP for Multilayer (With HN)



$$\delta_j(n) = -\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \tag{4.15}$$

$$= -\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} \varphi_j'(v_j(n)), \qquad \text{neuron } j \text{ is hidden}$$

$$\mathscr{E}(n) = \frac{1}{2} \sum_{k \in C} e_k^2(n), \qquad \text{neuron } k \text{ is an output node} \tag{4.16}$$

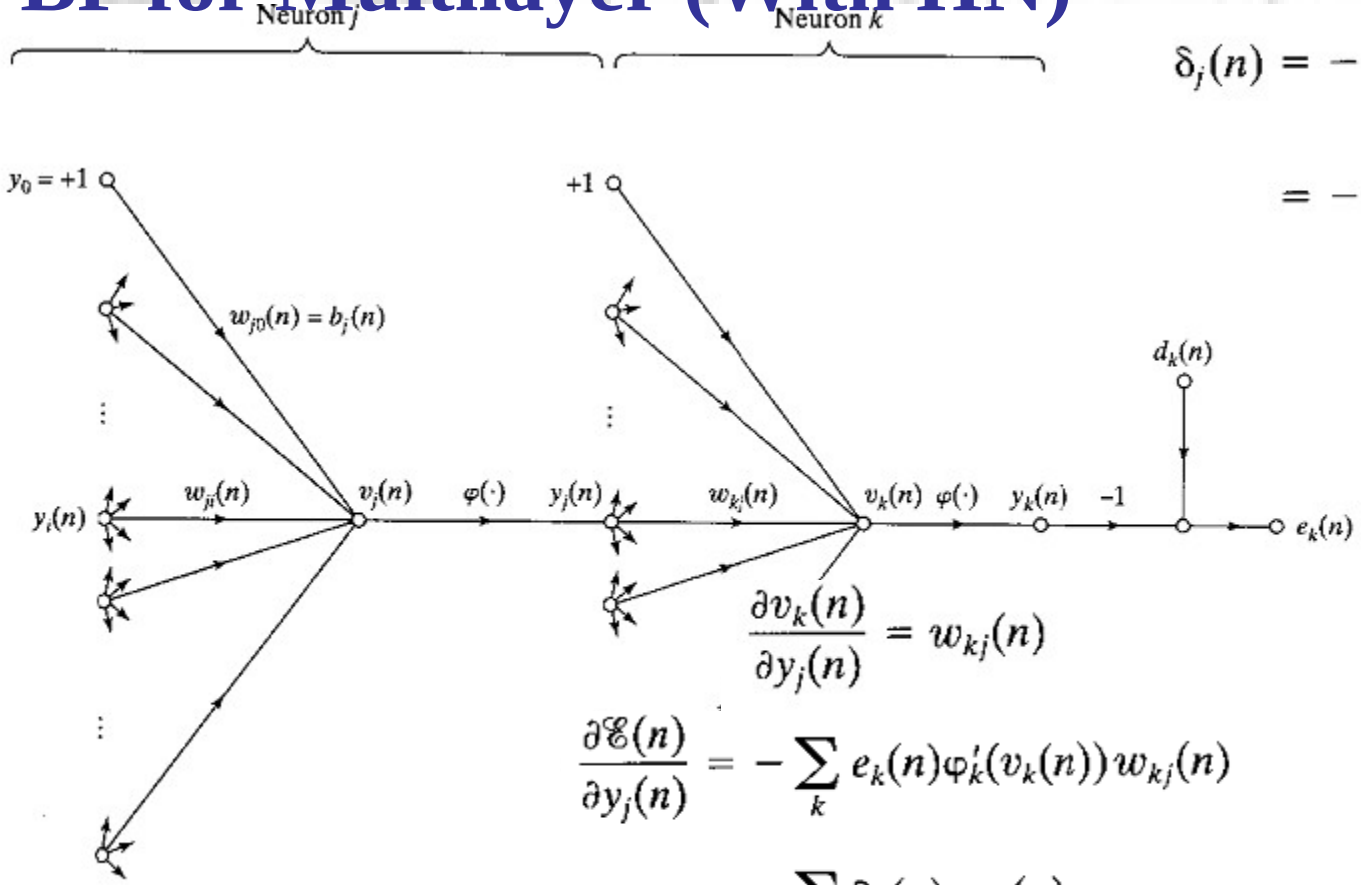
$$\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} = \sum_k e_k \frac{\partial e_k(n)}{\partial y_j(n)} \tag{4.17}$$

$$\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial v_k(n)} \frac{\partial v_k(n)}{\partial y_j(n)} \tag{4.18}$$

$$e_k(n) = d_k(n) - y_k(n)$$
$$= d_k(n) - \varphi_k(v_k(n)), \tag{4.19}$$

$$v_k(n) = \sum_{j=0}^{m} w_{kj}(n) y_j(n) \tag{4.21}$$

BP for Multilayer (With HN)



$$\delta_j(n) = -\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

$$= -\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} \varphi_j'(v_j(n)),$$

$$\frac{\partial v_k(n)}{\partial y_j(n)} = w_{kj}(n) \tag{4.22}$$

$$\frac{\partial \mathscr{E}(n)}{\partial y_j(n)} = -\sum_k e_k(n)\varphi_k'(v_k(n))w_{kj}(n) \tag{4.23}$$

$$= -\sum_k \delta_k(n)w_{kj}(n)$$

$$\delta_j(n) = \varphi_j'(v_j(n)) \sum_k \delta_k(n)w_{kj}(n), \qquad \text{neuron } j \text{ is hidden} \tag{4.24}$$

$$\begin{pmatrix} \textit{Weight} \\ \textit{correction} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \textit{learning-} \\ \textit{rate parameter} \\ \eta \end{pmatrix} \cdot \begin{pmatrix} \textit{local} \\ \textit{gradient} \\ \delta_j(n) \end{pmatrix} \cdot \begin{pmatrix} \textit{input signal} \\ \textit{of neuron } j \\ y_i(n) \end{pmatrix} \tag{4.25}$$

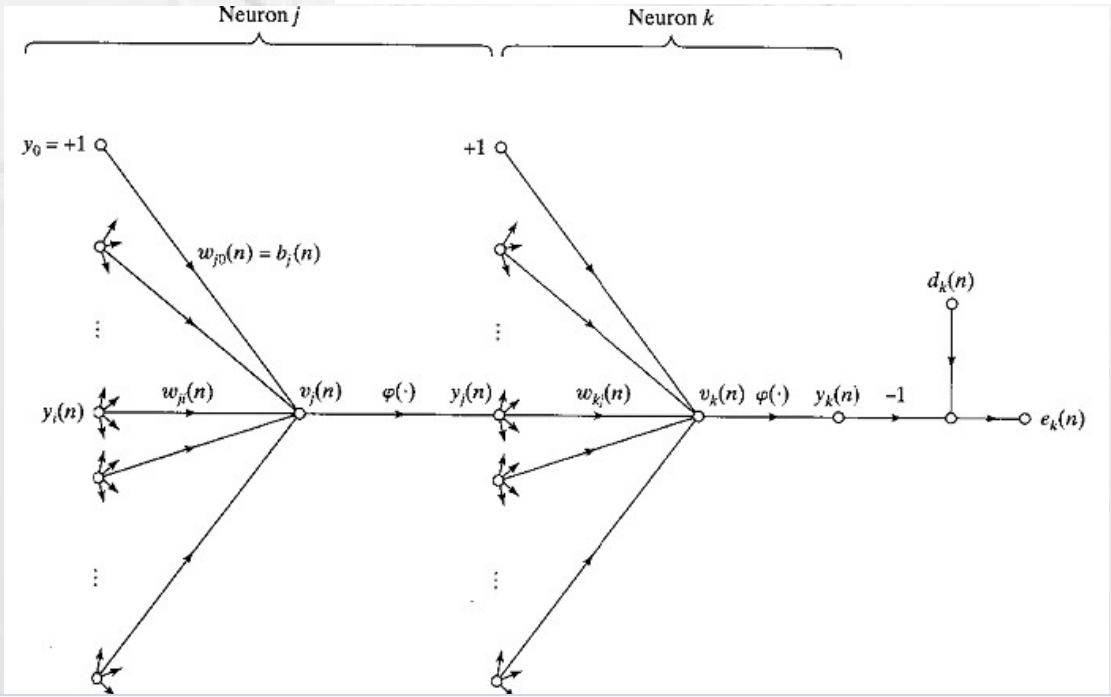
BP for Multilayer (With HN)

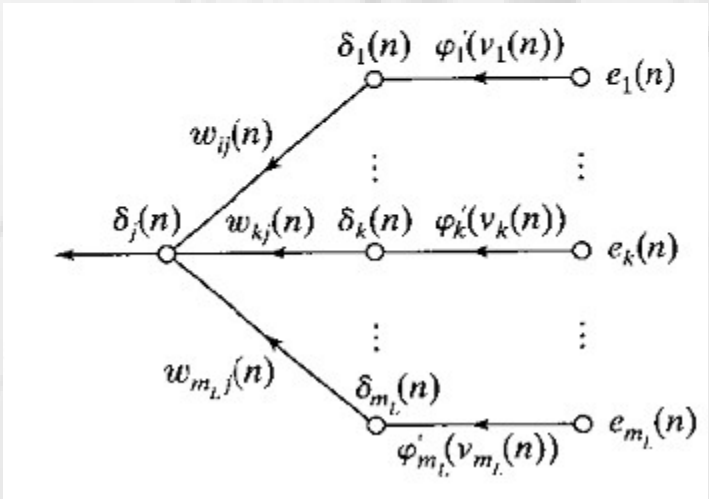
$$w_{ji}(t+1)=w_{ji}(t)+\Delta w_{ji}$$

$$\begin{pmatrix} \text{Weight} \\ \text{correction} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{learning-} \\ \text{rate parameter} \\ \eta \end{pmatrix} \cdot \begin{pmatrix} \text{local} \\ \text{gradient} \\ \delta_j(n) \end{pmatrix} \cdot \begin{pmatrix} \text{input signal} \\ \text{of neuron } j \\ y_i(n) \end{pmatrix}$$

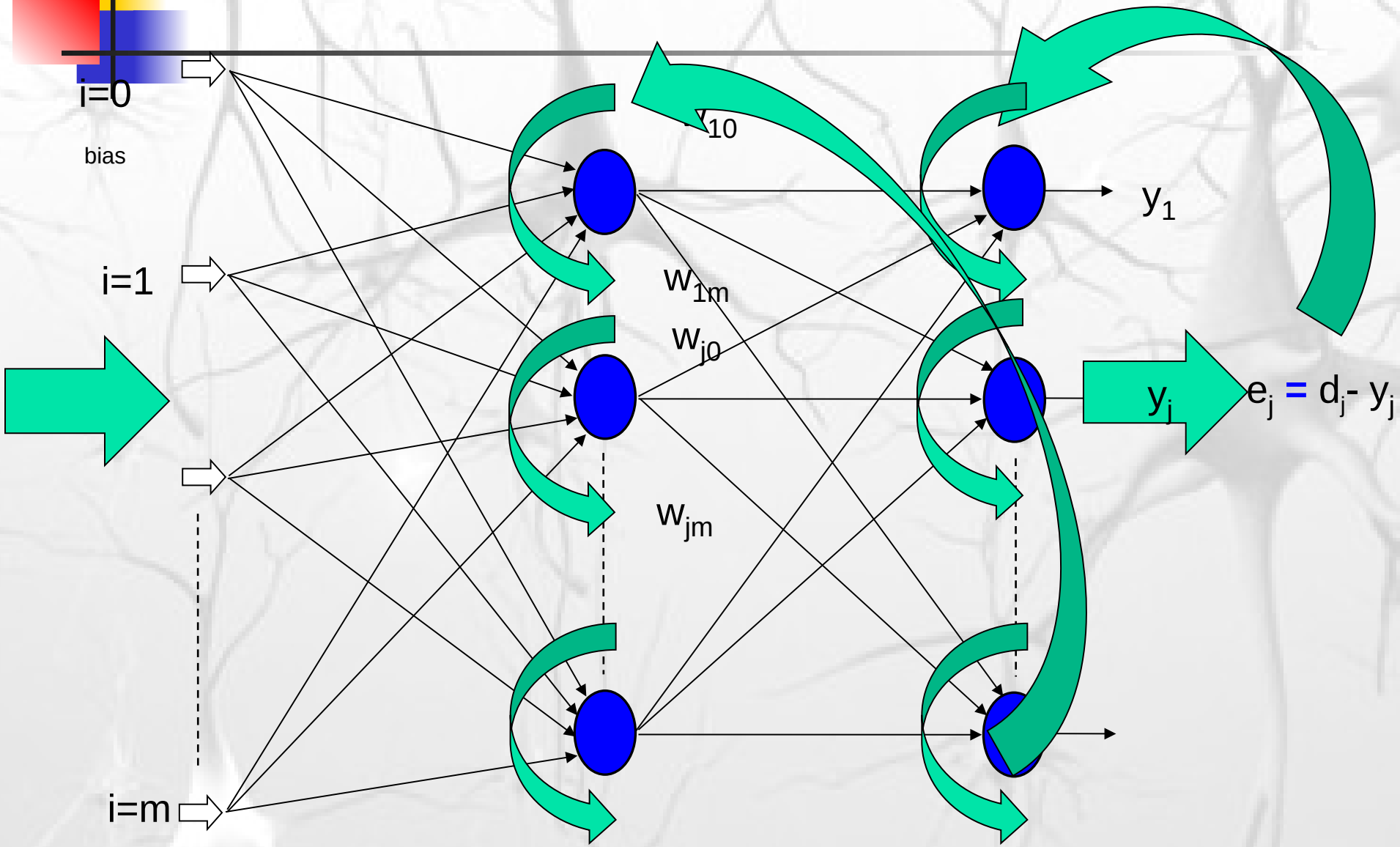
$\delta_k(n)=e_k(n)\varphi'(v_k(n))$ for k output neuron

$$\delta_j(n) = \varphi_j'(v_j(n)) \sum_k \delta_k(n) w_{kj}(n), \qquad \text{neuron } j \text{ is hidden}$$

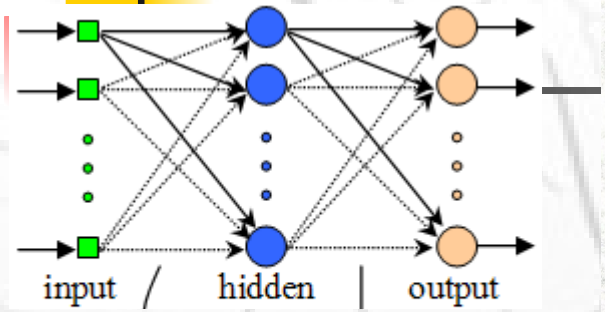




Weight Update Sequence in BP



Matter of Activation Function(AF)



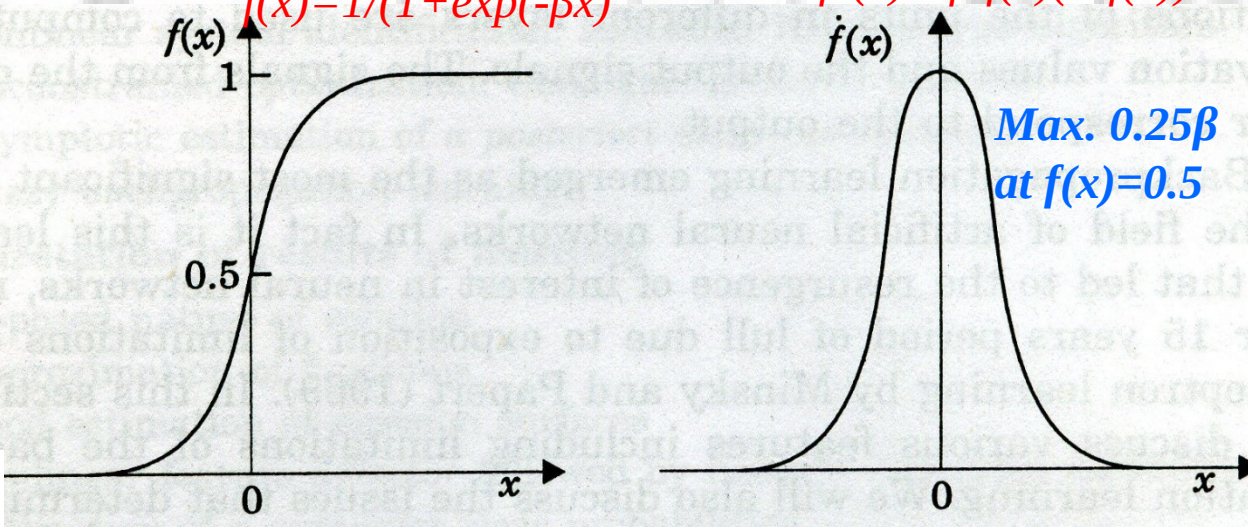
$\delta_k(n)=e_k(n)\varphi'(v_k(n))$

$$\delta_j(n) = \varphi_j'(v_j(n)) \sum_k \delta_k(n) w_{kj}(n)$$

Differentiability is the only condition for AF

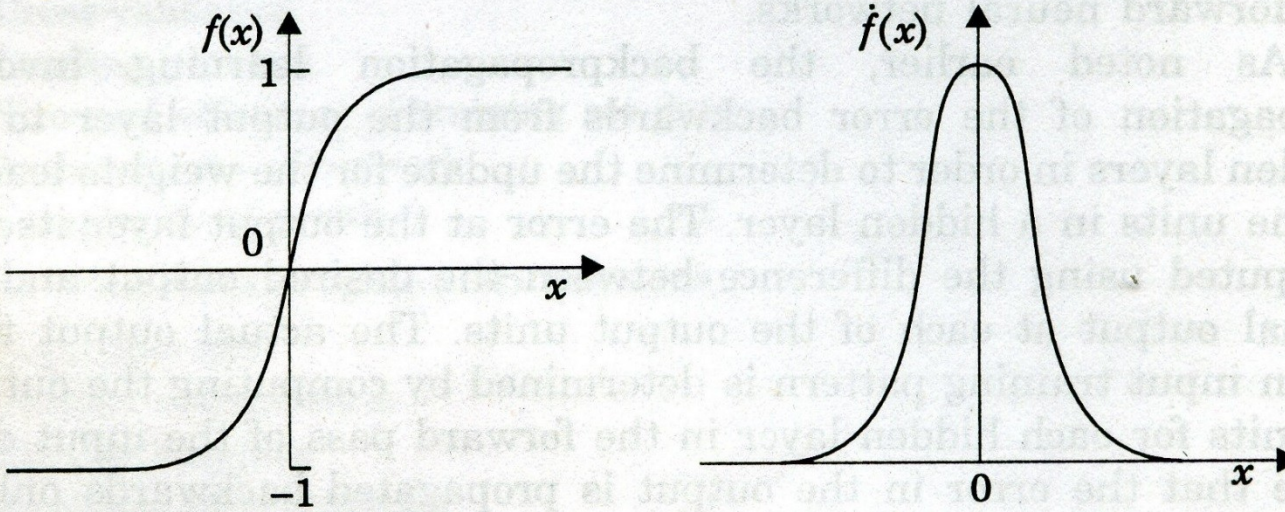
an example of continuously differentiable nonlinear AF is sigmoidal nonlinearity; its two forms are :

$f(x)=1/(1+\exp(-\beta x))$ $\quad f'(x)=\beta f(x)(1-f(x))$



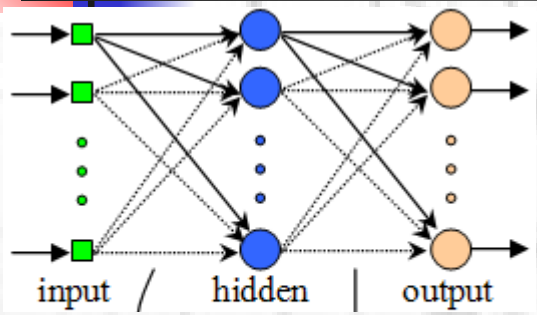
(a) Logistic function and its derivative

$f(x) = \tanh \beta x$ $\quad \dot{f}(x) = \beta(1 - f^2(x))$



(b) Hyperbolic tangent function and its derivative

BP at a Glance



$$\left(\begin{array}{c}\text{Weight}\\ \text{correction}\end{array}\right) = \left(\begin{array}{c}\text{learning-}\\ \text{rate parameter}\end{array}\right) \cdot \left(\begin{array}{c}\text{local}\\ \text{gradient}\end{array}\right) \cdot \left(\begin{array}{c}\text{input signal}\\ \text{of neuron } j\end{array}\right)$$

$$\Delta w = \eta\ \delta\ x$$

The local gradient of output unit (δ_o) and hidden unit (δ_h) are defined by:

$$\delta_o = -\frac{\partial e}{\partial f_o}\frac{\partial f_o}{\partial x_o} \qquad \delta_h = \sum_o \delta_o w_o \frac{\partial f_h}{\partial x_h}$$

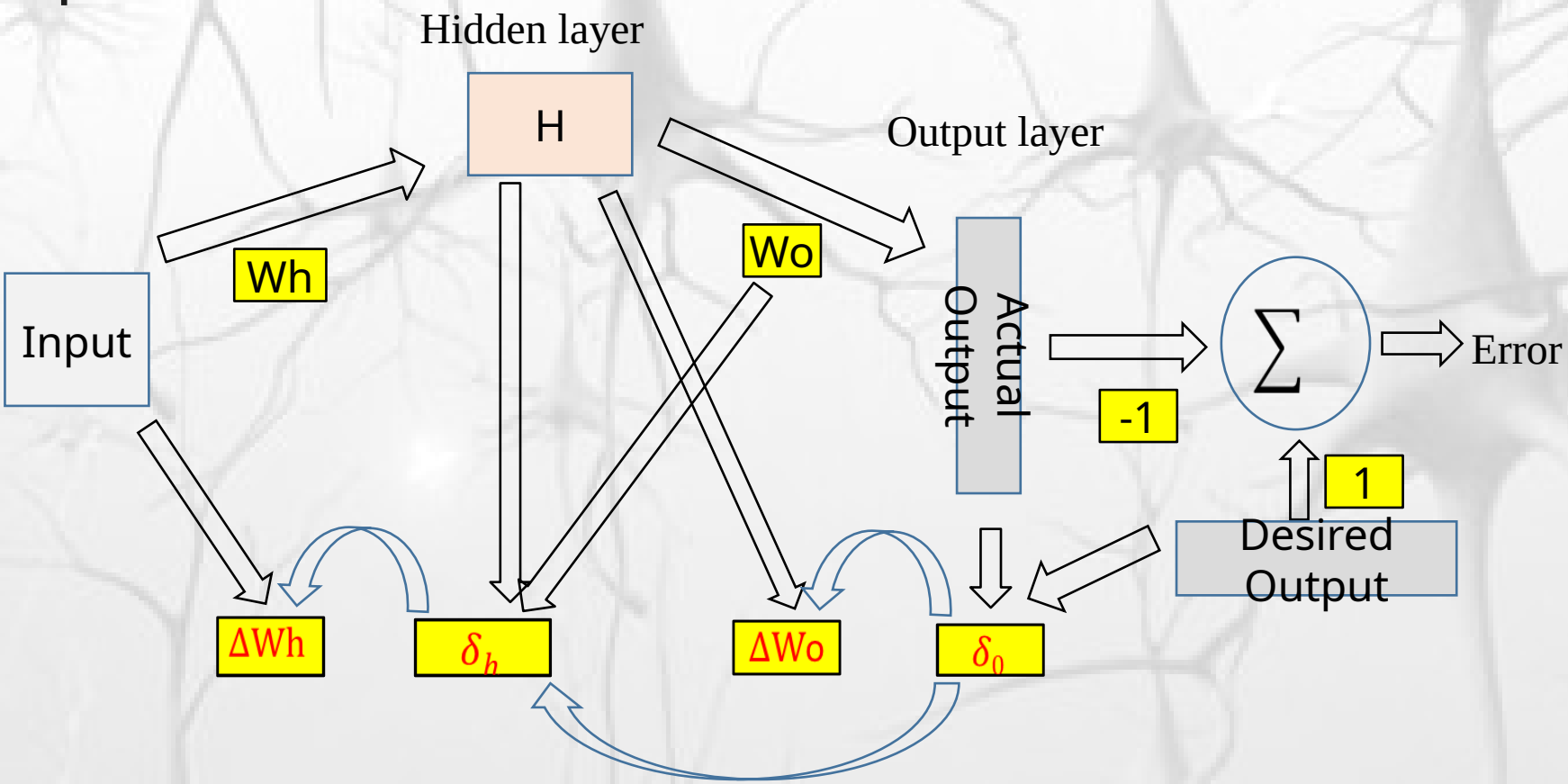
$$e(n) = \frac{1}{2}(d(n) - f_o(n))^2, \qquad \frac{\partial e_o(n)}{\partial f_o(n)} = -(d(n) - f_o(n))$$

For logistic sigmoid activation function $f_o(n) = 1/(1+\exp(-x_o(n)))$ $\dfrac{\partial f_o(n)}{\partial x_o(n)} = f_o(n)(1 - f_o(n))$

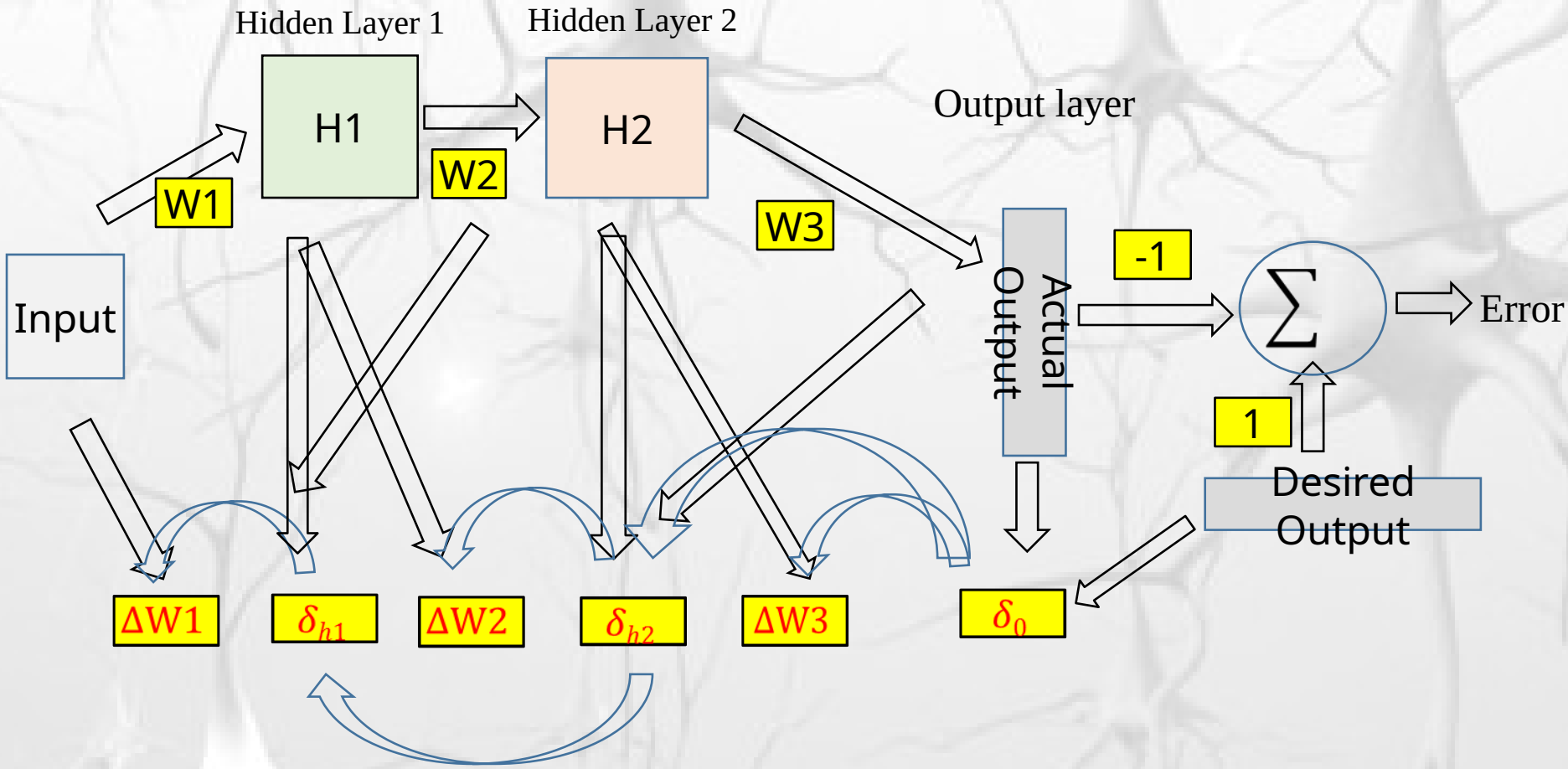
Now the local gradient of output unit (δ_o) becomes $\delta_o = (d(n) - f_o(n))f_o(n)(1 - f_o(n))$

For the same sigmoid activation function the local gradient of hidden unit (δ_h) becomes $\delta_h = f_h(n)(1 - f_h(n))\sum_o \delta_o w_o$

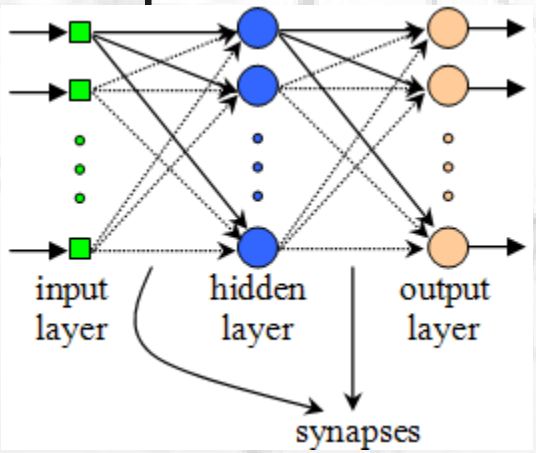
Operational flowchart of a NN with Single Hidden Layer

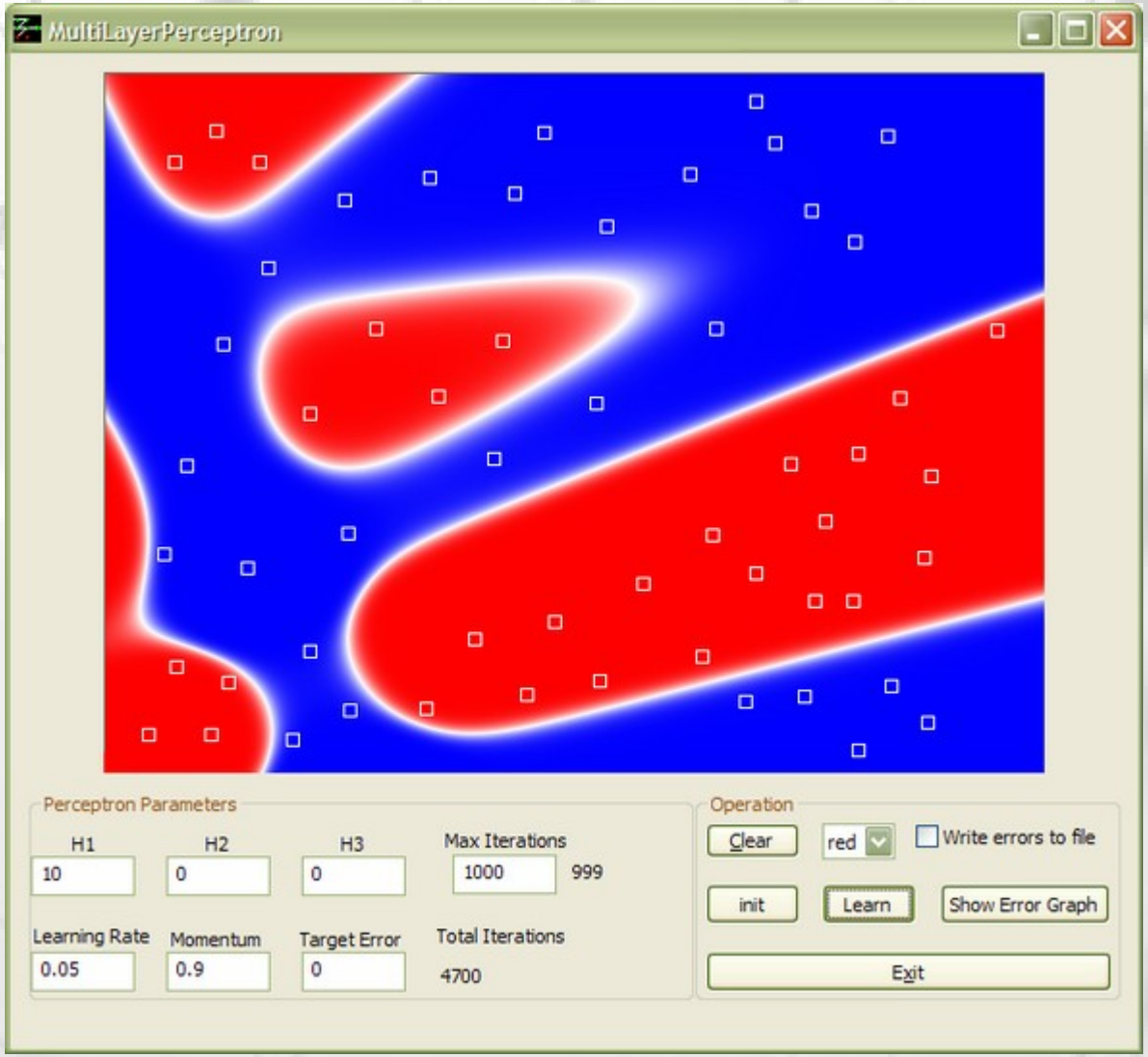


Operational flowchart of a NN with Two Hidden Layer



A MLP Simulator for Simple Application





Hossein Khosravi : The Developer of the Simulator

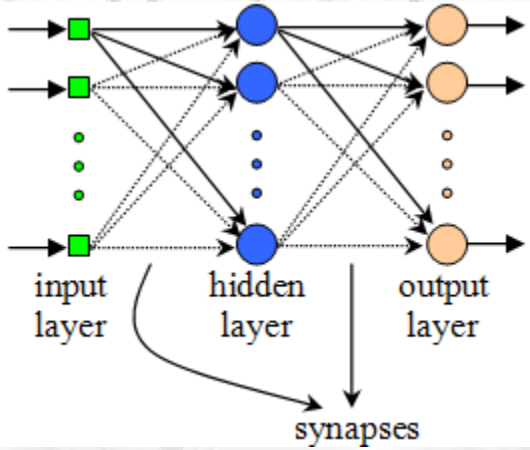
https://www.codeproject.com/Articles/9447/Neural-Network-Classifier

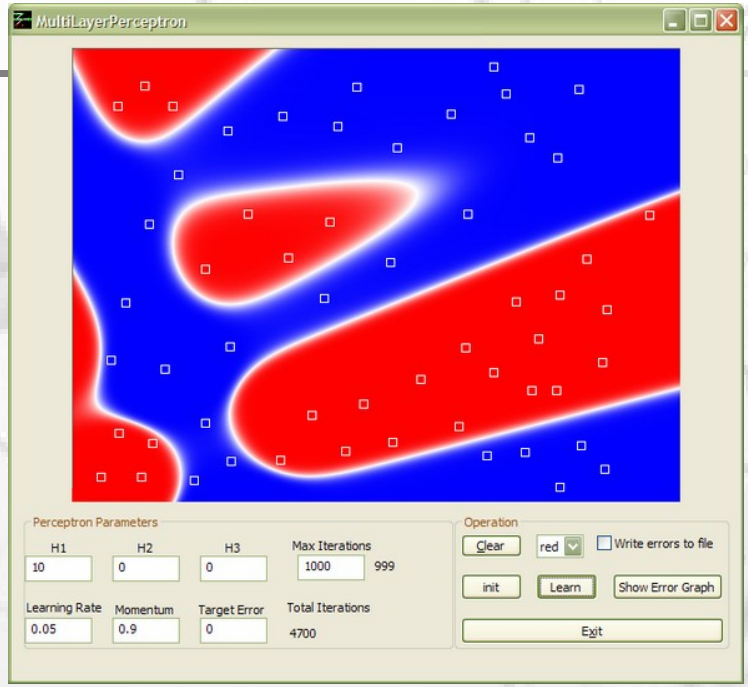
Member Profile: Hossein Khosravi

About Member

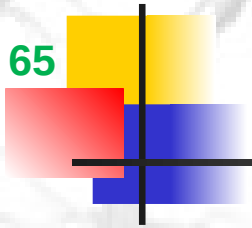
Friendly Url	http://www.codeproject.com/Members/Hossein-Khosravi
Status	**Silver**. Member No. 429414
	View Member's Blog
Messages Posted	26 - Poster
Articles Submitted	2 - Contributor
Biography	I am an electronic engineer intrested in Pattern Recognition (specially OCR), Neural Networks and Image Processing. I was born in Khezri, a small town in Khorasan, the largest province of Iran, I have taken My BS. from Sharif University of thechnology and by now I'm studying in Tarbiat Modarres University as a Ph.D. candidate.
Location	Iran, Islamic Republic Of
Job Title	Web Developer
Company	
Member since	Sunday, June 08, 2003 (6 years, 4 months)
Homepage	

Blog:



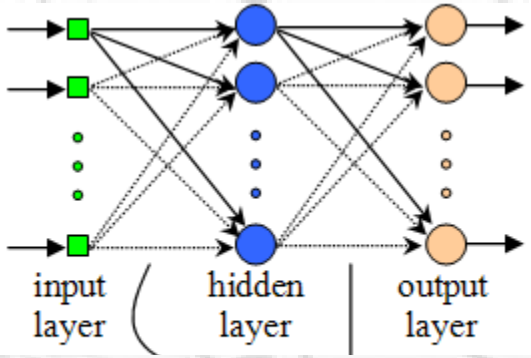


Lets run the simulator



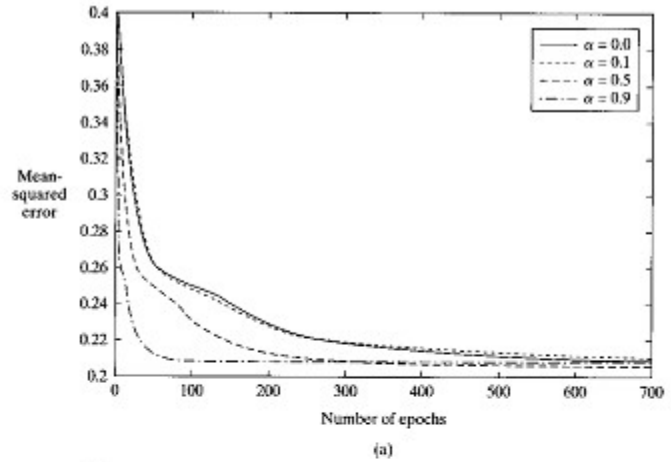
Practical Aspects of BP

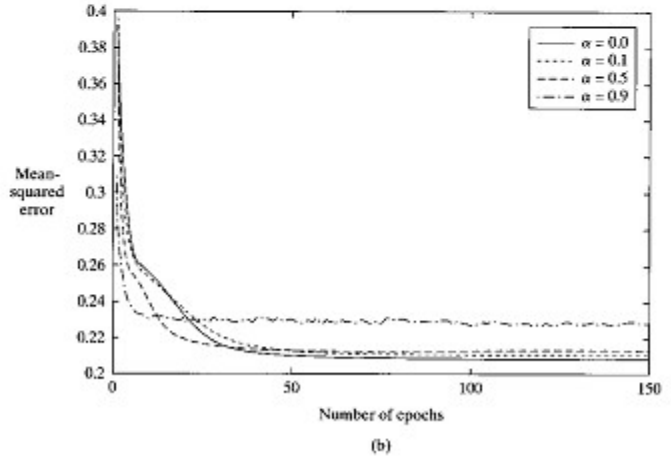
Practical Considerations

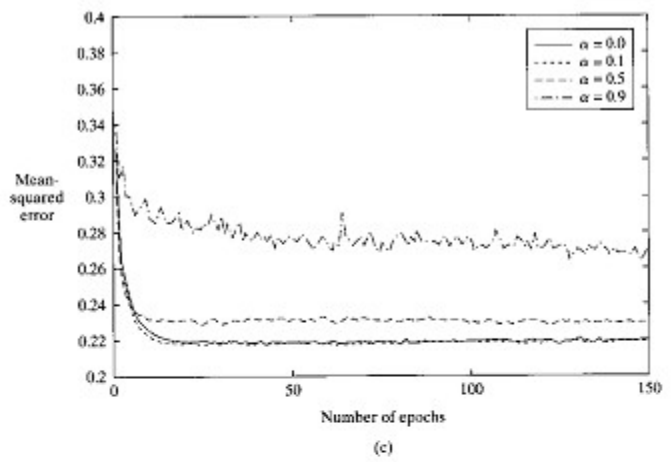


- **Training Data:** Sufficient / proper training data is require for proper input-output mapping.
- **Network Size:** Complex problem require more hidden neurons and may perform better with multiple hidden layers.
- **Weight Initialization:** NN works on numeric data. Before training all the synaptic weights are initialized with small random values, e.g., -0.5 to +0.5.
- **Learning Rate (η) and Momentum Constant (α):** A small η results slower convergence and its larger value gives oscillation. Momentum also speed up training but larger α with large η arise oscillation.
- **Stopping Training:** Training should stop at better generalization position.

Effect of Learning Rate (η) and Momentum Constant (α) values







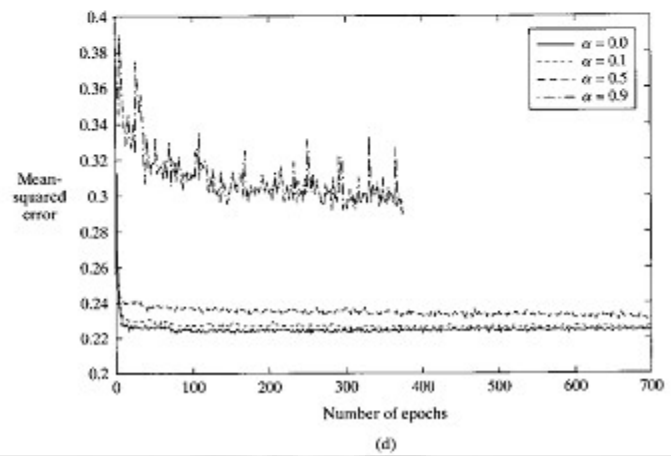


FIGURE 4.15 Ensemble-averaged learning curves for varying momentum α, and the following values of learning-rate parameters: (a) $\eta = 0.01$, (b) $\eta = 0.1$ (c) $\eta = 0.5$, and (d) $\eta = 0.9$.

Learning and Generalization

Learning and generalization are the most important topics in NN research. Learning is the ability to approximate the training data while generalization is the ability to predict well beyond the training data.

- Generalization is more desirable because the common use of a NN is to make good prediction on new or unknown objects.
- It measures on the testing set that is reserved from available data and not use in the training.
- Testing error rate (TER), i.e., rate of wrong classification on testing set, is widely acceptable quantifying measure, which value minimum is good.

$$TER = \frac{\text{Total testing set misclassified patterns}}{\text{Total testing set patterns}}$$

Learning and Overfitting

Available Data

Training Set
(Use for learning)

Testing Set
(Reserve to measure generalization)

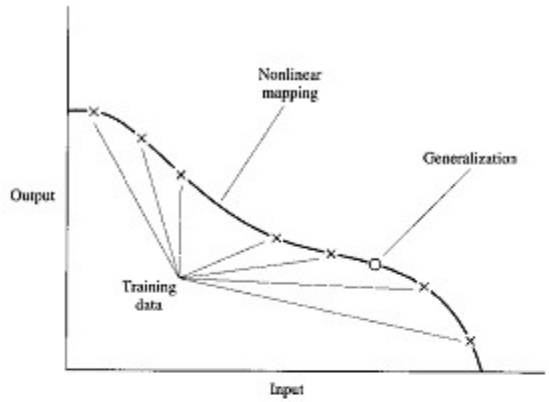
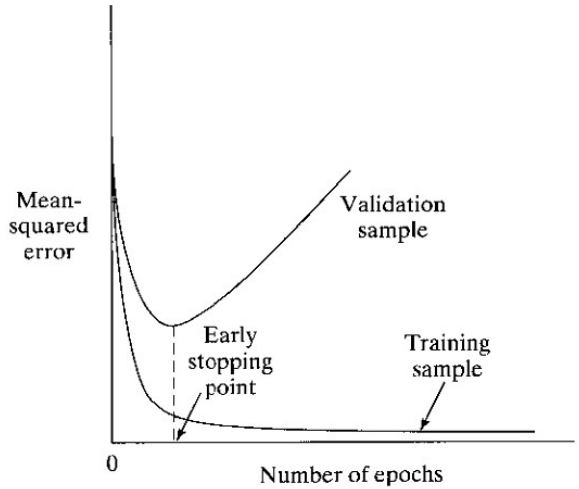
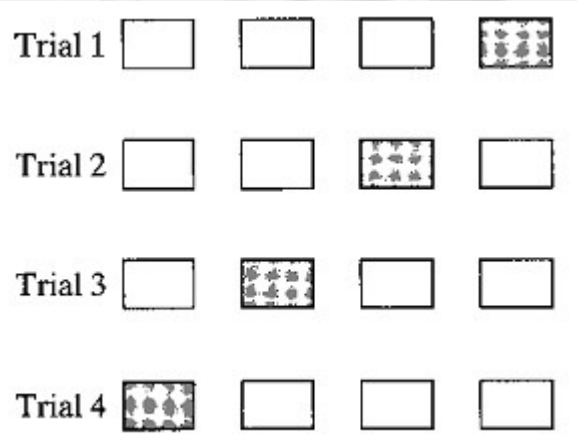


FIGURE 4.19 (a) Properly fitted data (good generalization) (b) Overfitted data (poor generalization).





Where we find benchmark data for testing NNs or machine learning?

Benchmark Problems for Evaluation

A benchmark is a point of reference by which something can be measured.

- For NN or machine learning, the most popular benchmark dataset collection is the University of California, Irvine (UCI) Machine Learning Repository (http://archive.ics.uci.edu/ml/).

- UCI contains raw data that require preprocessing to use in NN. Some preprocessed data is also available at Proben1 (ftp://ftp.ira.uka.de/pub/neuron/).

- Various persons or groups also maintain different benchmark datasets for specific purpose: Delve (www.cs.toronto.edu/~delve/data/datasets.html), Orange (www.ailab.si/orange/datasets.asp), etc.

UCI Machine Learning Repository - Windows Internet Explorer

http://archive.ics.uci.edu/ml/

UCI Machine Learning Repository

About Citation Policy Donate a Data Set Contact

Search

Repository Web

Machine Learning Repository

Center for Machine Learning and Intelligent Systems

View ALL Data Sets

Welcome to the UC Irvine Machine Learning Repository!

We currently maintain 174 data sets as a service to the machine learning community. You may **view all data sets** through our searchable interface. Our old web site is still available, for those who prefer the old format. For a general overview of the Repository, please visit our About page. For information about citing data sets in publications, please read our citation policy. If you wish to donate a data set, please consult our donation policy. For any other questions, feel free to contact the Repository librarians. We have also set up a mirror site for the Repository.

Supported By: In Collaboration With: Rexa.info

Latest News:

07-23-2008: Repository mirror has been set up.
03-24-2008: New data sets have been added!
06-25-2007: Two new data sets have been added: UJI Pen Characters, MAGIC Gamma Telescope
04-13-2007: Research papers that cite the repository have been associated to specific data sets.
04-09-2007: Three new data sets have been added: Poker Hand, Callt2 Building People Counts, Dodgers Loop Sensor.
09-08-2006: The Beta site has been launched.
09-01-2006: SPECTF.test has been modified by the donor.

Featured Data Set: Statlog (Shuttle)

Task: Classification
Data Type: Multivariate
 # Attributes: 9
 # Instances: 58000

The shuttle dataset contains 9 attributes all of which are numerical. Approximately 80% of the data belongs to class 1

Newest Data Sets:

06-26-2008: Parkinsons
04-21-2008: Ozone Level Detection
04-03-2008: Abscisic Acid Signaling Network
03-20-2008: Hill-Valley
03-12-2008: Bag of Words
03-08-2008: Reuters Transcribed Subset
02-29-2008: Gisette
02-29-2008: Dorothea
02-29-2008: Madelon

Most Popular Data Sets (hits since 2007):

39351: Iris
31585: Adult
26458: Wine
23553: Breast Cancer Wisconsin (Diagnostic)
18449: Abalone
18321: Poker Hand
13471: Yeast
12996: Internet Advertisements
11793: SPECT Heart

UCI Machine Learning Repository - Windows Internet Explorer

Thesis | idea of neural ne... | UCI Machine Lea... | FTP directory: /p... | Ensemble of Div... | H-Bibliography.d... | Desktop | 4:39 PM

Benchmark Problems

Problems Related to Human Life

Problem	Task
Breast Cancer Wisconsin	Predicts whether a tumor is benign (not dangerous to health) or malignant (dangerous) based on a sample tissue taken from a patient's breast.
BUPA Liver Disorder	Identify lever disorders based on blood tests along with other related information such as alcohol consumption.
Diabetes	Investigate whether the patient shows or not the signs of diabetes.
Heart Disease Cleveland	Predicting whether at least one of four major heart vessels is reduced in diameter by more than 50%.
Hepatitis	Anticipate status (i.e., live or die) of hepatitis patient.
Lymphography	Predict the situation of lymph nodes and lymphatic vessels.
Lungcancer	Identify types of pathological lung cancers.
Postoperative	Determine place to send patients for postoperative recovery.

Benchmark Problems

Problems Related to Finance

Problem	Task
Australian Credit Card	Classify people as good or bad credit risks depend on applicants' particulars.
Car	Evaluate cars based on price and facilities.
Labor Negotiations	Identify a worker as good or bad i.e., contract with him beneficial or not.
German Credit Card	Like Australian Card, this problem also concerns to predict the approval or non-approval of a credit card to a customer.

Problems Related to Plants

Problem	Task
Iris Plants	Classify iris plant types.
Mushroom	Identify whether a mushroom is edible or not based on a description of the mushroom's shape, color, odor, and habitat.
Soybean	Recognize 19 different diseases of soybeans.

Benchmark Problems and NN Architecture

After Preprocessing

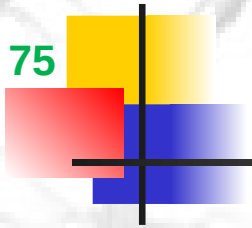
Depends on Problem

Abbr.	Problem	Total Examp	Input Features		NN Architecture		
			Cont.	Discr.	Inputs	Class	Hidd. Node
ACC	Australian Credit Card	690	6	9	51	2	10
BLN	Balance	625	-	4	20	3	10
BCW	Breast Cancer Wisconsin	699	9	-	9	2	5
CAR	Car	1728	-	6	21	4	10
DBT	Diabetes	768	8	-	8	2	5
GCC	German Credit Card	1000	7	13	63	2	10
HDC	Heart Disease Cleveland	303	6	7	35	2	5
HPT	Hepatitis (HPT)	155	6	13	19	2	5
HTR	Hypothyroid	7200	6	15	21	3	5
HSV	House Vote	435	-	16	16	2	5
INS	Ionosphere	351	34	-	34	2	10
KRP	King+Rook vs King+Pawn	3196	-	36	74	2	10
LMP	Lymphography	148	-	18	18	4	10
PST	Postoperative	90	1	7	19	3	5
SBN	Soybean	683	-	35	82	19	25
SNR	Sonar	208	60	-	60	2	10
SPL	Splice Junction	3175	-	60	60	3	10
WIN	Wine	178	13	-	13	3	5
WVF	Waveform	5000	21	-	21	3	10
ZOO	Zoo	101	15	1	16	7	10

Input Features of Diabetes

1. Number of times pregnant
2. Plasma glucose concentration
3. Diastolic blood pressure
4. Triceps skin fold thickness (mm)
5. 2-Hour serum insulin (mu U/ml)
6. Body mass index
7. Diabetes pedigree function
8. Age

❖ Problems show variations in number of examples, input features and classes.



Supportive Tools and Techniques

Textbooks: Hundreds of Books Available

Machine Learning Search!

Batch search for books

Download type: Resumed dl with original filename **Results per page** 25

View results: Simple Detailed **Search with mask (word*):** No Yes

Search in fields The column set default Title Author(s) Series

Publisher Year ISBN Language MD5 Tags

Library Genesis

Current alias domains are libgen.rs, libgen.is, libgen.st. Update your bookmarks!

Get involved to help the project!

A guide to effective catalog searching

Try Libgen Desktop application! Now with IPFS downloads!

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24

4565 files found | showing results from 26 to 50

also search "Machine Learning"

ID	Author(s)	Title	Publisher	Year	Pages	Language	Size	Extension	Mirror
59901	Du Zhang, Du Zhang; Jeffrey J. P. Tsai	Advances in Machine Learning Applications in Software Engineering *[illustrated edition]* *9781591409410, 1-59140-941-1, 159140942X, 9781591409427, 1591409438, 9781591409434*	Idea Group Pub	2007	499	English	12 Mb	pdf	[1] [2
63627	Holmes D. E., Jain L. C. (Ed)	Innovations in Machine Learning: Theory and Applications *0-387-94559-8*		2006	284	English	3 Mb	pdf	[1] [2
70985	Ethem Alpaydin	*Adaptive computation and machine learning* Introduction to machine learning *0262012111, 9780262012119*	MIT Press	2004	432	English	3 Mb	djvu	[1] [2
70998	Sridhar Mahadevan	*Synthesis Lectures on Artificial Intelligence and Machine Learning* Representation discovery using harmonic analysis *[1 ed.]* *1598296590, 9781598296594*	Morgan and Claypool Publishers	2008	160	English	5 Mb	pdf	[1] [2
73067	Kevin Leyton-Brown	*Synthesis Lectures on Artificial Intelligence and Machine Learning* Essentials of game theory *[1 ed.]* *1598295934, 9781598295931*	Morgan and Claypool Publishers	2008	104	English	1 Mb	pdf	[1] [2
78471	Ethem Alpaydin	*Adaptive Computation and Machine Learning* Introduction to Machine Learning *0262012111, 9780262012119*	The MIT Press	2004	432	English	36 Mb	pdf	[1] [2
95396	Michie D.	Machine learning, neural and statistical classification		1994	298	English	2 Mb	pdf	[1] [2
95419	Nilsson N.J.	Introduction to machine learning		1997	209	English	3 Mb	pdf	[1] [2
110667	Pedro Domingos, Daniel Lowd	*Synthesis Lectures on Artificial Intelligence and Machine Learning* Markov logic: an interface layer for artificial intelligence *[1 ed.]* *9781598296921, 1598296922*	Morgan and Claypool Publishers	2009	155	English	1 Mb	pdf	[1] [2
111061	Cyril Goutte, Nicola Cancedda, Marc Dymetman, George Foster	*Neural information processing series* Learning machine translation *0262072971, 9780262072977, 9780262255097*	MIT Press	2009	329	English	5 Mb	pdf	[1] [2
111507	Yaochu Jin	*Studies in Computational Intelligence* Multi-objective machine learning *[1 ed.]*	Springer	2006	656	English	8 Mb	pdf	[1] [2

Supportive Tools and Techniques

Library Genesis: http://libgen.is

Download ▾ Upload ▾ Updates ▾ All journals ▾ Community & support

Library Genesis: Scientific articles

Current alias domains are libgen.rs, libgen.is, libgen.st. Update your bookmarks!
Try Libgen Desktop application!

Decision Support System search

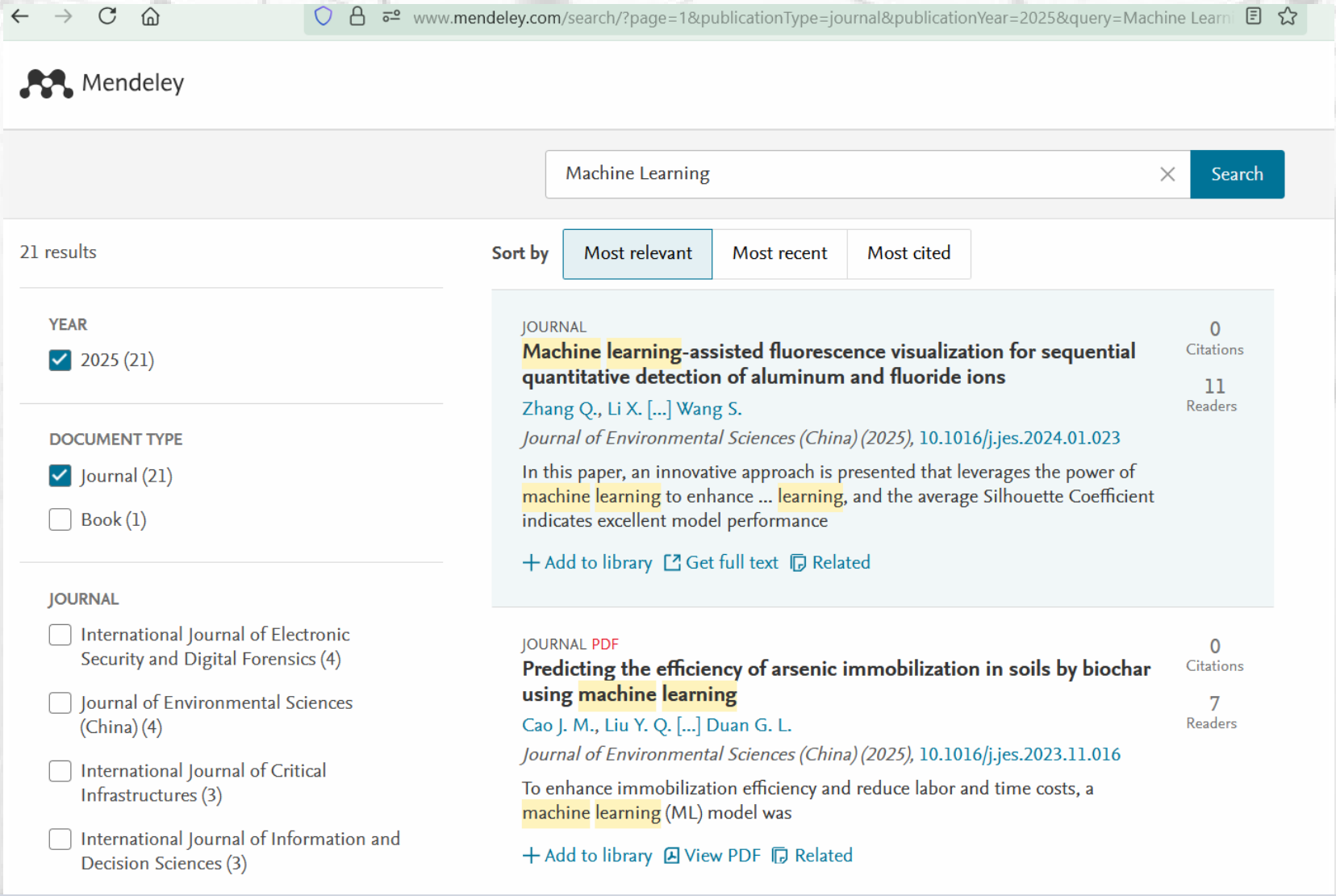
You can search by the article title, title + authors, DOI (Digital Object Identifier). Keep in mind that the text search is currently limited to first 100 results and not sorted by relevance, for better results try with a more precise query.

100 files found page 1 / 4 ▾ ▶

Author(s)	Article	Journal	File	Mirrors
John R. Ottensmann; Michael E. Gleeson	Implementation and testing of a decision support system for public library materials acquisition budgeting DOI: 10.1002/(sici)1097-4571(199303)44:2<83::aid-asi3>3.0.co;2-5	Journal of the American Society for Information Science and Technology volume 44, issue 2 (1993)	1.10 Mb [edit]	[Sci-Hub] [Libgen.rs] [Libgen.lc]
V. Loia; S. Sessa; A. Staiano; R. Tagliaferri	Merging fuzzy logic, neural networks, and genetic computation in the design of a decision-support system DOI: 10.1002/(sici)1098-111x(200007)15:7<575::aid-int1>3.0.co;2-a	International Journal of Intelligent Systems volume 15, issue 7 (2000)	160 Kb [edit]	[Sci-Hub] [Libgen.rs] [Libgen.lc]
Mie Nakatani; Takayuki Yamaoka; Yoshihisa Hosono; Shogo Nishida	Proposal of decision support system for cooperation in hierarchical structures DOI: 10.1002/(sici)1520-6440(199702)80:2<78::aid-ecjc7>3.0.co;2-2	Electronics and Communications in Japan (Part III: Fundamental Electronic Science)) volume 80, issue 2 (1997)	1.52 Mb [edit]	[Sci-Hub] [Libgen.rs] [Libgen.lc]
Per J. Agrell; Ralph E. Steuer	ACADEA—a decision support system for faculty performance reviews DOI: 10.1002/1099-1360(200009)9:5<191::aid-mcda276>3.0.co;2-5	Journal of Multi-Criteria Decision Analysis volume 9, issue 5 (2000)	149 Kb [edit]	[Sci-Hub] [Libgen.rs] [Libgen.lc]
Stephen T. Roehling; James	System dynamics modeling applied to software outsourcing decision	Software Process: Improvement	363 Kb	[Sci-Hub]

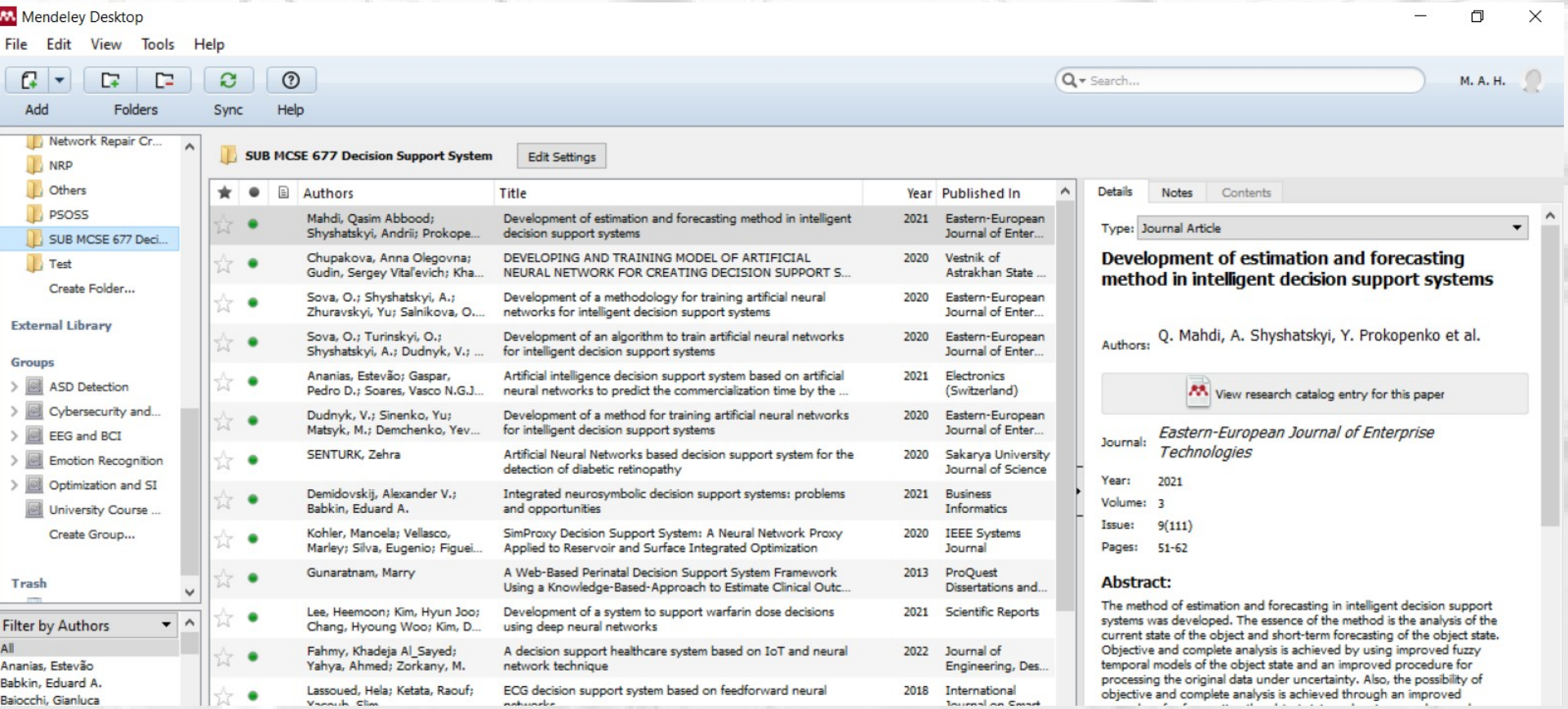
Supportive Tools and Techniques

Mendeley: https://www.mendeley.com



Supportive Tools and Techniques

Mendeley: Desktop Edition



An Application of Feed Forward Neural Network
Optical Character Recognition(OCR)

Optical Character Recognition(OCR)

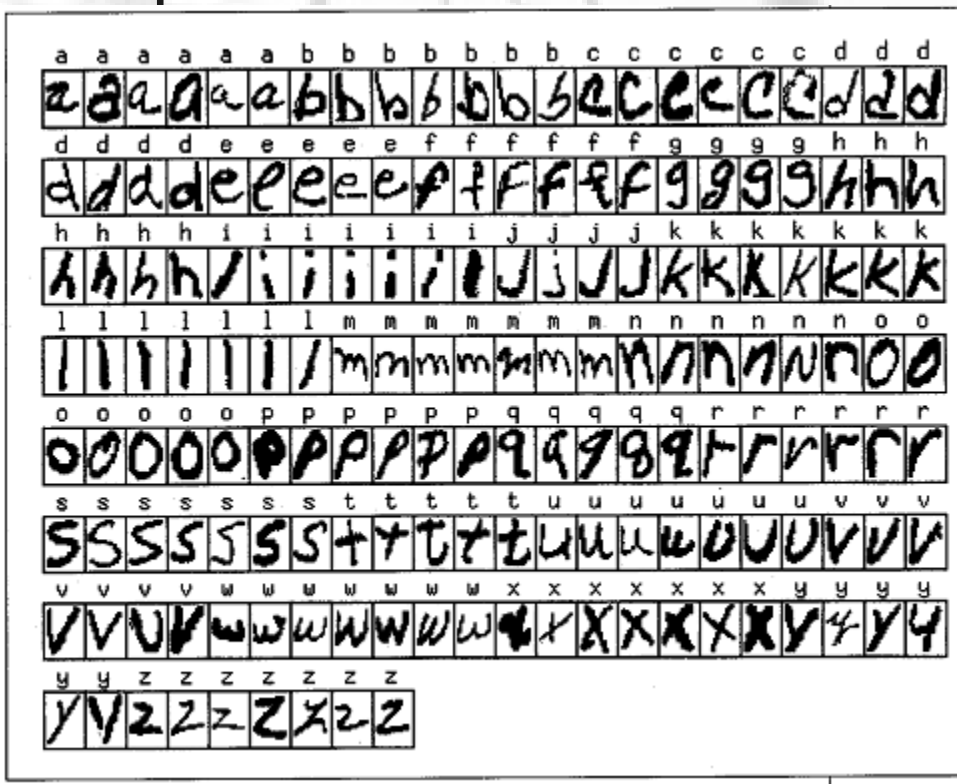


Figure 10. A sample set of characters in the NIST database.

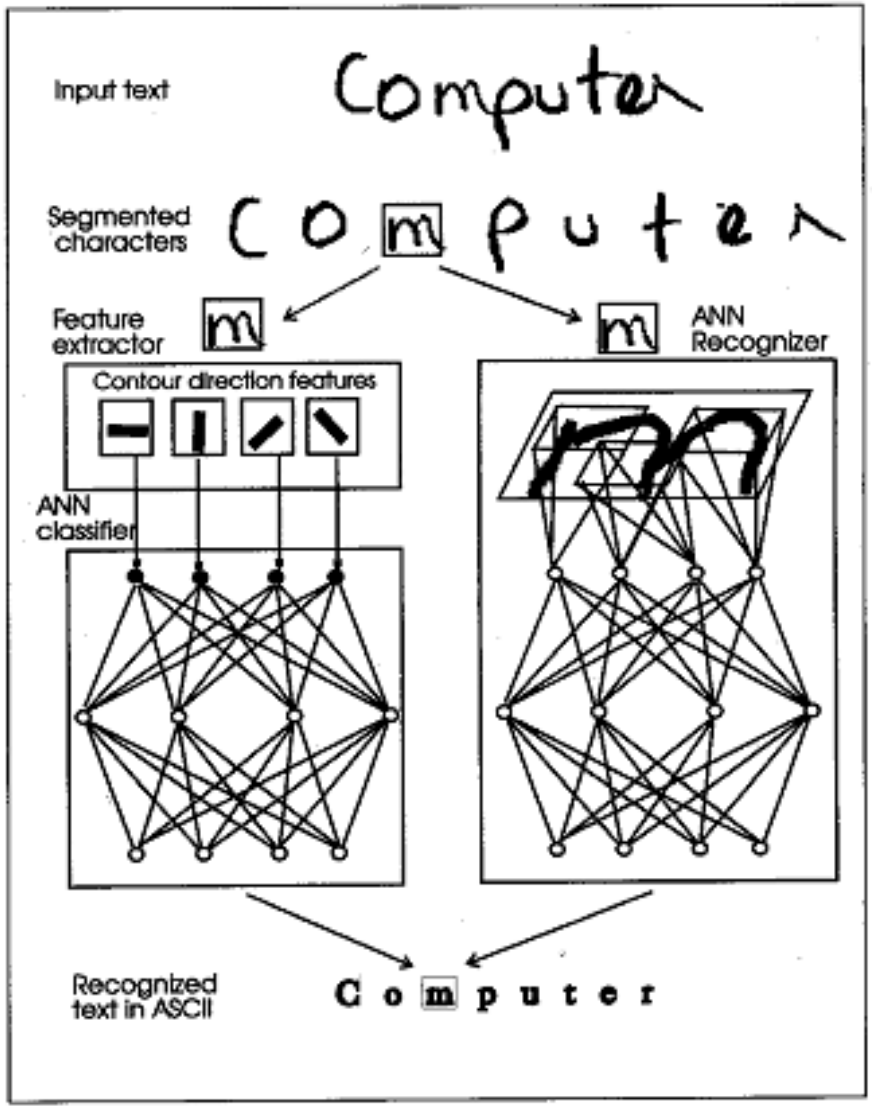


Figure 11. Two schemes for using ANNs in an OCR system.

Evaluation Methods for ML Classification Models

Metrics, validation protocols, ANN learning curves, and statistical comparisons with specific numerical examples

Classification example: Disease vs Healthy

Prepared as a lecture slide deck

What does model evaluation answer?

A classification model must be judged from several angles, not one score.

Example model: ANN outputs P(disease | patient) = 0.72.
Evaluation asks whether this score is correct, useful, calibrated, stable across data splits, and deployable.

Start with the confusion matrix

Specific example: 100 patients, 40 diseased and 60 healthy.

Predicted → / Actual ↓	Disease	Healthy
Disease	TP = 32 correct disease	FN = 8 missed disease
Healthy	FP = 6 false alarm	TN = 54 correct healthy

TP + TN = correct predictions
FP + FN = errors

100	40	60
Total	Actual positive	Actual negative

All common classification metrics are different summaries of this matrix.
For multiclass tasks, the matrix has one row and one column per class.

Accuracy is useful, but often insufficient

Predicted → / Actual ↓	Disease	Healthy
Disease	TP = 32 correct disease	FN = 8 missed disease
Healthy	FP = 6 false alarm	TN = 54 correct healthy

Specific example: 100 patients, 40 diseased and 60 healthy.

$$\text{Accuracy} = (TP + TN) / N = (32 + 54) / 100 = 86\%$$

Accuracy answers: How many cases are correct overall?
It is acceptable when classes are balanced and error types have similar cost.

Imbalance trap

1000 samples: 50 positive, 950 negative

Predict every sample as negative

95%
Accuracy

0%
Recall

Predicted → / Actual ↓	Disease	Healthy
Disease	TP = 32 correct disease	FN = 8 missed disease
Healthy	FP = 6 false alarm	TN = 54 correct healthy

Precision, recall, and specificity describe different mistakes

Same example: TP=32, FN=8, FP=6, TN=54.

Precision / PPV	TP / (TP + FP)	**32 / 38 = 84.2%**	Among predicted disease cases, 84.2% are truly diseased.
Recall / Sensitivity	TP / (TP + FN)	**32 / 40 = 80.0%**	The model detects 80.0% of diseased patients.
Specificity	TN / (TN + FP)	**54 / 60 = 90.0%**	The model correctly rejects 90.0% of healthy patients.
NPV	TN / (TN + FN)	**54 / 62 = 87.1%**	Among predicted healthy cases, 87.1% are truly healthy.

Screening studies usually emphasize sensitivity; confirmatory tests often emphasize precision and specificity.

F1, balanced accuracy, and MCC summarize performance under imbalance

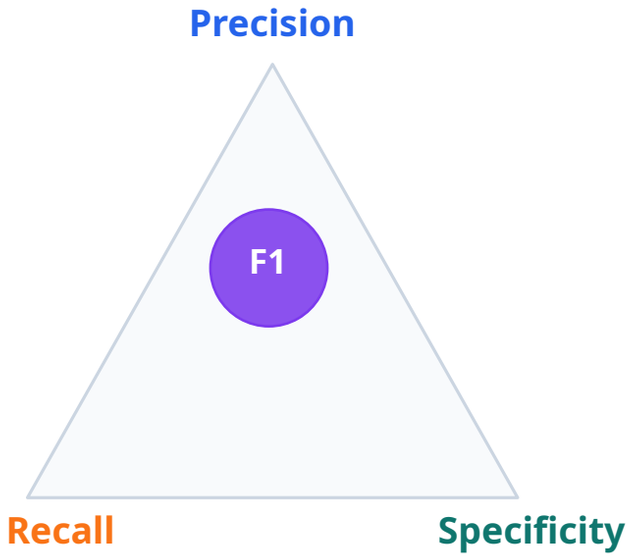
Use composite metrics when accuracy hides class-wise behavior.

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$
$$= 2 \times 0.842 \times 0.800 / (0.842 + 0.800) = 82.1\%$$

$$Balanced\ Accuracy = (Sensitivity + Specificity) / 2$$
$$= (80.0\% + 90.0\%) / 2 = 85.0\%$$

$$MCC \approx 0.70$$

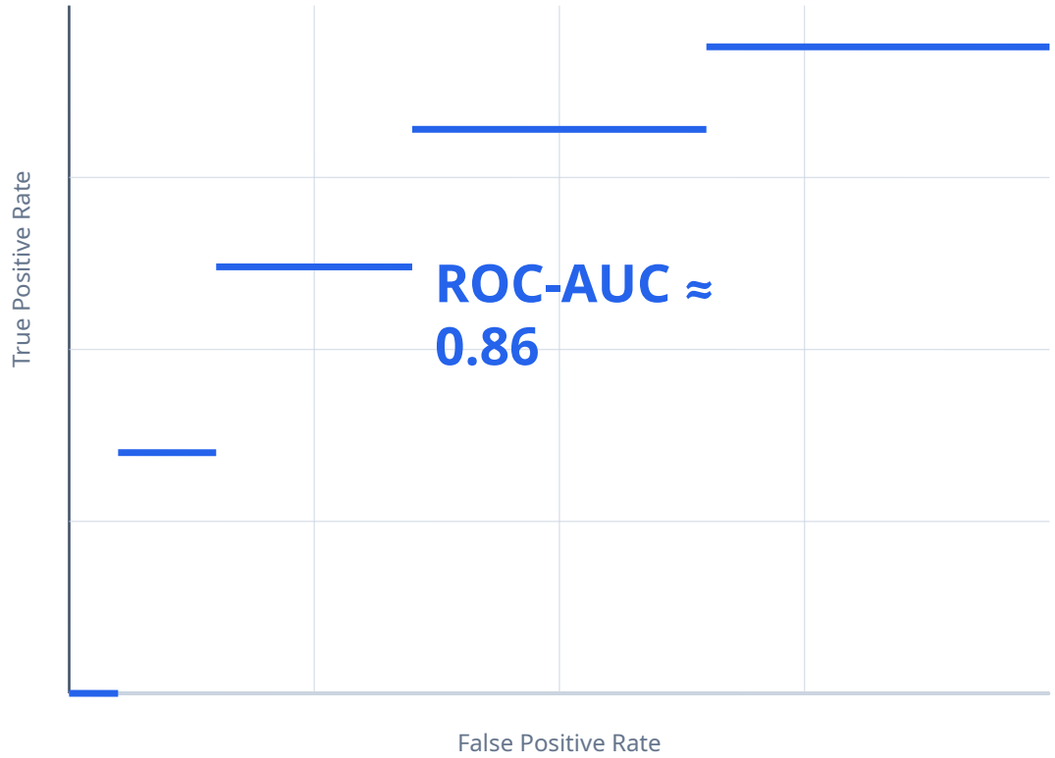
Range: −1 to +1; robust under imbalance



Practical choice:
Report F1 when precision and recall both matter.
Report balanced accuracy when sensitivity and specificity should be equally weighted.
Report MCC for a single robust score under imbalance.

ROC curve evaluates ranking across decision thresholds

ANNs usually output probabilities; the threshold controls FP/FN trade-off.



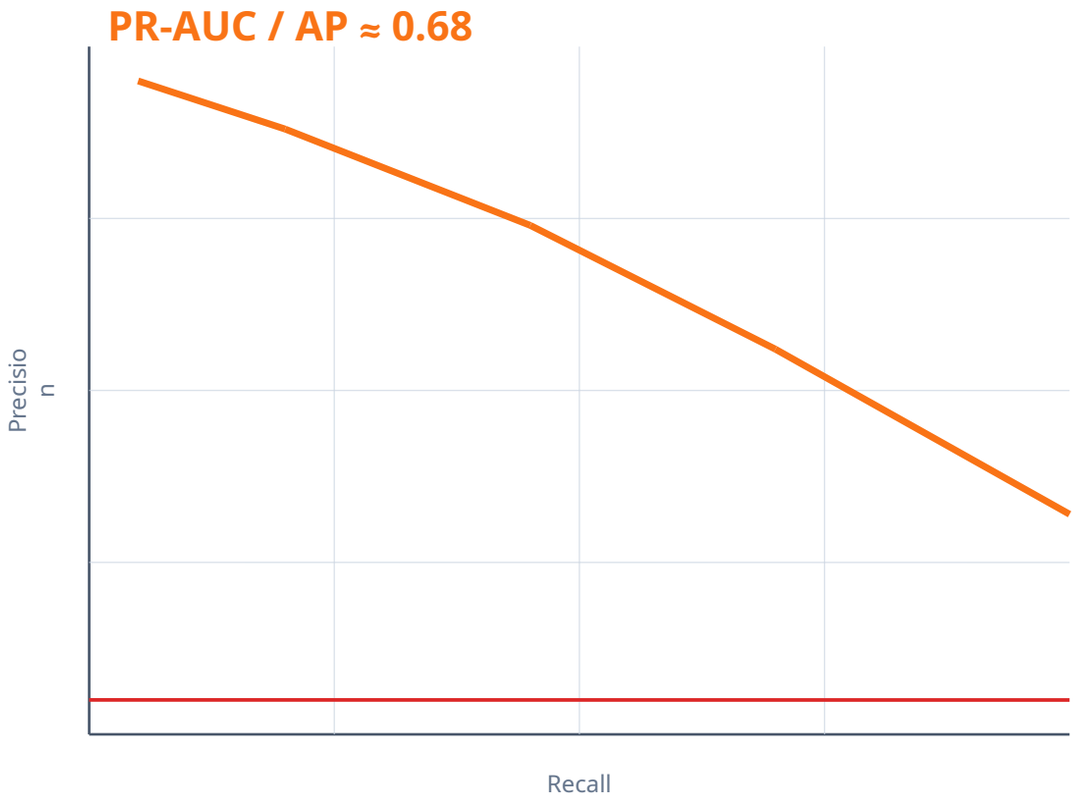
Example threshold choice

Threshold	Precision	Recall	Use case
0.30	68%	92%	Screening
0.50	84%	80%	Balanced
0.75	93%	54%	Confirmatory

AUC measures discrimination: whether positive cases receive higher scores than negative cases. AUC does not select the operating threshold by itself.

Precision–Recall curve is often better for rare positives

Example: only 5% positives; random PR baseline is 0.05.



Why ROC may look optimistic under imbalance

Dataset: 1000 samples; 50 positive, 950 negative.
A few false positives may barely change FPR because TN is large.
Precision directly penalizes false alarms among positive predictions.

Use PR-AUC when the positive class is rare and clinically important.

Probability quality: log loss, Brier score, calibration

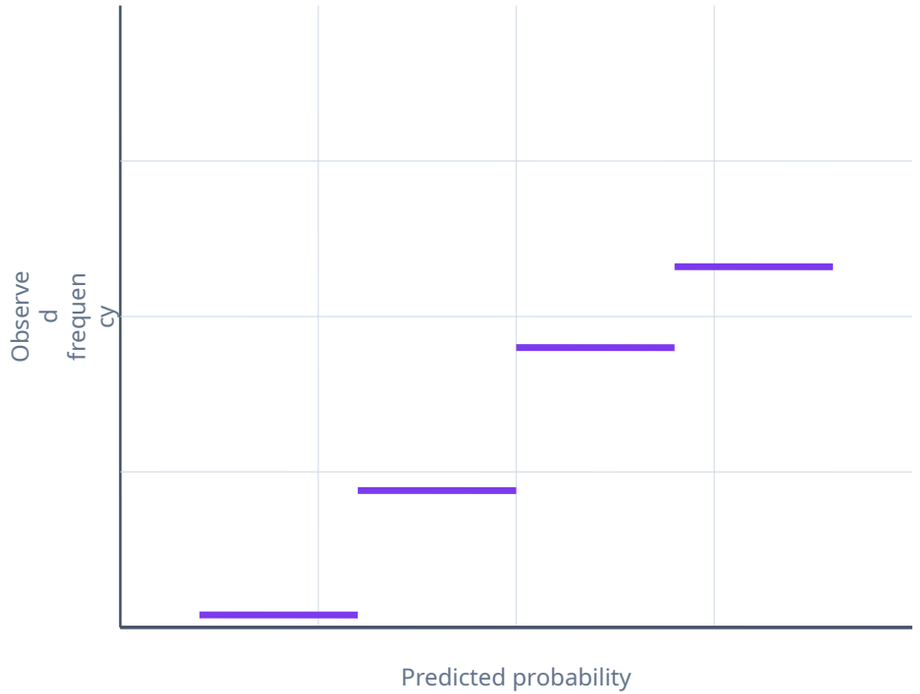
A model can classify well but still output unreliable probabilities.

For one positive case (y = 1):

p = 0.90	Loss = 0.105
p = 0.55	Loss = 0.598
p = 0.10	Loss = 2.303

Brier score = average $(p - y)^2$
Lower is better

Calibration curve



Below diagonal = overconfident probabilities

Multiclass classification: report per-class and averaged metrics

Example: sleep-stage classification with Wake, N1, and N2.

Actual ↓ / Predicted →	Wake	N1	N2
Wake	450	30	20
N1	45	40	15
N2	20	35	345

Averaging choices

Macro-F1 — **(0.91 + 0.62 + 0.84) / 3 = 0.79**

Each class gets equal weight.

Weighted-F1 — **≈ 0.84**

Classes weighted by support; majority classes dominate.

Micro-F1 — **≈ Accuracy for single-label multiclass**

Each sample gets equal weight.

Recommendation: always include per-class recall/F1, especially when minority classes matter.

Validation method determines whether the reported score is credible

Metrics are only meaningful if the split protocol is sound.

Train
70%
fit ANN weights

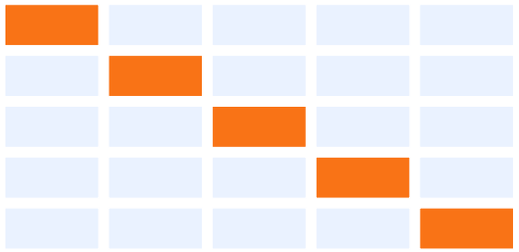
→

Validation
15%
tune hyperparameters

→

Test
15%
final unbiased score

Stratified 5-fold cross-validation



Orange block = test fold once; blue blocks = training folds.

Common protocols

- ✓ Hold-out: simple but split-dependent
- ✓ Stratified k-fold: stable class proportions
- ✓ Nested CV: tuning inside, evaluation outside
- ✓ External test: strongest generalization evidence

For medical, EEG, and subject data: split by subject, not by segment

Otherwise, the ANN can memorize patient-specific patterns.

Incorrect segment-wise split

Same subject appears in train and test

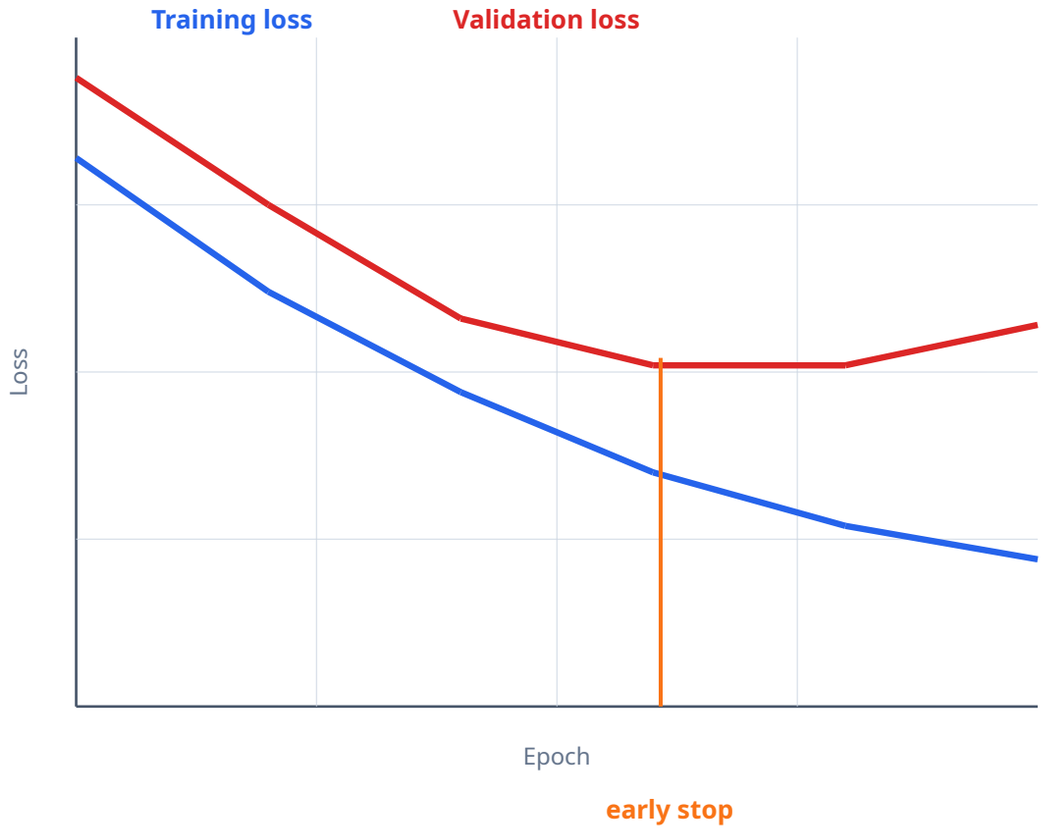
Correct subject-wise split

Test subjects are unseen during training

Use GroupKFold, leave-one-subject-out CV, or patient-level hold-out.
Example: EEG windows from patient 12 must all remain in either train, validation, or test—not split across them.

For ANN models, inspect learning curves

Training behavior explains underfitting, overfitting, and early-stopping decisions.



Interpretation

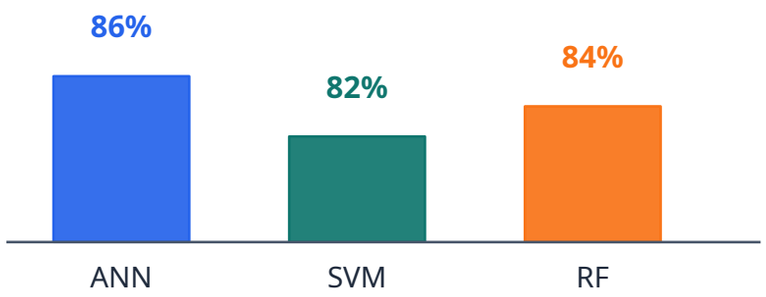
- ✓ Both losses high → underfitting
- ✓ Training improves, validation worsens → overfitting
- ✓ Validation minimum suggests early stopping point
- ✓ Repeat runs: random initialization affects ANN results

0.84	0.87	0.82
Run 1	Run 2	Run 3

Report mean ± SD: F1 = 0.84 ± 0.03

Compare models statistically, not only by best accuracy

Use paired evidence because classifiers are evaluated on the same test cases or folds.



Mean F1 across folds

Which test to use?

Confidence interval	Uncertainty around one metric
McNemar test	Two classifiers on same test set
DeLong test	Compare ROC-AUC values
Wilcoxon / paired t-test	Scores over matched folds or runs
Friedman + Nemenyi	More than two classifiers over datasets

Example report: ANN F1 = 0.86, 95% CI [0.81, 0.90]; McNemar p = 0.03 vs SVM

Robustness and computational evaluation matter for real deployment

A deployable classifier must remain useful under operational constraints.

Examples:
EEG: remove one channel and re-test.
Images: add acquisition noise and re-test.
Embedded model: report parameters, model size, inference time, memory use.

Minimum defensible evaluation package

For a classification paper, include metrics, protocol, uncertainty, and model behavior.

Metrics

- ✓ Confusion matrix
- ✓ Accuracy + balanced accuracy
- ✓ Precision, recall, specificity
- ✓ F1, MCC, ROC-AUC, PR-AUC
- ✓ Per-class metrics for multiclass

Methodology

- ✓ Stratified or subject-wise validation
- ✓ Independent test set if available
- ✓ Training/validation curves for ANN
- ✓ Mean ± SD and 95% CI
- ✓ Statistical comparison with baselines

Example final statement:
“The ANN achieved F1 = 0.84 ± 0.03 and ROC-AUC = 0.89 under stratified 5-fold CV.
On the independent test set, sensitivity = 0.80 and specificity = 0.90.”

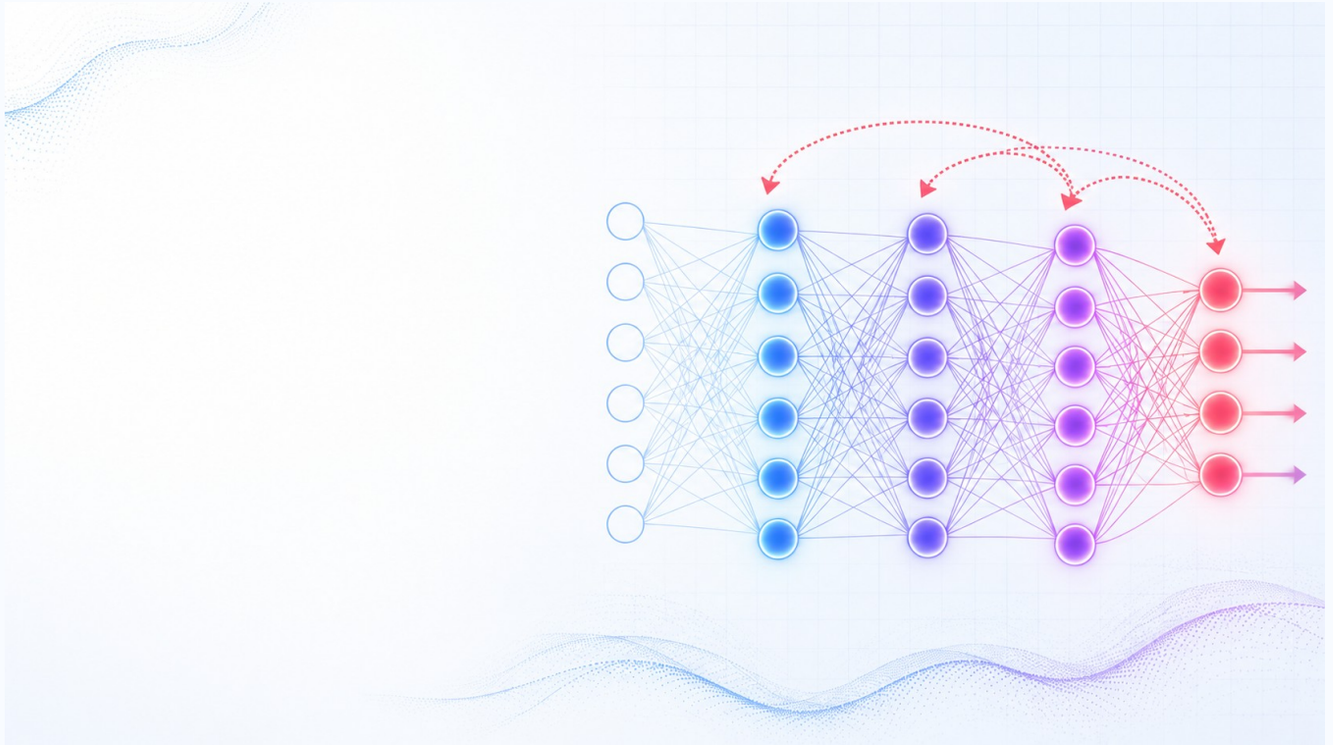
Choose metrics according to task risk: false negatives, false positives, imbalance, probability use, and deployment constraints.

Backpropagation Learning

Convergence • Training modes • Learning rate & momentum • Bias–variance trade-off

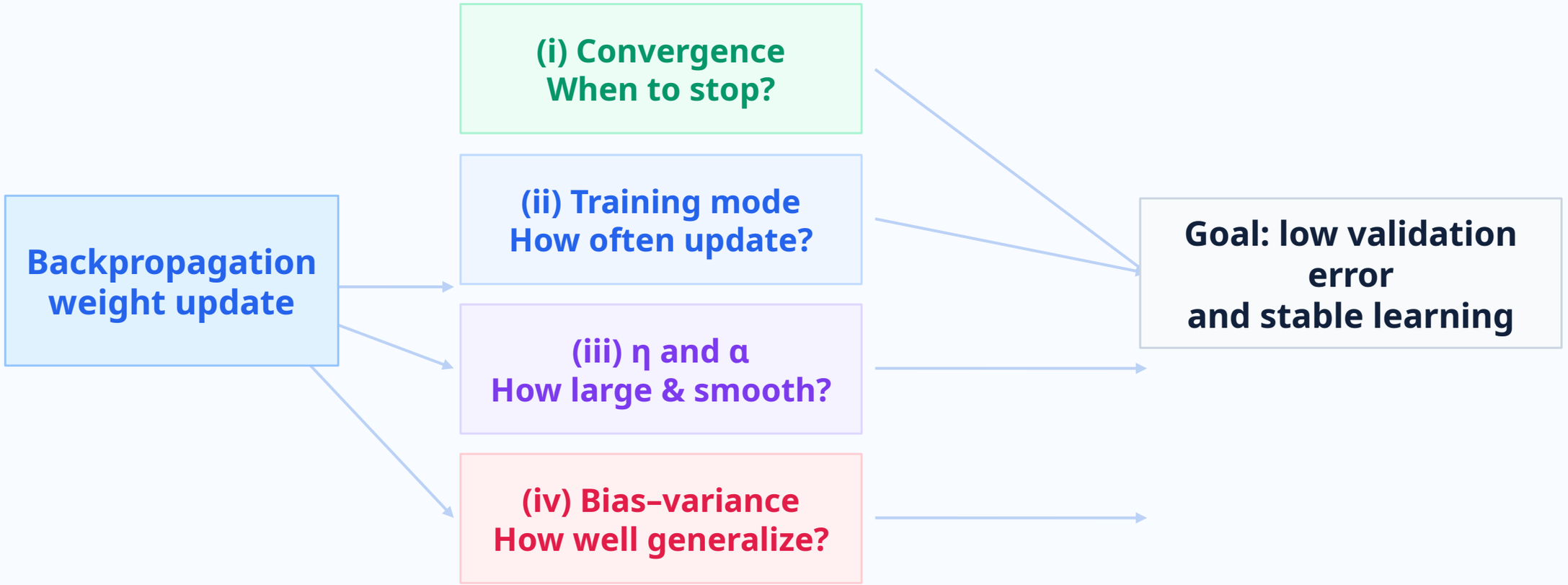
$$w(t+1) = w(t) + \Delta w(t)$$

Lecture slide set with examples



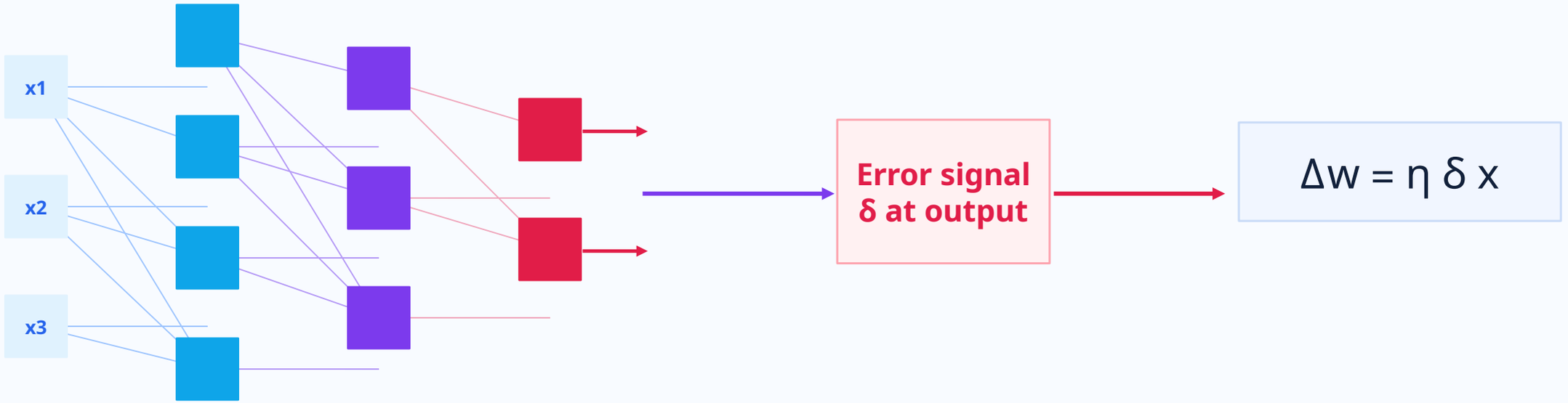
Concept map: what controls BP learning

The four issues interact: update frequency, step size, inertia, and generalization.



BP learning cycle and notation

Backpropagation computes error signals and converts them into weight corrections.



- x: input arriving at a weight
- δ: local error signal of the receiving neuron
- η: learning rate controlling update size
- Δw: correction added to current weight

(i) Convergence: meaning

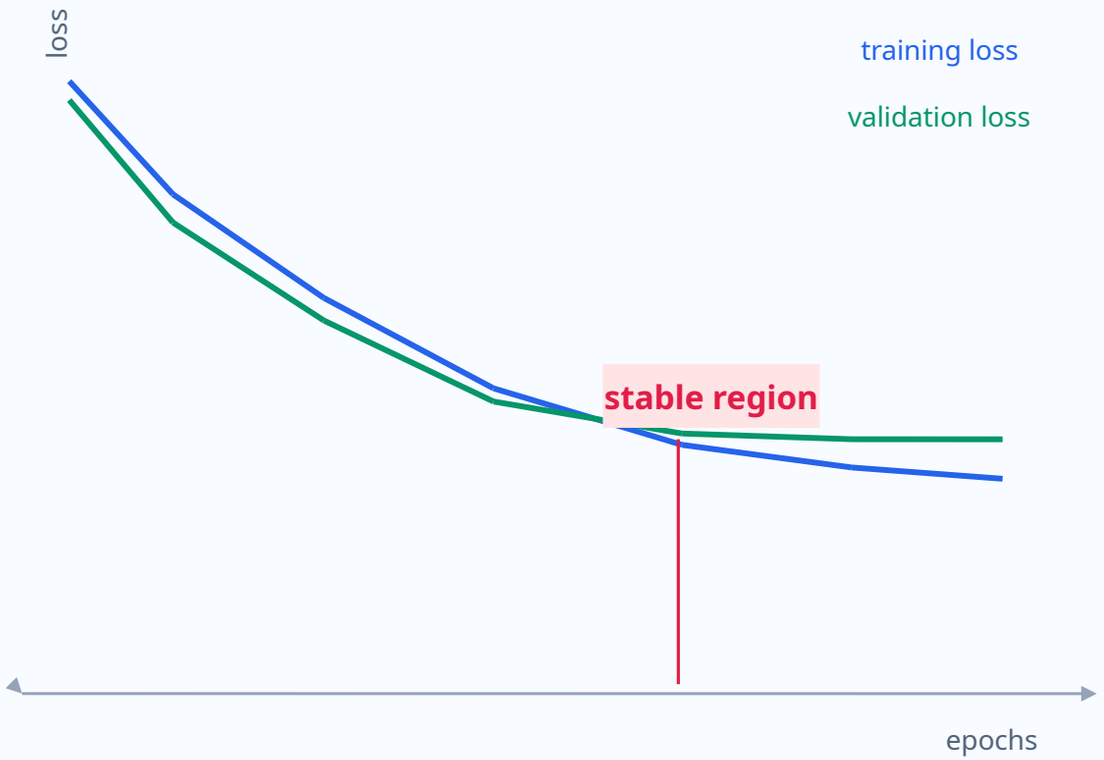
Convergence means the optimization process has reached a stable region.

$$w(t+1) = w(t) + \Delta w(t)$$

$$|E(t) - E(t-1)| < \varepsilon E$$

$$||\Delta w(t)|| < \varepsilon w$$

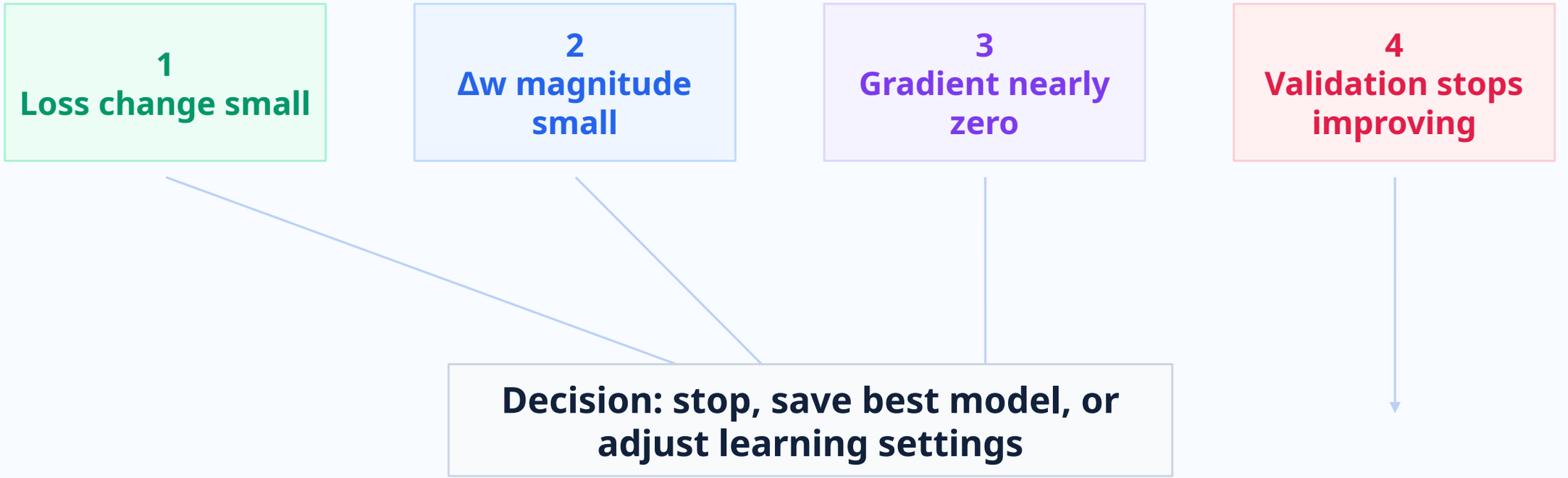
- The error no longer decreases meaningfully.
- Weight corrections become very small.
- A small gradient indicates a stationary region, not necessarily the global optimum.
- Good convergence must also be checked on validation data.



Example: loss improves only 0.00001 for many epochs → training is practically stable.

How convergence is identified

Use several stopping signals together rather than one number alone.



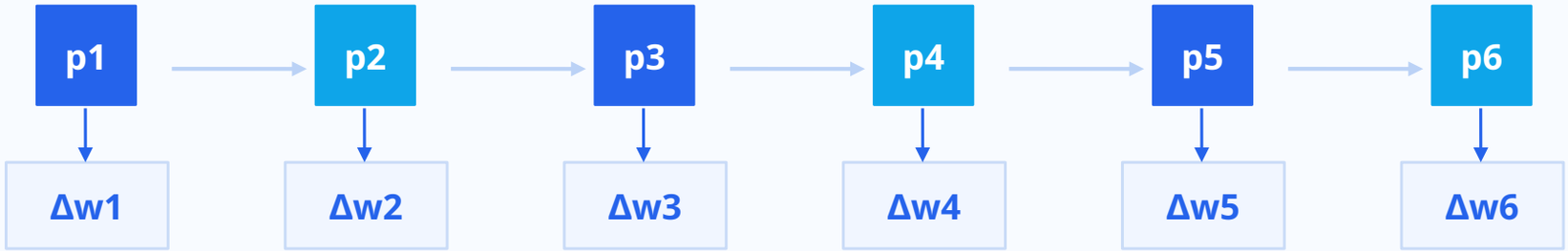
- Proper convergence: training and validation losses are both stable at low values.
- Overfitting: training loss decreases but validation loss increases.
- Non-convergence: losses oscillate or diverge.

(ii) Sequential mode training

Sequential, online, or stochastic BP updates weights after every training sample.

For sample p: $\Delta w(p) = \eta\, \delta(p)\, x(p)$

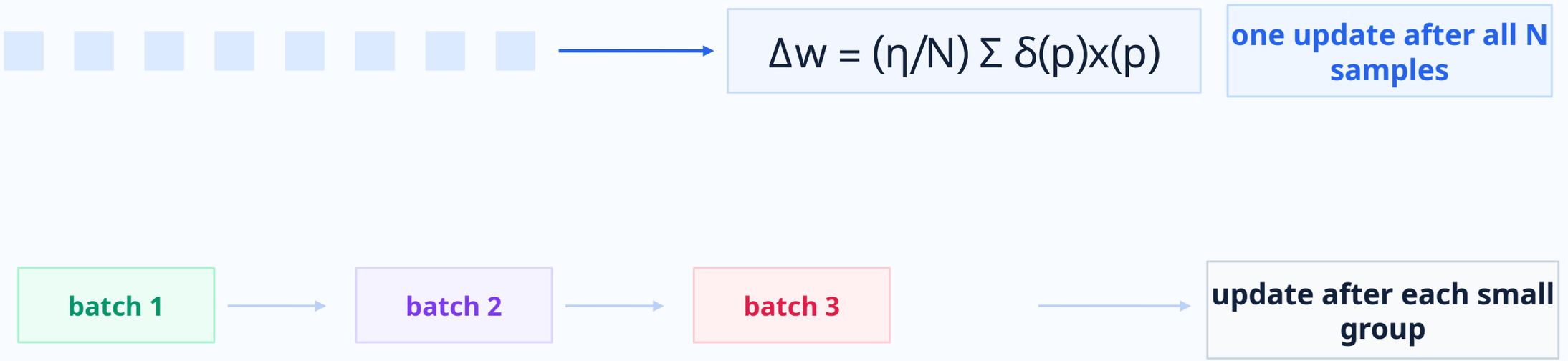
Example: 1,000 samples → 1,000 updates per epoch



- Fast feedback and low memory requirement.
- Noisy update trajectory; sample order matters.
- Noise may help escape shallow local minima or saddle regions.
- Useful for large or streaming datasets.

Batch mode and mini-batch training

Batch training accumulates corrections; mini-batch training is the practical compromise.



- Batch: stable direction, but memory and update latency are high.
- Mini-batch: less noisy than sequential and more frequent than full batch.
- Modern ANN training usually uses mini-batches for vectorized computation.

Sequential vs batch mode: worked comparison

Same dataset, different update frequency and optimization behavior.

Aspect	Sequential mode	Batch mode
Update point	After each sample	After full training set
Updates per epoch	N	1
Update path	Noisy / fluctuating	Smooth / deterministic
Memory	Low	Higher
Best suited for	Online or very large data	Small/medium data; parallel matrix computation
Example for N = 1,000	1,000 updates	1 update

high update frequency

stable global correction

(iii) Effect of learning rate η

η controls how strongly BP responds to the current error signal.

$$\Delta w = \eta \, \delta \, x$$

- Very small η: stable but slow convergence.
- Suitable η: fast and controlled loss reduction.
- Very large η: oscillation, overshooting, or divergence.

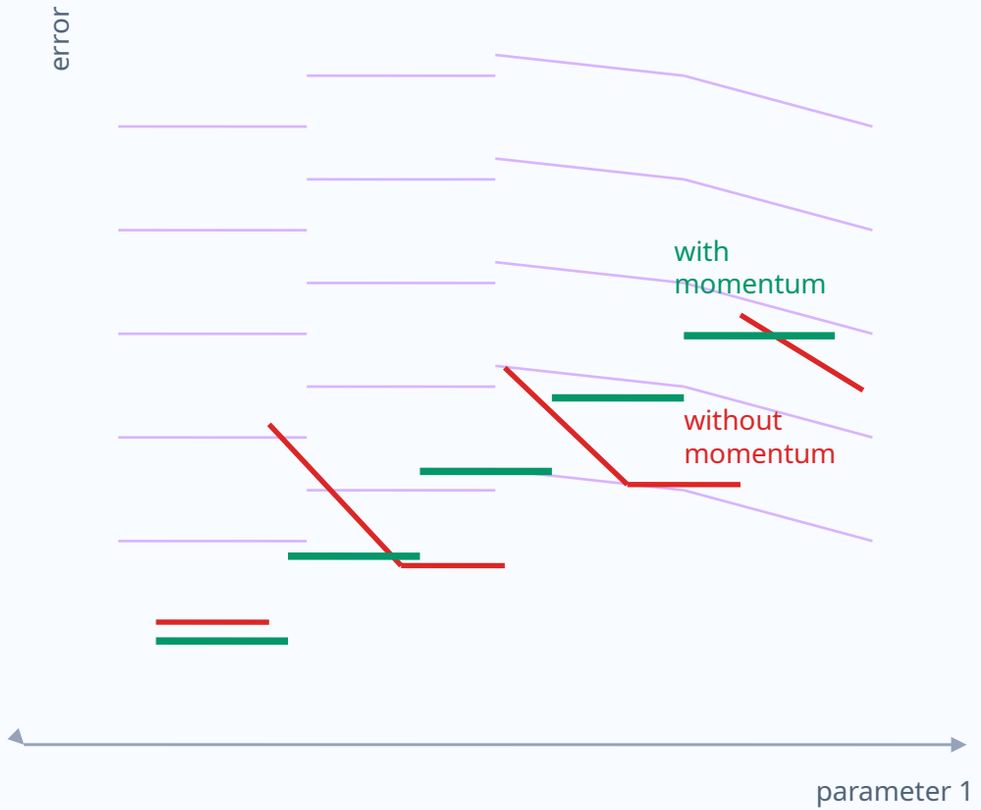
Practical diagnosis: if loss explodes or oscillates strongly, reduce η; if loss is flat and gradients are nonzero, increase η or normalize inputs.

Momentum α: adding inertia to BP

Momentum retains part of the previous correction to smooth the trajectory.

$$\Delta w(t) = \eta \delta(t) x(t) + \alpha \Delta w(t-1)$$

- Reinforces repeated movement in the same direction.
- Damps zigzag motion across steep error valleys.
- Can cross shallow local minima or flat regions.
- Too much α may overshoot and destabilize training.



Learning rate and momentum must be selected jointly

A good setting balances speed, stability, and generalization.

η	α	Likely behavior
Low	Low	Stable, but very slow
Low	Moderate/ high	Smoother and faster
Moderat e	Moderate	Often effective
High	Low	Oscillation or divergence
High	High	Overshooting and instability

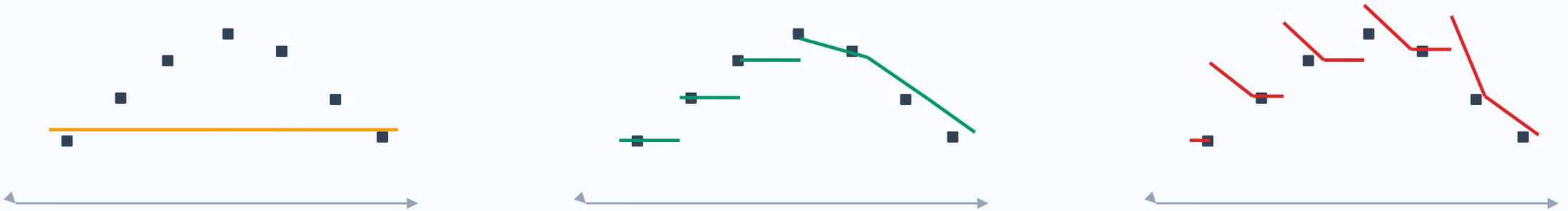
Example: if loss shows repeated up-and-down jumps, reduce η first. Then use moderate α to smooth the path.

Rule of thumb: η reacts to current error; α remembers the previous direction.

(iv) Bias–variance trade-off

Generalization error reflects model simplicity, model sensitivity, and irreducible noise.

$$\text{Expected error} \approx \text{Bias}^2 + \text{Variance} + \text{Noise}$$



underfitting: high bias

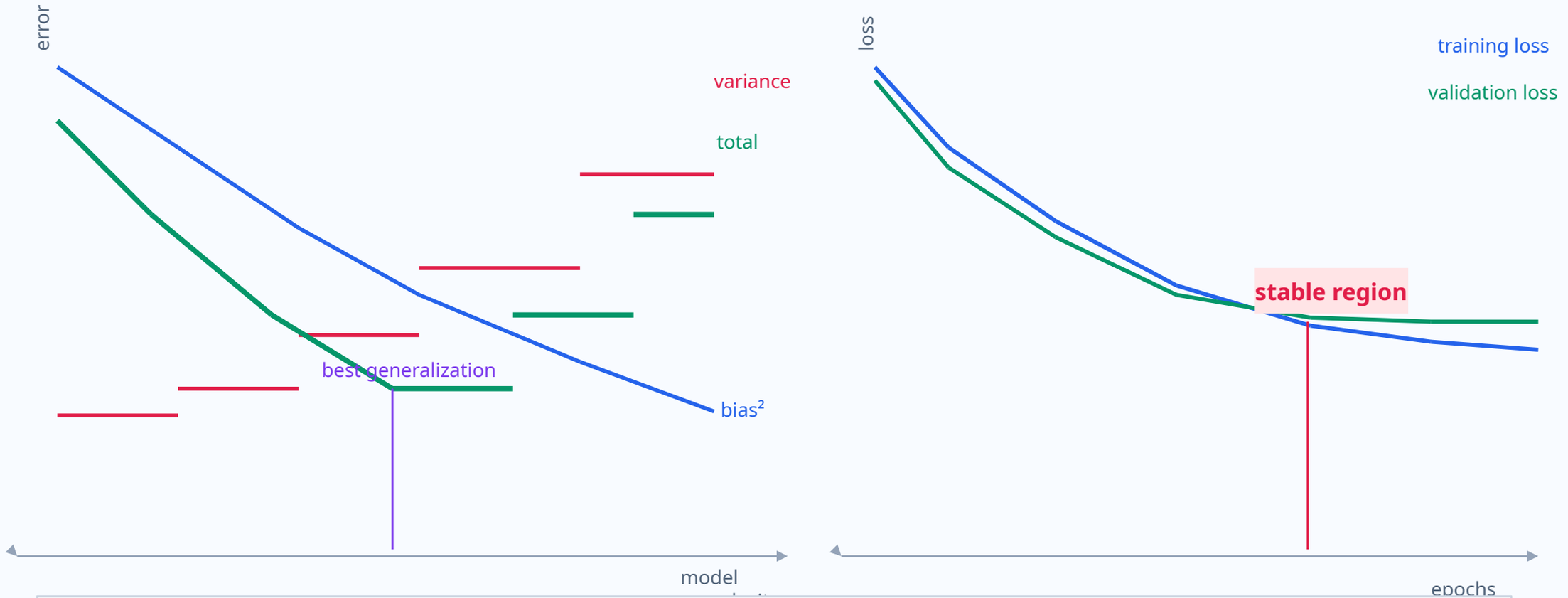
good fit: balance

overfitting: high variance

- High bias: model too simple; training and validation errors both high.
- High variance: model too complex; low training error but poor validation error.

Bias–variance through BP training

Training too little causes underfitting; training too long can cause overfitting.



Early stopping uses validation loss to keep the model near the best generalization point.

Practical checklist for BP learning

Use curves and examples to distinguish optimization problems from generalization problems.

Loss high training + validation	→ High bias underfitting	→ Increase capacity or train better
Training low validation high	→ High variance overfitting	→ Regularize, add data, or early stop
Loss oscillates or diverges	→ Optimization instability	→ Reduce η, normalize, check gradients

Final message: convergence must be stable and useful on unseen data.

For Back Propagation (BP) learning discuss (i) Sequential and Batch Modes Trainings, (ii) Effect of Learning rate and moment and (iii) bias variance tradeoff

Backpropagation Learning

Backpropagation (BP) trains a multilayer neural network by calculating the output error, propagating the error backward through the network, and adjusting the weights and biases to reduce the error.

A typical weight-update rule is

$$w_{ji}(t+1) = w_{ji}(t) + \Delta w_{ji}(t)$$

where the weight correction is

$$\Delta w_{ji}(t) = \eta \delta_j x_i$$

Here:

- η = learning rate
- δ_j = error signal of neuron j
- x_i = input from neuron i
- Δw_{ji} = change applied to the weight

(i) Sequential and Batch Modes of Training

1. Sequential Mode Training

Sequential training is also called **online**, **incremental**, or **stochastic** training. The network weights are updated after presenting each training sample.

For a training sample p,

$$\Delta w_{ji}^{(p)} = \eta \delta_j^{(p)} x_i^{(p)}$$

The weight is immediately updated as

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}^{(p)}$$

Procedure

For each training epoch:

1. Present one training sample.
2. Perform forward propagation.
3. Calculate the output error.
4. Backpropagate the error.
5. Update the weights immediately.
6. Repeat for the remaining samples.

Characteristics

- One weight update is performed for every sample.
- For N training samples, N updates occur in one epoch.
- The order of training samples affects the learning process.
- Samples are generally shuffled before each epoch.
- The error curve may fluctuate because each sample produces a different weight correction.

Advantages

- Begins learning immediately.
- Requires less memory.
- Suitable for large datasets and streaming data.
- The noisy weight updates may help the network escape shallow local minima, saddle points, or flat regions.
- Can adapt to changes in the data distribution.

Disadvantages

- Weight updates are noisy and unstable.
- The loss may oscillate around the minimum.
- Training results may depend on the order of samples.
- It is less efficient for parallel computation.

Example

Suppose there are 1,000 training samples. In sequential mode, the network weights are updated **1,000 times per epoch**.

---

2. Batch Mode Training

In batch training, all training samples are processed before the weights are updated. The individual weight corrections are accumulated over the complete training set.

For N training samples,

$$\Delta w_{ji} = \eta \sum_{p=1}^{N} \delta_j^{(p)} x_i^{(p)}$$

Frequently, the average correction is used:

$$\Delta w_{ji} = \frac{\eta}{N} \sum_{p=1}^{N} \delta_j^{(p)} x_i^{(p)}$$

The weights are updated only once at the end of the epoch:

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Procedure

For each epoch:

1. Present all training samples.
2. Calculate the error for every sample.
3. Accumulate or average the weight corrections.
4. Update the weights once.
5. Begin the next epoch.

Characteristics

- One weight update occurs per epoch.
- The weight correction represents the entire training dataset.
- The training trajectory is smoother and more deterministic.
- It requires storage and processing of the complete batch.

Advantages

- Produces stable and smooth weight updates.
- The update direction represents the overall dataset.
- Efficient for matrix operations and parallel hardware.
- Easier to analyze mathematically.
- Results do not depend strongly on the ordering of samples.

Disadvantages

- Requires more memory.
- Each update may take considerable time for a large dataset.
- Cannot naturally handle continuously arriving data.
- Stable updates may become trapped in local minima or flat regions.
- One abnormal sample can influence the accumulated update.

Example

For 1,000 training samples, batch mode performs only **one weight update per epoch**.

Sequential versus Batch Training

Property	Sequential mode	Batch mode
Weight update	After each sample	After all samples
Updates per epoch	N	1
Update stability	Noisy	Smooth
Memory requirement	Low	High
Convergence path	Fluctuating	Stable
Sample-order effect	Significant	Minimal
Online learning	Suitable	Not suitable
Parallelization	Less efficient	Highly efficient
Escape from local regions	More likely	Less likely
Computation per update	Low	High

Mini-batch training

Modern neural networks generally use **mini-batch training**, which is a compromise between sequential and batch modes.

For a mini-batch containing B samples,

$$\Delta w_{ji} = \frac{\eta}{B} \sum_{p=1}^{B} \delta_j^{(p)} x_i^{(p)}$$

The network weights are updated after every mini-batch.

Mini-batch training provides:

- less noise than sequential training;

- more frequent updates than full-batch training;
- efficient vectorized and GPU computation;
- moderate memory requirements.

(ii) Effect of Learning Rate and Momentum

1. Effect of Learning Rate

The learning rate η determines the magnitude of the weight correction:

$$\Delta w_{ji} = \eta \delta_j x_i$$

It therefore controls how quickly the network moves toward a minimum of the error function.

Small learning rate

When η is very small:

- weight changes are small;
- learning is stable;
- the error usually decreases smoothly;
- training requires many epochs;
- the network may progress very slowly through flat regions;
- it may appear to have stopped learning.

Thus, an excessively small learning rate produces **slow convergence**.

Large learning rate

When η is large:

- weight changes are large;
- initial learning may be fast;
- the network may overshoot the minimum;
- the error may oscillate;
- training may become unstable;
- the error may diverge instead of decreasing.

Thus, an excessively large learning rate can prevent convergence.

Appropriate learning rate

A suitable learning rate provides:

- sufficiently rapid error reduction;

- controlled weight changes;
- limited oscillation;
- stable convergence to a useful solution.

The appropriate value depends on:

- the dataset;
- input normalization;
- network architecture;
- activation functions;
- batch size;
- weight initialization;
- optimizer;
- curvature of the error surface.

Geometric interpretation

In a steep direction of the error surface, a large learning rate may cause repeated movement from one side of a valley to the other. In a shallow direction, a small learning rate may make progress extremely slow. Consequently, a single fixed learning rate may not be ideal for every direction or training stage.

---

2. Effect of Momentum

Momentum uses a fraction of the previous weight correction when calculating the current correction:

$$\Delta w_{ji}(t) = \eta \delta_j(t) x_i(t) + \alpha \Delta w_{ji}(t-1)$$

where:

- α = momentum coefficient;
- $\Delta w_{ji}(t-1)$ = previous weight correction;
- $0 \leq \alpha < 1$.

The weight is then updated by

$$w_{ji}(t+1) = w_{ji}(t) + \Delta w_{ji}(t)$$

Interpretation

The first term,

$$\eta \delta_j(t) x_i(t),$$

represents the correction calculated from the current error.

The second term,

$$\alpha \Delta w_{ji}(t-1),$$

maintains part of the previous direction of movement. Momentum therefore gives the weight update a form of **inertia**.

Benefits of momentum

1. Accelerates movement in consistent directions

When consecutive weight corrections have the same direction, momentum reinforces them. This increases the effective update magnitude and accelerates movement along a long, shallow error valley.

2. Reduces oscillation

When consecutive corrections alternate in direction, the momentum terms partially cancel the oscillation. This is particularly useful in steep directions of the error surface.

3. Helps cross small local minima

The accumulated movement may carry the network across:

- shallow local minima;
- small irregularities;
- saddle-point regions;
- flat regions of the error surface.

4. Reduces sensitivity to individual samples

In sequential or mini-batch training, individual samples can produce noisy corrections. Momentum smooths these corrections by incorporating information from previous updates.

Effect of the momentum coefficient

Small momentum coefficient

For a small α:

- previous updates have little influence;
- training behaves similarly to ordinary BP;
- limited acceleration and smoothing occur.

Large momentum coefficient

For a large α:

- previous updates strongly influence the current update;
- convergence may become faster;
- oscillation may be reduced;
- the network may overshoot the minimum;
- excessive momentum may cause instability.

Typical momentum values are often between 0.8 and 0.99, but the appropriate value is problem-dependent.

Combined effect of learning rate and momentum

Learning rate and momentum must be selected jointly.

Learning rate	Momentum	Likely behaviour
Low	Low	Stable but very slow
Low	Moderate/high	Smooth and faster
Moderate	Moderate	Usually effective
High	Low	Oscillation or divergence
High	High	Strong overshooting and instability

Momentum does not simply replace the learning rate:

- the **learning rate** controls the response to the current error;
- **momentum** controls how much previous movement is retained.

(iii) Bias–Variance Trade-off

The bias–variance trade-off explains the balance between a model that is too simple and a model that is too complex.

The expected prediction error can be conceptually decomposed as

$$\text{Expected error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$$

1. Bias

Bias is the error produced by restrictive or incorrect assumptions in the learning model.

A high-bias neural network:

- is too simple to represent the underlying relationship;
- fails to learn important patterns;
- produces high training error;
- produces high validation or test error;
- leads to underfitting.

Causes of high bias

- too few hidden neurons;
- too few hidden layers;
- overly strong regularization;
- insufficient training epochs;
- an excessively small learning rate that prevents effective training;
- inappropriate input features;
- excessive early stopping.

Underfitting pattern

$$\text{Training error: high}$$

$$\text{Validation error: high}$$

The two errors are often relatively close.

---

2. Variance

Variance measures how strongly the trained model changes when it is trained using a different sample of the data.

A high-variance neural network:

- is excessively complex relative to the available data;
- learns noise and sample-specific details;
- obtains very low training error;
- produces substantially higher validation or test error;
- leads to overfitting.

Causes of high variance

- too many hidden layers or neurons;
- too many trainable parameters;

- a small training dataset;
- noisy or unrepresentative data;
- training for too many epochs;
- insufficient regularization;
- data leakage or poor validation design.

Overfitting pattern

$$\text{Training error: very low}$$

$$\text{Validation error: high}$$

A large gap appears between training and validation performance.

---

3. Trade-off

Increasing model complexity generally produces the following effects:

- bias decreases because the network can represent more complicated relationships;
- variance increases because the network becomes more sensitive to the particular training samples.

Decreasing model complexity generally produces the opposite effects:

- variance decreases;
- bias increases.

Therefore, the objective is not merely to minimize training error. The objective is to find the model complexity that minimizes **validation or generalization error**.

Model condition	Bias	Variance	Result
Too simple	High	Low	Underfitting
Appropriate complexity	Moderate/low	Moderate/low	Good generalization
Too complex	Low	High	Overfitting

Bias and variance during BP training

At the beginning of BP training:

- both training and validation errors are usually high;
- the network has not learned sufficiently;

- bias is high.

As training proceeds:

- training error decreases;
- validation error also initially decreases;
- bias is reduced.

After excessive training:

- training error continues to decrease;
- validation error may begin to increase;
- the network starts learning noise;
- variance increases.

The point at which the validation error reaches its minimum is often considered an appropriate stopping point.

Methods for controlling the bias–variance trade-off

To reduce high bias

- increase the number of hidden neurons;
- add suitable hidden layers;
- train for more epochs;
- reduce excessive regularization;
- improve input features;
- use a more suitable activation function;
- improve optimization and learning-rate selection.

To reduce high variance

- collect more training data;
- use data augmentation;
- reduce the number of neurons or layers;
- apply L_1 or L_2 regularization;
- use weight decay;
- apply dropout;
- use early stopping;
- use cross-validation;
- use ensemble learning;
- remove irrelevant features.

Summary

1. **Sequential training** updates the weights after each sample. It learns quickly and supports online learning, but its updates are noisy.
2. **Batch training** updates the weights after processing the entire training set. It provides stable updates but requires more memory and performs fewer updates.
3. **Mini-batch training** combines the computational efficiency of batch training with the frequent updates of sequential training.
4. The **learning rate** determines the magnitude of the current weight correction. A low value produces slow learning, whereas a high value may cause oscillation or divergence.
5. **Momentum** retains part of the previous weight correction. It accelerates movement in consistent directions and reduces oscillation.
6. **High bias** results in underfitting, while **high variance** results in overfitting. Effective BP training seeks a network architecture and stopping point that provide low validation error and good generalization.

Convergence in Backpropagation Learning

Convergence means that the neural network's training process has reached a stable state where further iterations produce little or no meaningful improvement in the objective or error function.

During backpropagation, the weights are repeatedly updated:

$$w_{ji}(t+1) = w_{ji}(t) + \Delta w_{ji}(t)$$

The network is considered to be approaching convergence when the weight changes Δw_{ji} and the reduction in error become very small.

Convergence does **not necessarily mean that the error becomes zero**. It means the optimization process has reached a minimum or a sufficiently stable region of the error surface.

How convergence is identified

1. The training error stops decreasing significantly

Let $E(t)$ be the training error at epoch t. Convergence may be declared when

$$|E(t) - E(t-1)| < \varepsilon_E$$

where ε_E is a small predefined tolerance.

For example, if

$$E(t-1) = 0.02004$$

and

$$E(t) = 0.02003,$$

then the improvement is only

$$|0.02003 - 0.02004| = 0.00001.$$

If this small change persists for several epochs, training may be considered converged.

---

2. Weight changes become very small

Convergence may also be identified when

$$|\Delta w_{ji}(t)| < \varepsilon_w$$

for all or most weights, where ε_w is a small threshold.

A more general condition is

$$\|\Delta \mathbf{w}(t)\| < \varepsilon_w$$

where $\|\Delta \mathbf{w}(t)\|$ represents the magnitude of all weight changes.

Small weight updates indicate that backpropagation is no longer making substantial changes to the network.

---

3. The error signal becomes very small

In backpropagation,

$$\Delta w_{ji} = \eta \delta_j x_i$$

If the error signal δ_j becomes small, the resulting weight correction also becomes small. This indicates that the network output is close to the target, or that the optimization has reached a stationary region.

However, a small error signal may also occur because of the **vanishing-gradient problem**, so this criterion should not be used alone.

---

4. The gradient magnitude becomes close to zero

Since the weight correction is based on the error gradient, convergence is theoretically associated with

$$\left\| \frac{\partial E}{\partial \mathbf{w}} \right\| \approx 0.$$

This means the error surface is almost flat around the current weight values.

However, a near-zero gradient may indicate:

- a local minimum;
- a global minimum;
- a saddle point;
- a flat plateau.

Therefore, a small gradient indicates optimization stability, but it does not guarantee the best possible solution.

---

5. Validation error stops improving

For practical machine learning, convergence should also be monitored using validation data.

Suppose:

- training error continues decreasing;
- validation error initially decreases;
- validation error later becomes stable or starts increasing.

The model has reached its best generalization point near the minimum validation error.

This is commonly detected using **early stopping**.

For example:

- monitor validation loss after each epoch;
- save the model whenever validation loss improves;
- stop training if validation loss does not improve for a fixed number of epochs.

The number of allowed non-improving epochs is called **patience**.

Example:

$$\text{Patience} = 10$$

means training stops if validation loss does not improve for 10 consecutive epochs.

Typical convergence criteria

Training can be stopped when one or more of the following conditions are satisfied:

$$E(t) < E_{\text{target}}$$

The error has fallen below a predefined target.

$$|E(t) - E(t-1)| < \varepsilon_E$$

The change in error is negligible.

$$\|\Delta \mathbf{w}(t)\| < \varepsilon_w$$

The weight changes are negligible.

$$\left\| \frac{\partial E}{\partial \mathbf{w}} \right\| < \varepsilon_g$$

The gradient magnitude is sufficiently small.

$$t \geq t_{\max}$$

The maximum number of epochs has been reached.

In practice, several criteria are usually combined.

Convergence patterns

Proper convergence

- Training error decreases and then stabilizes.
- Validation error decreases and stabilizes.
- Weight updates become small.
- Training and validation performance remain reasonably close.

Underfitting

- Training error remains high.
- Validation error also remains high.
- The model may appear stable but has not learned the underlying pattern sufficiently.

This is not desirable convergence.

Overfitting

- Training error continues decreasing.
- Validation error begins increasing.
- The model is fitting noise in the training data.

In this case, the best model is normally the one obtained at the minimum validation error, not the final epoch.

Non-convergence

- Error fluctuates strongly.
- Error increases or diverges.
- Weight updates remain large.

- Training accuracy does not stabilize.

This may result from:

- an excessively large learning rate;
- poor feature scaling;
- unsuitable weight initialization;
- exploding gradients;
- an inappropriate architecture.

Loss-curve interpretation

Training loss	Validation loss	Interpretation
Both decreasing	Both decreasing	Learning is progressing
Both stable at low values	Stable at low values	Good convergence
Both stable at high values	Stable at high values	Underfitting
Decreasing	Increasing	Overfitting
Strongly oscillating	Strongly oscillating	Unstable or non-convergent training
Increasing rapidly	Increasing rapidly	Divergence

Important distinction

Optimization convergence

The training algorithm has reached a point where the loss, gradients, or weights change very little.

Generalization convergence

The model has reached the point of best performance on unseen or validation data.

A neural network may converge in terms of optimization but still generalize poorly. Therefore, convergence should usually be identified by examining both:

$$\text{training loss}$$

and

$$\text{validation loss.}$$

In one sentence

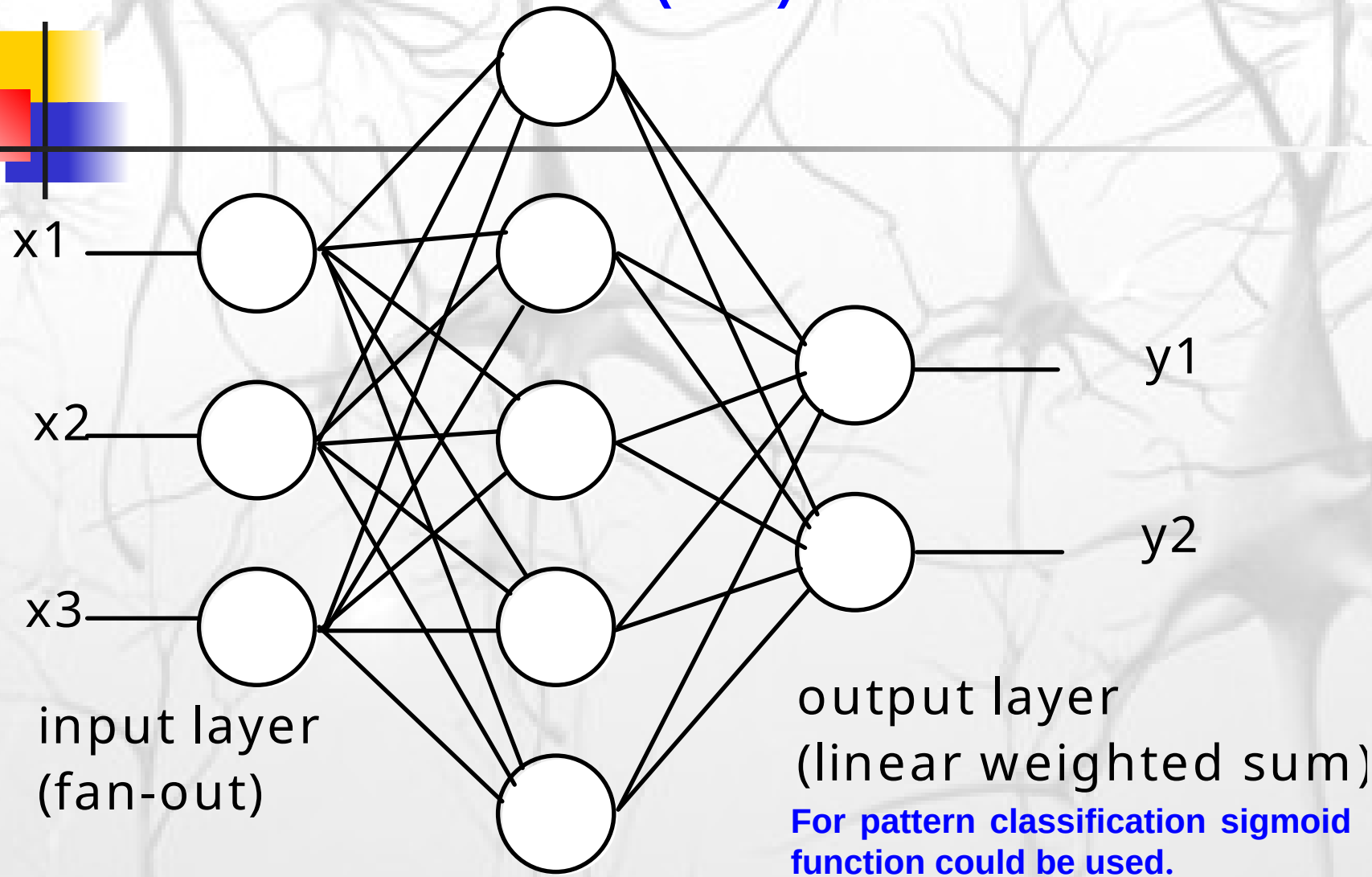
In backpropagation, convergence is the condition in which further weight updates produce negligible improvement, and it is identified by stable loss values, small weight changes, small gradients, and—most importantly—no further improvement in validation performance.

Radial Basis Function (RBF) Networks

Dr. Md. Aminul Haque Akhand

Dept. of CSE, KUET

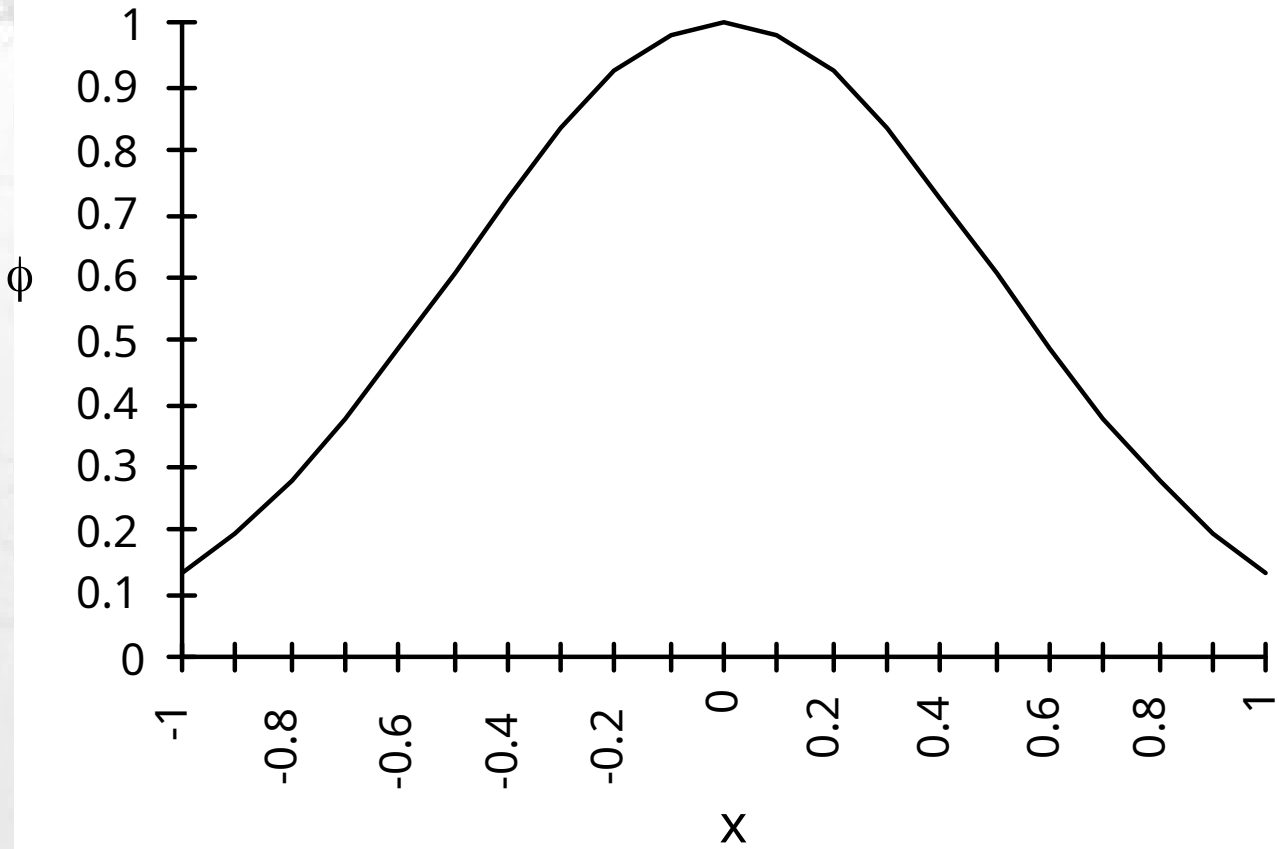
Radial Basis Function (RBF) Networks



Clustering

- The unique feature of the RBF network is the process performed in the hidden layer.
- The idea is that the patterns in the input space form clusters.
- If the centres of these clusters are known, then the distance from the cluster centre can be measured.
- Furthermore, this distance measure is made non-linear, so that if a pattern is in an area that is close to a cluster centre it gives a value close to 1.
- Beyond this area, the value drops dramatically.
- The notion is that this area is radially symmetrical around the cluster centre, so that the non-linear function becomes known as the radial-basis function.

Gaussian function



- The most commonly used radial-basis function is a Gaussian function
- In a RBF network, *r* is the distance from the cluster centre.

Distance Measure

- The distance measured from the cluster centre is usually the Euclidean distance.
- For each neuron in the hidden layer, the weights represent the co-ordinates of the centre of the cluster.
- Therefore, when that neuron receives an input pattern, X, the distance is:

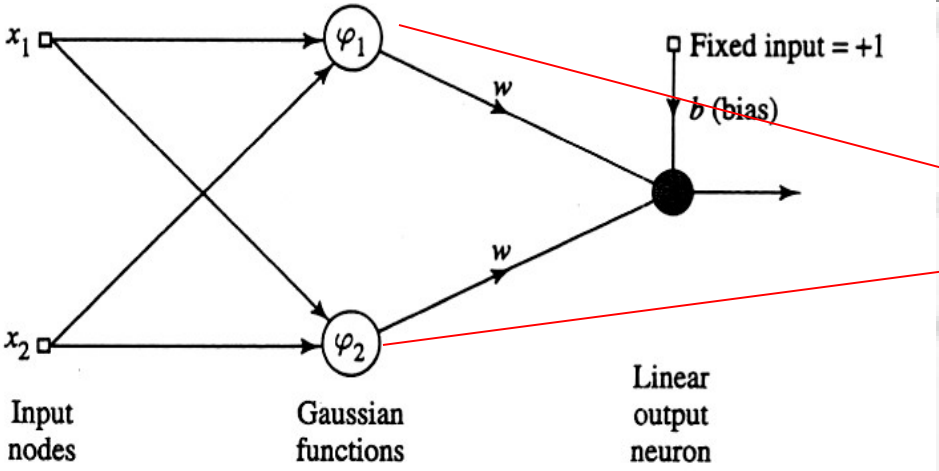
$$r_j = \sqrt{\sum_{i=1}^{n} (x_i - w_{ij})^2}$$

Width of hidden unit basis function

$$(hidden_unit)\varphi_j = \exp\left(-\frac{\sum_{i=1}^{n}(x_i - w_{ij})^2}{2\sigma^2}\right)$$

- The variable sigma, σ, defines the width or radius of the bell-shape and is something that has to be determined empirically.
- When the distance from the centre of the Gaussian reaches σ, the output drops from 1 to 0.6.

XOR Problem Solution



$$G(\| x - t_i \|) = \exp(- \| x - t_i \|^2) \ , \ i = 1,2$$

$$\varphi_1 = \exp(- \| x - t_1 \|^2)$$

$$\varphi_2 = \exp(- \| x - t_2 \|^2)$$

Suppose Centres

$t_1 = [1,1]^T$ and $t_2 = [0,0]^T$

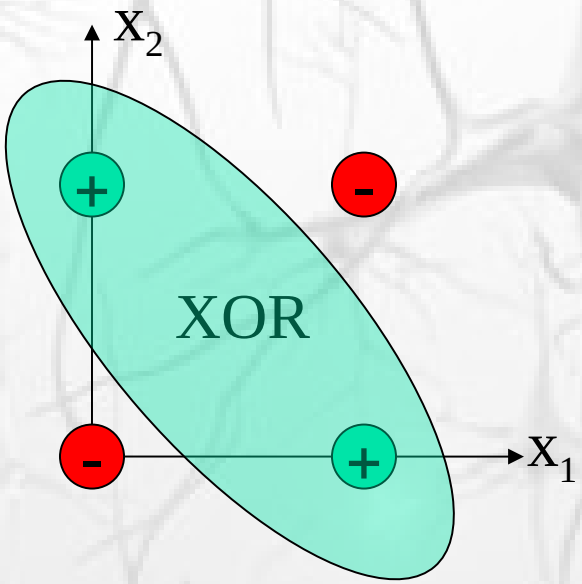
The value of radius σ is chosen such that $2\sigma^2 = 1$

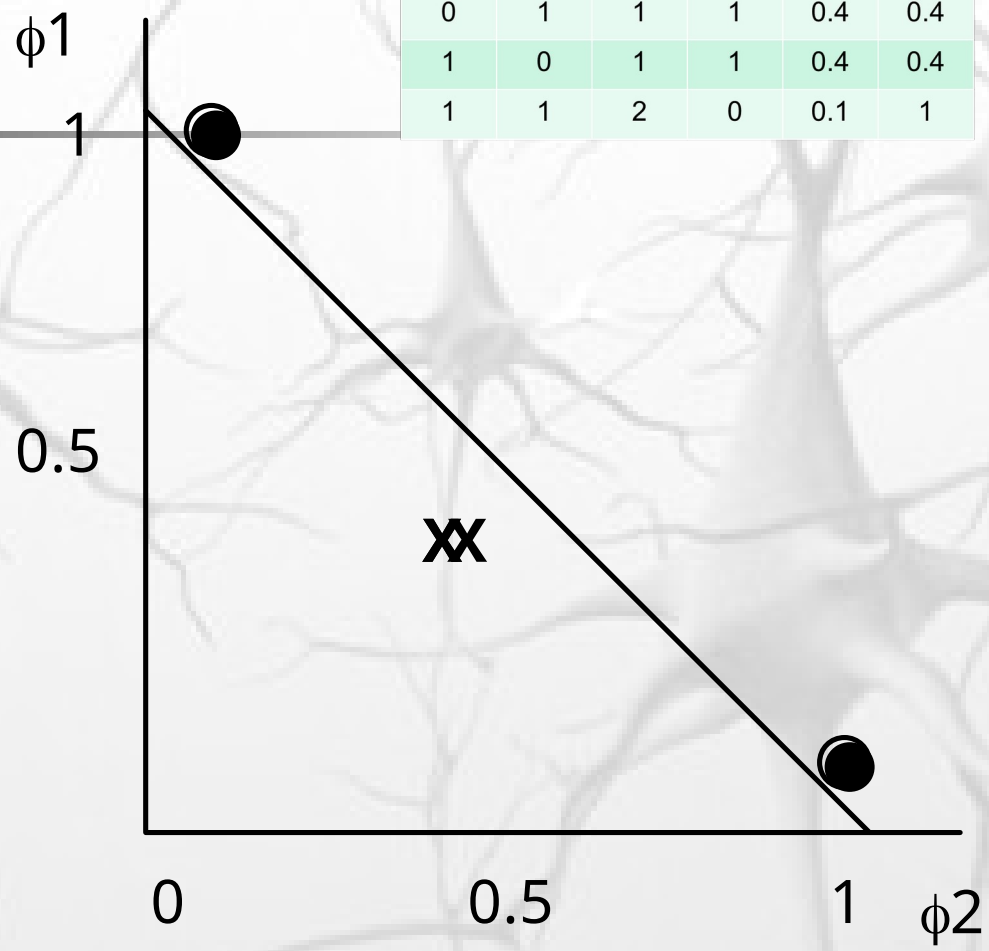
Distance (r_1 & r_2) calculated using Hamming distance

x_1	x_2	r_1	r_2	φ_1	φ_2
0	0	0	2	1	0.1
0	1	1	1	0.4	0.4
1	0	1	1	0.4	0.4
1	1	2	0	0.1	1

XOR Problem Solution

x_1	x_2	r_1	r_2	φ_1	φ_2
0	0	0	2	1	0.1
0	1	1	1	0.4	0.4
1	0	1	1	0.4	0.4
1	1	2	0	0.1	1





- This is an ideal solution - the centres were chosen carefully to show this result.
- The learning of an RBF network finds those centre values

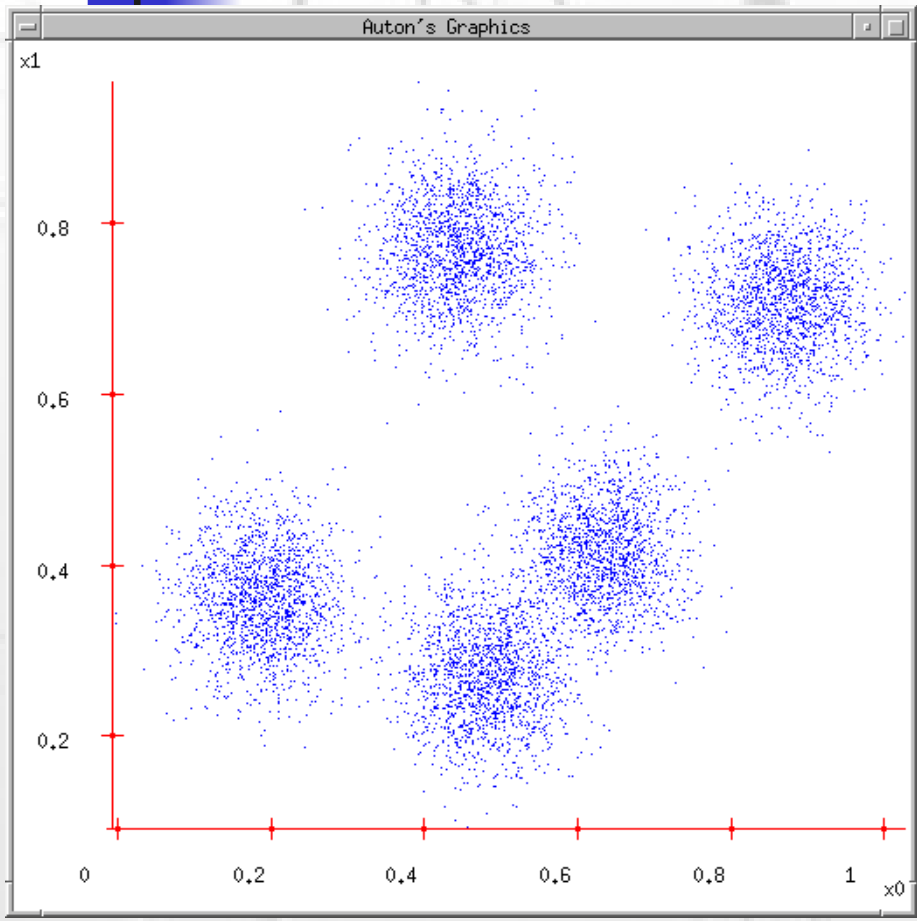
Training hidden layer

- The hidden layer in a RBF network has units which have weights that correspond to the vector representation of the centre of a cluster.
- These weights are found either using a traditional clustering algorithm such as the k-means algorithm, or adaptively using essentially the Kohonen algorithm.
- In either case, the training is unsupervised but the number of clusters that you expect, k, is set in advance. The algorithms then find the best fit to these clusters.

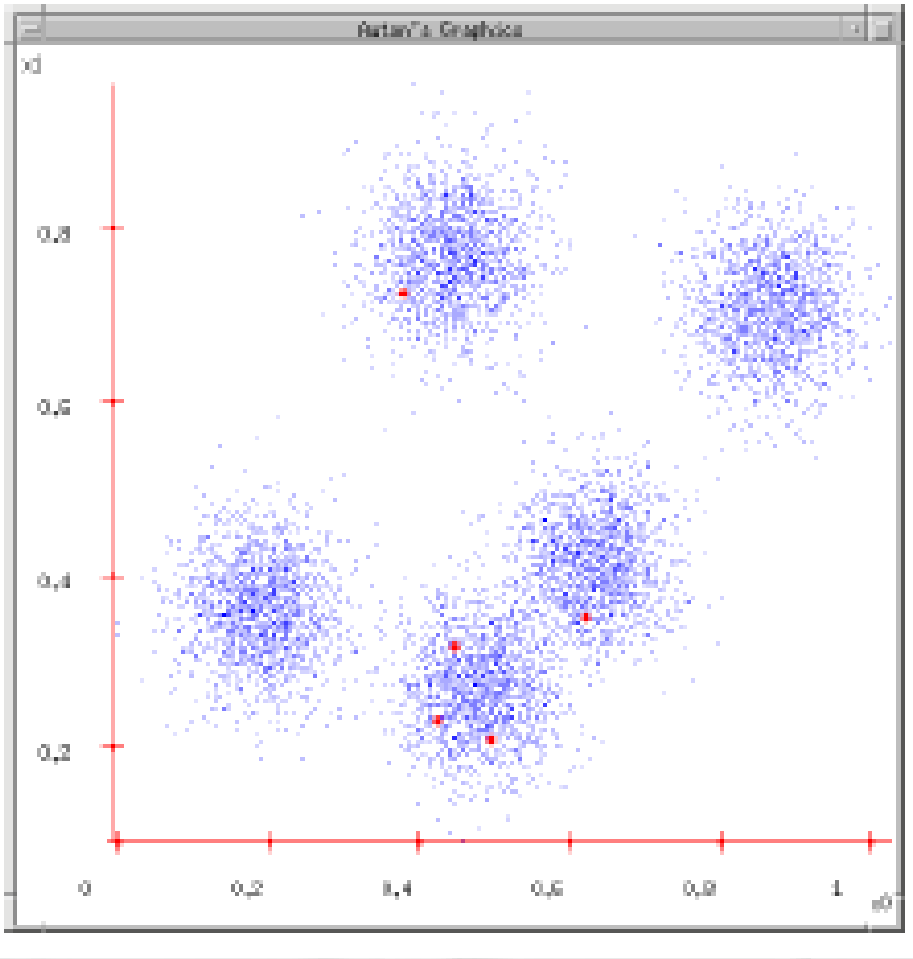
k -means algorithm

- Initially k points in the pattern space are randomly set.
- Then for each item of data in the training set, the distances are found from all of the k centres.
- The closest centre is chosen for each item of data - this is the initial classification, so all items of data will be assigned a class from 1 to k.
- Then, for all data which has been found to be class 1, the average or mean values are found for each of co-ordinates.
- These become the new values for the centre corresponding to class 1.
- Repeated for all data found to be in class 2, then class 3 and so on until class k is dealt with - we now have k new centres.
- Process of measuring the distance between the centres and each item of data and re-classifying the data is repeated until there is no further change – i.e. the sum of the distances monitored and training halts when the total distance no longer falls.

k -means algorithm

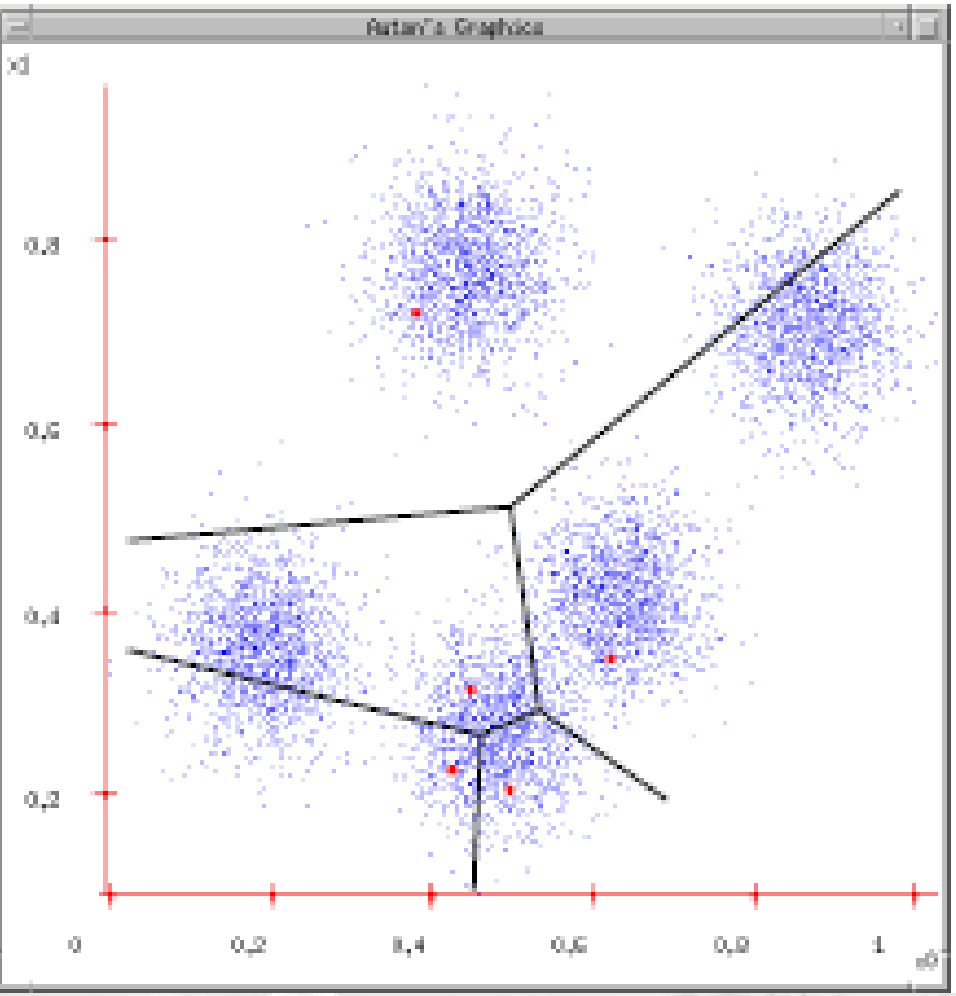


1. Ask user how many clusters they'd like. (e.g. k=5)

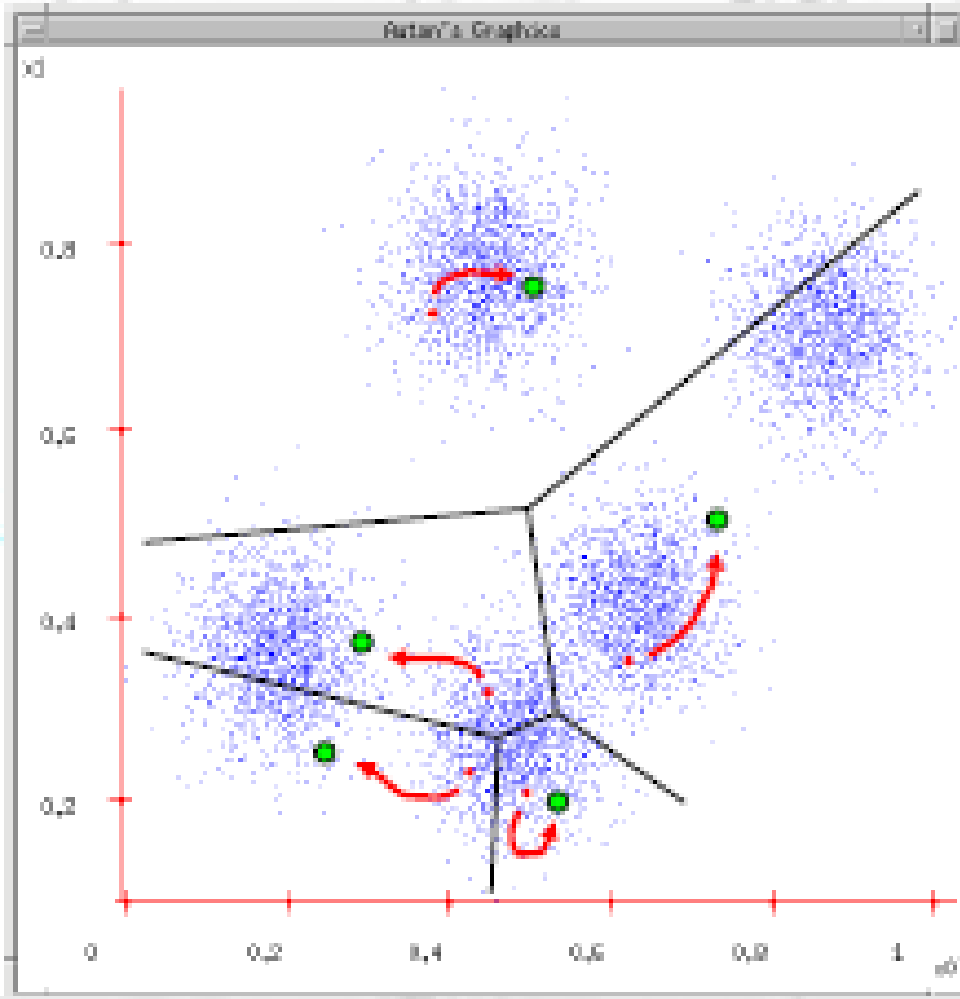


2. Randomly guess k cluster Center locations

k -means algorithm

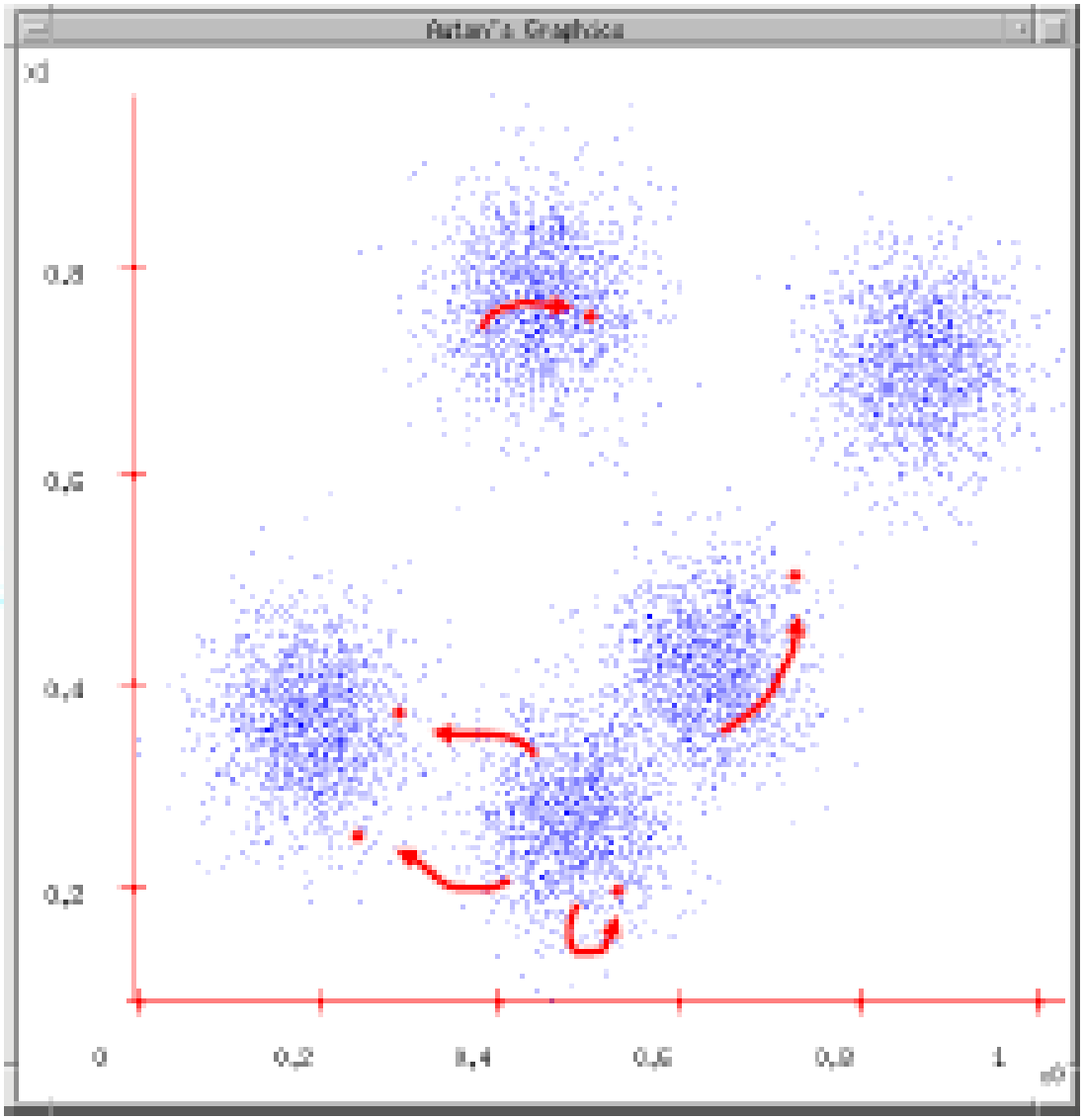


3. Each data point finds out which Center it's closest to.



4. Each Center finds the centroid of the points it owns

k -means algorithm



5. …and jumps there

6. …Repeat until terminated!

Adaptive k-means

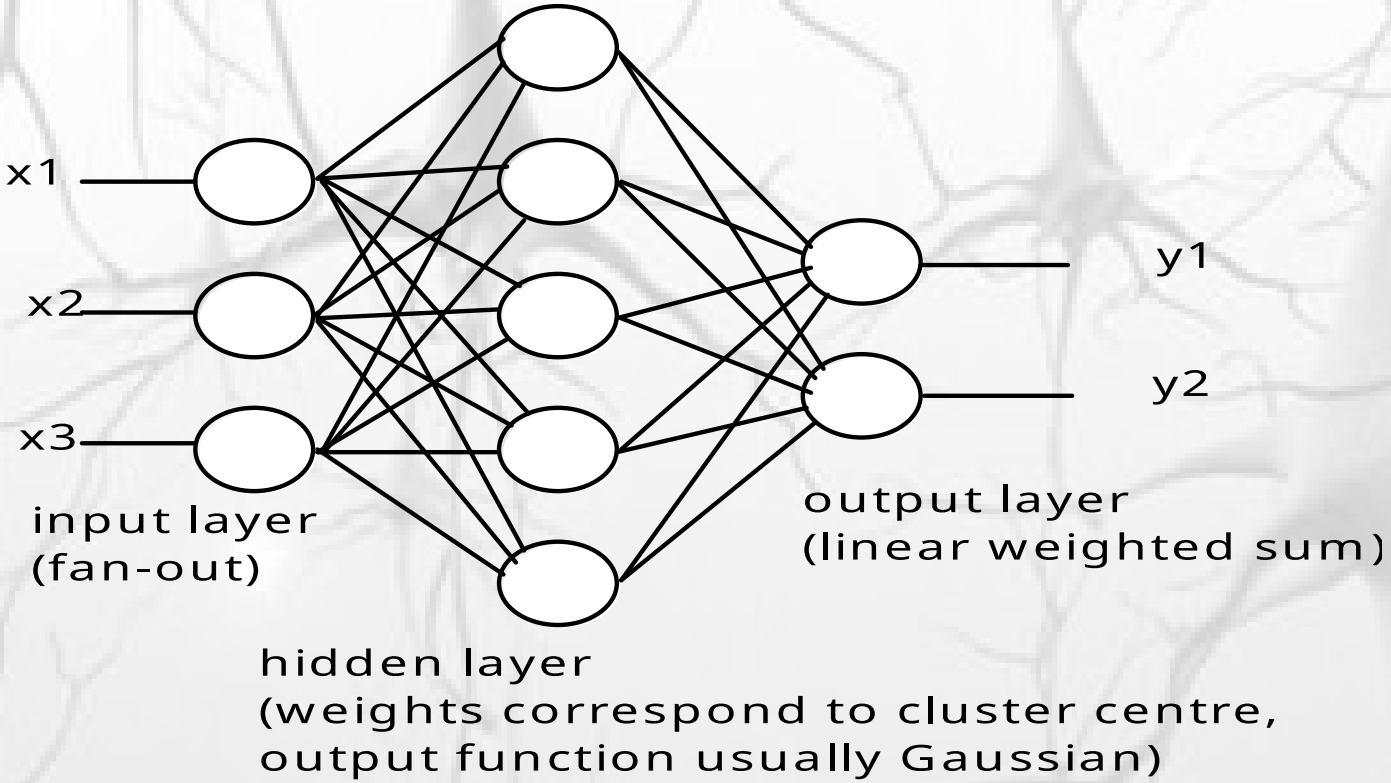
- The alternative is to use an adaptive k-means algorithm which similar to Kohonen learning.
- Input patterns are presented to all of the cluster centres one at a time, and the cluster centres adjusted after each one. The cluster centre that is nearest to the input data wins, and is shifted slightly towards the new data.
- This has the advantage that you don’t have to store all of the training data so can be done on-line.

Finding radius of Gaussians

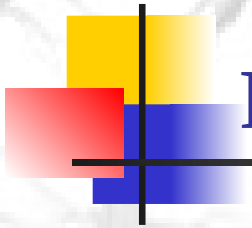
- Having found the cluster centres, the next step is determining the radius of the Gaussian curves.
- This is usually done using the P-nearest neighbour algorithm.
- A number P is chosen (a typical value for P is 2), and for each centre, the P nearest centres are found.
- The root-mean squared distance between the current cluster centre and its P nearest neighbours is calculated, and this is the value chosen for σ.
- So, if the current cluster centre is c_j, the value is:

$$\sigma_j = \sqrt{\frac{1}{P}\sum_{i=1}^{P}(c_j - c_i)^2}$$

Training output layer

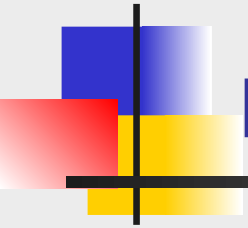


- Having trained the hidden layer with some unsupervised learning, the final step is to train the output layer using a standard gradient descent technique such as the Least Mean Squares algorithm.



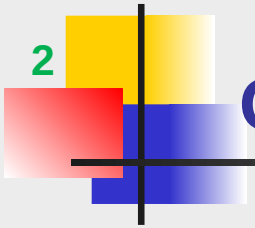
RBF vs. MLP

- An RBF network (in its most basic form) has a single hidden layer, whereas an MLP may have one or more hidden layers
- RBF trains faster than a MLP
- In RBF network the hidden layer is easier to interpret than the hidden layer in an MLP.
- Although the RBF is quick to train, when training is finished and it is being used it is slower than a MLP, so where speed is a factor a MLP may be more appropriate.



Ensembles of Diverse Neural Networks

Prof. Dr. Md. Aminul Haque Akhand
Dept. of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET)
www.kuet.ac.bd/cse/akhand



Outline of the Presentation

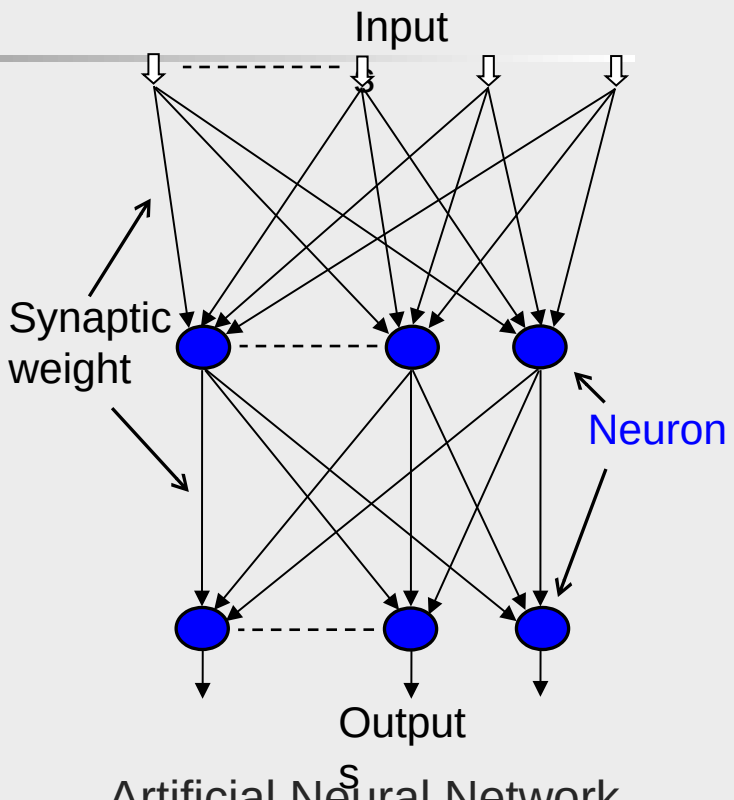
1. Introduction
2. Background of Ensemble Construction
3. Data Sampling for Ensemble Construction
4. Candidate Ensemble Methods
5. Experimental Studies
6. Motivation to better NNE Construction

Artificial Neural Network

According to Haykin, an artificial neural network (NN) is a collection of simple processing units and it has ability to store experimental knowledge.

NN resembles the human brain in two respects:

1. *Knowledge is acquired by a NN from its environment through a learning process.*
2. *Interneuron connection strengths, known as synaptic weights, are used to store the knowledge.*



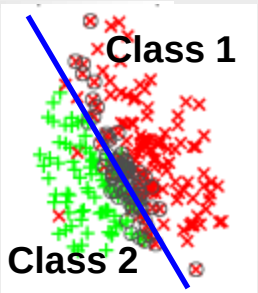
Artificial Neural Network

❖ Functionality of a NN depends on the synaptic weight values and the aim of learning is to get proper weight set.

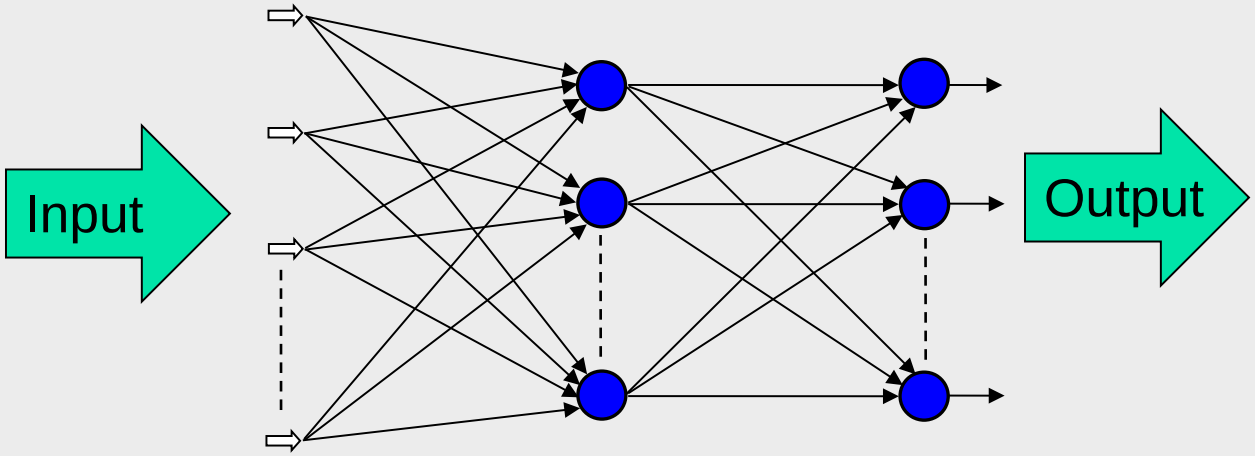
NNs have been successfully applied for various task; for classification it performs well.

NNs for Classification Tasks

- Classification is one of the most frequently encountered decision making tasks of human activity.
  - It occurs when an object needs to be assigned into a predefined group or class based on a number of observed attributes related to that object.



Back-Propagation (BP) Learning Algorithm for Classification



Error = Desired Output - Actual Output

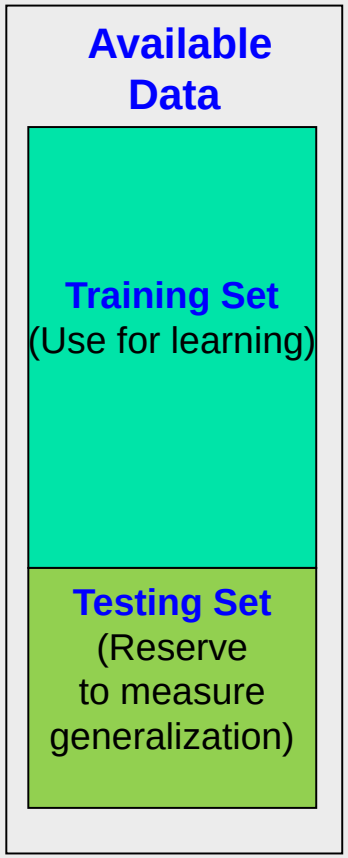
Forward Pass: Error is calculated based on desired and actual output.

Backward Pass: Synaptic weights are adjusted based on calculated error.

- At a time small fraction of a weight is corrected w. r. t demand correction for smooth learning; a parameter learning rate(η) defines the relative size to change.

Performance Measures

Learning and generalization are the most important topics in NN research. Learning is the ability to approximate the training data while generalization is the ability to predict well beyond the training data.



- Generalization is more desirable because the common use of a NN is to make good prediction on new or unknown objects.
- It measures on the testing set that is reserved from available data and not use in the training.
- Testing error rate (TER), i.e., rate of wrong classification on testing set, is widely acceptable quantifying measure, which value minimum is good.

$$\text{TER} = \frac{\text{Total testing set misclassified patterns}}{\text{Total testing set patterns}}$$

Benchmark problems are used to measure TER.

Benchmark Problems for Evaluation

A benchmark is a point of reference by which something can be measured.

- For NN or machine learning, the most popular benchmark dataset collection is the University of California, Irvine (UCI) Machine Learning Repository (http://archive.ics.uci.edu/ml/).
- UCI contains raw data that require preprocessing to use in NN. Some preprocessed data is also available at Proben1 (ftp://ftp.ira.uka.de/pub/neuron/).
- Various persons or groups also maintain different benchmark datasets for specific purpose: Delve (www.cs.toronto.edu/~delve/data/datasets.html), Orange (www.ailab.si/orange/datasets.asp), etc.
- In this study 32 benchmark problems are considered from UCI. Well defined benchmark methodology is followed for preprocessing.

UCI Machine Learning Repository - Windows Internet Explorer

http://archive.ics.uci.edu/ml/

UCI Machine Learning Repository

UCI Machine Learning Repository

Center for Machine Learning and Intelligent Systems

Welcome to the UC Irvine Machine Learning Repository!

We currently maintain 174 data sets as a service to the machine learning community. You may **view all data sets** through our searchable interface. Our old web si format. For a general overview of the Repository, please visit our About page. For information about citing data sets in publications, please read our citation policy. If donation policy. For any other questions, feel free to contact the Repository librarians. We have also set up a mirror site for the Repository.

Supported By: In Collaboration With: Rexa.info

Latest News:

07-23-2008: Repository mirror has been set up.

03-24-2008: New data sets have been added!

06-25-2007: Two new data sets have been added: UJI Pen Characters, MAGIC Gamma Telescope

04-13-2007: Research papers that cite the repository have been associated to specific data sets.

04-09-2007: Three new data sets have been added: Poker Hand, Callt2 Building People Counts, Dodgers Loop Sensor.

09-08-2006: The Beta site has been launched.

09-01-2006: SPECTF.test has been modified by the donor.

Newest Data Sets:

06-26-2008: Parkinsons

04-21-2008: Ozone Level Detection

04-03-2008: Abscisic Acid Signaling Network

03-20-2008: Hill-Valley

Most Popular Data

39351:

31585:

26458:

23553:

Benchmark Problems

Problems Related to Human Life

Problem	Task
Breast Cancer Wisconsin	Predicts whether a tumor is benign (not dangerous to health) or malignant (dangerous) based on a sample tissue taken from a patient's breast.
BUPA Liver Disorder	Identify lever disorders based on blood tests along with other related information such as alcohol consumption.
Diabetes	Investigate whether the patient shows or not the signs of diabetes.
Heart Disease Cleveland	Predicting whether at least one of four major heart vessels is reduced in diameter by more than 50%.
Hepatitis	Anticipate status (i.e., live or die) of hepatitis patient.
Lymphography	Predict the situation of lymph nodes and lymphatic vessels.
Lungcancer	Identify types of pathological lung cancers.
Postoperative	Determine place to send patients for postoperative recovery.

Benchmark Problems

Problems Related to Finance

Problem	Task
Australian Credit Card	Classify people as good or bad credit risks depend on applicants' particulars.
Car	Evaluate cars based on price and facilities.
Labor Negotiations	Identify a worker as good or bad i.e., contract with him beneficial or not.
German Credit Card	Like Australian Card, this problem also concerns to predict the approval or non-approval of a credit card to a customer.

Problems Related to Plants

Problem	Task
Iris Plants	Classify iris plant types.
Mushroom	Identify whether a mushroom is edible or not based on a description of the mushroom's shape, color, odor, and habitat.
Soybean	Recognize 19 different diseases of soybeans.

Summary of Benchmark Problems

Abbr.	Problem	Total Examp	Input Features		NN Architecture		
			Cont.	Discr.	Inputs	Class	Hidd. Node
ACC	Australian Credit Card	690	6	9	51	2	10
BLN	Balance	625	-	4	20	3	10
BCW	Breast Cancer Wisconsin	699	9	-	9	2	5
CAR	Car	1728	-	6	21	4	10
DBT	Diabetes	768	8	-	8	2	5
GCC	German Credit Card	1000	7	13	63	2	10
HDC	Heart Disease Cleveland	303	6	7	35	2	5
HPT	Hepatitis (HPT)	155	6	13	19	2	5
HTR	Hypothyroid	7200	6	15	21	3	5
HSV	House Vote	435	-	16	16	2	5
INS	Ionosphere	351	34	-	34	2	10
KRP	King+Rook vs King+Pawn	3196	-	36	74	2	10
LMP	Lymphography	148	-	18	18	4	10
PST	Postoperative	90	1	7	19	3	5
SBN	Soybean	683	-	35	82	19	25
SNR	Sonar	208	60	-	60	2	10
SPL	Splice Junction	3175	-	60	60	3	10
WIN	Wine	178	13	-	13	3	5
WVF	Waveform	5000	21	-	21	3	10
ZOO	Zoo	101	15	1	16	7	10

Input Features of Diabetes

1. Number of times pregnant
2. Plasma glucose concentration
3. Diastolic blood pressure
4. Triceps skin fold thickness (mm)
5. 2-Hour serum insulin (mu U/ml)
6. Body mass index
7. Diabetes pedigree function
8. Age

- Problems show variations in number of examples, input features and classes.

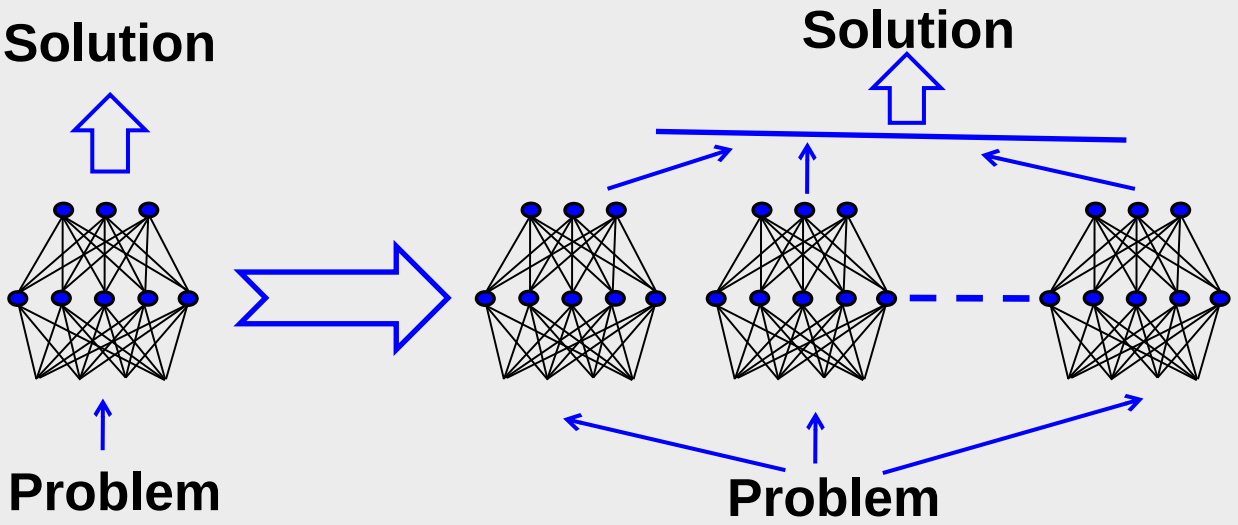
Why Ensemble of Neural Networks?

- The idea of building an ensemble with several NNs is taken from sociology.
- A committee of people for an important task or building board of doctors for a major operation is a common matter.

- Each member of the committee should be as competent as possible, but they should be complementary to one another. If one or a few members make an error, the probability is high that the remaining members can correct his error.
- Several NNs together might perform better than single NN when they maintain proper diversity to compensate failure of one by others.

The goal of ensemble is to achieve better generalization (i.e., lower TER) through producing diverse NNs.

Neural Network Ensemble (NNE)



In an NNE, component NNs solve the problem individually and combine their outputs for NNE's output. For better performance diversity among NNs is important.

- Diversity means disagreement among the NNs. Pair wise plain disagreement (*PD*) measure is the most popular among various measuring techniques.
- For a NNs pair, diversity *(div)* is equal to the proportion of the patterns on which NNs reply different class predictions. The total NNE diversity (*div_ens)* is the average for all the pairs.

$$div_{i.j} = \frac{1}{N}\sum_{n=1}^{N} Diff\,(C_i(x_n), C_j(x_n)),$$

$$div_ens = \frac{\left[div_{i.j} \text{ for all NNs pairs}\right]}{\text{Total NNs pair}} = \frac{\sum_{i=1}^{M-1}\sum_{j=i+1}^{M} div_{i.j}}{\sum_{m=1}^{M-1} m}$$

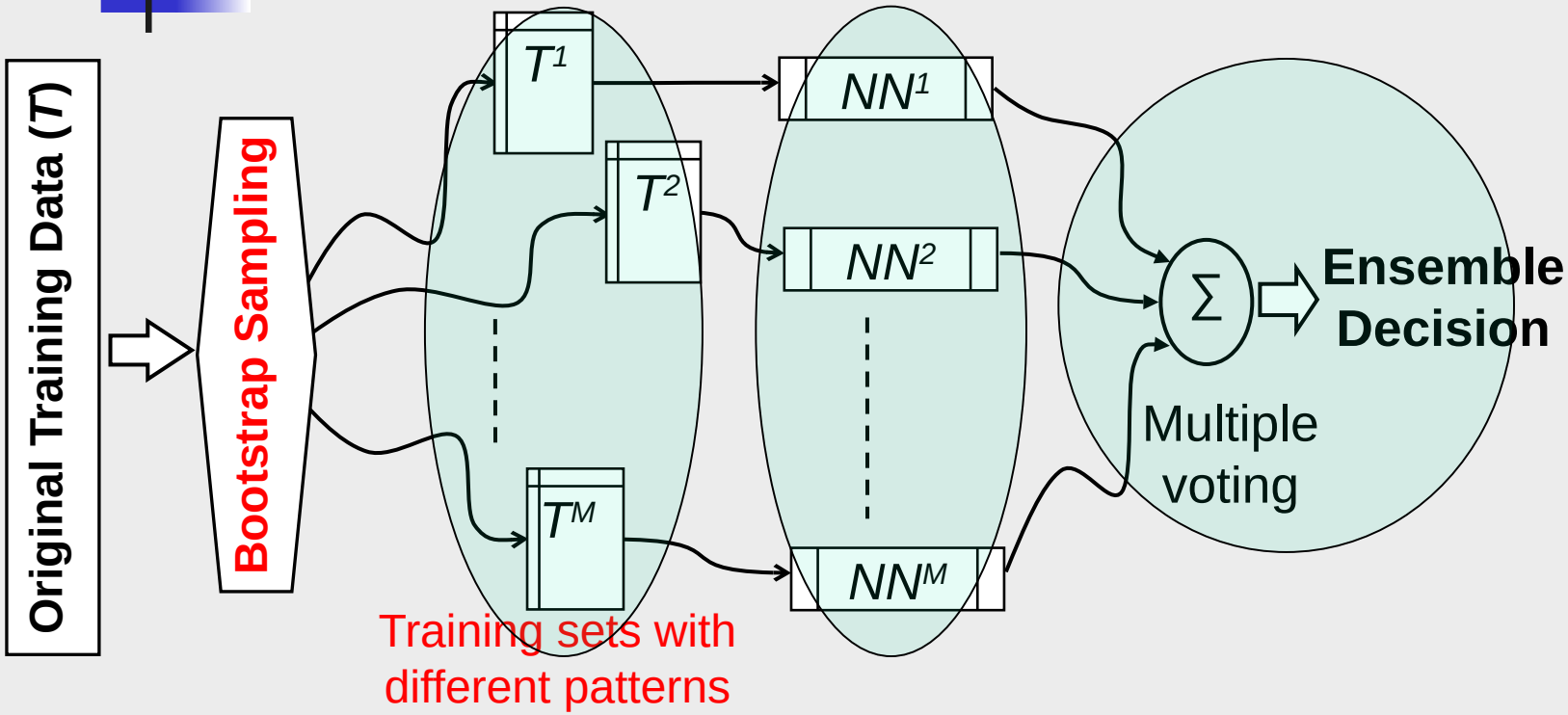
Data Sampling for NNE Construction

Proper diversity among component NNs is an important parameter for ensemble construction so that failure of one may compensate by others.

- There are various ways one can produce diverse NNs, such as varying initial random weights, algorithm employed and training data.
- Since functionality of a NN depends on its training data, data sampling is considered as the most effective for diversity than other approaches.
- Data sampling (i.e. variation in training data) involves: bootstrapping of original training data, generation artificial data, sampling of input features, etc.

1. Bagging (Breiman, 1996)

L. Breiman, "Bagging Predictors" *Machine Learning,* vol. 24, pp. 123 –140, 1996.



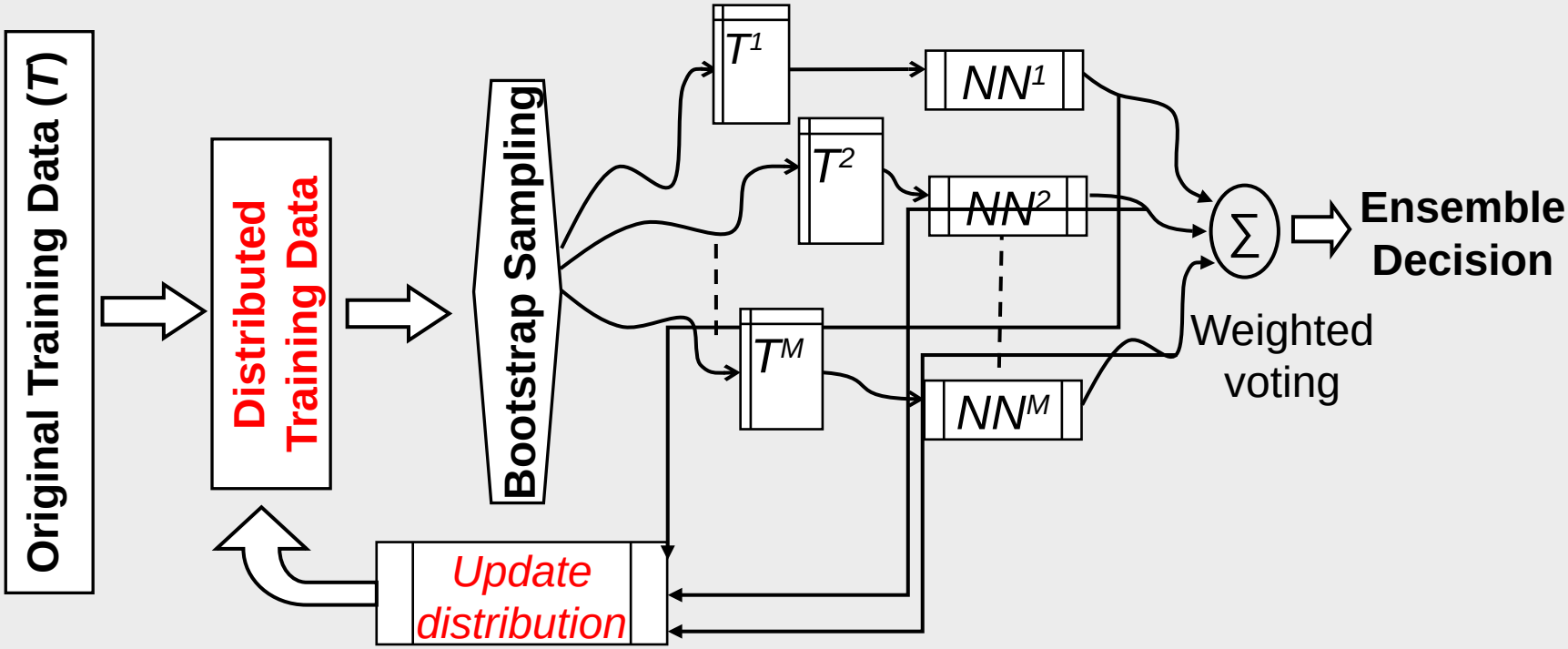
T	T¹	T²
1	4	10
2	9	6
3	6	3
4	8	9
5	1	2
6	8	7
7	5	3
8	1	4
9	2	2
10	4	9

Bootstrap sampling: Randomly selects a pattern and replace it again in its original place for later use.

In a training set many patterns appear multiple times while others are left.

2. AdaBoost (Freund & Schapire, 1996)

Y. Freund and R. E. Schapire, "Experiments with a new boosting algorithm", *in Proc. of the 13th Int. Conf. on Machine Learning(Morgan Kaufmann, 1996)*, pp. 148–156



- NNs are trained one after another sequentially and after training a NN distribution of training data updates.
- Existence of previously miss classified patterns increases in coming training sets due to error base distribution.

Bagging and AdaBoost

1. Let M be the number of networks to be trained for an ensemble

Take original training set $T= \{(x(1), d(1)),\ldots, (x(N), d(N))\}$ with class label

2. *for i=1 to M* {

   a. Make a training set, T_i by sampling N patterns uniformly at random with replacement from T

   b. Train network NN_i by T_i

}

3. Ensemble decision is made in multiple voting way

Bagging

1. Let M be the number of networks to be trained for an ensemble.

Take original train ing set $T= \{(x(1), d(1)),\ldots, (x(N), d(N))\}$ with class label

Assign weight for each pattern of T, initially weights are the same, i.e., $w_1(n)=1/N$

2. *for i=1 to M* {

   a. Make a training set, T_i by sampling N patterns at random with replacement from T based on weight distribution $w_i(n)$

   b. Train network NN_i with T_i

   c. (weighted error on training set)

   d.

   e. *for each*

   if *then* , *otherwise*

   f. Normalize the weights of T

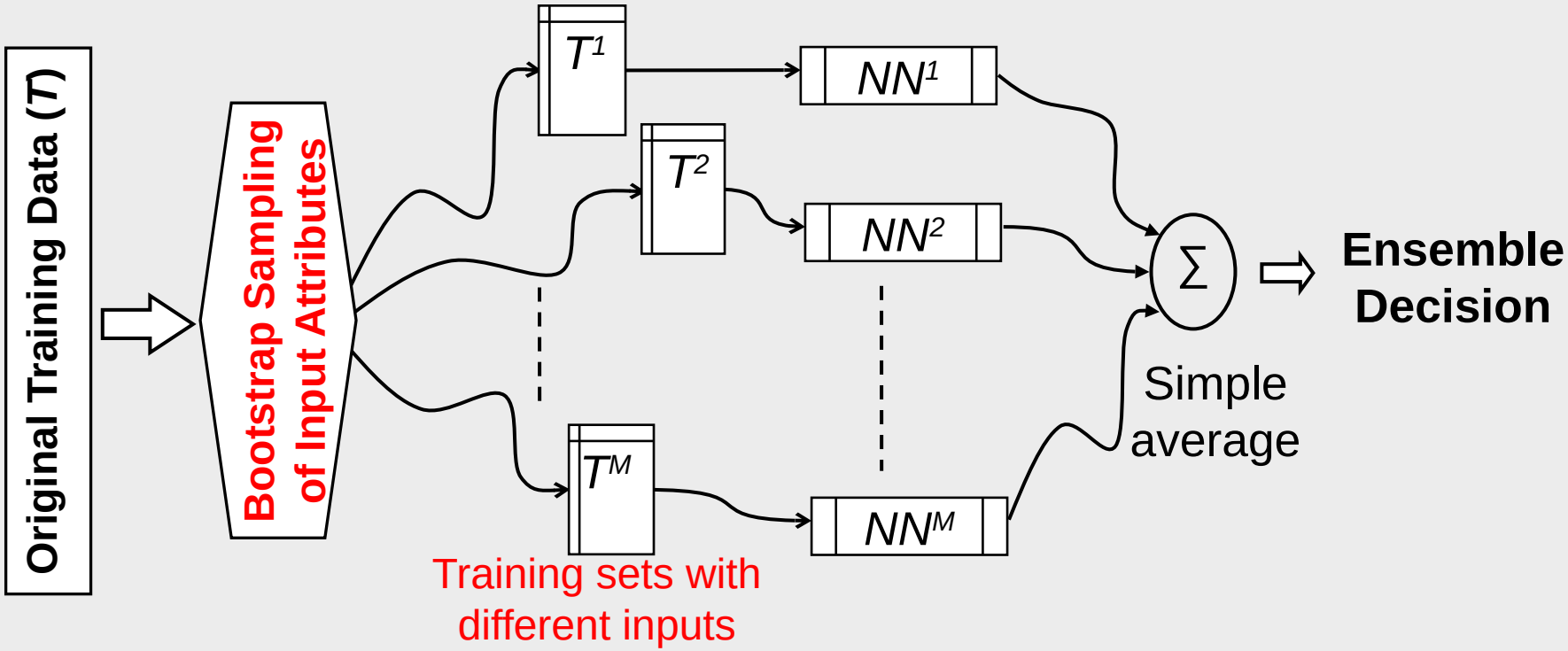
}

3. Ensemble decision is made in weighted voting way

AdaBoost

3. Random Subspace Method(RSM) (Ho, 1998)

T. K. Ho, "The random subspace method for constructing decision forests" *IEEE Trans. on Pattern Analysis and Machine Intelligence, vol.* 20, pp. 832–844, 1998



RSM require to maintain tagging with original inputs for every NN's inputs.

RSM

1. Let M be the number of networks to be trained for an ensemble

   Take original training set $T= \{(x(1), d(1)),\ldots, (x(N), d(N))\}$ with class label

   and feature set

2. *for i=1 to M* {

   a. Make a feature subset, F_i by sampling $|F|$ features uniformly at random with replacement from F

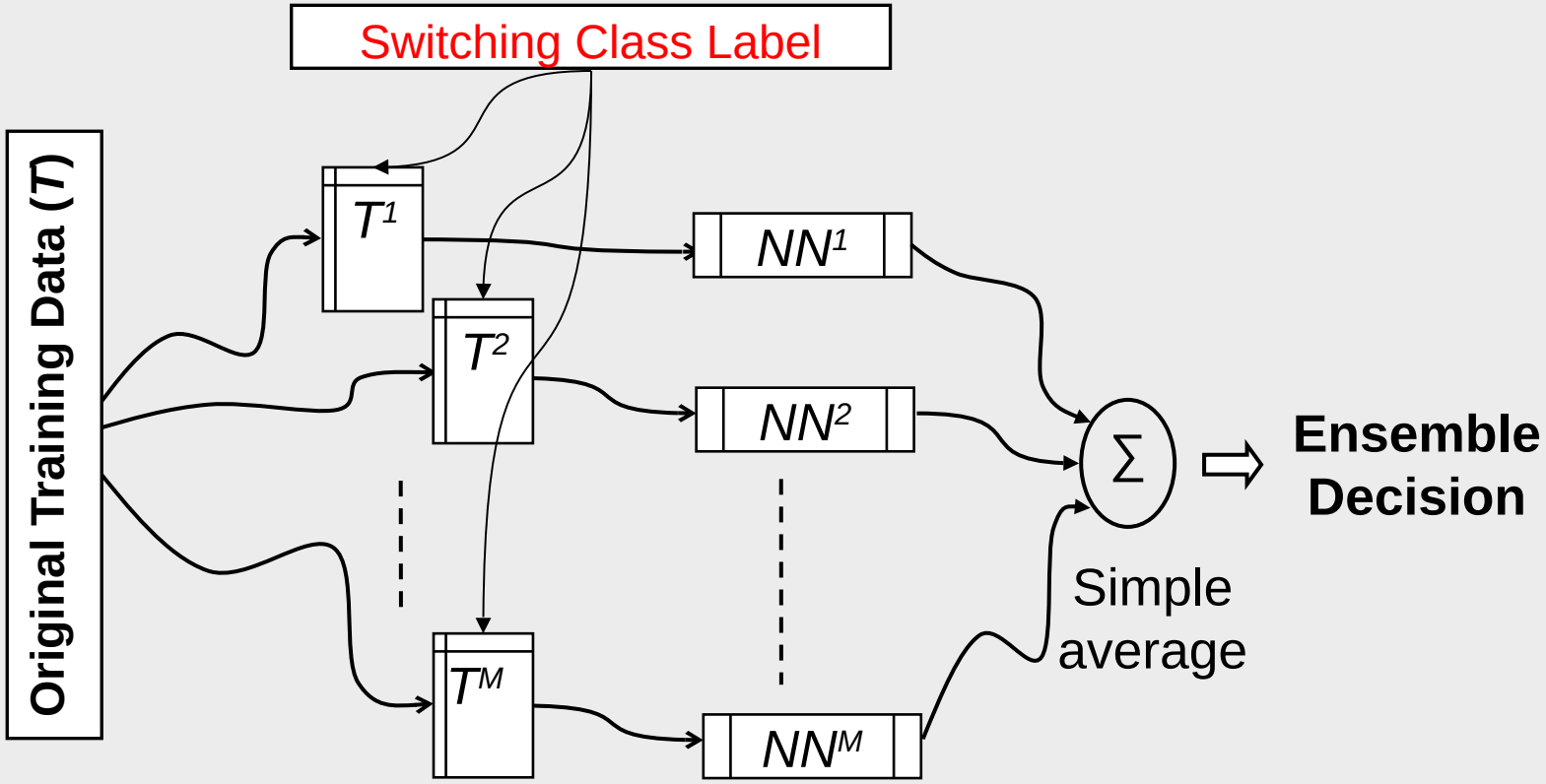
   b. Train network NN_i by T with feature set F_i

   }

3. Ensemble decision is made in simple average way

4. Class Label Switching (Martínez-Muñoz & Suárez, 2005)

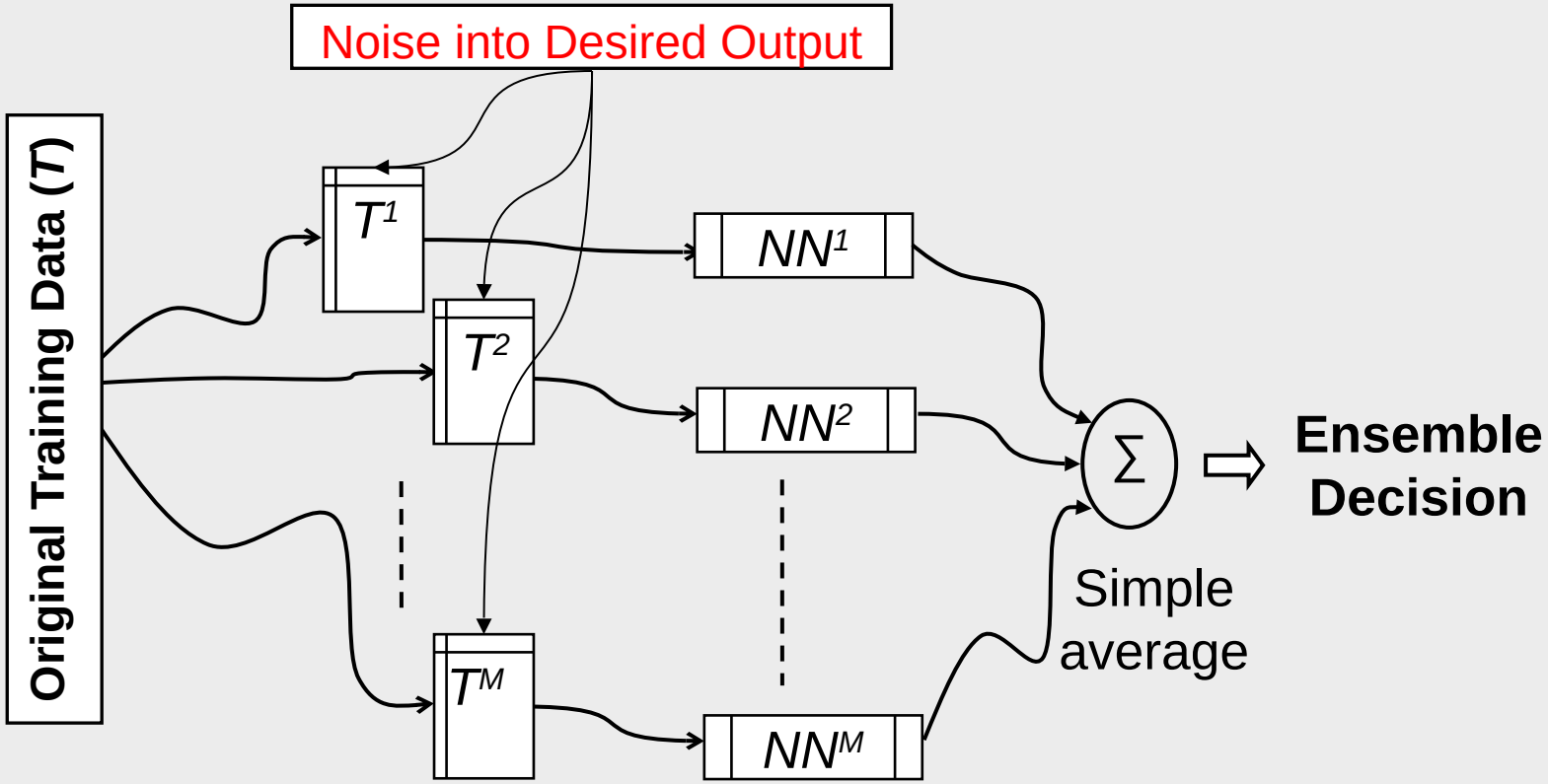
G. Mart´ınez-Mu˜noz and A. Su´arez, "Switching class labels to generate classification ensembles," *Pattern Recognition*, vol. 38, pp. 1483–1494, 2005.



A parameter $S_{fraction}$ maintains number of examples to change class label.

5. Smearing (Breiman, 2000)

L. Breiman, "Randomizing outputs to increase prediction accuracy," Machine Learning, vol 40, pp. 229–242, 2000.



Class Label Switching and Smearing

1. Let M be the number of networks to be trained for an ensemble

   Take original training set $T= \{(x(1), d(1)),\ldots, (x(N), d(N))\}$ with class label

   Take $S_{Fraction}$ -factor that determines number of pattern to alter class label

   Number of patters to switch the class label, $S = S_{Fraction} * N$

2. *for i=1 to M* {

   a. Make a training set, $T_i = T$

   b. Randomly select S examples in T_i and assign different class label randomly

   c. Train network NN_i by T_i

   }

3. Ensemble decision is made in simple average way

Class Label Switching

1. Let M be the number of networks to be trained for an ensemble

   Take original training set $T= \{(x(1), d(1)),\ldots, (x(N), d(N))\}$ with class label

   Compute standard deviation measure (sd_k) is each class based on Eq. (2.1)

2. *for i=1 to M* {

   a. Make a training set, $T_i = T$

   b. Change the desired output of T_i based on Eq. (2.2)

   c. Train network NN_i by T_i

   }

3. Ensemble decision is made in simple average way

If p_k is the proportion of the patterns in class k, then standard deviation measure (sd)

$$sd_k = 2\,(p_k\,(1-p_k))\text{^}0.5 \quad (2.1))$$

The new desired output for the k-th class for the n-th training pattern for a network is

$$d_k'(n) = d_k(n) + z_k(n) sd_k \quad k=1,\ldots,K\ n=1,\ldots,N\ (2.2)$$

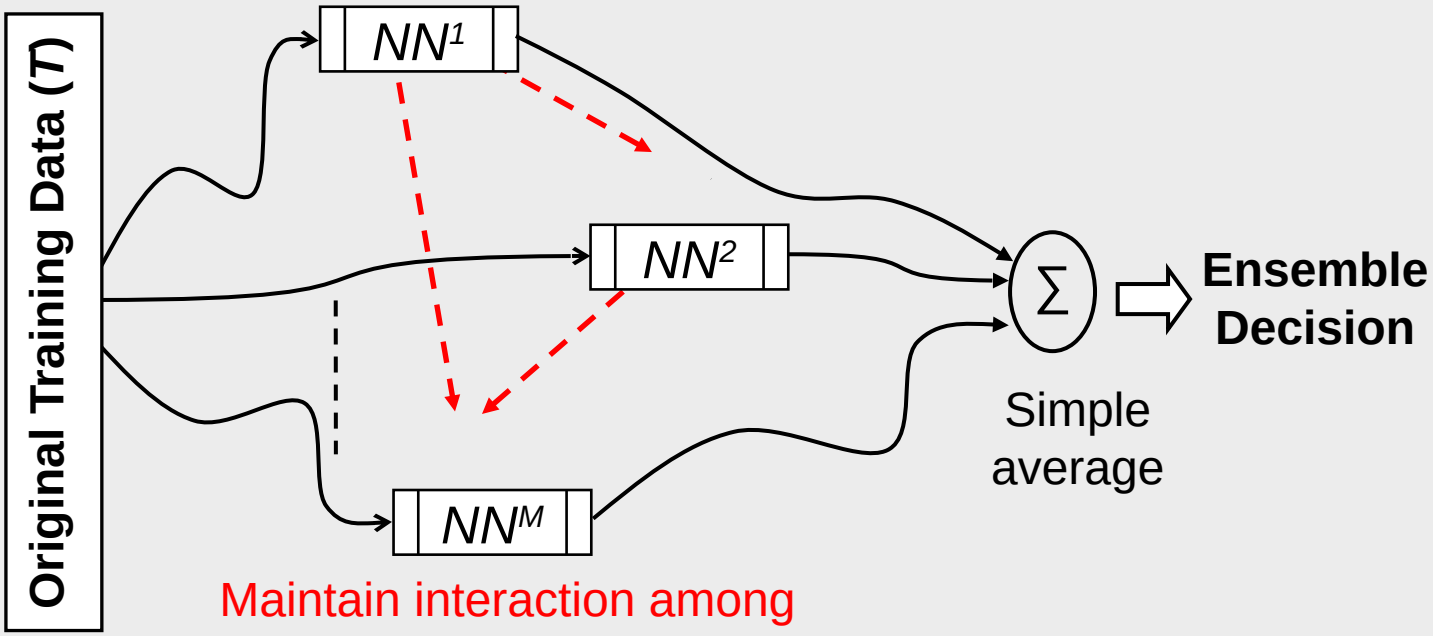
Where $z_k(n)$ is the random number from Gaussian distribution with a mean zero and a variance of one.

Smearing

6. Negative Correlation Learning(NCL) (Liu & Yao, 1999)

Y. Liu and X. Yao, "Ensemble learning via negative correlation," *Neural Networks*, vol. 12 pp. 1399–1404, 1999.

Y. Liu and X. Yao, "Simultaneous training of negatively correlated neural networks in an ensemble," *IEEE Trans. Systems, Man, and Cybernetics — Part B*, vol. 29, pp. 716–725, 1999.



Maintain interaction among component NNs

$$\text{Error function, } e_i(n) = \frac{1}{2}(f_i(n) - d(n))^2 + \lambda\left((f_i(n) - \bar{f}(n))\sum_{j \neq i}(f_j(n) - \bar{f}(n))\right)$$

Normal BP portion — Correlation penalty term

- Interaction produces negatively correlated NNs and therefore NNs motivate different functional spaces.
- The coefficient of the penalty term (λ) maintain strength of interaction.

Negative Correlation Learning(NCL)

1. Let *M* be the number of networks to be trained for an ensemble

   Take original training set *T= {(x(1), d(1)),…, (x(N), d(N))}* with class label

   Create *M* networks, NN_1 ----NN_M

2. *for n=1 to N* {

   Prepare ensemble output for pattern *n*

   for i=1 to M {

   Train network NN_i for pattern *n* using error definition of Eq. (2.3)

   }

}

3. Ensemble decision is made in simple average way

Negative Correlation Learning (NCL)

$$e_i(n) = \frac{1}{2}(d(n) - f_i(n))^2 + \lambda p_i(n) \quad \text{------- Eq. 2.3}$$

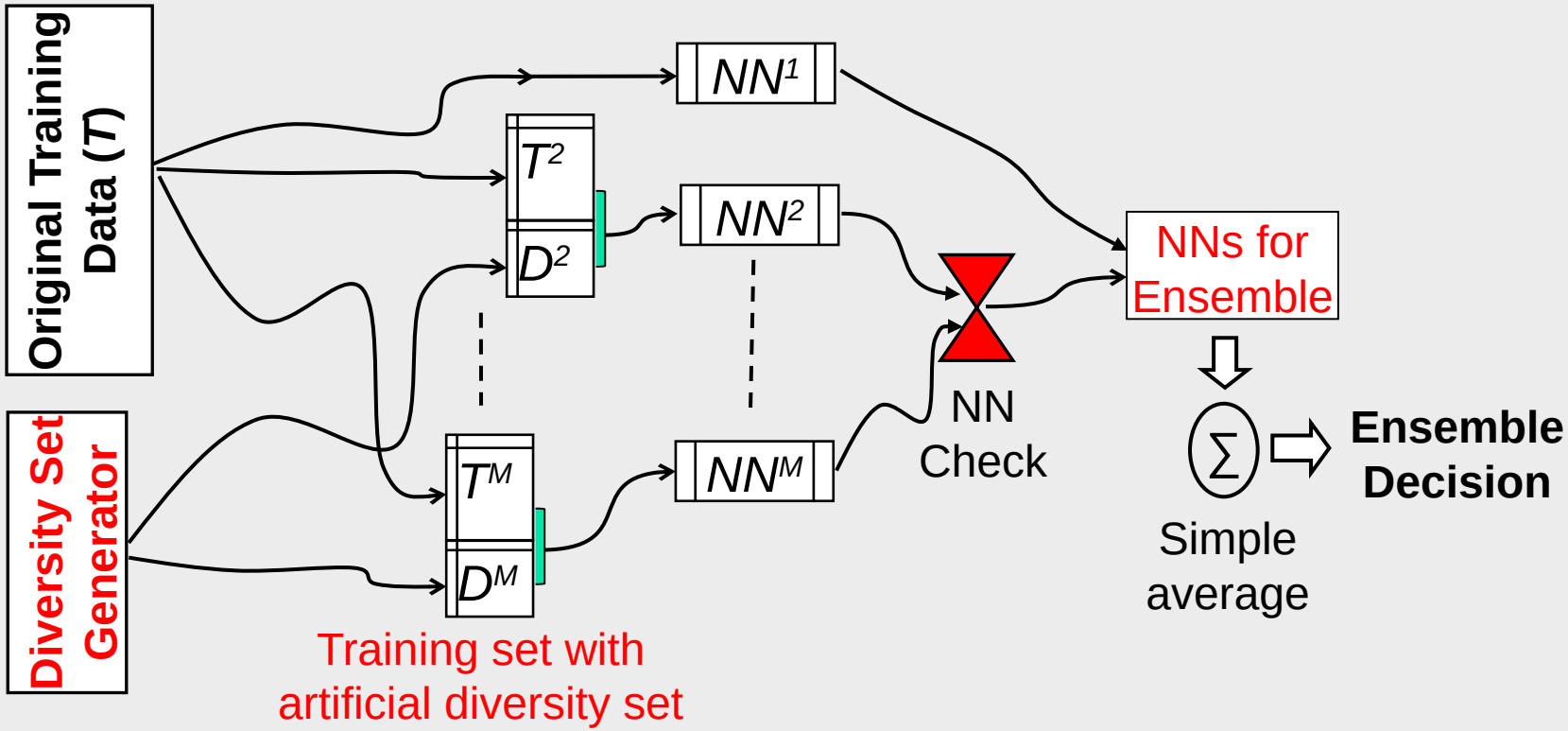
$$p_i(n) = (f_i(n) - \bar{f}(n)) \sum_{j \neq i} (f_j(n) - \bar{f}(n))$$

$$\bar{f}(n) = \frac{1}{M} \sum_{i=1}^{M} f_i(n)$$

7. DECORATE (Melville & Mooney, 2005)

Diverse Ensemble Creation by Oppositional Relabeling of Artificial Training Examples

P. Melville and R. J. Mooney, "Creating diversity in ensembles using artificial data," *Information Fusion, vol.* 6, pp. 99–111, 2005.



- Trains predefined large number of NNs to select NNs for final NNE
- A parameter R_{size} maintains size of diversity set.
- Training of large number of NNs and additional diversity set increase training time.

DECORATE

1. Let I_{max} be the number of networks to be trained for an ensemble and M is the desired number of networks

   Take original training set $T = \{(x(1), d(1)),\ldots, (x(N), d(N))\}$ with class label

   Take R_{Size} -factor that determines size of diversity set

   i=1 (for network in ensemble) and *trails* =1 (for trial network)

   Train network NN_i with T (first network is trained with original training data)

   Initialize ensemble, $Ens = \{ NN_i\}$

   Compute ensemble error,

2. *while trials* < I_{max} and $i < M$ {

   a. Generate $R_{Size} \times |T|$ training examples, R

   b. Label examples in R with probability of class labels inversely proportional to predictions of *Ens*

   c. Prepare training set T_i, $T_i = T \cup R$ and train network NN_i with T_i

   d. $Ens = Ens \cup \{ NN_i\}$

   e. Compute ε' based on Step 1

   f. *if* $\varepsilon' \leq \varepsilon$ *then* $i = i + 1$ *and* $\varepsilon = \varepsilon'$, *otherwise* $Ens = Ens - \{ NN_i\}$

   g. *trials = trials +1*

}

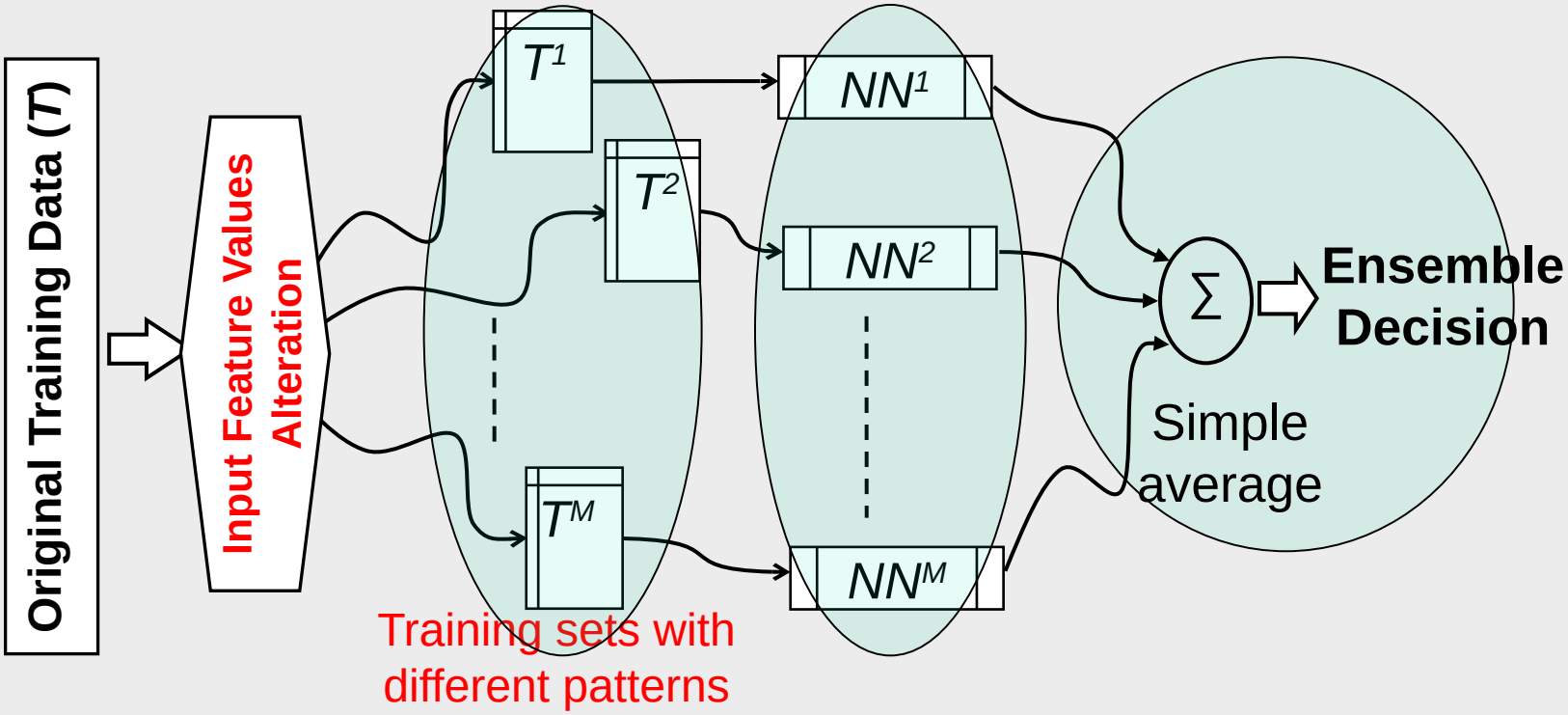
3. Ensemble decision is made in simple average way

DECORATE

8. Ensemble through Input Values Alteration (EIVA)

(M.A.H. Akhand & K. Murase, 2012)

M. A. H. Akhand, and K. Murase , "Ensembles of Neural Networks based on the Alteration of Input Feature Values " *International Journal of Neural Systems*, vol. 22, no.1, pp. 77-87, 2012.



Feature Values Alteration: Randomly selects a pattern and alter some of its feature values with the value of another one.

Ensemble through Input Values Alteration (EIVA)

$$P_a: \; I_{a,1} \; I_{a,2} \; I_{a,3} \; \text{-----} \; | \; \text{-----} \; I_{a,f-2} \; I_{a,f-1} \; I_{a,f} \; cls_a$$
$$P_b: \; I_{b,1} \; I_{b,2} \; I_{b,3} \; \text{-----} \; | \; \text{-----} \; I_{b,f-2} \; I_{b,f-1} \; I_{b,f} \; cls_b$$
$$cp$$
$$P'_a: \; I_{a,1} \; I_{a,2} \; I_{a,3} \; \text{-----} \; I_{b,cp} \; \text{-----} \; I_{b,f-1} \; I_{b,f} \; cls_a$$

Fig. 1. Input values alteration based pattern generation.

1. Take the original training set $T = \{(I(1), d(1)),\ldots,(I(n), d(n)),\ldots, (I(N), d(N))\}$ with class label $d(n)$ $K = \{1,2,\ldots,k\}$ and feature set $F = \{1,2,\ldots, f\}$ // Problem to classify N patterns into k different classes based on f input feature values.

   Let M be the number of networks to be trained for an ensemble.

   Define AT to be the Alteration Type and AF to be the Alteration Factor.

   Number of patterns in which to alter input values in a training set *is* $P=N*AF$

2. *For* $i=1$ *to* M {

   a. $T_i = T$ // Copy from original training set

   b. *for* $j = 1$ *to* P {

      i. Select a pattern P_a in T_i

      ii. Select a pattern P_b in T (for $AT = 0$)

      // P_b will be in the **same class** as P_a for $AT = 1$ and a **different class** for $AT = 2$.

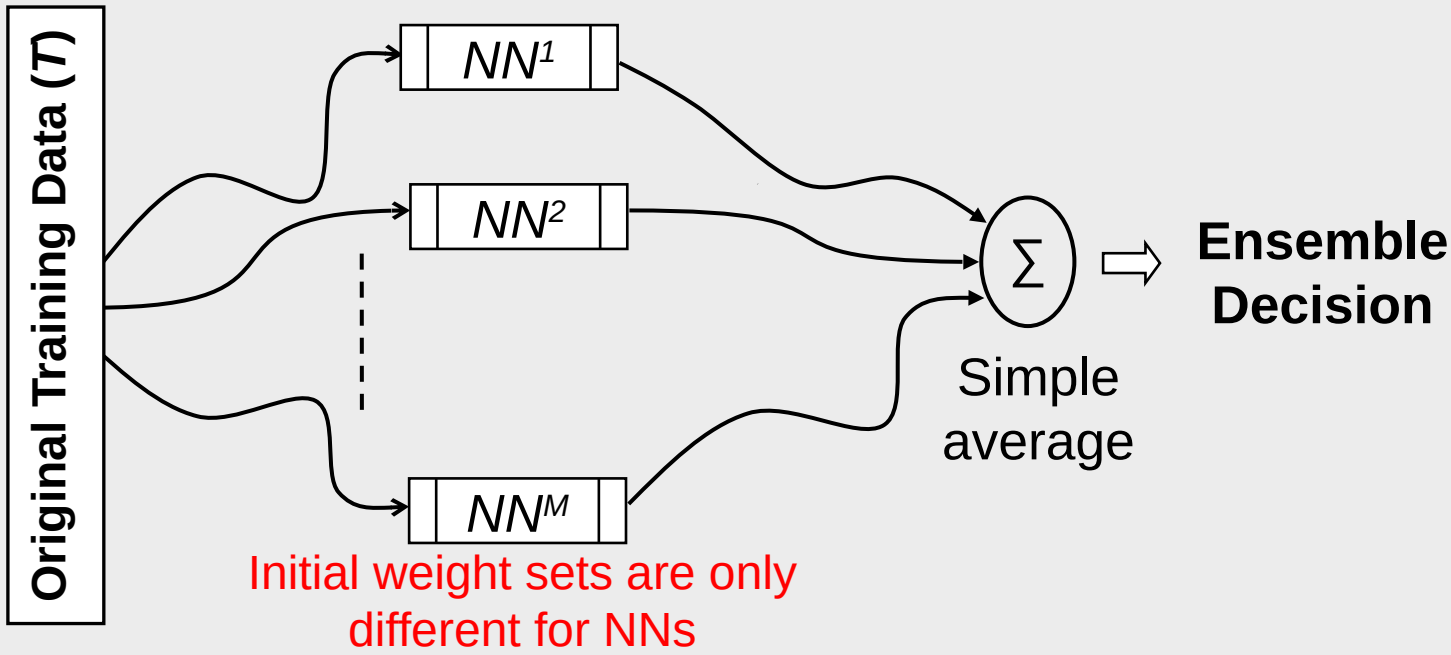
      iii. Alter some input values of P_a using corresponding values from P_b }

   c. Train network NN_i with T_i}

3. Ensemble decision averaging outputs of M networks

Fig. 2. Algorithm for neural network ensembles based on input values alteration (EIVA).

9. Simple NNE(sNNE)



- To evaluate performance of data sampling based NNE methods, sNNE is considered as base line.
- Less diversity due to same data for all NNs; therefore, performance is worse.

Motivation to Comparative Study

- A number of ensemble methods already proposed using various data sampling techniques.
- Proposed methods are investigated on heterogeneous test bed.
- Some methods are proposed for decision trees and empirical result for NN is not available.

Comparative study of the proposed methods on a common ground is necessary to evaluate their effectiveness.

(Eight prominent NNE methods are considered for investigation.)

Experimental Studies

Common Settings:

- For an NNE 20 NNs are considered.
- Each NN is trained for equal 50 / 75 / 100 iteration.
- Learning rate of back propagation was set 0.15.
- 10-fold cross validations was followed for result presentation.

Built in Parameter Settings:

- For DECORATE R_{Size} value was 0.5, 0.75 or 1. Maximum trial NNs was 30.
- In class label switching $S_{fraction}$ was 0.1, 0.2 or 0.3.
- NCL was tested with λ value 0.25, 0.5 or 0.75.

TER Comparison over 50 Indp. Runs

Problem	sNNE 20NNs/NNE	Bagging 20NNs/NNE	AdaBoost 20NNs/NNE	DECORATE (NNs/NNE)	RSM 20NNs/NNE	Switching 20NNs/NNE	Smearing 20NNs/NNE	NCL 20NNs/NNE	EIVA 20NNs/NNE
ACC	0.1522	0.1417	0.1568	**0.14**(8.34)	0.1461	0.1423	0.1414	0.1443	0.1394
BCW	0.0348	0.0322	0.0322	0.0299(6.40)	0.0296	**0.029**	0.0319	0.0313	0.0304
CAR	0.1128	0.0995	**0.0799**	0.1203(2.00)	0.1647	0.118	0.1193	0.1036	-
DBT	0.2379	0.2321	**0.2305**	0.2342(1.14)	0.2318	0.2382	0.2366	0.2308	0.23
GCC	0.2462	0.2424	0.2476	0.2652(1.88)	0.2414	0.2416	0.2482	**0.2402**	0.237
HDC	0.1633	0.1573	0.1653	**0.152**(6.78)	**0.152**	0.1567	0.1567	0.1627	0.152
HPT	0.1547	0.1627	0.172	0.16(1.16)	0.1573	0.1587	0.1547	**0.152**	0.1427
HTR	0.0531	0.0518	**0.0263**	0.0528(1.02)	0.0559	0.056	0.0558	0.0522	0.052
INS	0.1343	0.1297	0.1034	**0.0606**(12.9)	0.1429	0.132	0.1806	0.1366	0.1531
IRP	**0.0267**	0.0293	0.028	**0.0267**(1.00)	0.0293	**0.0267**	**0.0267**	**0.0267**	0.0227
LMP	0.1486	0.1529	0.1729	**0.1371**(4.74)	0.1486	0.14	0.1586	0.15	0.1529
PRM	0.068	0.068	0.072	**0.066**(20.00)	0.068	0.07	0.078	0.068	0.064
SGM	0.07	0.0648	**0.0432**	0.0761(3.14)	0.0681	0.0724	0.0765	0.0677	0.0779
SNR	0.195	0.194	0.181	**0.166**(7.58)	0.20	0.201	0.209	0.195	0.198
SPL	0.1419	0.1556	0.1529	0.1823(2.36)	**0.0873**	0.1527	0.1725	0.1409	0.1357
STL	0.144	0.1379	**0.1358**	0.1525(2.98)	0.1435	0.1419	0.1556	0.145	0.14285
WVF	0.1327	**0.1297**	0.132	0.1308(4.18)	0.1352	0.1339	0.1345	0.1312	0.1286

Result Summary over 30 Problems (Without EIVA)

	sNNE	Bagging	AdaBoost	DECORATE	RSM	Switching	Smearing	NCL
Average TER	**0.1505**	**0.1393**	**0.1380**	**0.1441**	**0.1562**	**0.1512**	**0.1581**	**0.1462**
Best/Worst	**2/3**	**5/1**	**11/6**	**7/2**	**2/7**	**2/3**	**1/9**	**6/0**
NNE Method	**Pair Wise Win/Draw/Loss Summary**							
sNNE	-	23/1/6	19/0/11	17/1/12	12/2/16	11/1/18	8/2/20	20/4/6
Bagging		-	16/1/13	11/0/19	10/2/18	7/1/22	5/0/25	10/1/19
AdaBoost			-	13/0/17	11/0/19	12/0/18	10/0/20	15/0/15
DECORATE				-	11/1/18	10/1/19	5/1/24	16/1/13
RSM					-	17/0/13	12/0/18	22/1/7
Switching						-	6/2/22	19/1/10
Smearing							-	24/1/5

Conclusions from the Comparative Study:

- No one is superior to others for all the problems.
- DECORATE performs better for problems with limited examples, e.g., INS, LPM, PRM. NCL also good for small problems.
- For large sized problems bagging and AdaBoost are the best, e.g., CAR, HRT, WVF. AdaBoost might show very good result for very large problems.
- RSM performs well for sufficient input set, e.g., SPL.

TER Comparison of EIVA over 50 Indp. Runs

Table 2. TER comparisons over five standard 10-fold cross-validation runs. A plus (or minus) sign indicates that the TER is found to be significantly better (or worse) than that of an sNNE by t-test. The bottom of the table contains a pairwise Win/Draw/Lose summary and method Best/Worst summary.

Prob.	sNNE	DECOR.	RSM	CLS	NCL	EIVA
ACC	0.1522	0.14^{+}	0.1461^{+}	0.1423^{+}	0.1443	$\mathbf{0.1394}^{++}$
BCW	0.0348	0.0299^{++}	0.0296^{++}	$\mathbf{0.029}^{++}$	0.0313^{++}	0.0304^{++}
BLD	0.3429	0.3453	0.3494	0.3612^{--}	0.3382	**0.3359**
DBT	0.2379	0.2342	0.2318	0.2382	0.2308^{++}	$\mathbf{0.23}^{++}$
GCC	0.2462	0.2652^{--}	0.2414	0.2416	0.2402^{+}	$\mathbf{0.237}^{++}$
HDC	0.1633	$\mathbf{0.152}^{+}$	$\mathbf{0.152}^{+}$	0.1567	0.1627	$\mathbf{0.152}^{+}$
HPT	0.1547	0.16	0.1573	0.1587	0.152	**0.1427**
HRC	0.1141	0.1112	0.1094	0.1147	0.1106	$\mathbf{0.1053}^{+}$
HSV	0.0489	0.0442	0.0428	$\mathbf{0.0386}^{++}$	0.0396^{+}	0.0414^{++}
HTR	0.0531	0.0528	0.0559^{--}	0.056^{--}	0.0522^{++}	$\mathbf{0.052}^{++}$
INS	0.1343	$\mathbf{0.0606}^{++}$	0.1429	0.132	0.1366	0.1531^{--}
IRP	0.0267	0.0267	0.0293	0.0267	0.0267	**0.0227**
LMP	0.1486	**0.1371**	0.1486	0.14	0.15	0.1529
LNG	0.54	0.4733	0.5267	0.4867	0.5333	**0.4667**
PGB	0.0582	0.0577	0.0669^{-}	0.0576	0.0589	**0.0569**
PRM	0.068	0.066	0.068	0.07	0.068	**0.064**
PST	0.3111	0.3133	0.2955^{++}	0.3155	$\mathbf{0.2889}^{++}$	0.3066
SBN	**0.0541**	0.0559^{-}	0.0562	0.0556	0.0544	0.057
SGM	0.07	0.0761^{-}	0.0681	0.0724^{-}	$\mathbf{0.0677}^{++}$	0.0779
SNR	0.195	$\mathbf{0.166}^{++}$	0.20	0.201	0.195	0.198
SPL	0.1419	0.1823^{--}	$\mathbf{0.0873}^{++}$	0.1527^{--}	0.1409	0.1357^{++}
STL	0.144	0.1525^{--}	0.1435	**0.1419**	0.145	0.14285
WIN	**0.0118**	0.0153	0.0129	0.0165^{-}	**0.0118**	**0.0118**
WVF	0.1327	0.1308	0.1352	0.1339	0.1312	$\mathbf{0.1286}^{++}$
ZOO	0.11	$\mathbf{0.072}^{++}$	0.118^{-}	0.094	0.102	0.1
Avg	**0.1478**	**0.1408**	**0.1446**	**0.1453**	**0.1445**	**0.1416**

NNE Method	Pairwise Win/Draw/Lose Summary					
	sNNE	DEC.	RSM	CLS	NCL	EIVA
sNNE	—	15/1/9	12/2/11	11/1/13	16/4/5	19/1/5
DECOR.		—	11/1/13	9/1/15	13/1/11	17/1/7
RSM			—	12/0/13	17/1/7	17/1/7
CLS				—	14/1/10	17/0/8
NCL					—	17/1/7
Best/Worst	2/5	5/4	2/4	3/8	3/0	14/4

Recent Ensemble Methods

Method / year	Concise method summary	Full citation and DOI
Deep Ensembles 2017	Train networks independently from different initializations and average predictive distributions; a strong accuracy, calibration, and uncertainty baseline.	Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. Advances in Neural Information Processing Systems, 30. DOI: 10.48550/arXiv.1612.01474.
Snapshot Ensembles 2017	Use cyclic learning rates to visit several local minima and save checkpoints from one training run; reduces training cost but retains multi-model inference.	Huang, G., Li, Y., Pleiss, G., Liu, Z., Hopcroft, J. E., & Weinberger, K. Q. (2017). Snapshot ensembles: Train 1, get M for free. International Conference on Learning Representations. DOI: 10.48550/arXiv.1704.00109.
ADP 2019	Regularize ensemble entropy and diversity of non-maximal predictions; reduces adversarial transfer while preserving clean accuracy.	Pang, T., Xu, K., Du, C., Chen, N., & Zhu, J. (2019). Improving adversarial robustness via promoting ensemble diversity. Proceedings of Machine Learning Research, 97, 4970–4979. DOI: 10.48550/arXiv.1901.08846.
BatchEnsemble 2020	Represent each member with shared weights plus member-specific rank-one factors; lowers memory and computation while supporting parallel training.	Wen, Y., Tran, D., & Ba, J. (2020). BatchEnsemble: An alternative approach to efficient ensemble and lifelong learning. International Conference on Learning Representations. DOI: 10.48550/arXiv.2002.06715.
MIMO 2021	Train multiple independent input-output subnetworks inside one model; obtains ensemble-like robustness and calibration in a single forward pass.	Havasi, M., Jenatton, R., Fort, S., Liu, J. Z., Snoek, J., Lakshminarayanan, B., Dai, A. M., & Tran, D. (2021). Training independent subnetworks for robust prediction. International Conference on Learning Representations. DOI: 10.48550/arXiv.2010.06610.

Recent Ensemble Methods(Cont.)

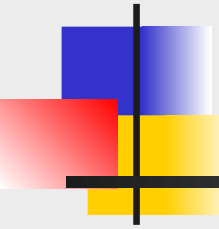
Method / year	Concise method summary	Full citation and DOI
DVERGE 2020	Distill and separate non-robust features so member vulnerabilities differ; improves robustness to transfer attacks as ensemble size grows.	Yang, H., Zhang, J., Dong, H., Inkawhich, N., Gardner, A., Touchet, A., Wilkes, W., Berry, H., & Li, H. (2020). DVERGE: Diversifying vulnerabilities for enhanced robust generation of ensembles. Advances in Neural Information Processing Systems, 33. DOI: 10.48550/arXiv.2009.14720.
Global NCL 2022	Optimize the final ensemble's global objective instead of only the component errors; supplies a unified negative-correlation framework.	Perales-Gonzalez, C., Fernandez-Navarro, F., Carbonero-Ruz, M., & Perez-Rodriguez, J. (2022). Global negative correlation learning: A unified framework for global optimization of ensemble models. IEEE Transactions on Neural Networks and Learning Systems, 33(8), 4031–4042. DOI: 10.1109/TNNLS.2021.3055734.
DNCC 2024	Decompose classification loss into individual accuracy and correlation with the ensemble; explicitly controls the accuracy-diversity trade-off.	Zhang, L., Hou, Q., Liu, Y., Bian, J.-W., Xu, X., Zhou, J. T., & Zhu, C. (2024). Deep negative correlation classification. Machine Learning, 113, 7223–7241. DOI: 10.1007/s10994-024-06604-0.
Dynamic NCL 2025	Update the accuracy-diversity weighting parameter during learning according to the relative losses; adapts the trade-off to model and task difficulty.	Takama, H., Tsuge, S., & Hasegawa, T. (2025). Dynamic negative correlation learning in deep ensemble learning. Proceedings of the 39th Annual Conference of the Japanese Society for Artificial Intelligence, 1S3-GS-2-04. DOI: 10.11517/pjsai.JSAI2025.0_1S3GS204.
Bayesian DE 2026 preprint	Combine independently trained predictors through analytic Bayesian linear regression; provides interpretable weights and calibrated posterior uncertainty.	Fard, S. A. D., Maros, M., & Lee, J. (2026). Efficient Bayesian deep ensembles via analytic predictive inference. arXiv preprint arXiv:2607.06776. DOI: 10.48550/arXiv.2607.06776.

Recommended Survey:

Y. Yang, H. Lv, and N. Chen, 'A Survey on Ensemble Learning under the Era of Deep Learning,' Artificial Intelligence Review, 56, 5545-5589, 2023. DOI: 10.1007/s10462-022-10283-5

Conclusions

- Basic introduction about NNs and NNE is presented.
- Comparative study of prominent existing methods is given and identified effectiveness of the methods.
- Given an Outline for better NNE construction.



Classification with Decision Trees

Prof. Dr. Md. Aminul Haque Akhand
Dept. of CSE, KUET

Reference/Source Slides: Maria Simi Michael Crawford

Inductive Inference with Decision Trees

- *Decision Trees (DT)* is one of the most widely used and practical methods of *inductive inference*
- Features:
  - Method for approximating *discrete-valued* functions (including boolean)
  - Learned functions are represented as *decision trees* (or *if-then-else* rules)
  - Expressive hypotheses space, including disjunction
  - Robust to noisy data

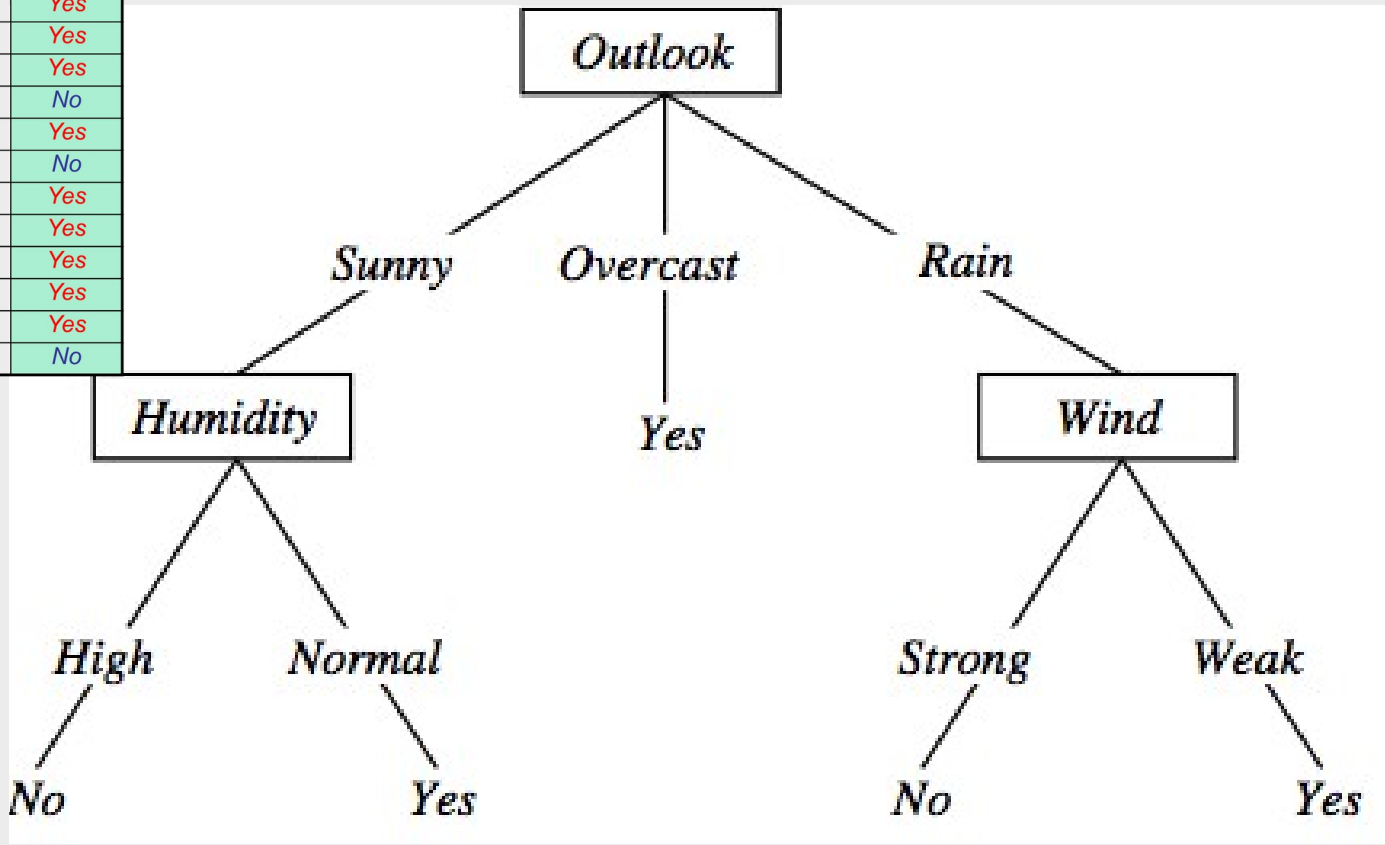
Inductive Inference: A type of reasoning where a general conclusion is drawn from specific observations or evidence

Play Tennis based on Weather Condition

Day	Outlook	Temp	Humidity	Wind	Tennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

DT Representation (Play Tennis)

Day	Outlook	Temp	Humidity	Wind	Tennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



D2 Sample from Table:

⟨Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Strong⟩ *No*

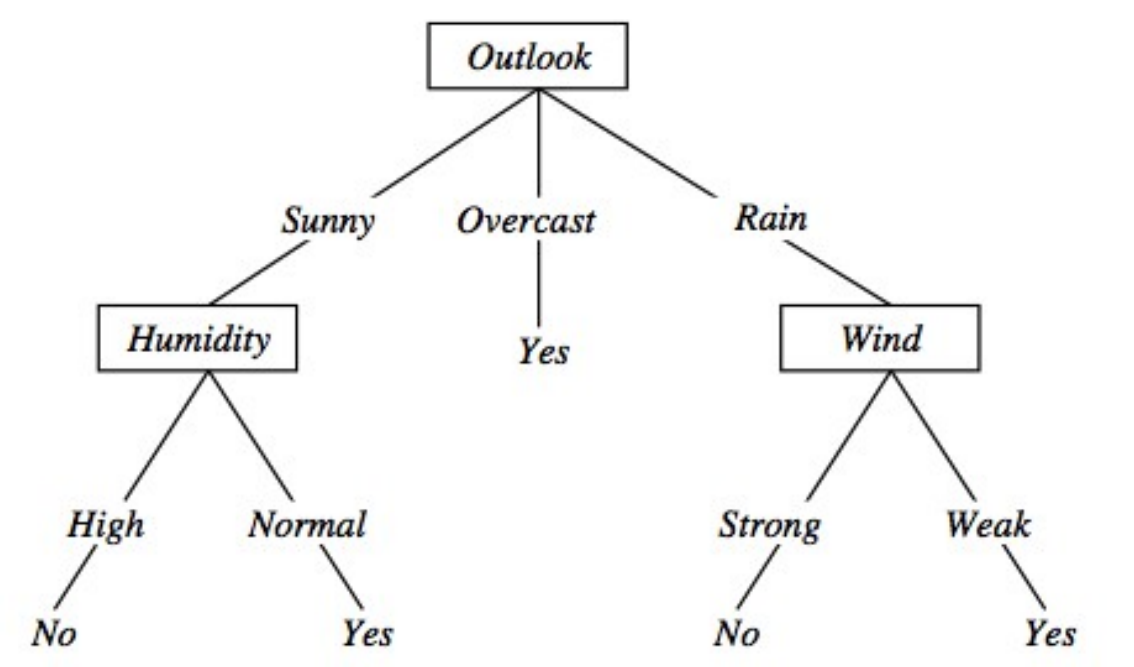
DT Expressivity

DTs represent a disjunction of conjunctions on constraints on the value of attributes:

($Outlook = Sunny \wedge Humidity = Normal$) $\vee$

($Outlook = Overcast$) $\vee$

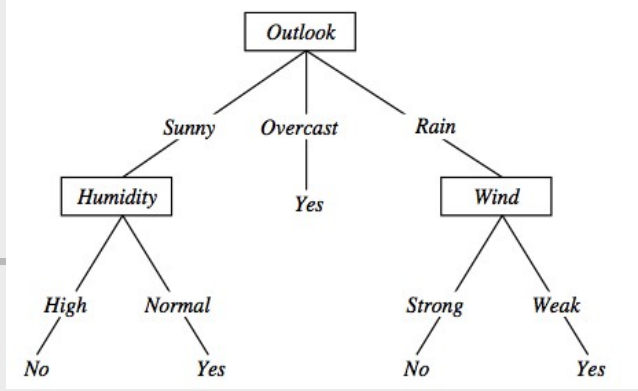
($Outlook = Rain \wedge Wind = Weak$)



When to use Decision Trees

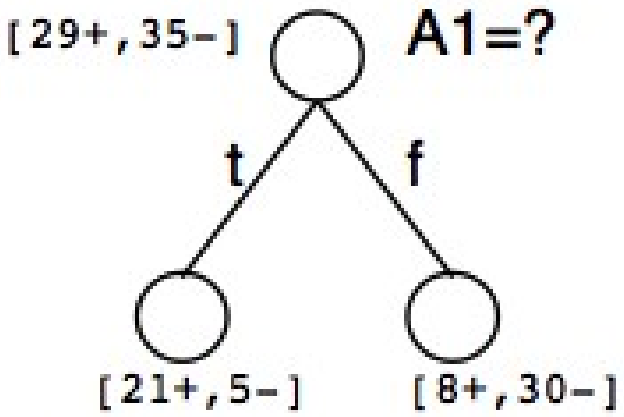
- **Problem characteristics:**
  - Instances can be described by attribute value pairs
  - Target function is discrete valued
  - Disjunctive hypothesis may be required
  - Possibly noisy training data samples
    - Robust to errors in training data
    - Missing attribute values
- **Different classification problems:**
  - Medical diagnosis
  - Credit risk analysis
  - ----------

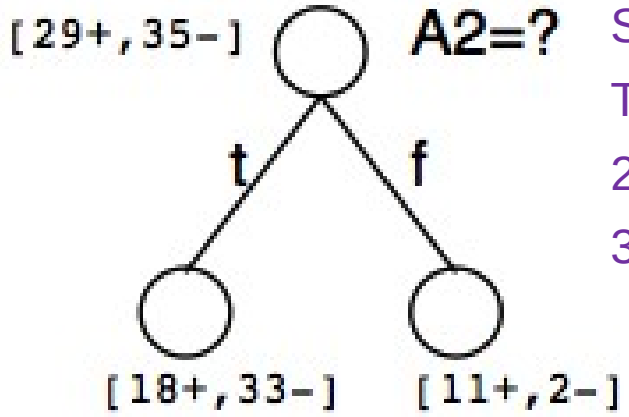
Top-Down Induction of DTs



- ID3 (Quinlan, 1986) is a basic algorithm for learning DT's
- Given a training set of examples, the algorithms for building DT performs search in the space of DTs
- The construction of the tree is top-down. The algorithm is greedy.
- The fundamental question is "which attribute should be tested next? Which question gives us more information?"
- Select the *best* attribute
- A descendent node is then created for each possible value of this attribute and examples are partitioned according to this value
- The process is repeated for each successor node until all the examples are classified correctly or there are no attributes left

Which Attribute is the best Classifier?





Suppose

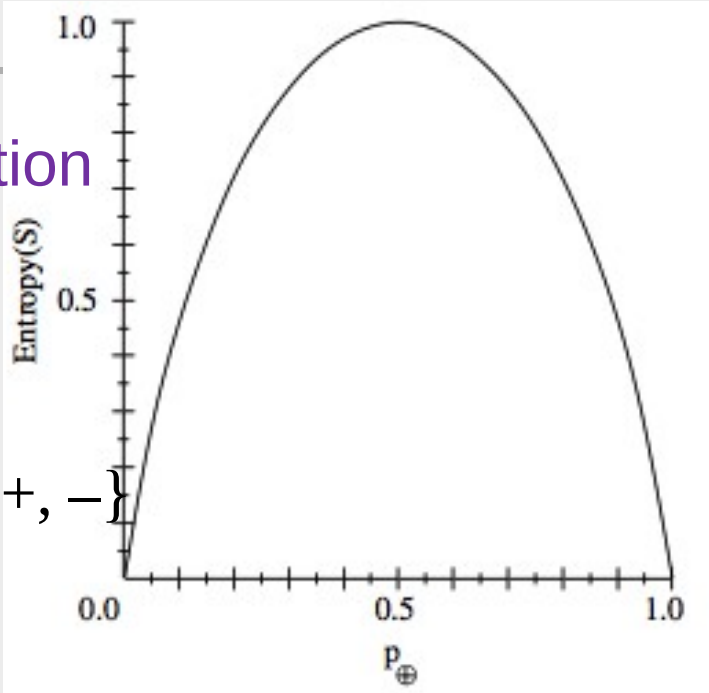
Total instance : 64

29 for Class 1(+)

35 for Class 2 (-)

- A statistical property called *information gain*, measures how well a given attribute separates the training examples
- Information gain uses the notion of *entropy*, commonly used in information theory
- *Information gain = expected reduction of entropy*

Entropy in General



Entropy measures the amount of information in a random variable

- For binary classification
  [two-valued random variable]

$$H(X) = -p_+ \log_2 p_+ - p_- \log_2 p_- \qquad X = \{+, -\}$$

- For classification in *c* classes

$$H(X) = -\sum_{i=1}^{c} p_i \log_2 p_i = \sum_{i=1}^{c} p_i \log_2 1/p_i \qquad X = \{i, \ldots, c\}$$

Example: rolling a die with 8 equally probable sides

$$H(X) = -\sum_{8} 1/8 \log_2 1/8 = -\log_2 1/8 = \log_2 8 = 3$$

Entropy in Binary Classification

Entropy measures the impurity of a collection of examples. It depends from the distribution of the random variable *p*.

- *S* is a collection of training examples
- p_+ the proportion of positive examples in *S*
- p_- the proportion of negative examples in *S*

$$Entropy\ (S) \equiv -p_+ \log_2 p_+ - p_- \log_2 p_- \qquad [0 \log_2 0 = 0]$$

Case 1: $Entropy\ ([14+, 0-]) = -14/14 \log_2 (14/14) - 0 \log_2 (0) = 0$

Case 2: $Entropy\ ([9+, 5-]) = -9/14 \log_2 (9/14) - 5/14 \log_2 (5/14) = 0.94$

Case 3: $Entropy\ ([7+, 7-]) = -7/14 \log_2 (7/14) - 7/14 \log_2 (7/14) =$

$$= 1/2 + 1/2 = 1 \qquad [\log_2 1/2 = -1]$$

Note: the log of a number < 1 is negative $0 < p < 1$ $0 < Entropy < 1$

Information Gain as Entropy Reduction

- *Information gain* is the *expected* reduction in entropy caused by partitioning the examples on an attribute.
- The higher the information gain the more effective the attribute in classifying training data.
- Expected reduction in entropy knowing *A*

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|Sv|}{|S|} Entropy(Sv)$$

$Values(A)$ possible values for A

Sv subset of S for which A has value v

Example: Expected Information Gain

- Let
  - $Values(Wind) = \{Weak, Strong\}$
  - $S = [9+, 5-]$
  - $S_{Weak} = [6+, 2-]$
  - $S_{Strong} = [3+, 3-]$

Day	Outlook	Temp	Humidity	Wind	Tennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- Information gain due to knowing *Wind*:

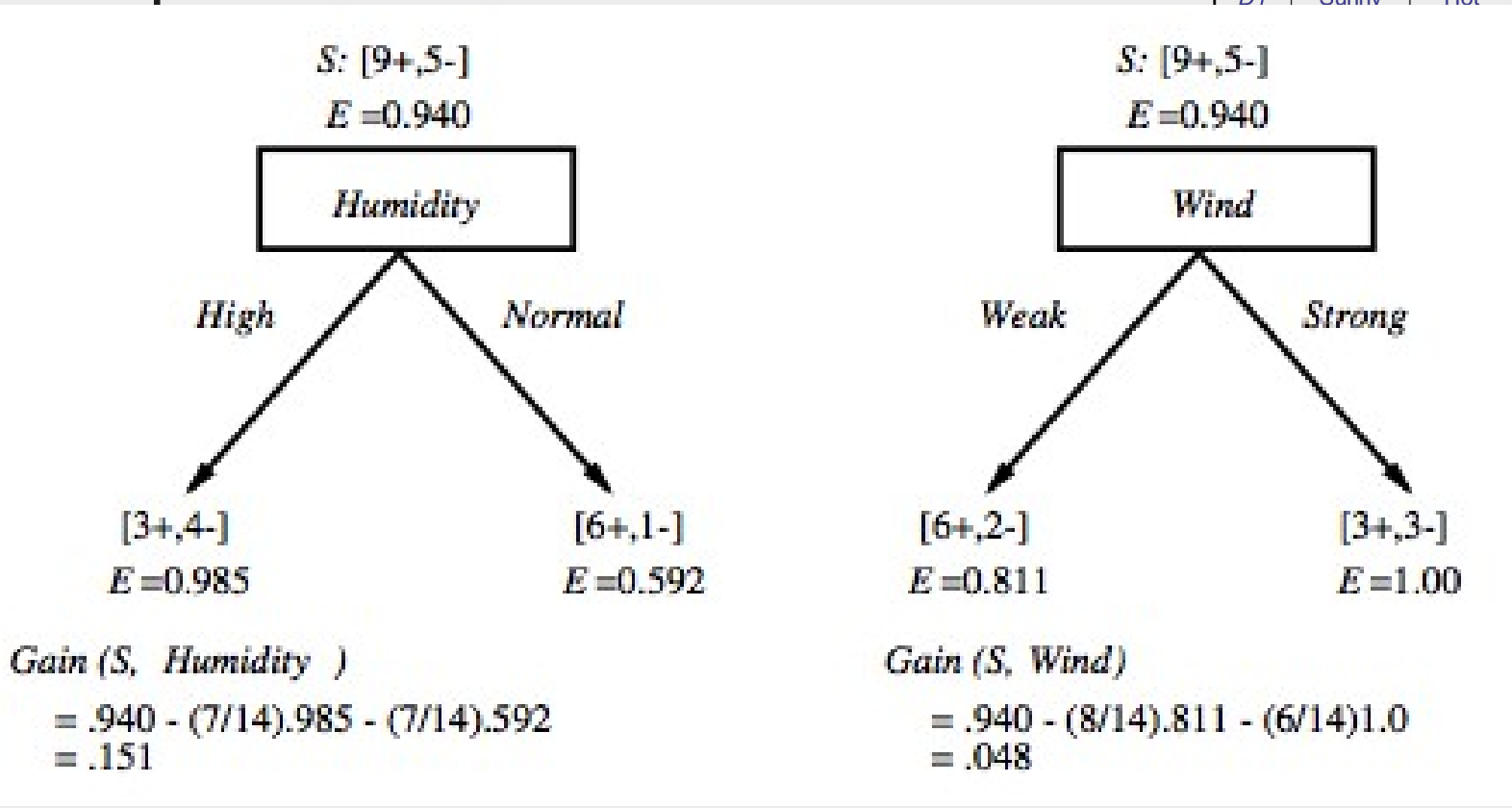
$$Gain(S, Wind) = Entropy(S) - 8/14\ Entropy(S_{Weak}) - 6/14\ Entropy(S_{Strong})$$

$$= 0.94 - 8/14 \times 0.811 - 6/14 \times 1.00$$

$$= 0.048$$

Which Attribute is the best Classifier?

Day	Outlook	Temp	Humidity	Wind	Tennis?
D1	Sunny	Hot	High	Weak	No
			High	Strong	No
			High	Weak	Yes
			High	Weak	Yes
			Normal	Weak	Yes
			Normal	Strong	No
			Normal	Strong	Yes
			High	Weak	No
			Normal	Weak	Yes
			Normal	Weak	Yes
			Normal	Strong	Yes
			High	Strong	Yes
			Normal	Weak	Yes
			High	Strong	No



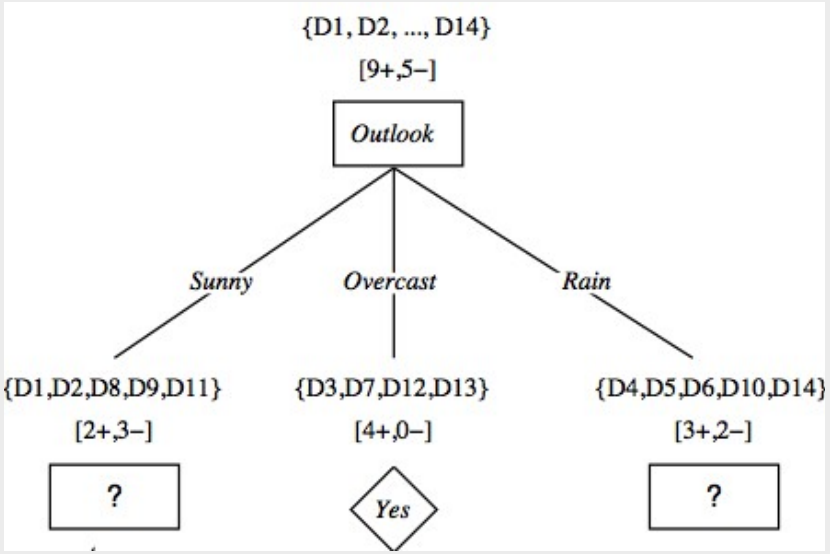
Gain (S, Humidity)
= .940 - (7/14).985 - (7/14).592
= .151

Gain (S, Wind)
= .940 - (8/14).811 - (6/14)1.0
= .048

First Step: Root Attribute Selection

Which attribute should be tested at the root?

Gain(*S*, *Outlook*) = 0.246
Gain(*S*, *Humidity*) = 0.151
Gain(*S*, *Wind*) = 0.084
Gain(*S*, *Temperature*) = 0.029

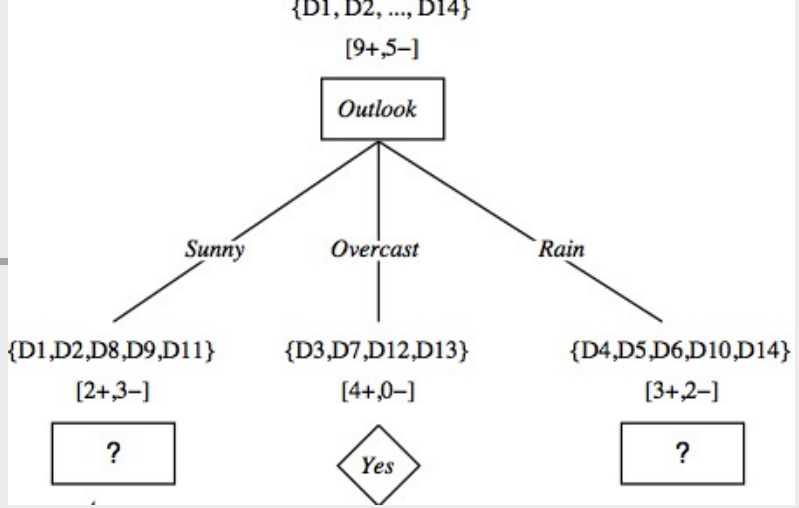


Outlook provides the best prediction for the target

Lets grow the tree:

- Add to the tree a successor for each possible value of *Outlook*
- Partition the training samples according to the value of *Outlook*

Second Step



Outlook={Sunny, Overcast, Rain}

- Working on *Outlook=Sunny* node:

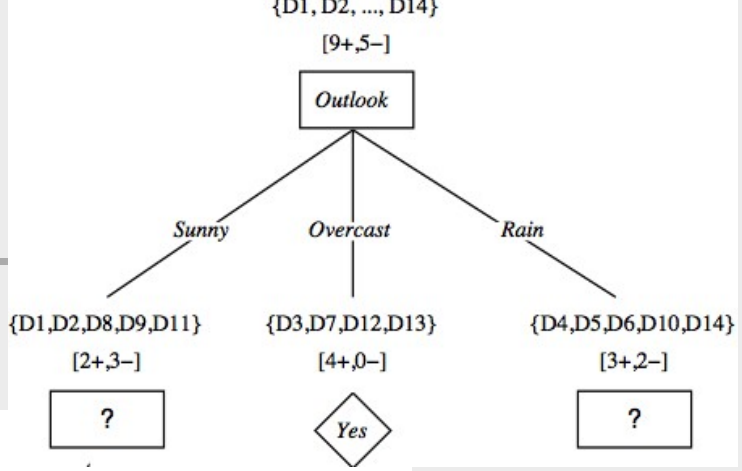
  $Gain(S_{Sunny}, Humidity) = 0.970 - 3/5 \times 0.0 - 2/5 \times 0.0 = 0.970$

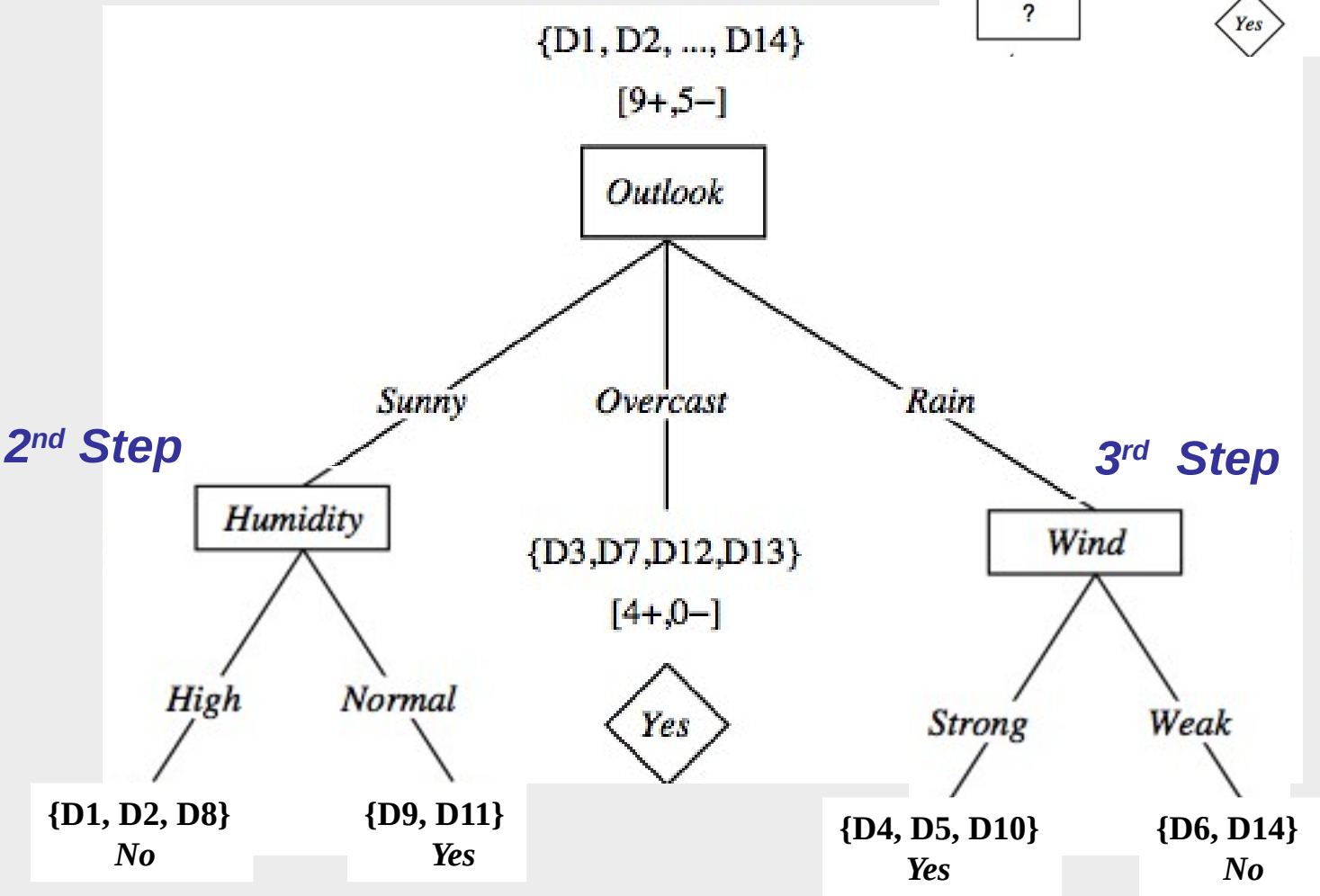
  $Gain(S_{Sunny}, Wind) = 0.970 - 2/5 \times 1.0 - 3/5 \times 0.918 = 0\ .019$

  $Gain(S_{Sunny}, Temp.) = 0.970 - 2/5 \times 0.0 - 2/5 \times 1.0 - 1/5 \times 0.0 = 0.570$

- *Humidity* provides the best prediction for the target
- Lets grow the tree:
  - add to the tree a successor for each possible value of *Humidity*
  - partition the training samples according to the value of *Humidity*

Second and Third Steps





2nd Step

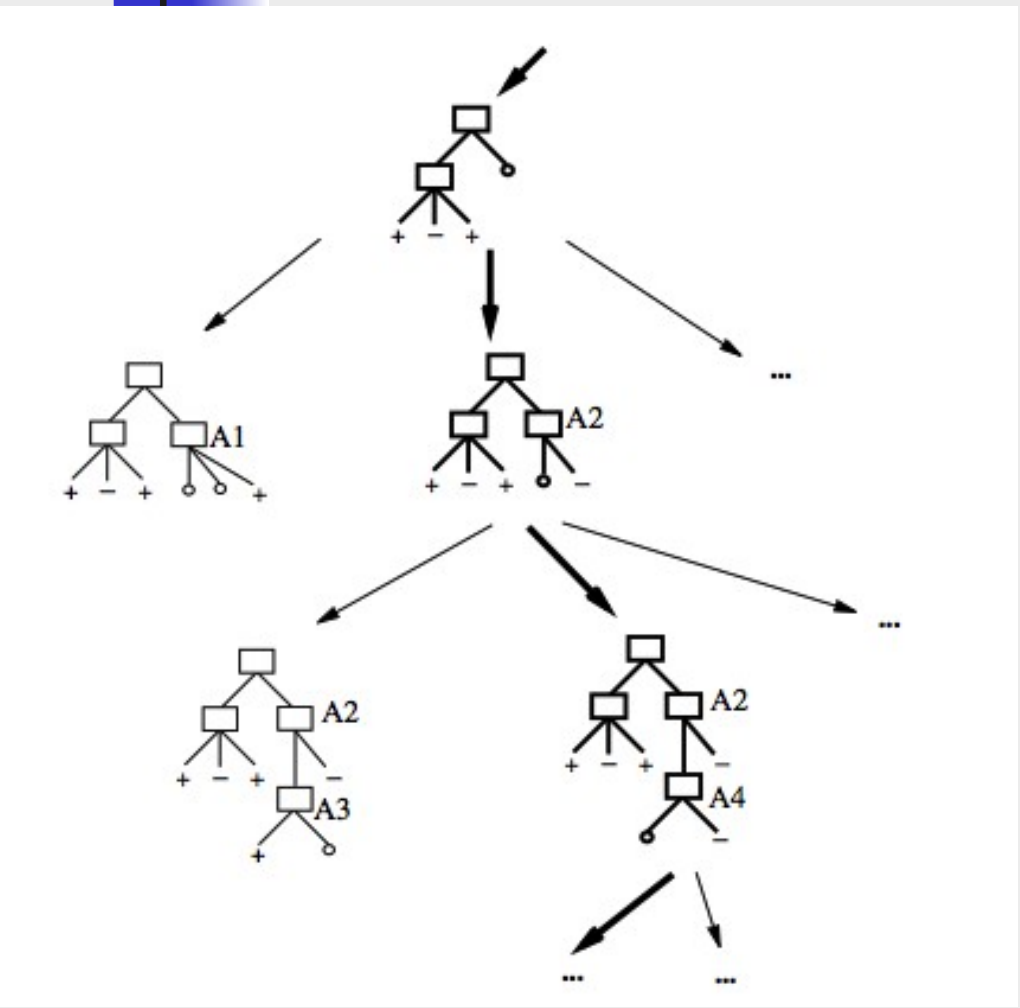
3rd Step

ID3: Algorithm

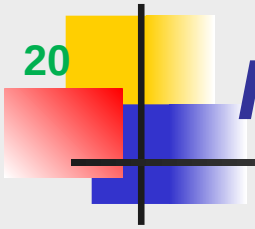
ID3(*X, T, Attrs*) *X*: training examples:
T: target attribute (e.g., *PlayTennis*),
Attrs: other attributes, initially all attributes

Create *Root* node
If all X's are +, *return Root* with class +
If all X's are –, *return Root* with class –
If Attrs is empty *return Root* with class most common value of *T* in *X*
else
  A ← best attribute; decision attribute for Root ← *A*
  For each possible value v_i of *A:*
   - add a new branch below *Root,* for test $A = v_i$
   - X_i ← subset of *X* with $A = v_i$
   - *If* X_i is empty *then* add a new leaf with class the most common value of *T* in *X*
     else add the subtree generated by ID3(X_i, *T, Attrs* - {*A*})
return Root

Search Space in DT Learning



- The search space is made by partial decision trees
- The algorithm is *hill-climbing*
- The evaluation function is *information gain*
- The hypotheses space is complete (represents all discrete-valued functions)
- The search maintains a single current hypothesis
- No backtracking; no guarantee of optimality
- It uses all the available examples (not incremental)
- May terminate earlier, accepting noisy classes



Inductive Bias in DT Learning

What is the inductive bias of DT learning?

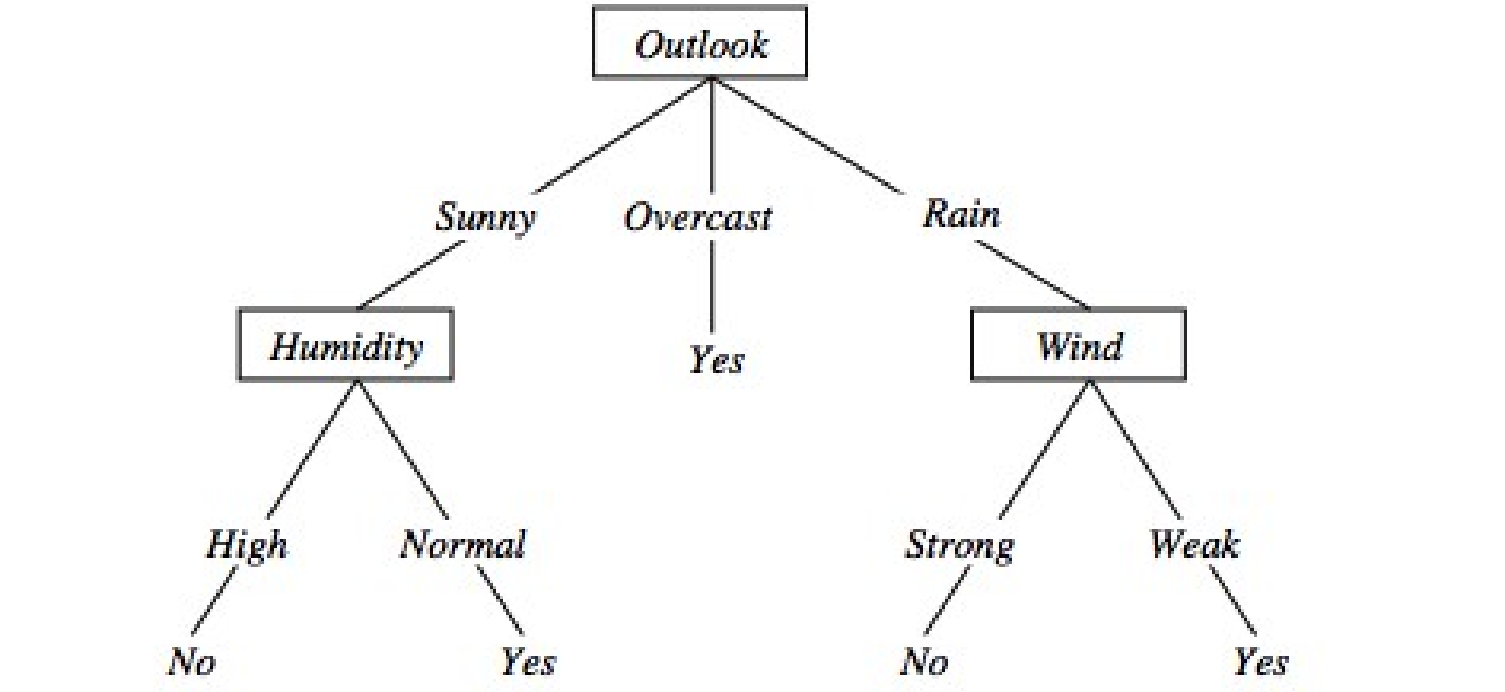
1. *Shorter trees are preferred over longer trees*

   This is the bias exhibited by a simple Breadth First Algorithm generating all DT's and selecting the shorter one

2. *Prefer trees that place high information gain attributes close to the root*

Note: DT's are not limited in representing all possible functions

Converting to Rules



Rule 1: (*Outlook=Sunny*)^(*Humidity=High*) ⇒ (*PlayTennis=No*)

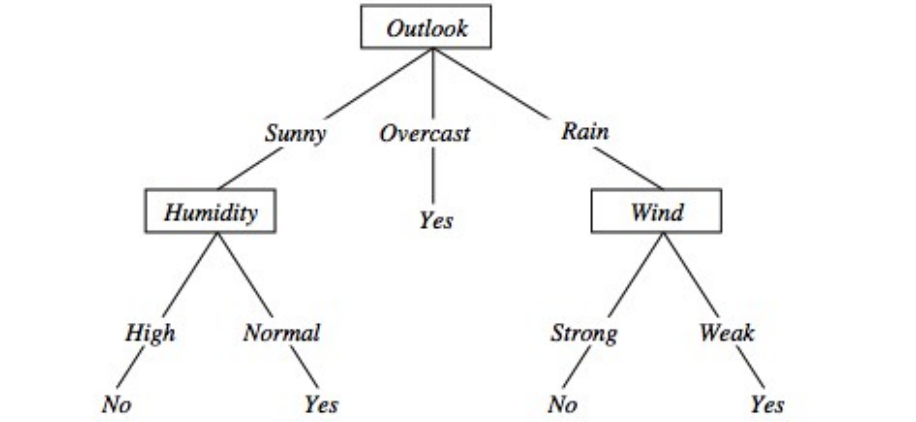
Rule 2: (*Outlook=Sunny*)^(*Humidity=Normal*) ⇒ (*PlayTennis=Yes*)

Rule 3: ??????

Rule 4: ??????

Rule 5: ??????

Why Converting to Rules?



- Each distinct path produces a different rule: a condition removal may be based on a local (contextual) criterion. Node pruning is global and affects all the rules

- In rule form, tests are not ordered and there is no book-keeping involved when conditions (nodes) are removed

- Converting to rules improves readability for humans

Issues in DTs Learning

- Overfitting
  - Reduced Error Pruning (REP)
  - Rule post-pruning (RPP)

- Extensions
  - Continuous valued attributes
  - Alternative measures for selecting attributes
  - Handling training examples with missing attribute values
  - Handling attributes with different costs
  - Improving computational efficiency
  - Most of these improvements in C4.5 (Quinlan, 1993)

Key Improvements in C4.5 over ID3

Feature	ID3	C4.5
Splitting criterion	Information Gain	Gain Ratio
Attribute types	Categorical only	Categorical + Continuous
Missing values	Not handled	Handled with probabilities
Pruning	Not available	Post-pruning supported
Output	Decision Trees	Trees + IF-THEN Rules
Overfitting	High risk	Reduced via pruning

The attribute with the maximum Gain Ratio is selected as the splitting attribute in C4.5

Information Gain (IG) vs. Gain Ratio(GR)

Information Gain (IG): Measures the reduction in entropy (uncertainty/disorder) when splitting the dataset based on an attribute.

$$IG(S,A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

Sv subset of *S* for which *attribute A* has value *v*

Bias issue:

- Information Gain favors attributes with many distinct values.
- Example: An attribute like *Student_ID* would perfectly split the dataset (unique value per record), giving high IG but useless for prediction.

Gain Ratio (GR): An improved measure that adjusts IG by considering how broadly and uniformly an attribute splits the data.

$$GR(A) = \frac{IG(S,A)}{SplitInformation(A)}$$

$$SplitInformation(A) = -\sum_{v \in Values(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

- This measures how "spread out" the attribute values are.
- If an attribute has many distinct values, SplitInfo will be large, thus reducing GR

Benefit:

- Removes the bias of IG toward attributes with many values.
- Chooses attributes that not only reduce entropy but also produce balanced, meaningful splits.

Comparing Attribute Selection Measures

Three measures, in general, return good results but

- Information Gain(IG):
  - biased towards multivalued attributes
- Gain Ratio(GR):
  - tends to prefer unbalanced splits in which one partition is much smaller than the others
- Gini index (GI):
  - biased to multivalued attributes
  - has difficulty when # of classes is large
  - tends to favor tests that result in equal-sized partitions and purity in both partitions

- Gini index is a measure of impurity or heterogeneity used by the CART (Classification and Regression Trees).
- Other Attribute Selection Measures: Chi-Square Statistic, MDL (Minimal Description Length), etc.

Overfitting in DTs

- Building DTs that “adapt too much” to the training examples may lead to “overfitting”.

- Consider error of hypothesis *h* over
  - training data: $error_D(h)$ — empirical error
  - entire distribution *X* of data: $error_X(h)$ — expected error

- Hypothesis *h* *overfits* training data if there is an alternative hypothesis $h' \in H$ such that

$$error_D(h) < error_D(h') \quad \text{and} \quad error_X(h') < error_X(h)$$

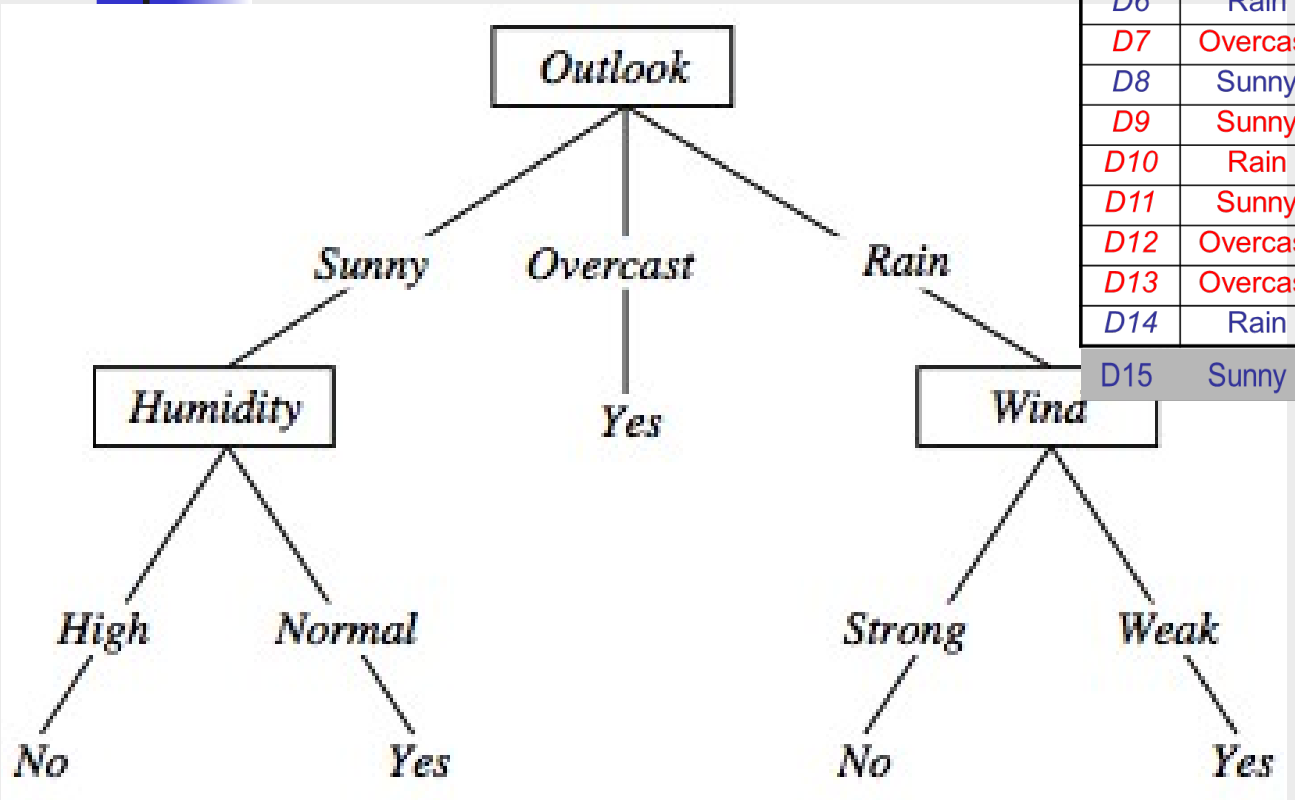
i.e. *h’* behaves better over unseen data

Example

Day	Outlook	Temp	Humidity	Wind	Tennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No
D15	Sunny	Hot	Normal	Strong	No

Overfitting in DTs

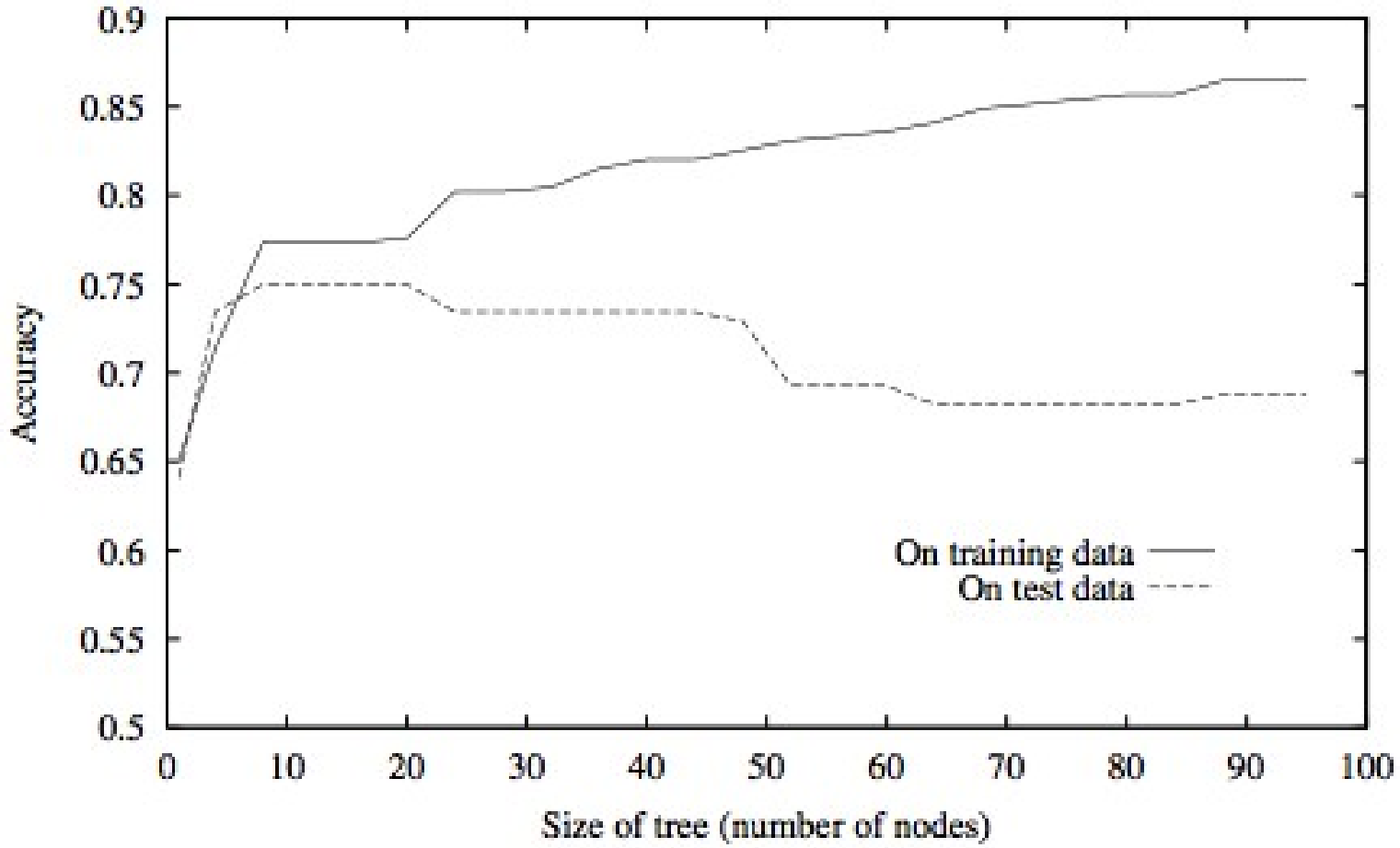
Day	Outlook	Temp	Humidity	Wind	Tennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No
D15	Sunny	Hot	Normal	Strong	No



⟨*Outlook=Sunny, Temp=Hot, Humidity=Normal, Wind=Strong* ⟩ *PlayTennis=No*

New noisy example causes splitting of second leaf node.

Overfitting in DTs



Avoid Overfitting in DTs

- Two Strategies:
  1. Stop growing the tree earlier, before perfect classification
  2. Allow the tree to *overfit* the data, and then *post-prune* the tree

- Training and validation set
  - Split the training in two parts (training and validation) and use validation to assess the utility of *post-pruning*
    - Reduced error pruning (REP)
    - Rule Post-Pruning (RPP)

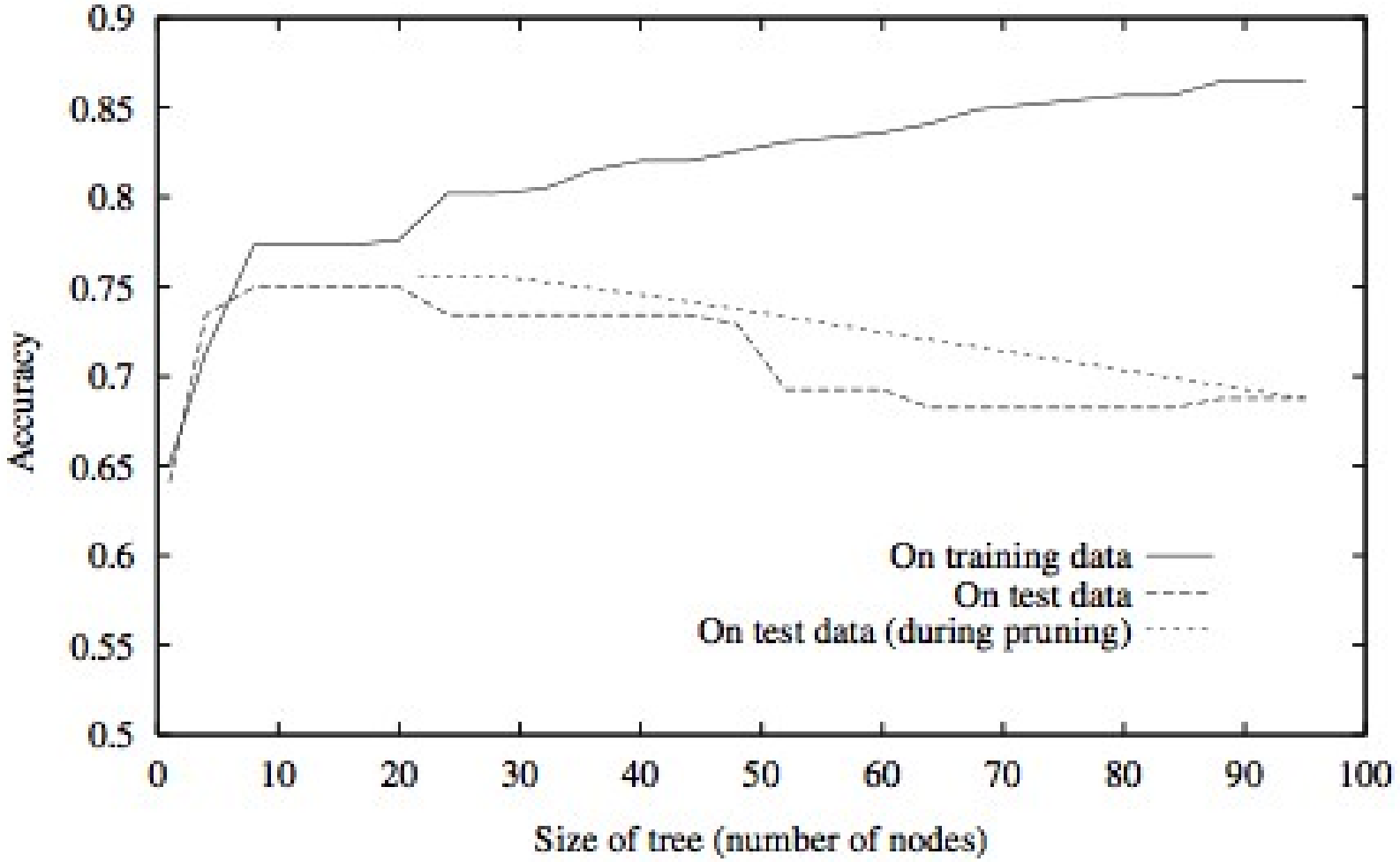
- Other approaches

  Use a statistical test to estimate effect of expanding or pruning
  Minimum description length principle: uses a measure of complexity of encoding the DT and the examples, and halt growing the tree when this encoding size is minimal

Reduced-Error Pruning (Quinlan 1987)

- Each node is a candidate for pruning
- *Pruning* consists in removing a subtree rooted in a node: the node becomes a leaf and is assigned the most common classification
- Nodes are removed only if the resulting tree performs no worse on the validation set.
- Nodes are pruned iteratively: at each iteration the node whose removal most increases accuracy on the validation set is pruned.
- Pruning stops when removing any further nodes does not lead to an improvement in accuracy on the validation set

Effect of Reduced Error Pruning



Rule Post-Pruning (RPP)

1. Create the decision tree from the training set
2. Convert the tree into an equivalent set of rules
   - Each path corresponds to a rule
   - Each node along a path corresponds to a pre-condition
   - Each leaf classification to the post-condition (i.e., Conclusion)
3. Prune (generalize) each rule by removing those preconditions whose removal improves accuracy …
   - … over validation set
   - … over training with a pessimistic, statistically inspired, measure
4. Sort the rules in estimated order of accuracy, and consider them in sequence when classifying new instances

Dealing with Continuous-valued Attributes

- So far discrete values for attributes and for outcome.
- Given a continuous-valued attribute A, dynamically create a new attribute A_c

$$A_c = \text{True } if\ A < c, \text{ False } otherwise$$

- How to determine threshold value c ?
- Example. *Temperature* in the *PlayTennis* example

Sort the examples according to *Temperature*

Temperature	40	48	54	60	72	80	85	90
PlayTennis	No	No		Yes	Yes	Yes		No

- Determine candidate thresholds by averaging consecutive values where there is a change in classification: (48+60)/2=54 and (80+90)/2=85
- Evaluate candidate thresholds (attributes) according to information gain. The best is $Temperature_{>54}$.The new attribute competes with the other ones

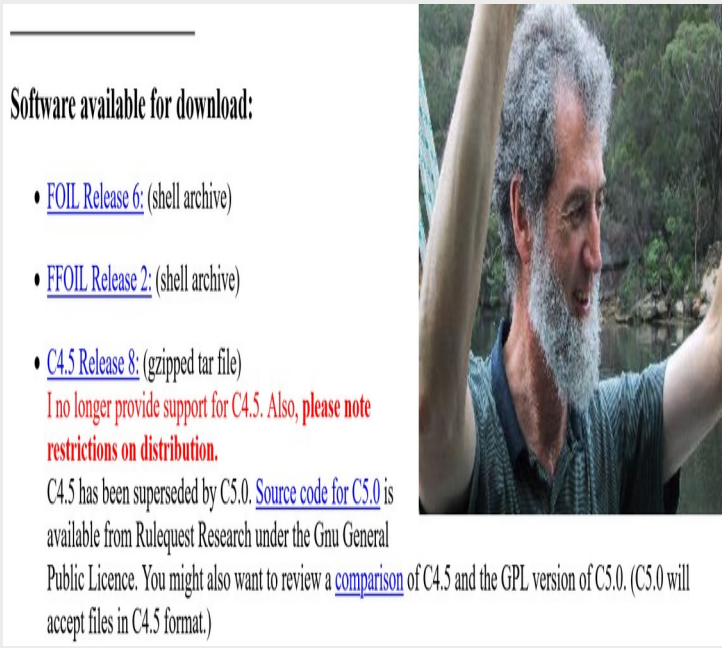
Handling Incomplete Training Data

- How to cope with the problem that the value of some attribute may be missing?
  - *Example*: Blood-Test-Result in a medical diagnosis problem
- The strategy: use other examples to guess attribute
  - Assign the value that is most common among the training examples at the node
  - Assign a probability to each value, based on frequencies, and assign values to missing attribute, according to this probability distribution
- Missing values in new instances to be classified are treated accordingly, and the most probable classification is chosen (C4.5)

Reference and Software

Important Reference: Machine Learning, Tom Mitchell, Mc Graw-Hill International Editions, 1997 (Cap 3).

- In R:
  - Packages tree and rpart
- C4.5:
  - http://www.cse.unwe.edu.au/~quinlan
  - https://www.rulequest.com/Personal/
- Weka
  - http://www.cs.waikato.ac.nz/ml/weka



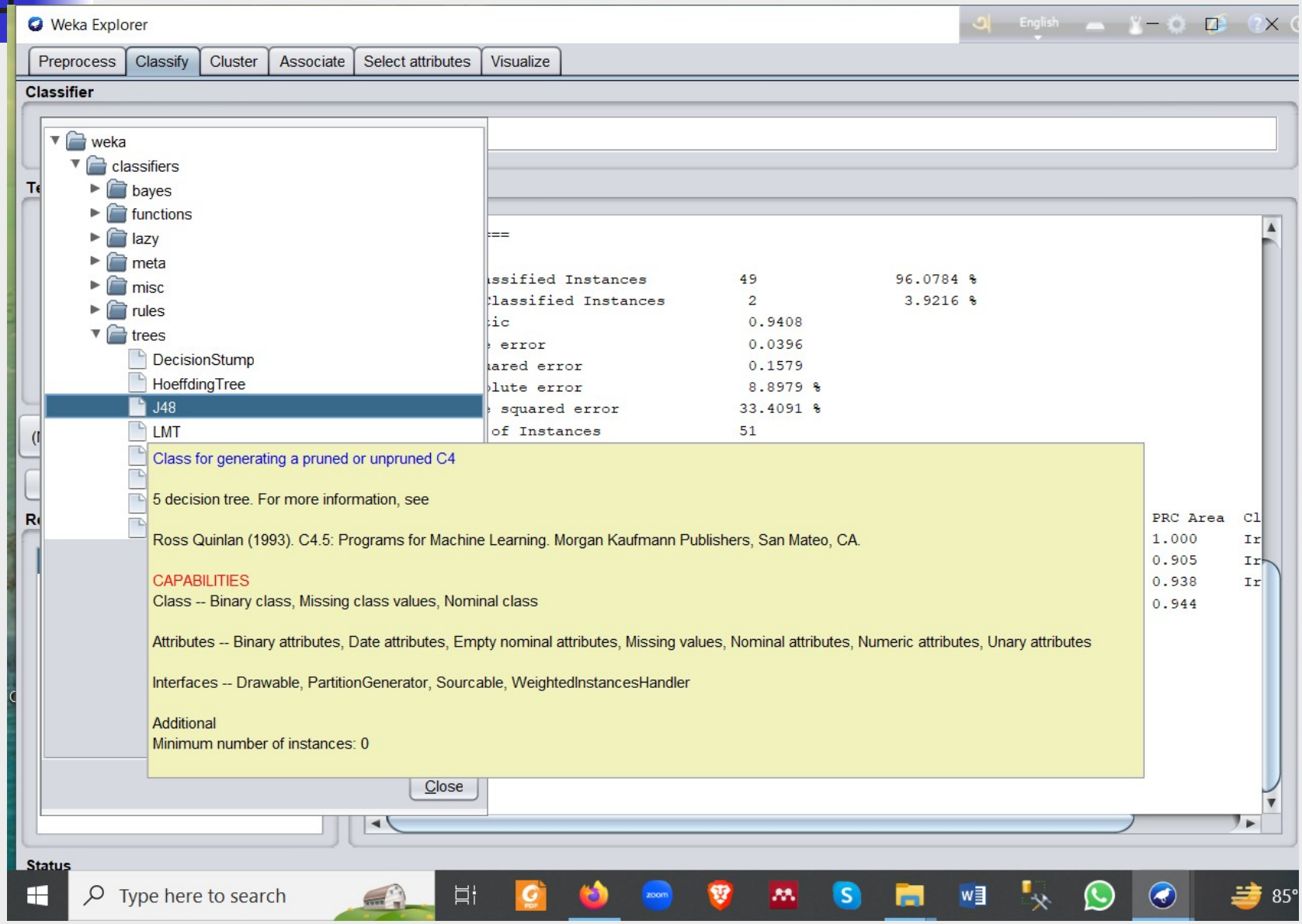
Software available for download:

- FOIL Release 6: (shell archive)
- FFOIL Release 2: (shell archive)
- C4.5 Release 8: (gzipped tar file)

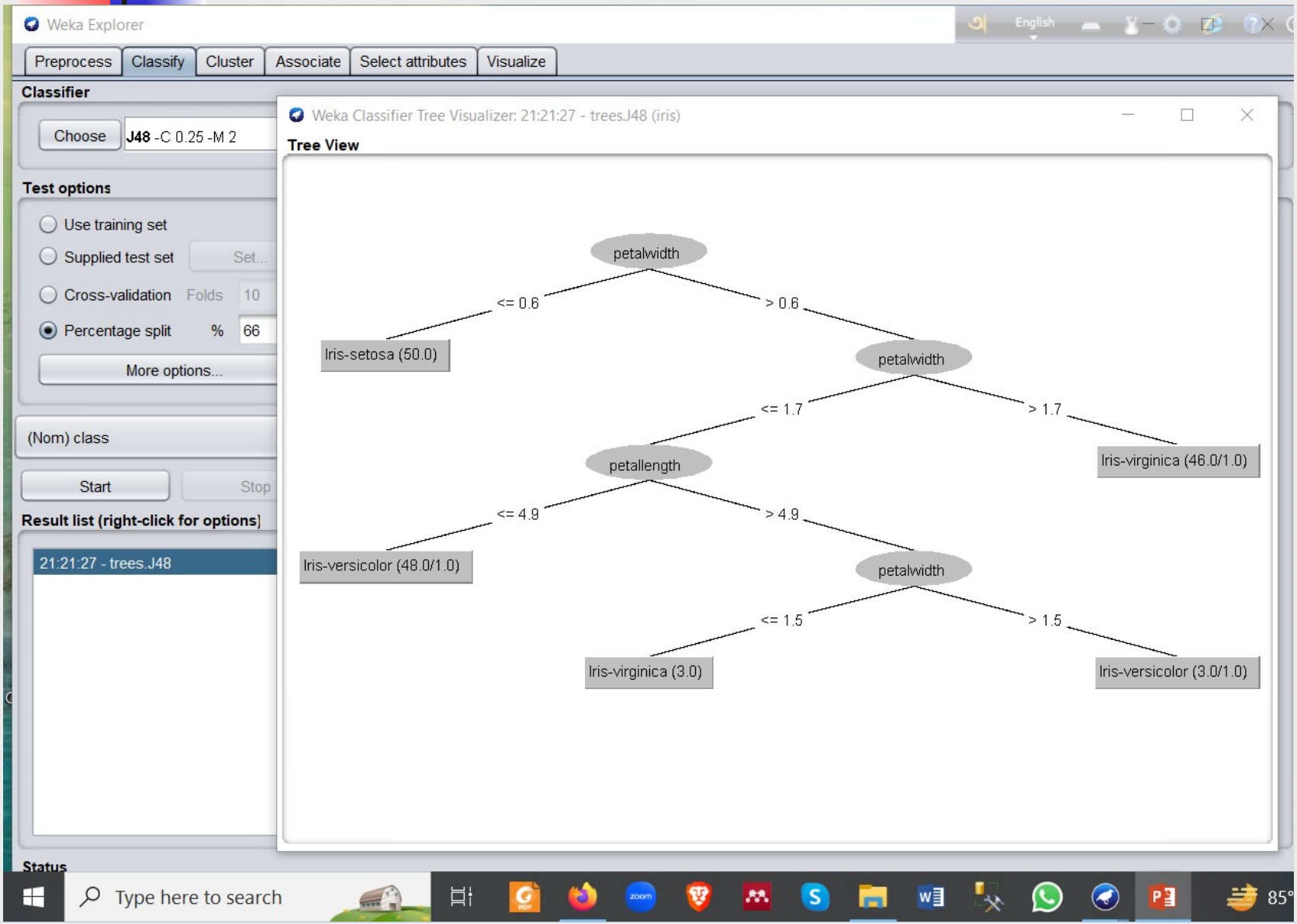
I no longer provide support for C4.5. Also, **please note restrictions on distribution.**

C4.5 has been superseded by C5.0. Source code for C5.0 is available from Rulequest Research under the Gnu General Public Licence. You might also want to review a comparison of C4.5 and the GPL version of C5.0. (C5.0 will accept files in C4.5 format.)

DT for Iris Dataset in Weka J48 (C4.5)



DT for Iris Dataset in Weka J48 (C4.5)



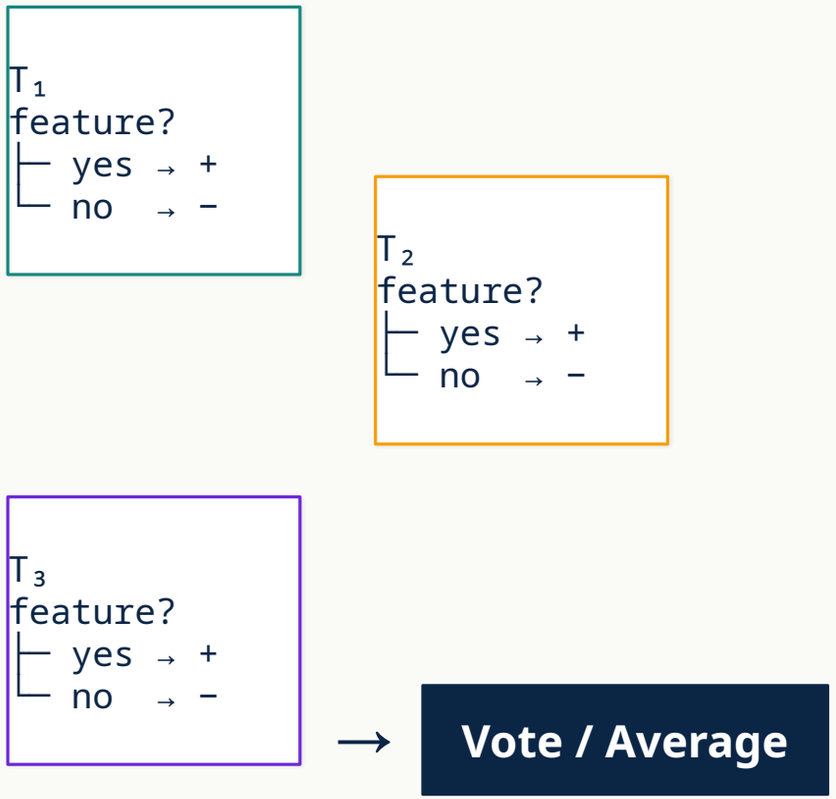
Open Discussion

Ensemble Learning with Decision Trees

Bagging • Random Forest • Extra Trees • Boosting • GBDT

Core question

How do we turn unstable trees into accurate, robust classifiers?



Lecture deck

Learning objectives

overview

After this lecture, students should be able to explain and compare the main DT ensemble families.

1. Explain why a single decision tree is often high-variance.
2. Describe how voting and averaging combine multiple trees.
3. Differentiate Bagging, Random Forest, and Extra Trees.
4. Differentiate AdaBoost and Gradient Boosting Decision Trees.
5. Select a suitable DT ensemble for a given classification/regression problem.

Key organizing idea

Parallel ensembles mainly reduce variance.
Sequential ensembles mainly build corrective models.

Why ensemble decision trees?

A single deep tree can fit many local irregularities. Small changes in data may change the split sequence.

```
Single DT
feature?
├─ yes → +
└─ no  → −
```

→

High variance model

Ensemble remedy

```
T₁
feature?
├─ yes → +
└─ no  → −
```

```
T₂
feature?
├─ yes → +
└─ no  → −
```

```
T₃
feature?
├─ yes → +
└─ no  → −
```

↓

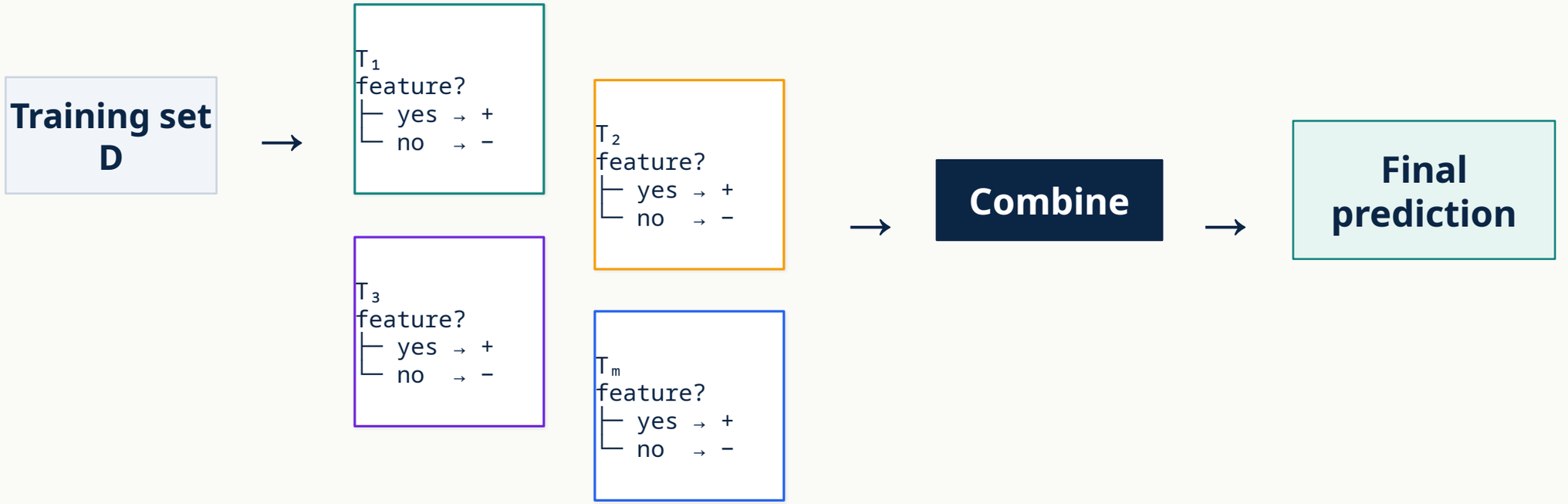
aggregate decisions more stable boundary

Core benefit: reduce variance, improve generalization, and make predictions less dependent on one tree.

Basic ensemble mechanism

core

Train multiple trees, then combine their predictions.



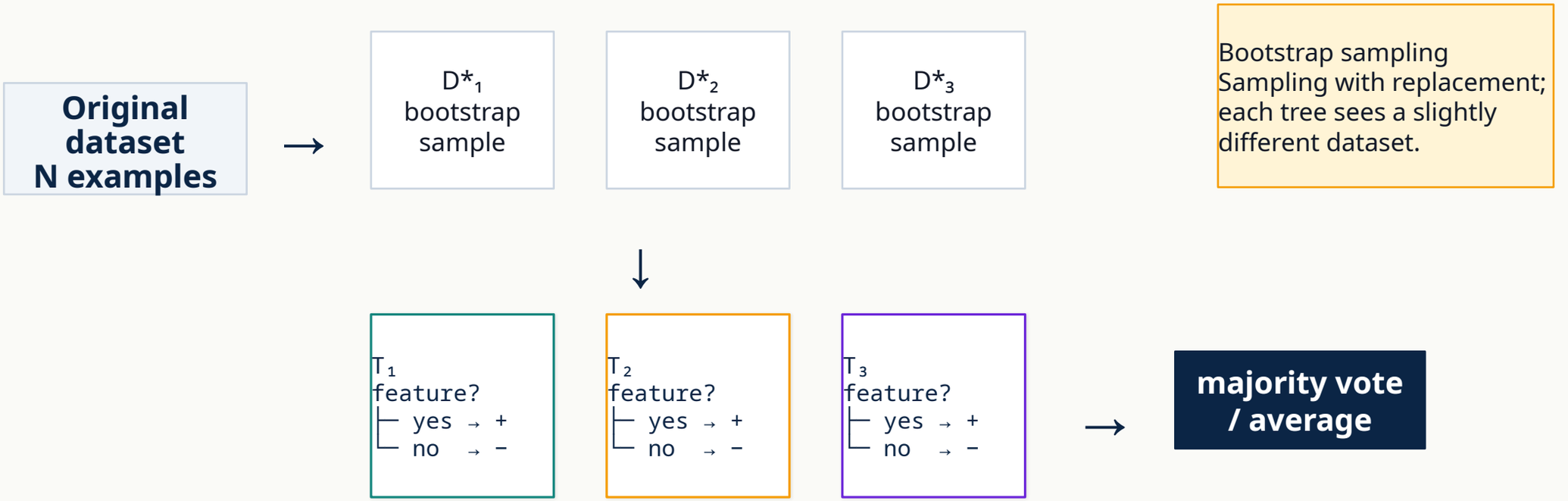
Classification:
$\hat{y} = \text{mode}\{T_1(x),\ldots,T_m(x)\}$

Regression:
$\hat{y} = (1/M)\ \Sigma\ T_m(x)$

Bagging: Bootstrap Aggregating

parallel

Build many trees independently using bootstrap samples, then vote or average.



Bagging is most useful when the base learner is unstable, as decision trees usually are.

Random Forest: Bagging + RSM idea

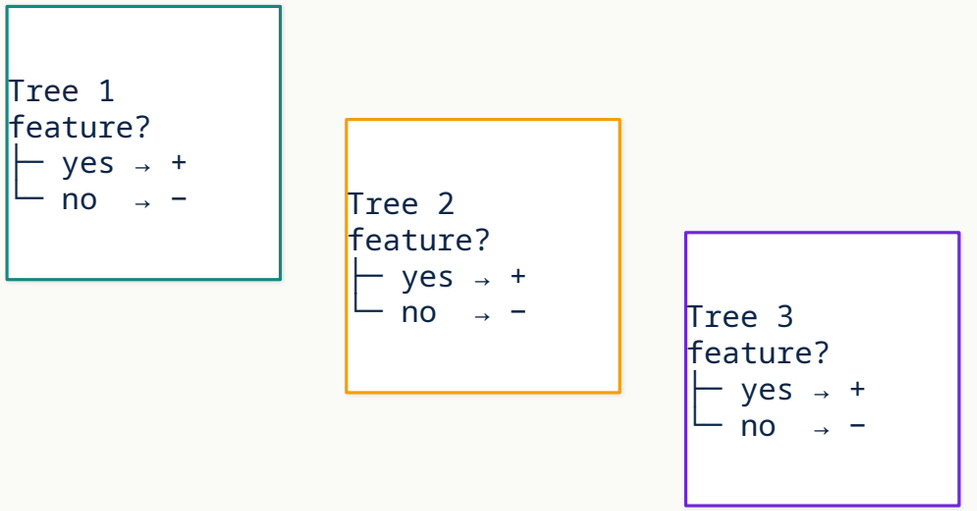
parallel

Random Forest uses bootstrap samples and random feature subsets; in standard RF, the feature subset is sampled at each split/node.

At each RF split

randomly choose m features from p
select the best split only among them

→



Important distinction

Conceptually yes: RF = Bagging + RSM.
Technically, classical RSM may use one feature subset per learner; RF usually resamples candidate features at every node.

RF = Bagging + RSM idea
precise: random features at each split

Extra Trees: more randomization

parallel

Extremely Randomized Trees add randomness to split thresholds as well as feature selection.

Method	Feature subset	Split threshold	Main effect
Random Forest	Random	Best threshold among candidates	Lower tree correlation
Extra Trees	Random	Randomized threshold	Even stronger randomization

Why it matters

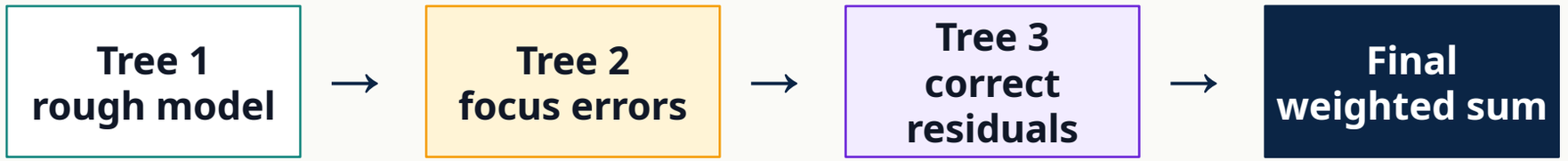
More randomness can reduce variance, but a single tree becomes less optimized. The ensemble compensates by averaging many such trees.

Use when many trees are acceptable and additional randomization helps stabilize predictions.

Boosting: sequential correction

sequential

Boosting builds an additive model. Later trees focus on what earlier trees handled poorly.



Additive form:

$$F_m(x) = F_{m-1}(x) + \eta\, T_m(x)$$

η is the learning rate; each new tree contributes only part of the correction.

Contrast with bagging

Bagging: independent trees.
Boosting: ordered trees; the next tree depends on previous errors.

AdaBoost with decision stumps

sequential

AdaBoost repeatedly reweights examples: misclassified examples receive more attention in the next round.

1. Start with equal example weights
2. Train weak tree usually stump
3. Increase weights of misclassified examples
4. Weighted vote by tree accuracy

Final classifier

$$H(x) = \text{sign}(\Sigma\ \alpha_m\ h_m(x))$$

α_m gives higher influence to more accurate weak trees.

Teaching point

AdaBoost changes the distribution of training examples, not the feature subset at each node.

Gradient Boosting Decision Trees

sequential

GBDT fits each new tree to the gradient of a loss function, often described as learning residual errors.

Model so far $F_{m-1}(x)$ → **Compute residuals / negative gradient** → **Fit new tree** $T_m(x)$ → **Update** $F_m = F_{m-1} + \eta T_m$

Regression view

If current prediction is too low for an example, the next tree learns a positive correction; if too high, a negative correction.

Key hyperparameters

number of trees, learning rate, tree depth, subsampling, regularization

Modern GBDT implementations

practice

These are implementation families built around gradient-boosted trees, with different engineering choices.

Tool	Distinctive emphasis	Typical strength
XGBoost	regularized scalable tree boosting	strong general-purpose tabular baseline
LightGBM	histogram-based learning and efficient growth	large datasets and faster training
CatBoost	ordered boosting and categorical feature handling	datasets with many categorical features

Caution for students These are not different base learners; they are optimized frameworks for building boosted tree ensembles.

Bagging vs. Boosting: main comparison

comparison

Aspect	Bagging / RF / Extra Trees	Boosting / GBDT
Training order	parallel or independent	sequential
Main mechanism	resample examples; randomize features/splits	correct errors or gradients
Main effect	variance reduction	strong additive model; often bias + variance reduction
Sensitivity	usually robust to noisy samples	can be sensitive if overfit or noisy labels dominate
Output	vote / average many trees	weighted sum of trees

Simple rule of thumb Use Random Forest as a strong, stable baseline. Use GBDT when careful tuning and top tabular performance are required.

Numerical classroom example: majority vote

example

Five decision trees predict the class of the same sample x.

Tree	T_1	T_2	T_3	T_4	T_5
Prediction	+	+	−	+	−
Weight	1	1	1	1	1

Majority vote

Class + receives 3 votes. Class − receives 2 votes. Therefore, the ensemble prediction is +.

Extension

For weighted boosting, replace simple counts with $\Sigma\alpha_m$ for each class.

Exam prompt

Ask students to compute unweighted and weighted ensemble decisions from the same prediction table.

Numerical classroom example: weighted vote

example

Boosting assigns different importance to different trees.

Tree	T_1	T_2	T_3	T_4
Prediction	+	−	+	−
Weight α_m	0.8	0.3	0.6	0.2

Class + score = 0.8 + 0.6 = 1.4

Class − score = 0.3 + 0.2 = 0.5

→

Prediction = +

This illustrates why a boosted ensemble may disagree with an unweighted majority vote.

Summary: basic DT ensembles

wrap-up

Family	Representative methods	Core idea
Bagging	Bagging, Random Forest, Extra Trees	many independent randomized trees
Boosting	AdaBoost, GBDT, XGBoost, LightGBM, CatBoost	ordered trees that correct earlier errors

Three takeaways

1. DT ensembles are effective because individual trees are diverse.
2. Randomization and resampling create diversity in parallel ensembles.
3. Sequential boosting creates diversity by focusing on previous errors.

The basic **ensemble methods using Decision Trees (DTs)** are:

Ensemble method	Core idea	How individual DTs differ	Final prediction
Bagging (Bootstrap Aggregating)	Train many trees independently on different bootstrap samples	Different sampled training sets	Majority voting for classification; averaging for regression
Random Forest (RF)	Bagging + random feature selection	Different samples **and** different subsets of features at each split	Voting / averaging
AdaBoost with DTs	Train trees sequentially, emphasizing previously misclassified examples	Each new tree focuses more on difficult samples	Weighted voting
Gradient Boosting Decision Trees (GBDT)	Each new tree learns to correct the errors/ residuals of the current ensemble	Trees are fitted sequentially to residual/error information	Sum of weighted tree outputs
Extra Trees (Extremely Randomized Trees)	Similar to RF, but introduces more randomness in selecting split points	Random features and more-random split thresholds	Voting / averaging

1. Bagging

Suppose we train M decision trees:

$$T_1(x), T_2(x), \ldots, T_M(x)$$

Each tree is trained using a **bootstrap sample** of the original dataset.

For classification:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \ldots, T_M(x)\}$$

For regression:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^{M} T_m(x)$$

The major purpose is to **reduce variance** and make decision trees more stable.

2. Random Forest

Random Forest extends bagging by adding **feature randomness**.

At each node, instead of considering all p features, the algorithm randomly selects $m < p$ features and determines the best split only among those features.

So:

$$\boxed{\text{Random Forest} = \text{Bagging of DTs} + \text{Random Feature Selection}}$$

This reduces correlation among the trees and generally improves ensemble performance.

---

3. AdaBoost

AdaBoost builds decision trees **sequentially**.

Initially:

$$w_i = \frac{1}{N}$$

for every training example.

After each weak tree is trained, the weights of incorrectly classified examples are increased. Therefore, the next tree pays more attention to difficult examples.

The final classifier is approximately:

$$H(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(x)\right)$$

where α_m represents the importance of tree m.

Often very small trees or **decision stumps** are used as weak learners.

---

4. Gradient Boosting Decision Trees

Gradient Boosting also constructs trees sequentially, but rather than simply increasing sample weights, each new tree attempts to correct the errors of the previous model.

Conceptually:

$$F_m(x) = F_{m-1}(x) + \eta T_m(x)$$

where:

- $F_{m-1}(x)$: current ensemble
- $T_m(x)$: new tree fitted to the errors/residuals
- η: learning rate

Popular extensions include **XGBoost, LightGBM, and CatBoost**.

---

5. Extra Trees

Extra Trees introduces greater randomness than Random Forest.

Random Forest:

- randomly select features;
- find the **best split** among those features.

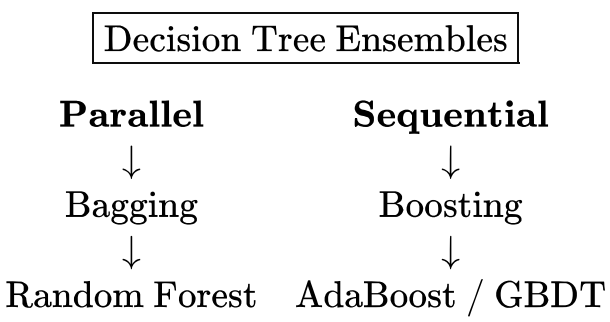
Extra Trees:

- randomly select features;
- also select split thresholds more randomly.

Greater randomization can further reduce variance.

Most fundamental classification

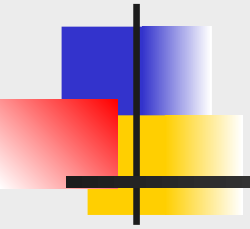
The basic DT ensembles can therefore be organized as:



The most important distinction for students is:

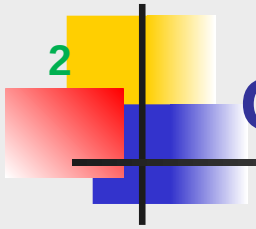
Bagging/RF: trees are mainly built **independently and in parallel** → primarily reduce **variance**.

Boosting: trees are built **sequentially**, with later trees correcting earlier errors → primarily improves the overall predictive model by progressively reducing residual errors/bias.



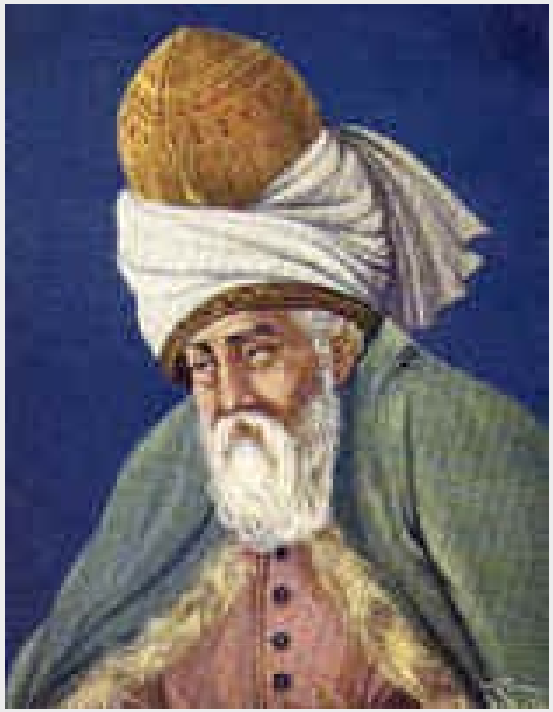
Deep Learning Aspects

Prof. Dr. Md. Aminul Haque Akhand
Dept. of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET)
www.kuet.ac.bd/cse/akhand



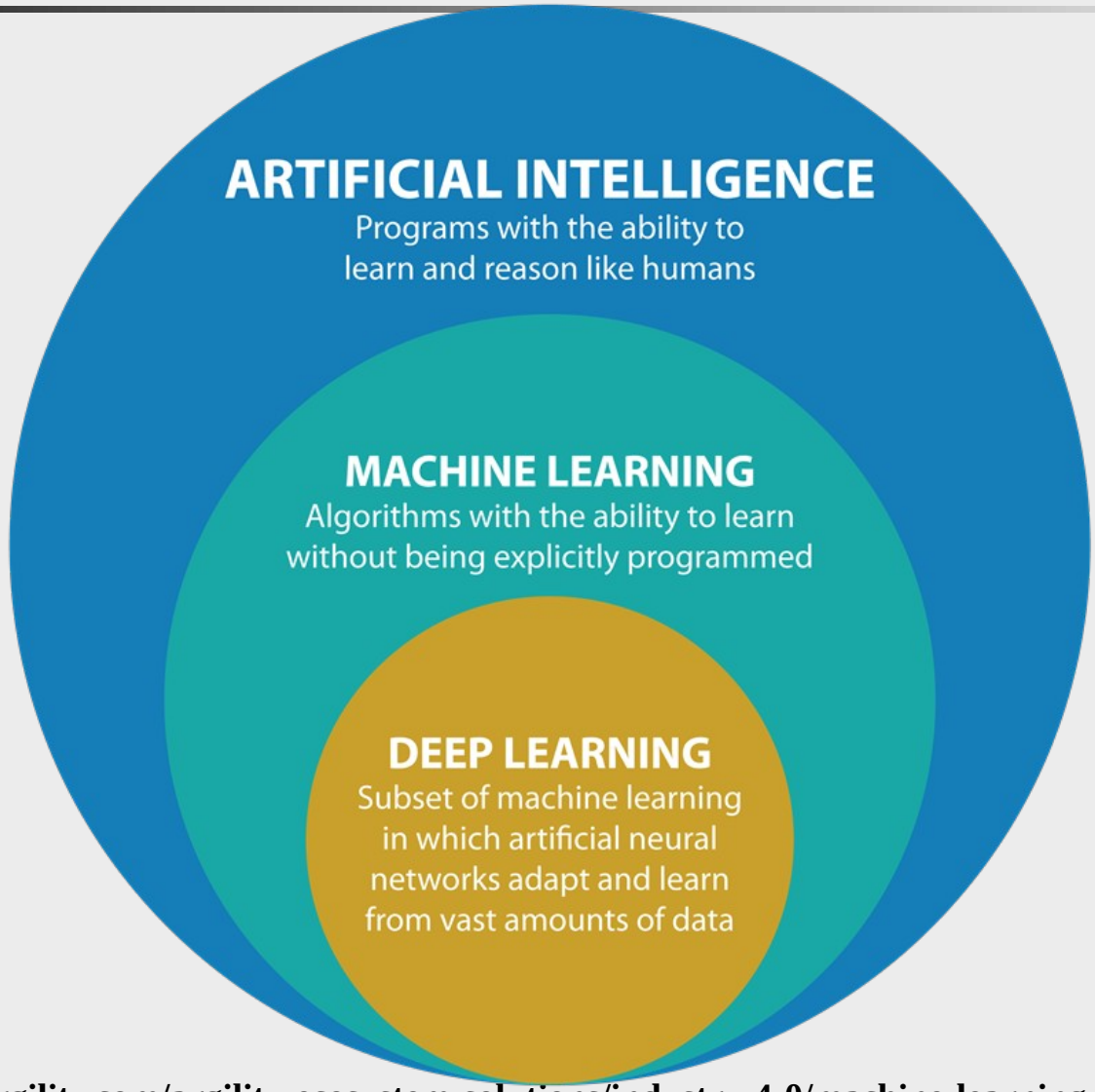
Outline of the Presentation

1. Theme
2. Machine Learning
3. Deep Learning
4. Fundamental Deep Learning Methods



“Yesterday I was clever,
so I wanted to change the world. Today I am wise,
so I am changing myself.”
— **Rumi**

AI ->Machine Learning->Deep Learning



https://www.argility.com/argility-ecosystem-solutions/industry-4-0/machine-learning-deep-learning/

AI ->Machine Learning->Deep Learning

Artificial Intelligence (AI)

AI is the broadest term, applying to any technique that enables computers to mimic human intelligence, using logic, if-then rules, decision trees, and machine learning (including deep learning).

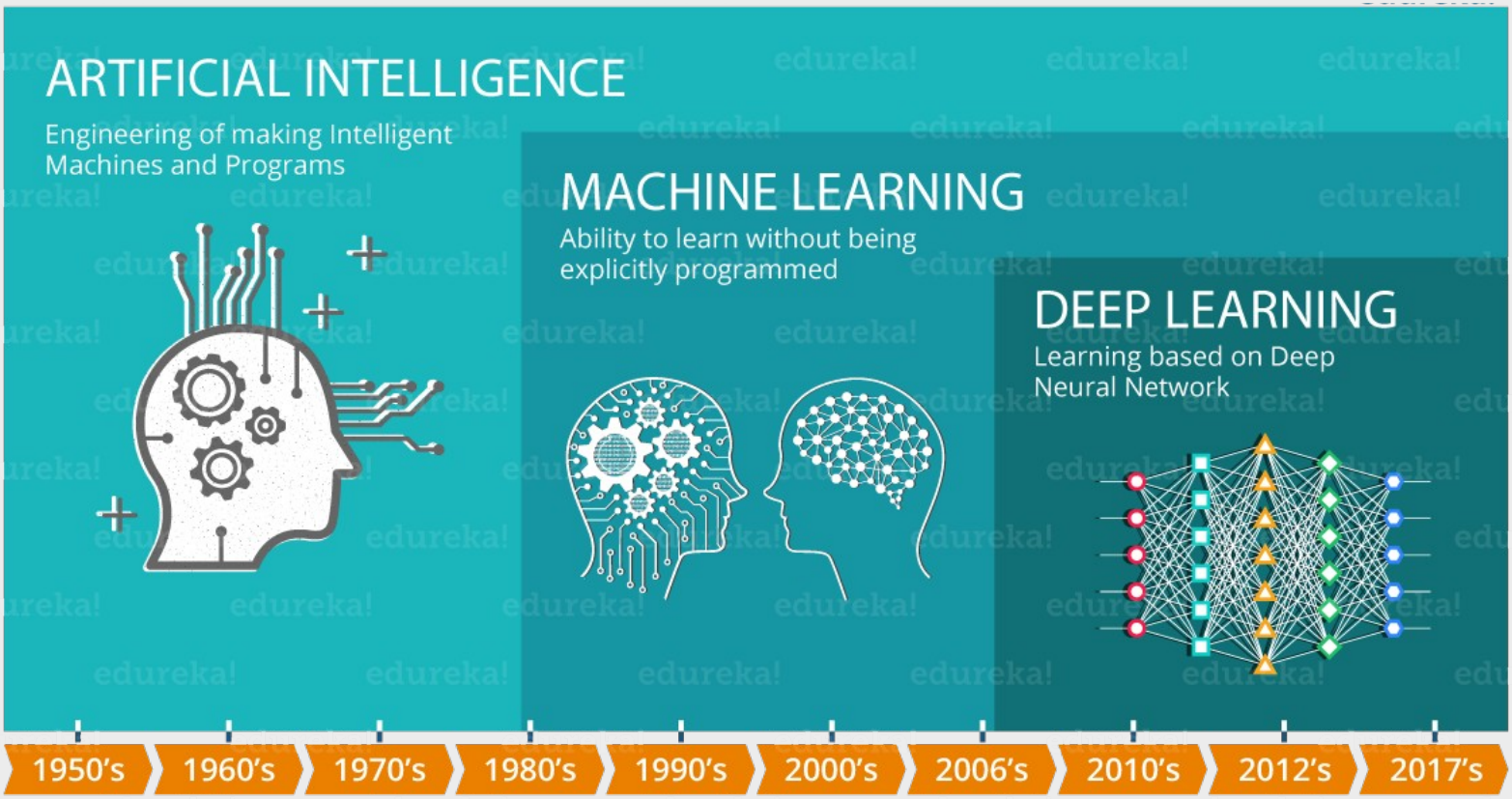
Machine Learning

The subset of AI that includes that enable machines to improve at tasks with experience. The category includes deep learning.

Deep Learning

The subset of machine learning composed of algorithms to train itself to perform tasks, like speech and image recognition, by exposing multilayered neural networks to vast amounts of data.

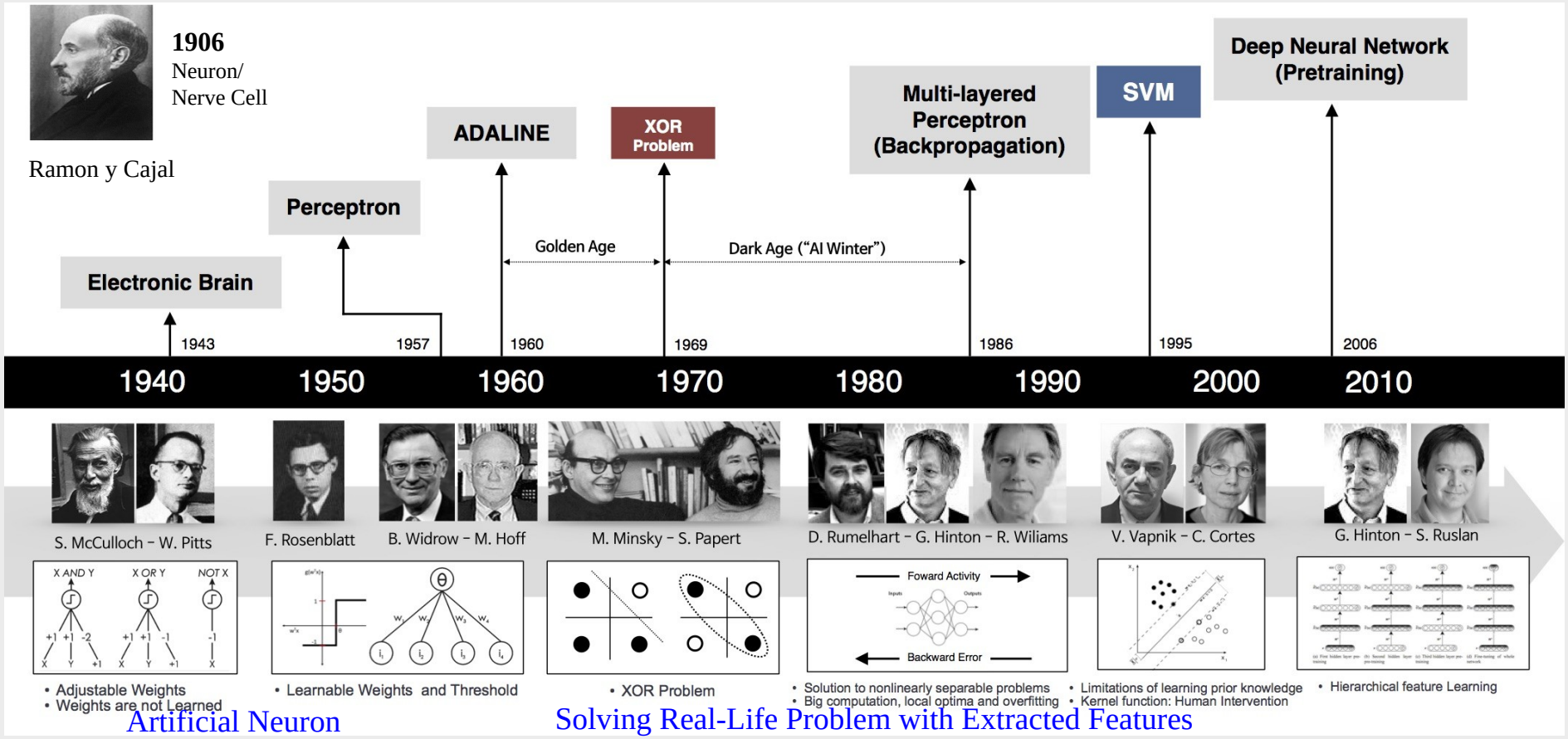
AI ->Machine Learning->Deep Learning



History and Background of Deep Learning

1906
Neuron/
Nerve Cell

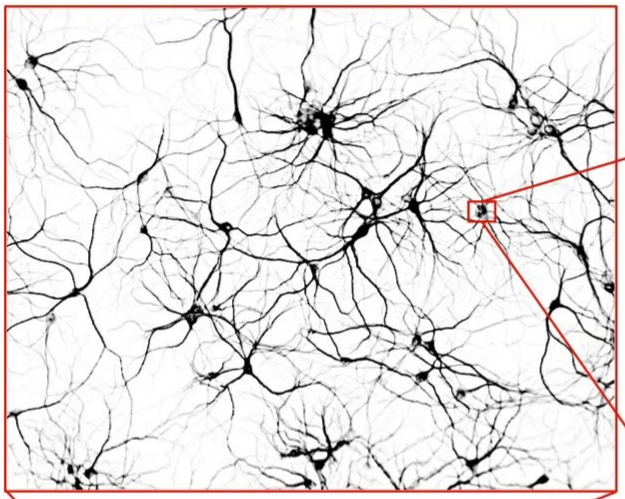
Ramon y Cajal

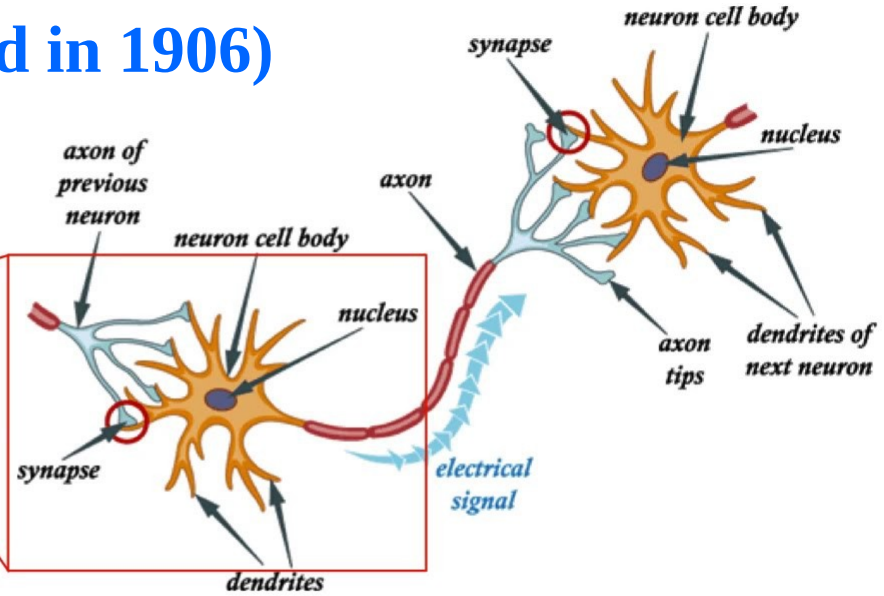


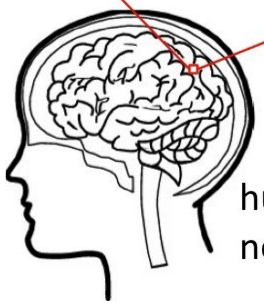
https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html

Neuron : Learning Element in Brain

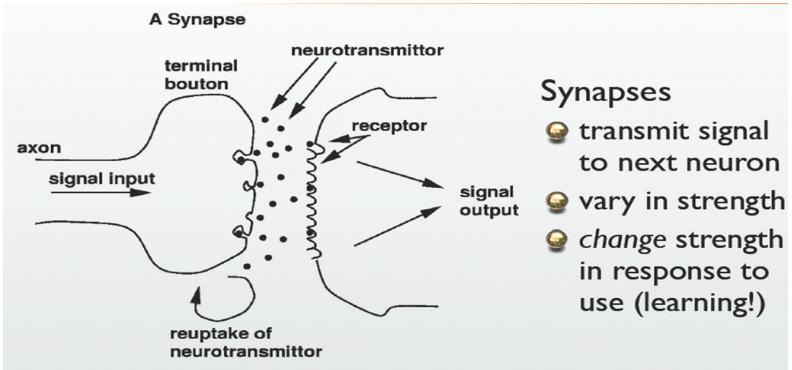
(Discovered in 1906)







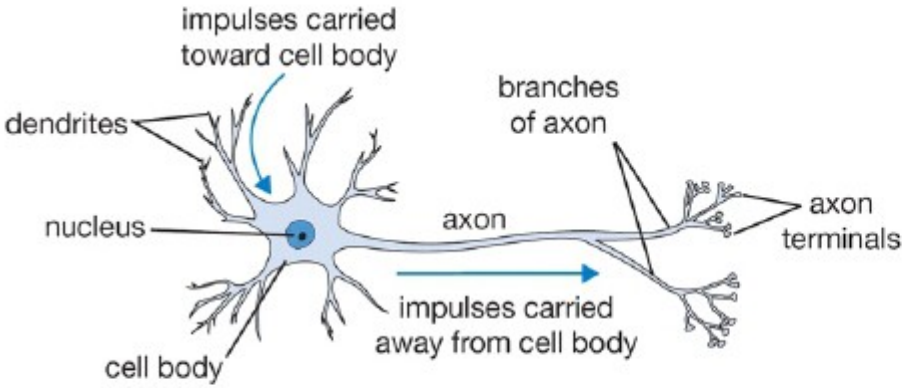
humans don't need features

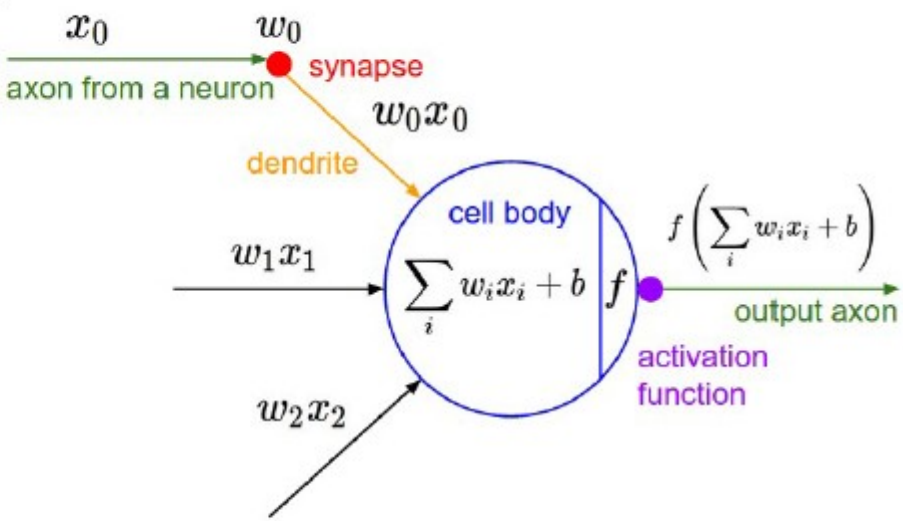


Copyright © 2014 Victor Lavrenko

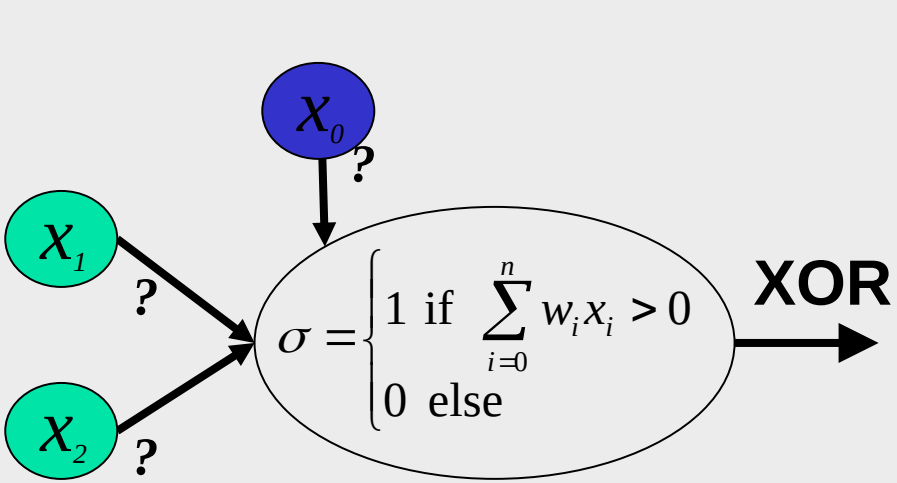
McCulloch-Pitts model of a Neuron

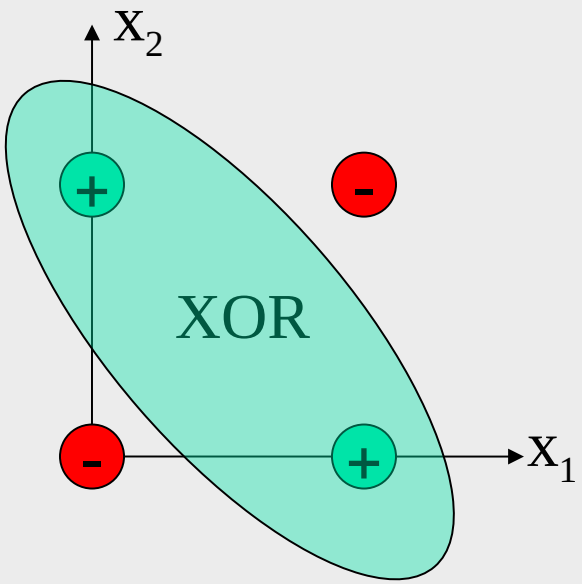
(Proposed in 1943)





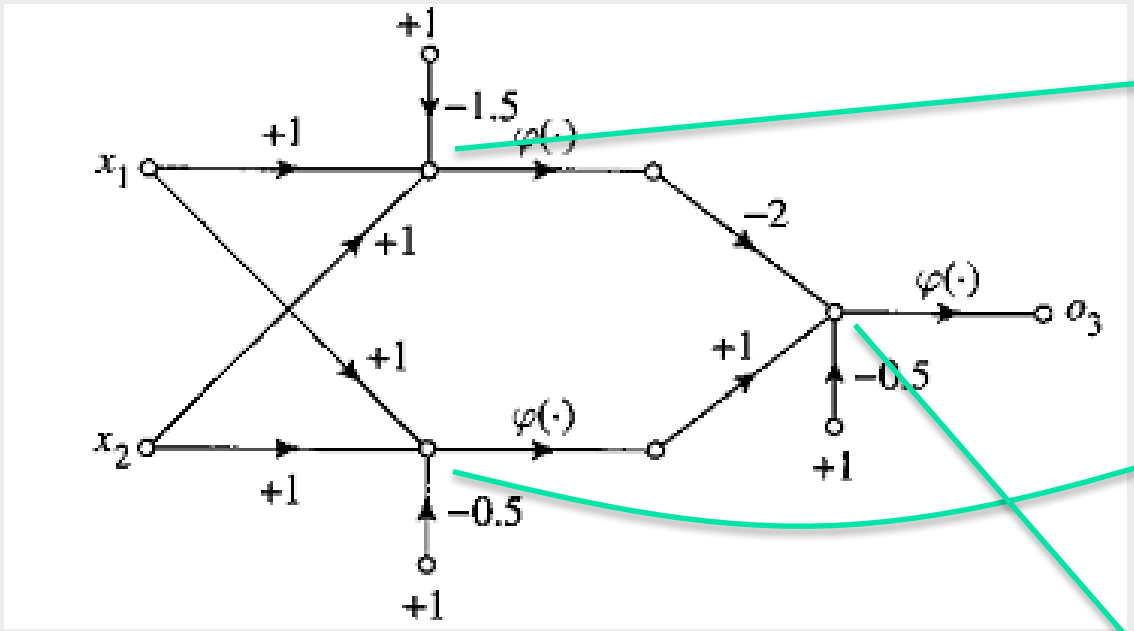
Ability of Single Neuron: Unable to Solve Nonlinear Problem

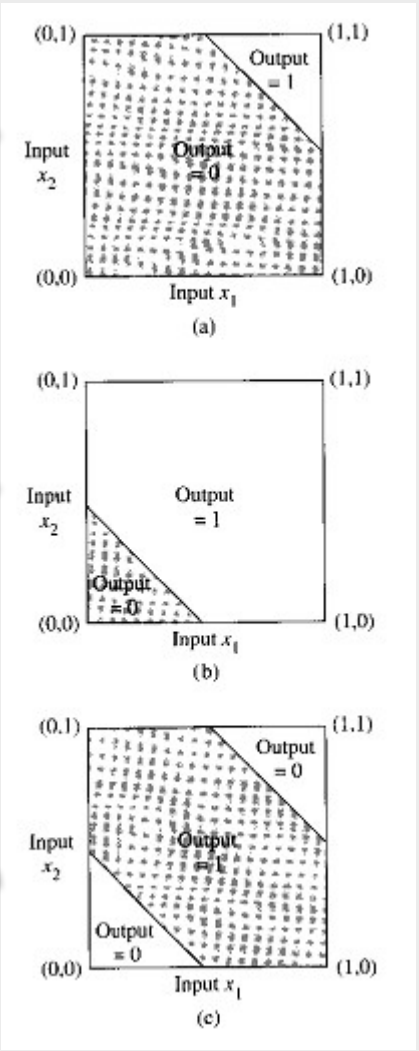




No arrangement work, not linearly separable. Require two boundary lines.

XOR solution with Hidden Neuron





AND

OR

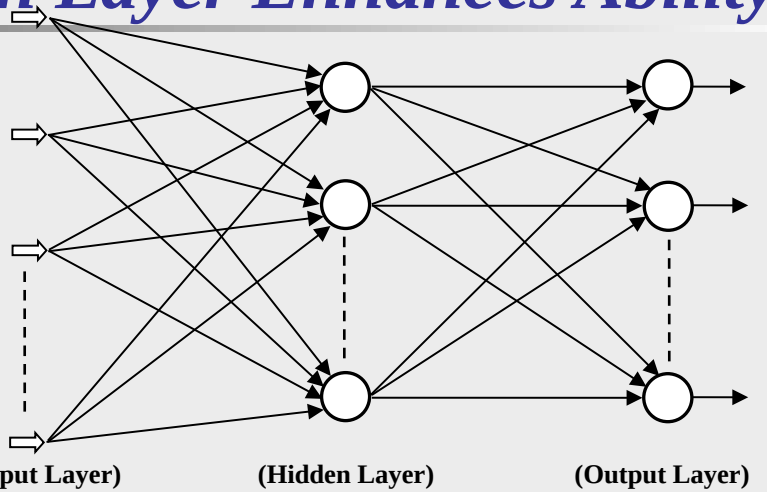
XOR

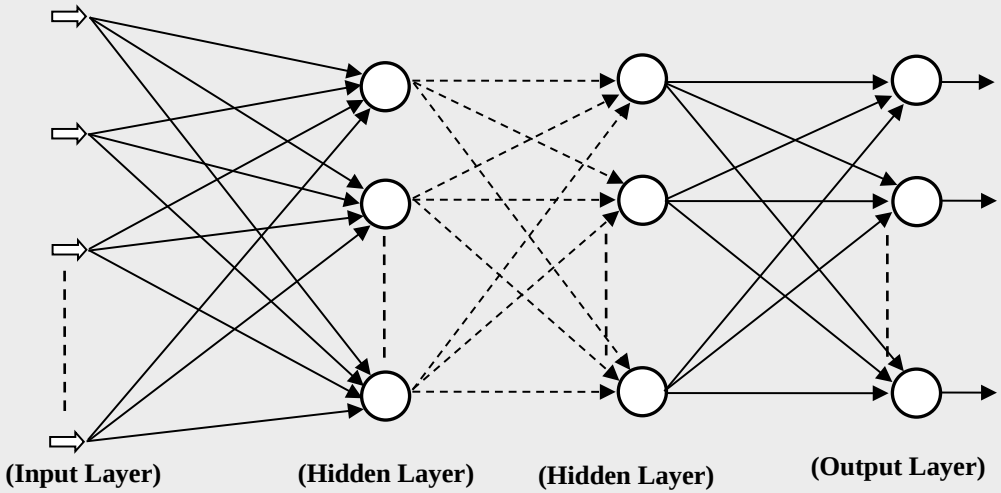
XOR logic through manipulation by hidden neurons

Open doors to Solve Real life Nonlinear Problem

Neural Network: Hidden Layer Enhances Ability

Hidden Layer(s) Enhances Ability to Solve Real Life Nonlinear Problems



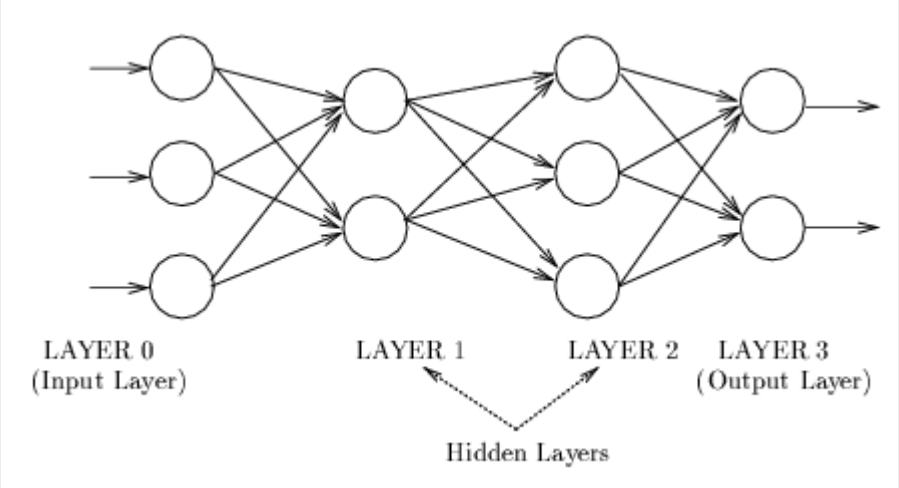


How to Select Appropriate Weight ?
Learning?

How to get appropriate weight set?

Back-propagation is the famous algorithm for training feed-forward networks

(Proposed in 1986)



http://en.wikipedia.org/wiki/Backpropagation

Backpropagation, or propagation of error, is a common method of teaching artificial neural networks how to perform a given task.

It was first described by Paul Werbos in 1974, but it wasn't until 1986, through the work of David E. Rumelhart, Geoffrey E. Hinton and Ronald J. Williams, that it gained recognition, and it led to a **"renaissance"** in the field of artificial neural network research.

nature publishing group

nature.com | about npg | news@nature.com | naturejobs | natureevents | help | site index

nature

my account | e-alerts | subscribe | register

SEARCH JOURNAL Go

Tuesday 06 October 2009

Journal Home
Current Issue
AOP
Archive

THIS ARTICLE

Download PDF
References

Export citation
Export references

Send to a friend

More articles like this

Table of Contents
< Previous | Next >

letters to nature

Nature **323**, 533 - 536 (09 October 1986); doi:10.1038/323533a0

Learning representations by back-propagating errors

DAVID E. RUMELHART*, GEOFFREY E. HINTON† & RONALD J. WILLIAMS*

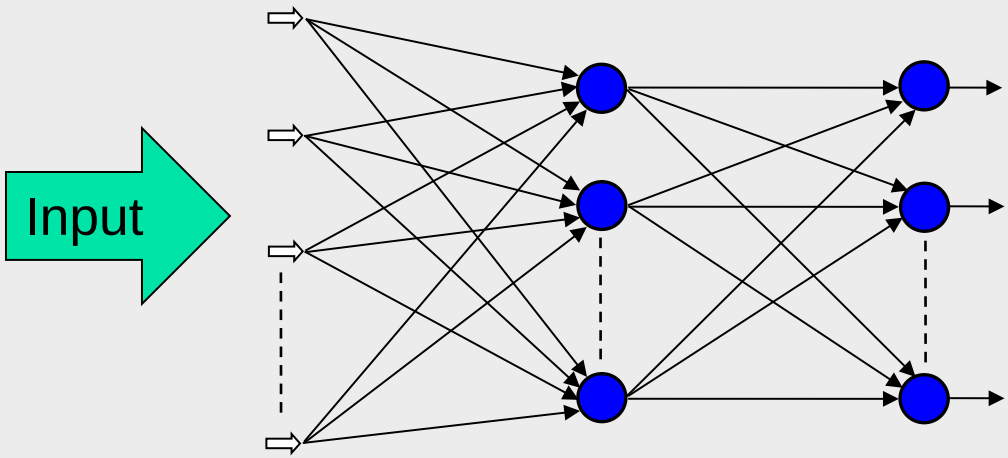
*Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
†Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA
†To whom correspondence should be addressed.

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

References

1. Rosenblatt, F. *Principles of Neurodynamics* (Spartan, Washington, DC, 1961).
2. Minsky, M. L. & Papert, S. *Perceptrons* (MIT, Cambridge, 1969).
3. Le Cun, Y. *Proc. Cognitiva* **85**, 599–604 (1985).

Back-propagation(BP)

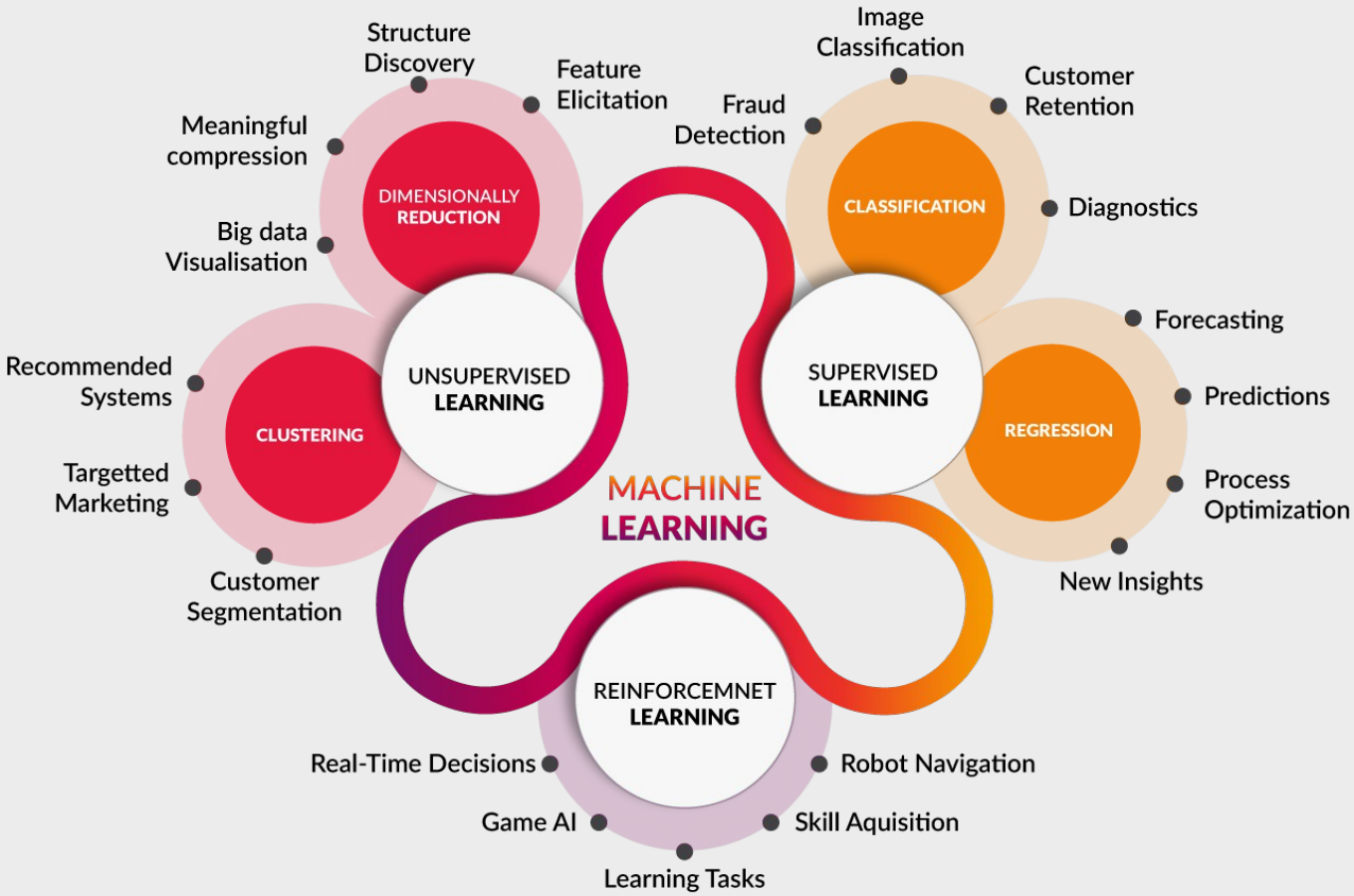


Error =
Desired Output
- Actual Output

Forward Pass: Error is calculated based on desired and actual output.

Backward Pass: Synaptic weights are adjusted based on calculated error.

Applications of Machine Learning



UCI Machine Learning Repository - Windows Internet Explorer

http://archive.ics.uci.edu/ml/

UCI Machine Learning Repository

About Citation Policy Donate a Data Set Contact

Search

Repository Web

View ALL Data Sets

Machine Learning Repository

Center for Machine Learning and Intelligent Systems

Welcome to the UC Irvine Machine Learning Repository!

We currently maintain 174 data sets as a service to the machine learning community. You may **view all data sets** through our searchable interface. Our old web site is still available, for those who prefer the old format. For a general overview of the Repository, please visit our About page. For information about citing data sets in publications, please read our citation policy. If you wish to donate a data set, please consult our donation policy. For any other questions, feel free to contact the Repository librarians. We have also set up a mirror site for the Repository.

Supported By: In Collaboration With: Rexa.info

Latest News:

07-23-2008: Repository mirror has been set up.
03-24-2008: New data sets have been added!
06-25-2007: Two new data sets have been added: UJI Pen Characters, MAGIC Gamma Telescope
04-13-2007: Research papers that cite the repository have been associated to specific data sets.
04-09-2007: Three new data sets have been added: Poker Hand, Callt2 Building People Counts, Dodgers Loop Sensor.
09-08-2006: The Beta site has been launched.
09-01-2006: SPECTF.test has been modified by the donor.

Featured Data Set: Statlog (Shuttle)

Task: Classification
Data Type: Multivariate
 # Attributes: 9
 # Instances: 58000

The shuttle dataset contains 9 attributes all of which are numerical. Approximately 80% of the data belongs to class 1

Newest Data Sets:

06-26-2008: Parkinsons
04-21-2008: Ozone Level Detection
04-03-2008: Abscisic Acid Signaling Network
03-20-2008: Hill-Valley
03-12-2008: Bag of Words
03-08-2008: Reuters Transcribed Subset
02-29-2008: Gisette
02-29-2008: Dorothea
02-29-2008: Madelon

Most Popular Data Sets (hits since 2007):

39351: Iris
31585: Adult
26458: Wine
23553: Breast Cancer Wisconsin (Diagnostic)
18449: Abalone
18321: Poker Hand
13471: Yeast
12996: Internet Advertisements
11793: SPECT Heart

UCI Machine Learning Repository - Windows Internet Explorer

Thesis | idea of neural ne... | UCI Machine Lea... | FTP directory: /p... | Ensemble of Div... | H-Bibliography.d... | Desktop | 4:39 PM

Benchmark Problems

Problems Related to Human Life

Problem	Task
Breast Cancer Wisconsin	Predicts whether a tumor is benign (not dangerous to health) or malignant (dangerous) based on a sample tissue taken from a patient's breast.
BUPA Liver Disorder	Identify lever disorders based on blood tests along with other related information such as alcohol consumption.
Diabetes	Investigate whether the patient shows or not the signs of diabetes.
Heart Disease Cleveland	Predicting whether at least one of four major heart vessels is reduced in diameter by more than 50%.
Hepatitis	Anticipate status (i.e., live or die) of hepatitis patient.
Lymphography	Predict the situation of lymph nodes and lymphatic vessels.
Lungcancer	Identify types of pathological lung cancers.
Postoperative	Determine place to send patients for postoperative recovery.

Input Features of Diabetes

1. Number of times pregnant
2. Plasma glucose concentration
3. Diastolic blood pressure
4. Triceps skin fold thickness (mm)
5. 2-Hour serum insulin (mu U/ml)
6. Body mass index
7. Diabetes pedigree function
8. Age

Benchmark Problems

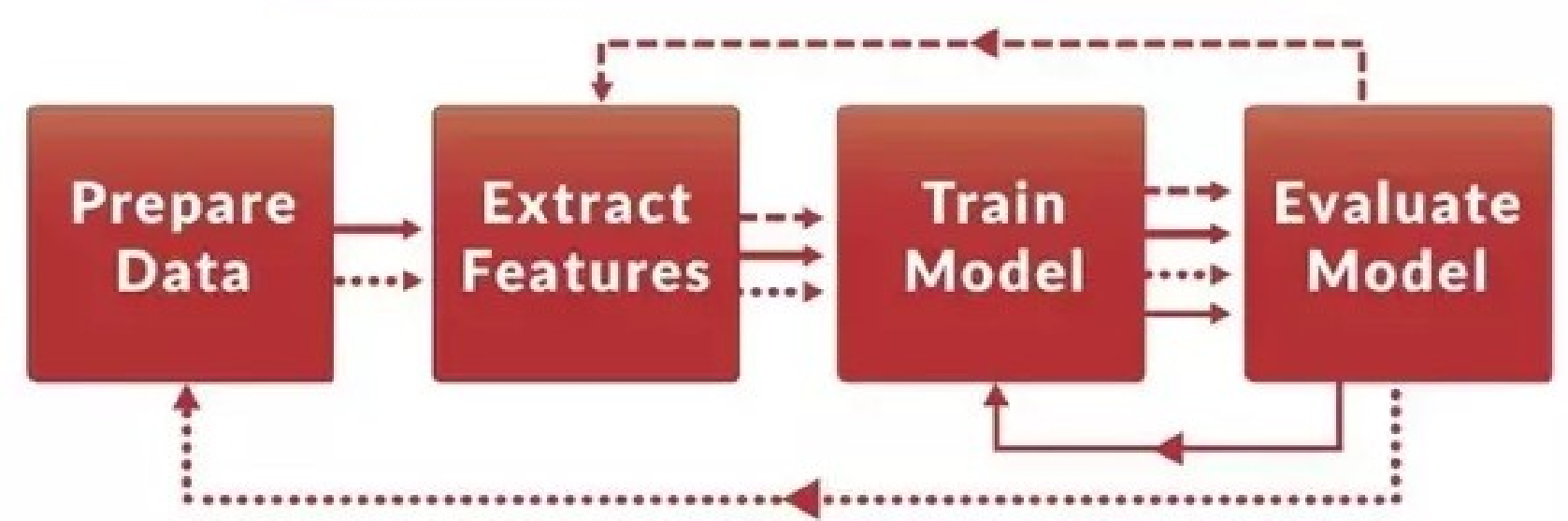
Problems Related to Finance

Problem	Task
Australian Credit Card	Classify people as good or bad credit risks depend on applicants' particulars.
Car	Evaluate cars based on price and facilities.
Labor Negotiations	Identify a worker as good or bad i.e., contract with him beneficial or not.
German Credit Card	Like Australian Card, this problem also concerns to predict the approval or non-approval of a credit card to a customer.

Problems Related to Plants

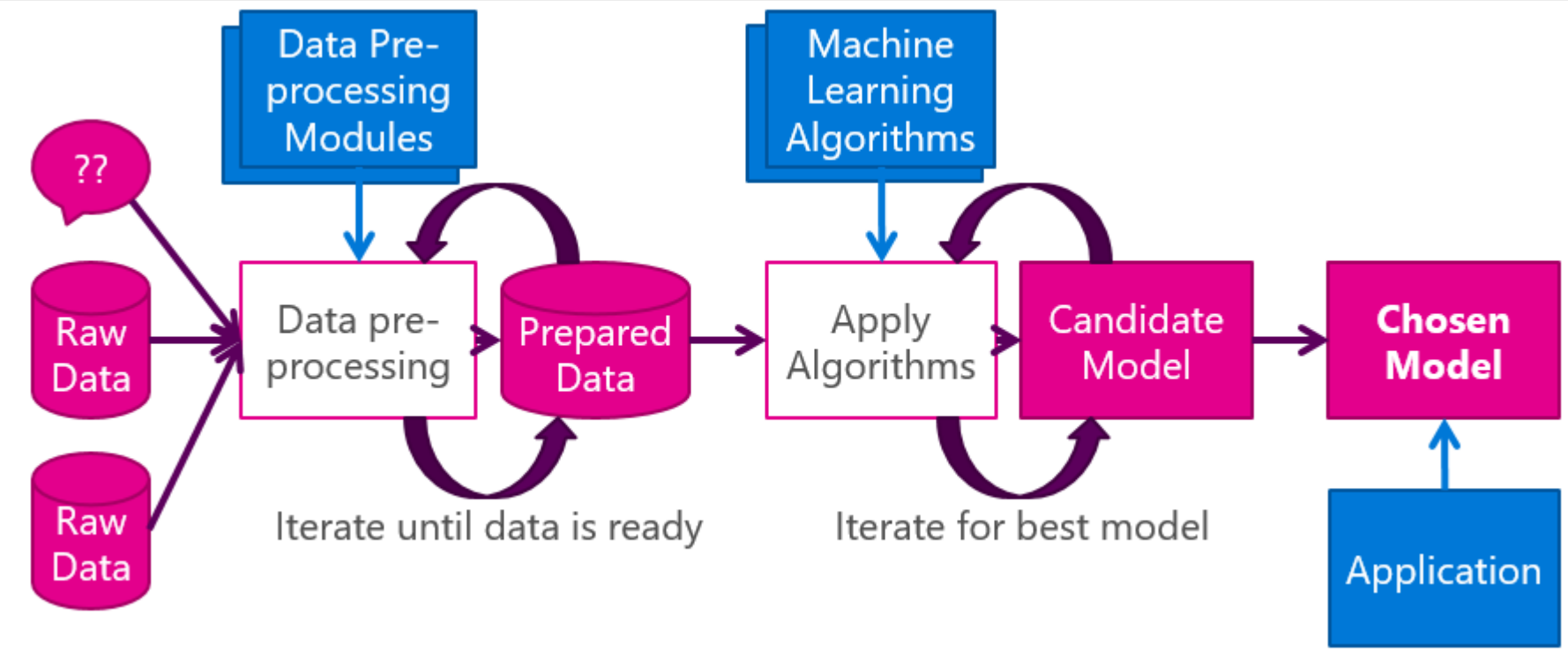
Problem	Task
Iris Plants	Classify iris plant types.
Mushroom	Identify whether a mushroom is edible or not based on a description of the mushroom's shape, color, odor, and habitat.
Soybean	Recognize 19 different diseases of soybeans.

Machine Learning Process



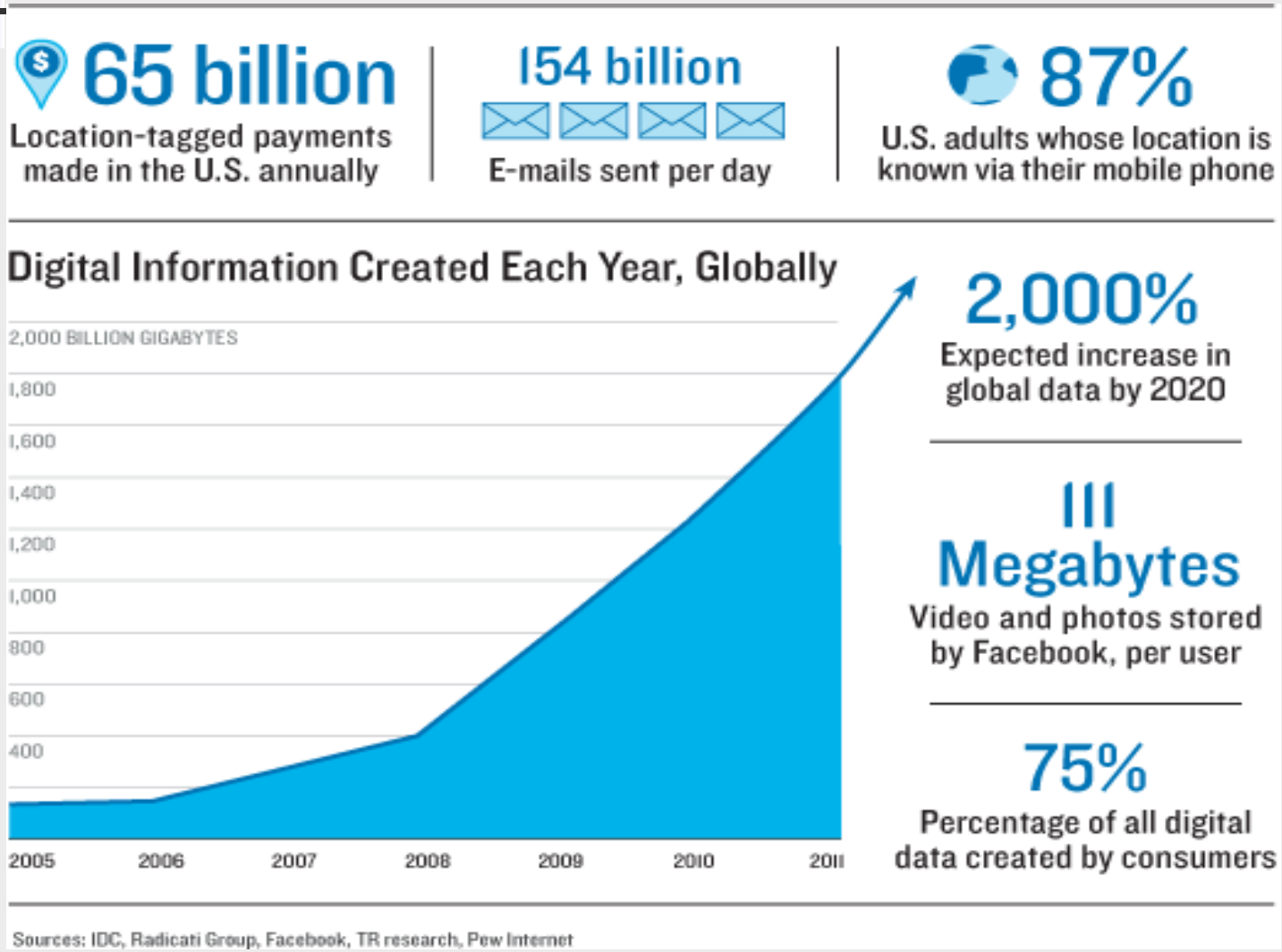
Performance depends on feature extraction which requires heuristics and, most notably, a great deal of time

Machine Learning Process (Different View)



Performance depends on data pre-processing (i.e., feature extraction) which requires human expertize

Why Deep Architecture?



For huge heterogeneous data feature extraction is challenging and training time unacceptable

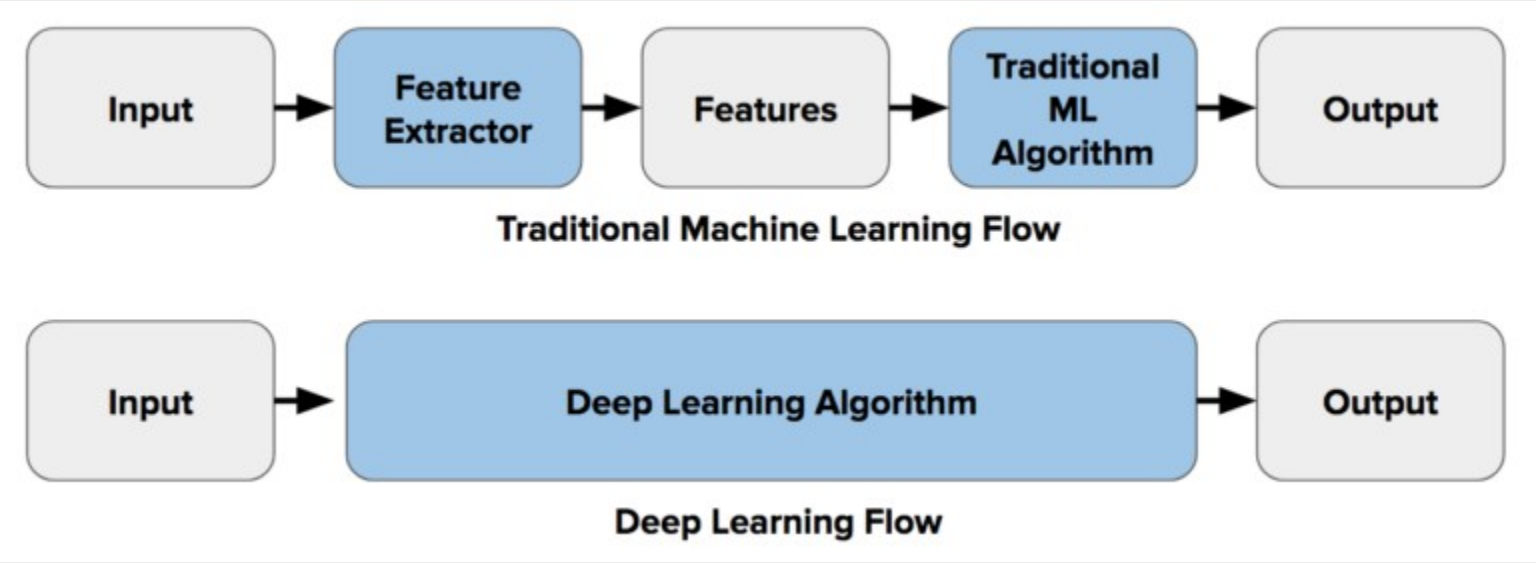
Motivation of Deep Learning

The main motivations to learning deep architectures are:

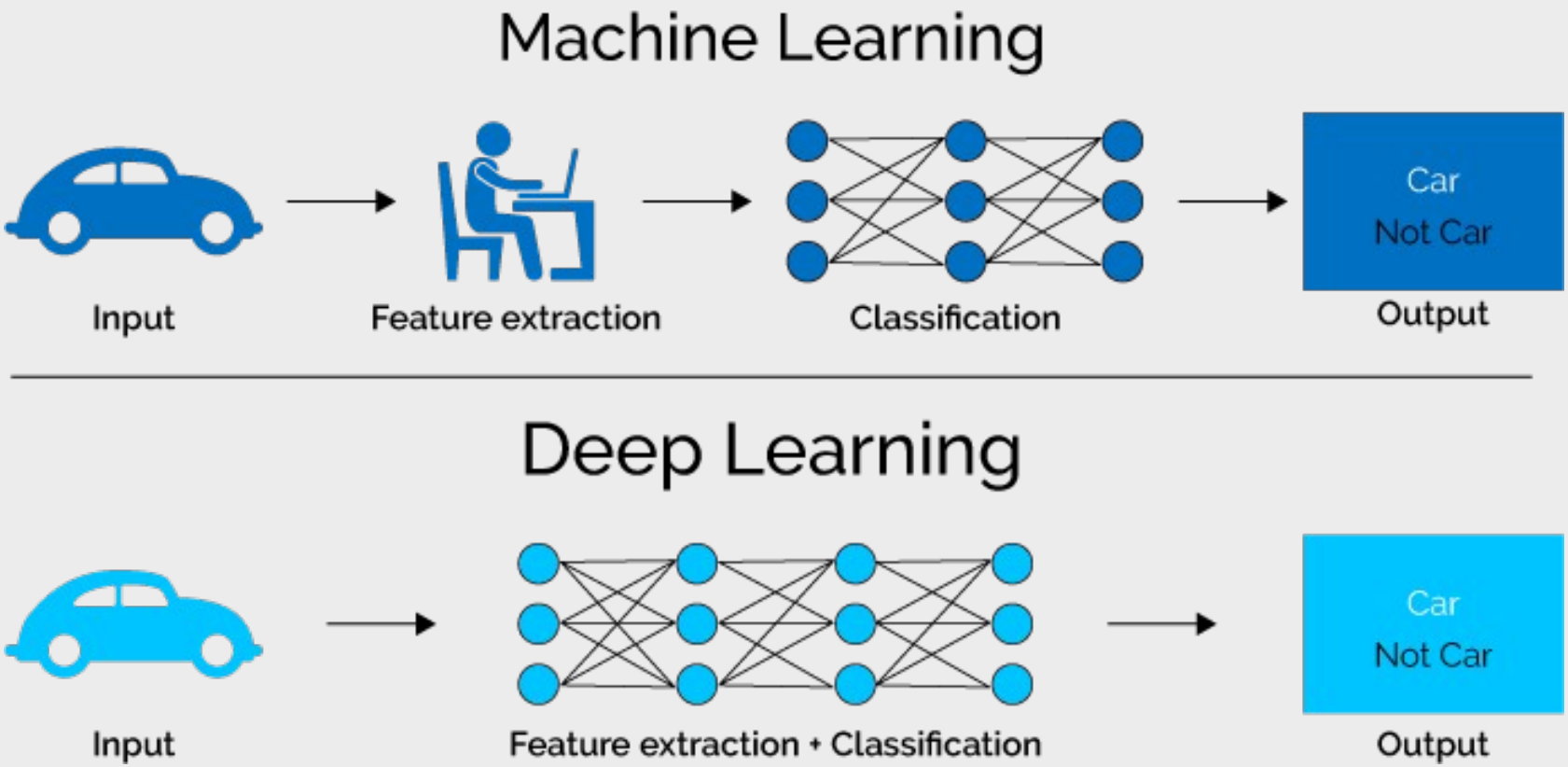
(1) Insufficient depth can hurt,

(2) The brain has a deep architecture and

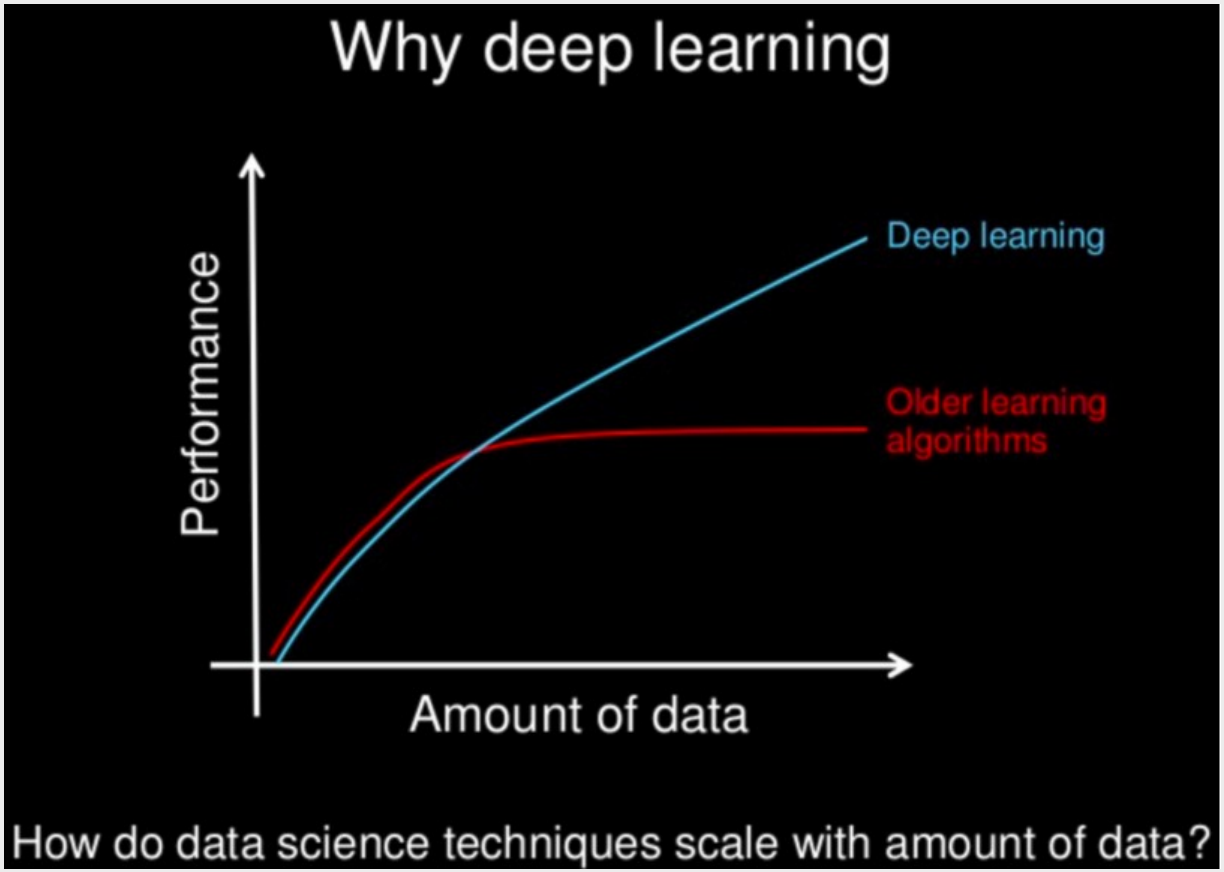
(3) Cognitive processes seem deep.



Machine Learning vs Deep Learning



Machine Learning vs Deep Learning



Understanding Deep Learning

1. what exactly is deep learning ? And,

2. why is it generally better than other methods on image, speech and certain other types of data?

The short answers

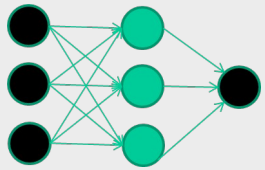
1. ***'Deep Learning' means using a neural network with several layers of nodes between input and output***

2. ***the series of layers between input & output do feature identification and processing in a series of stages, just as our brains seem to.***

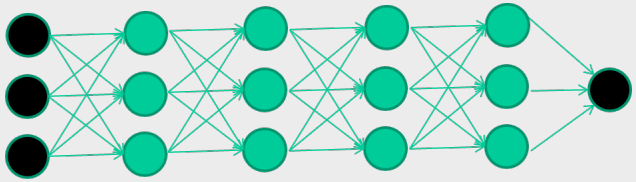
Understanding Deep Learning

3. multilayer neural networks have been around for 25 years. What's actually new?

we have always had good algorithms for learning the weights in networks with 1 hidden layer



but these algorithms are not good at learning the weights for networks with more hidden layers



what's new is: ***<u>algorithms for training many-layer networks</u>***

When to use Deep Learning (DL)?

- **DL** outperforms other techniques if the ***data size is large***.
- When there is ***lack of domain understanding for feature introspection***, **DL** techniques outshines others as you have to worry less about feature engineering.
- **DL** really shines when it comes to ***complex problems such as image classification, natural language processing, and speech recognition***.

Deep Learning Review in Nature

rnal/v521/n7553/full/nature14539.html

nature International weekly journal of science

Search Advanced search

Home | News & Comment | Research | Careers & Jobs | Current Issue | Archive | Audio & Video | For Authors

Archive > Volume 521 > Issue 7553 > Insights > Reviews > Article

NATURE | INSIGHT | REVIEW

Deep learning

Yann LeCun, Yoshua Bengio & Geoffrey Hinton

Affiliations | Corresponding author

Nature **521**, 436–444 (28 May 2015) | doi:10.1038/nature14539
Received 25 February 2015 | Accepted 01 May 2015 | Published online 27 May 2015

PDF | Citation | Rights & permissions | Article metrics

Abstract

Abstract · Supervised learning · Backpropagation to train multilayer architectures · Convolutional neural networks · Image understanding with deep convolutional networks · Distributed representations and language processing · Recurrent neural networks · The future of deep learning · References · Acknowledgements · Author information

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

Subject terms: Mathematics and computing · Computer science

At a glance

nl#insight

TOP TEN
Business Intelligence
TRENDS FOR 2017
GET THE WHITEPAPER
+ableau

Editors' pick

Image credit: Gabriela Hasbun for Nature

The rise and fall and rise again of 23andMe

Science jobs | Science events

naturejobs.com

Professor and Faculty Positions at the Academy of Medical Sciences (AMS), Zhengzhou University
The Academy of Medical Sciences of Zhengzhou University

Associate Editor / Senior Editor roles, Nature Research - Talent Pool 2017
Springer Nature

Postdocs, Key Lab for Neuroinformation, University of Electronic Sciences and

Research library / Report -- 2016
The Deep Learning Market Map: 60+ Startups Working Across E-Commerce, Cybersecurity, Sales, And More

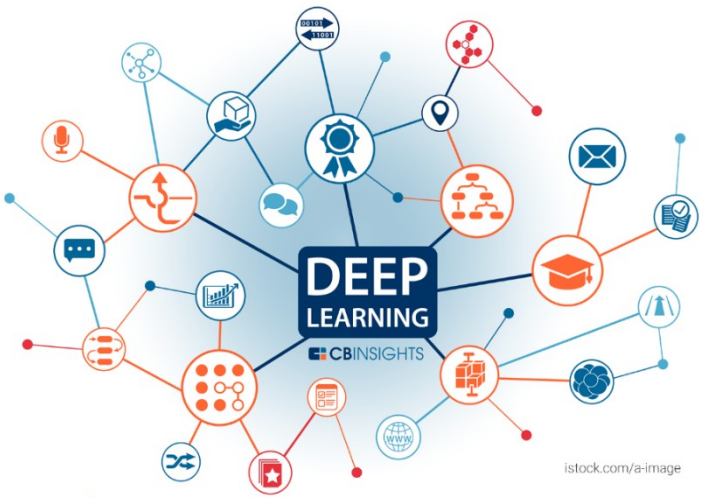
60+ STARTUPS USING DEEP LEARNING

CORE AI: COMPUTER VISION
clarifai, Tractable, netra, HYPERVERGE, captricity, deepomatic, sentient, Sighthound, Terraloupe, imagenii, pilot.ai

CORE AI: OTHER
TERADEEP, affectiva, Wave Computing, DigitalGenius, LEAPMIND, Arya, twentybn

BI, SALES & CRM
SKYMIND, TalkIQ, intranetum, ripjar, FRACTAL INDUSTRIES, MARIANA, DEEPGRAM, True AI, KOVID, ditto, Gridspace

CORE AI: VOICE INTERFACE
VIV, Mobvoi 出门问问, MaluubA, OBEN



E-COMMERCE
Reflektion, CORTEXICA, ViSENZE, STAQU

ROBOTICS & AUTO
netradyne, AdasWorks, drive.ai, Rokid, 图灵机器人 TURING ROBOT, comma.ai

ACQUIRED
(2014-2016YTD)
Etsy Blackbird, a Orbeus, IBM AlchemyAPI, intel Movidius, nervana SYSTEMS, Perceptio, vocaliq, turi, DEEPMIND, DNNresearch, Dark Blue Labs, Vision Factory, MAGIC PONY TECHNOLOGY, MADBITS, MetaMind

HEALTHCARE
imagia, Mindshare MEDICAL, BAYLABS, BUTTERFLY Network, Inc., deep genomics, Atomwise, zebra, PATHWAY GENOMICS, AVALON AI, Behold.ai, MedyMatch, ADVENIO, enlitic, Lunit, SIGTUPLE

SECURITY
deepinstinct, umbo CV, signalsense

OTHER
Alpaca, indico, Iris Automation

CBINSIGHTS

প্রথম আলো

সাংবাদিক ছাড়াই সংবাদ

৩০ মে ২০১৮, ১৯:৫০

আপডেট : ০২ জুন ২০১৮, ১১:৪০

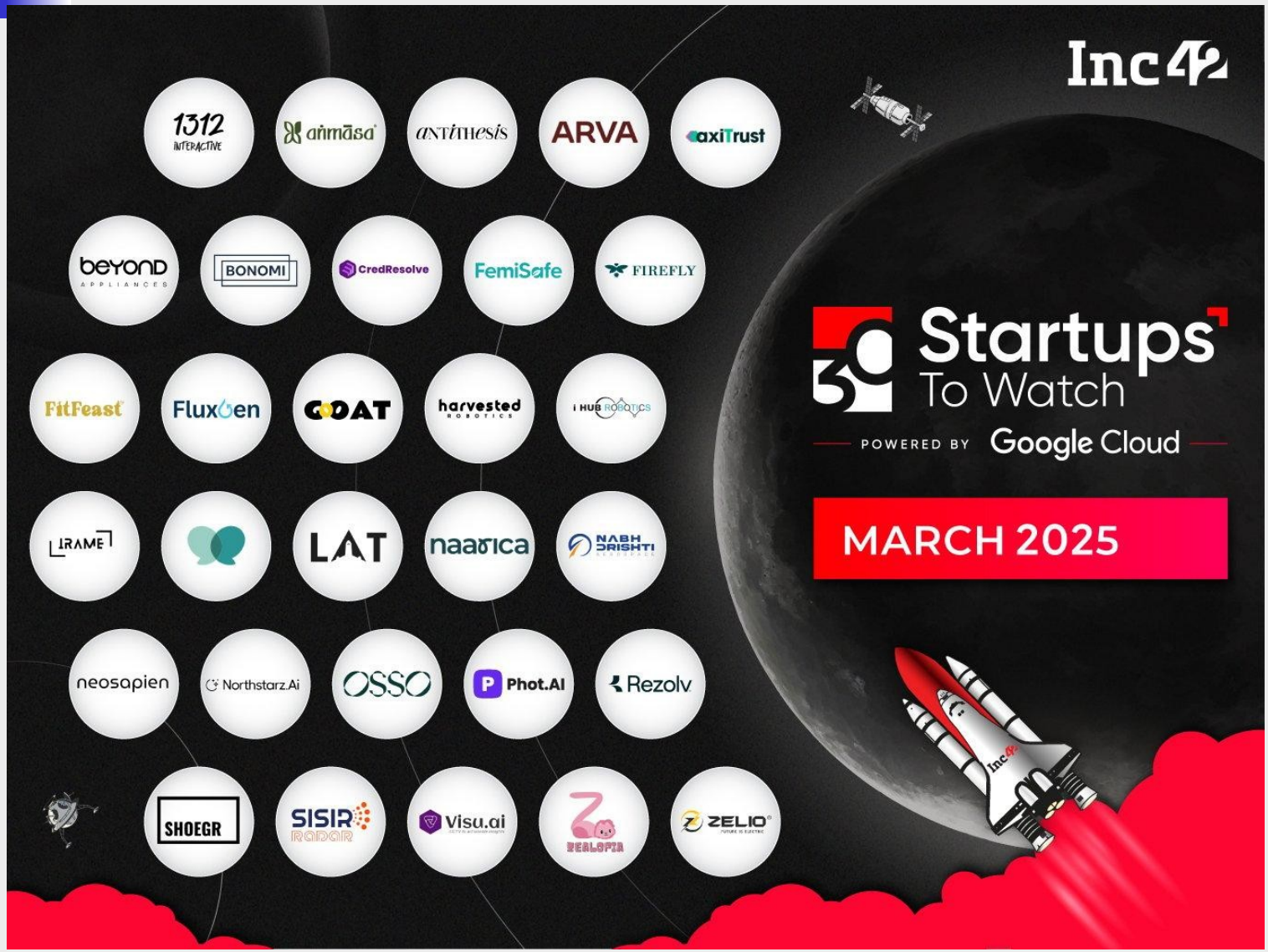
6/21/2018 About JX PRESS - English - JX 通信社 https://jxpress.net/about/about-jx-press/ 2/5

The major technological areas where our company excels in the development of this kind of service are **machine learning, deep learning, and natural language processing technology**. Using these technologies, we aim to achieve qualitative improvements in journalism while simultaneously reducing costs through this “mechanization of the news

Sophia uses voice recognition (speech-to-text) technology from Alphabet Inc of Google.

--- Sophia does utilize AI methods including face tracking, emotion recognition, and robotic movements generated by deep neural networks. Sophia's dialogue is generated via a decision tree, but is integrated with these outputs uniquely.

30 Startups To Watch: Startups That Caught Our Eyes In March 2025



Fundamental Deep Learning Methods

- Stacked Denoising Autoencoder (SDAE)
- Deep Belief Network (DBN)

} Final Architecture same as NN but preparation differs

- Convolutional Neural Network (CNN)

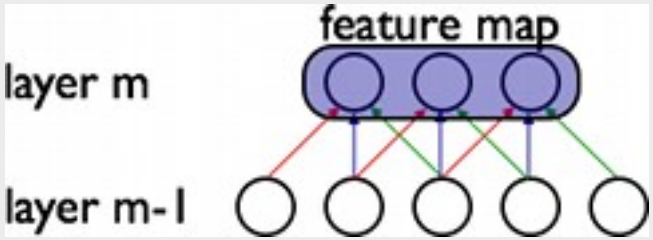
Efficient for 2D data e.g., Image

- Long Short-Term Memory (LSTM)

Developed for Sequential Data

Convolutional Neural Network (CNN)

CNNs exploit spatially-local correlation by enforcing a local connectivity pattern between neurons of adjacent layers.

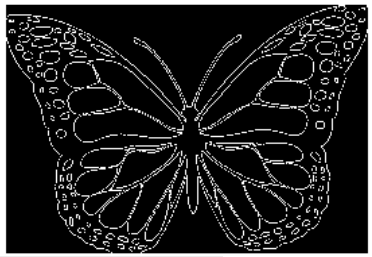


Weights of the same color are shared — constrained to be identical.

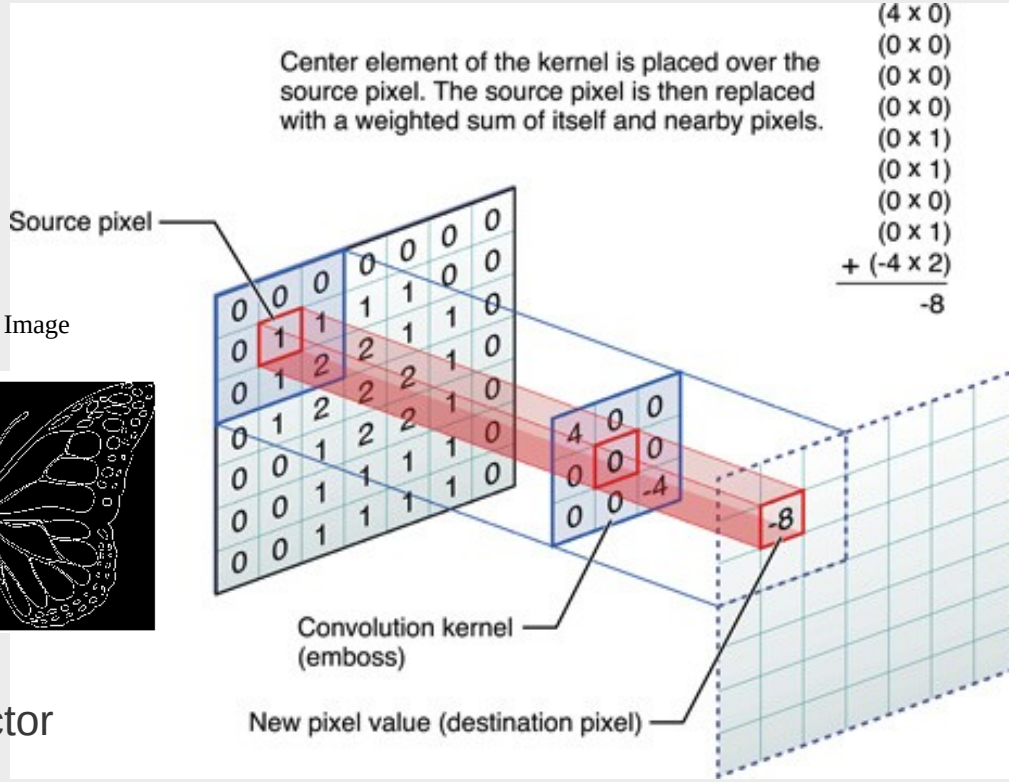
CNN's are found to be very effective with images

Input Image | Convolution Kernel | Convolved Image

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

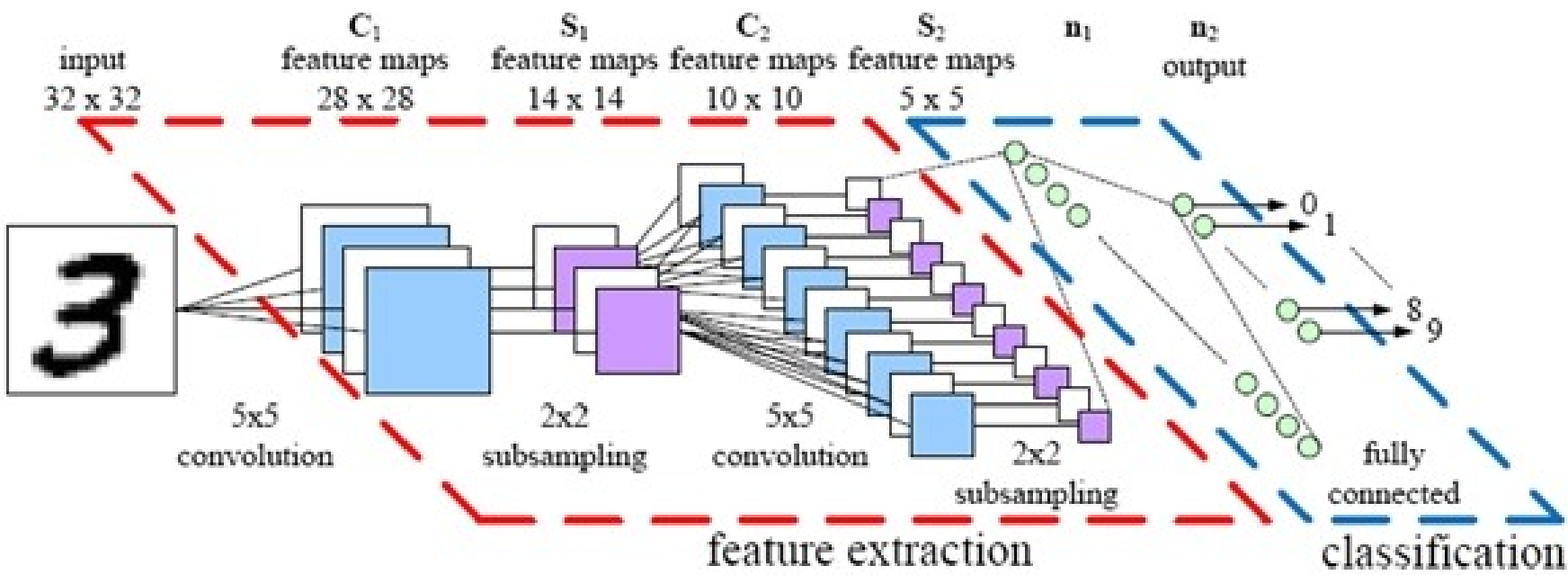


Convolution of an image with an edge detector convolution kernel



Convolution operation explore

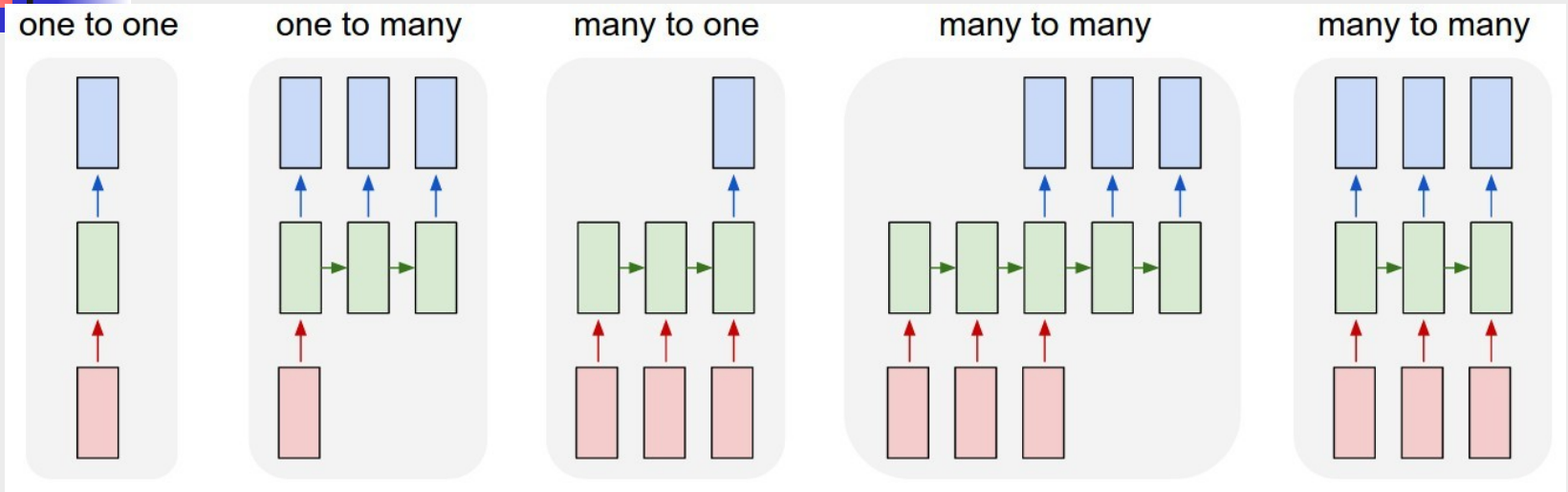
CNN: Sample Network for 32X32 Image Classification



CNN takes 2D input and is shown effective for operation on image and 2D representation of Data.

How about sequential events? Speech

Aspects of Sequential Data



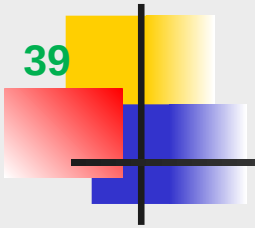
(a) fixed-sized input to fixed-sized output. ordinary classification(e.g., **image classification**)

(b) Sequence output for fixed size input. **Image captioning** from a fixed-size image where outputs is a sentence with a sequence of words.

(c) Sequence input and single vector fixed output. **Sentiment analysis** where a given sentence is classified as expressing positive or negative sentiment.

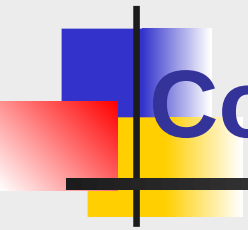
(d) Sequence input and sequence output. **Machine Translation:** System reads a sentence in a language (e.g., English) and then outputs a sentence in another language (e.g., Bengali).

(e) Synced sequence input and output. Labeling of each frame in a video sequence



Thanks for your attention

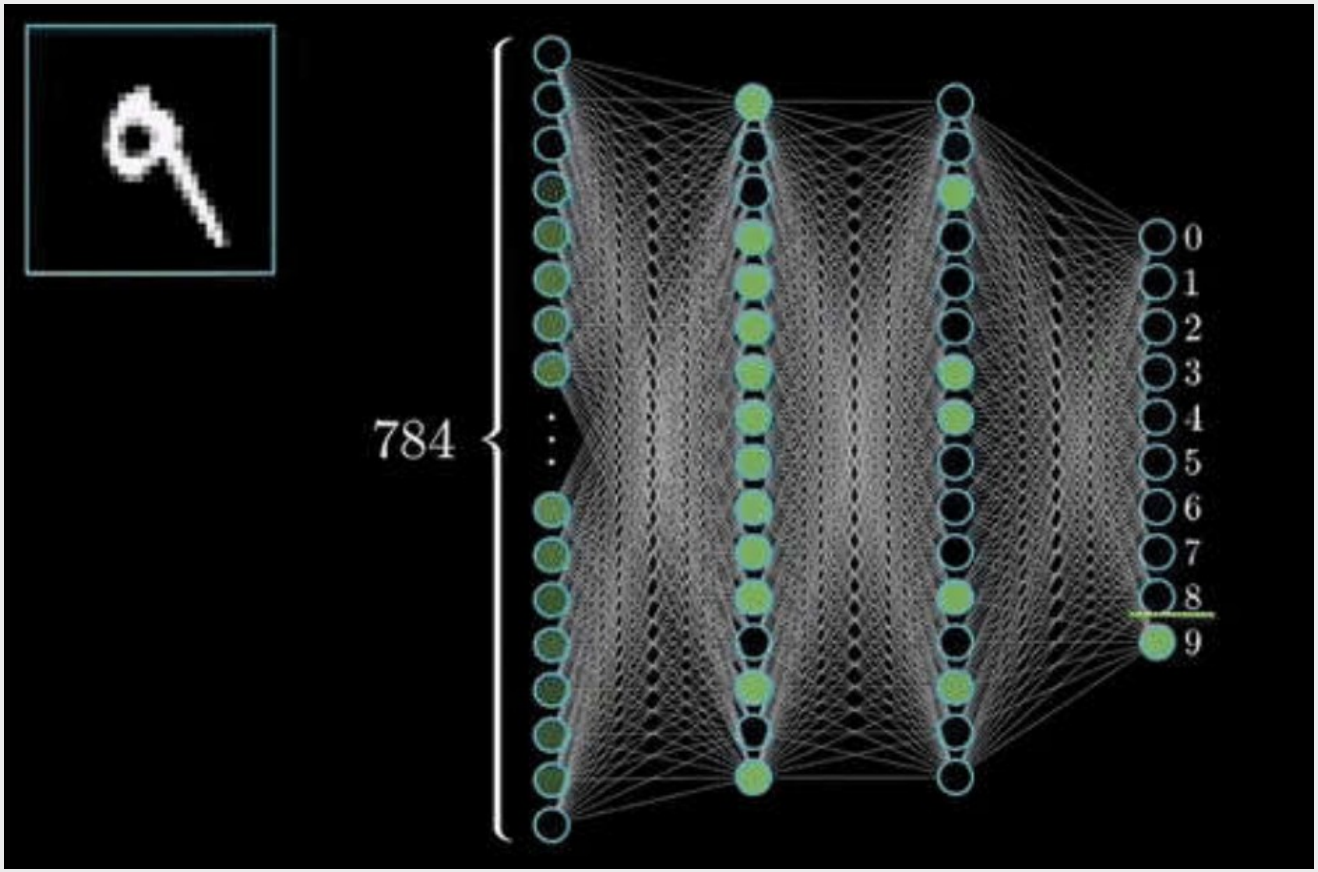
Question and Answer



Convolutional Neural Network (CNN)

Prof. Dr. Md. Aminul Haque Akhand
Dept. of Computer Science and Engineering (CSE)
Khulna University of Engineering & Technology (KUET)
www.kuet.ac.bd/cse/akhand

Handwritten Numeral Recognition with NN



Input I =784 for 28x28 Image

Limitations of NN and Motivation to CNN

- First, the number of trainable parameters for NN becomes extremely large.
- Secondly, NN offers little or no invariance to shifting, scaling, and other forms of distortion.
- Third, the topology of the input data is completely ignored, yielding similar training results for all permutations of the input vector.
- To overcome these problems, a classifier is often split up into a hand-crafted feature extractor and the actual trainable classifier module.
- Designing the feature extractor by hand requires a lot of heuristics and, most notably, a great deal of time.

Convolutional Neural Network(CNN) Basic

- CNNs are a family of multi-layer neural networks particularly designed for use on two-dimensional data, such as images.
- CNNs use three basic ideas: local connections, shared weights, and pooling.
- The naming significance of CNNs is the use convolutional operation based on local connections and weight sharing.
- The structure of a CNN comprises several pairs of convolutional - subsampling (i.e., pooling) layers and a fully connected layer.

Distinguishing Features and Operations:Convolution

- In mathematics, convolution is an operation which describes a rule of how to mix two functions or pieces of information: The feature map (or input data) and the convolution kernel mix together to form a transformed feature map.
- Convolution is often interpreted as a filter, where the kernel filters the feature map for information of a certain kind.
- An image may consider as a 2D pixel values and a particular kernel might filter for edges and discard other information. Kernel size is always square and smaller than image.

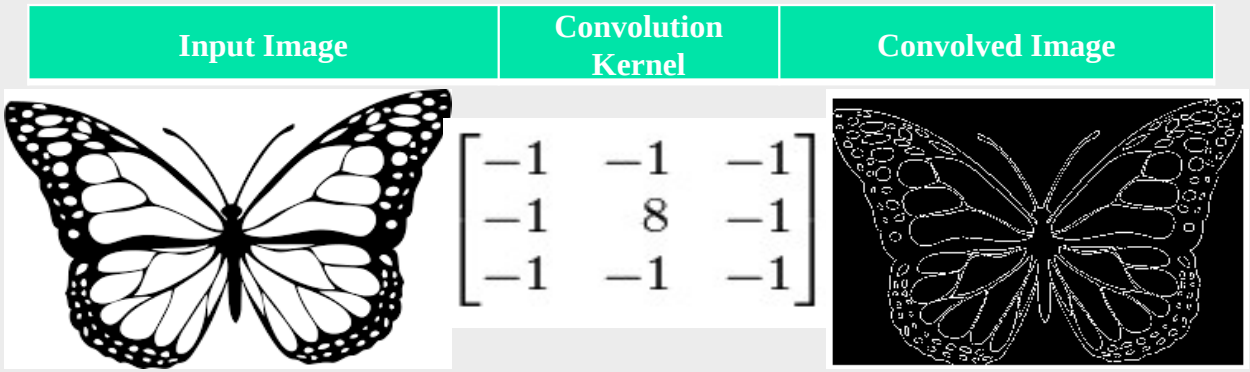
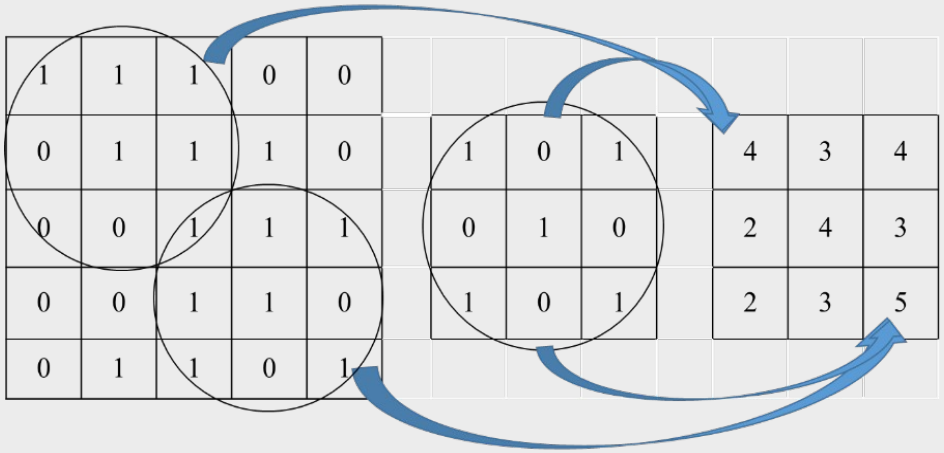


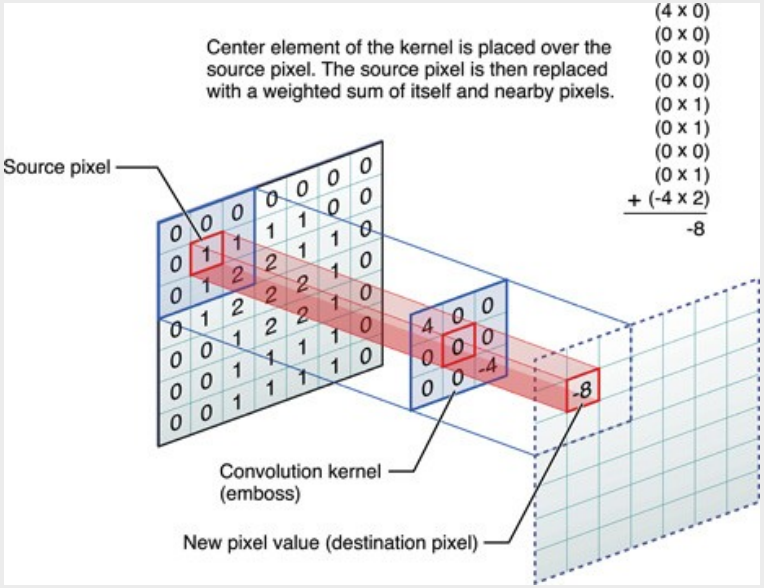
Figure 4.1: Convolution of an image with an edge detector convolution kernel..

Distinguishing Features and Operations :Convolution



5×5 Input Image **3×3 Kernel** **3×3 Convolved Feature**

Figure 4.2: Demonstration of convolution operation with sample image and kernel.



Distinguishing Features and Operations:Subsampling

- In general term, subsample means a sample drawn from a larger sample.
- In CNNs, subsampling is performed commonly after convolution. It is also termed as pooling or down sampling.
- Subsampling takes small rectangular patches or blocks from the input feature map (produced by convolutional) and produce a single output from each block.
- It considers non overlapping patches and therefore output feature map is reduced in size in both direction by the factor of patch size.

- For 2×2 patch size, the output feature map size is half of input feature map in both directions and contains only 25% of information.
- There are several ways to calculate the subsampled value: mean-pooling, max-pooling and stochastic-pooling.

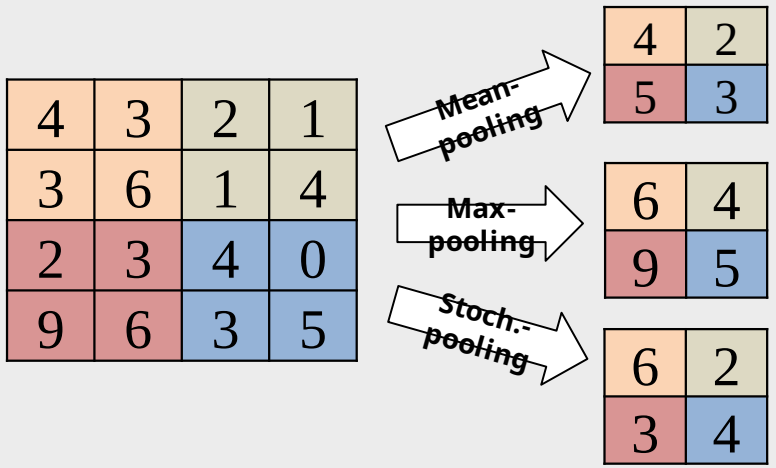


Figure 4.3: Different pooling outcomes on a sample feature map.

CNN Architecture

$$I \rightarrow \{C - S\} \rightarrow \{C - S\} \ldots \ldots \ldots \rightarrow \{C - S\} \rightarrow O$$

where *I, C, S* and *O* corresponds Input, Convolution, Subsampling and Output, respectively.

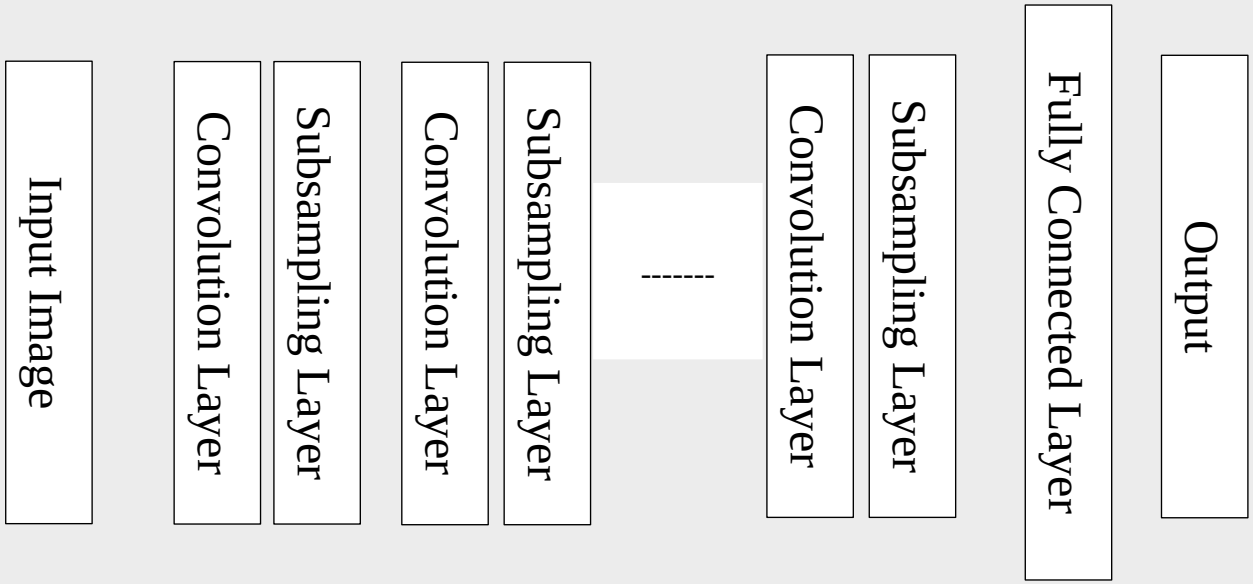


Figure 4.4: Basic structure of a CNN.

Close View of Convolution and Subsampling Process

$$H_{CFM} = H_I - H_K + 1$$

$$W_{CFM} = W_I - W_K + 1$$

$$H_{SFM} = H_{CFM} / H_P$$

$$W_{SFM} = W_{CFM} / W_P$$

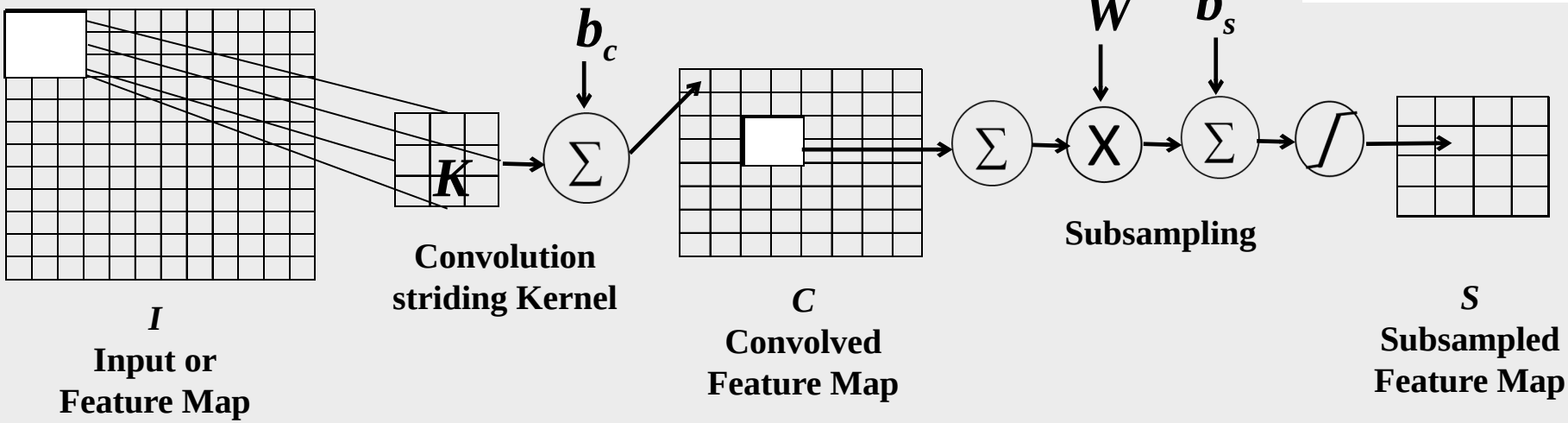


Figure 4.5: Convolution and Subsampling Process.

Details of a Sample CNN Structure and Training

For better understanding with a standard CNN architecture to classify handwritten numeral from 28×28(=784) pixel image is

$$I_{28\times28} \rightarrow \{6K1_{5\times5}C1_{24\times24} - S_{2\times2}6S1_{12\times12}\} \rightarrow \{12K2_{5\times5}C2_{8\times8} - S_{2\times2}12S2_{4\times4}\} \rightarrow \{Wo_{192\times10}\} \rightarrow I_{10}.$$

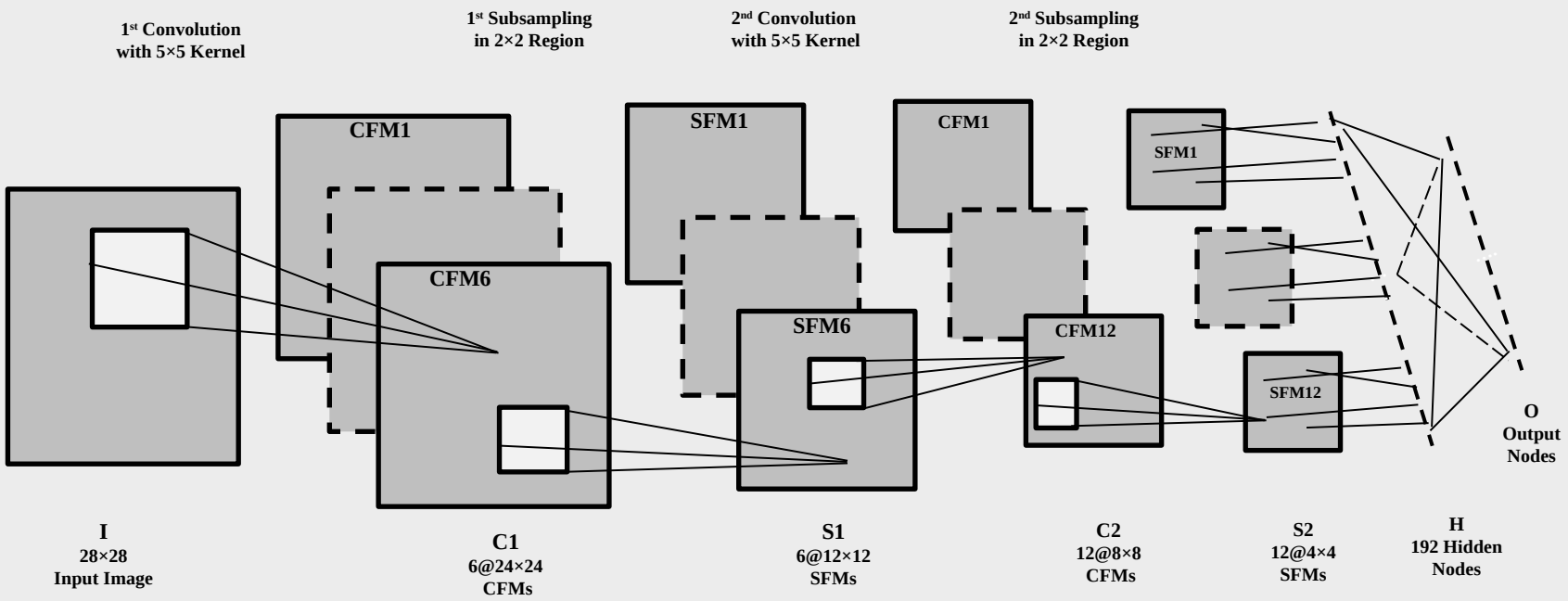


Figure 4.6: Detailed Structure of a Sample CNN.

Details of a Sample CNN Structure and Training

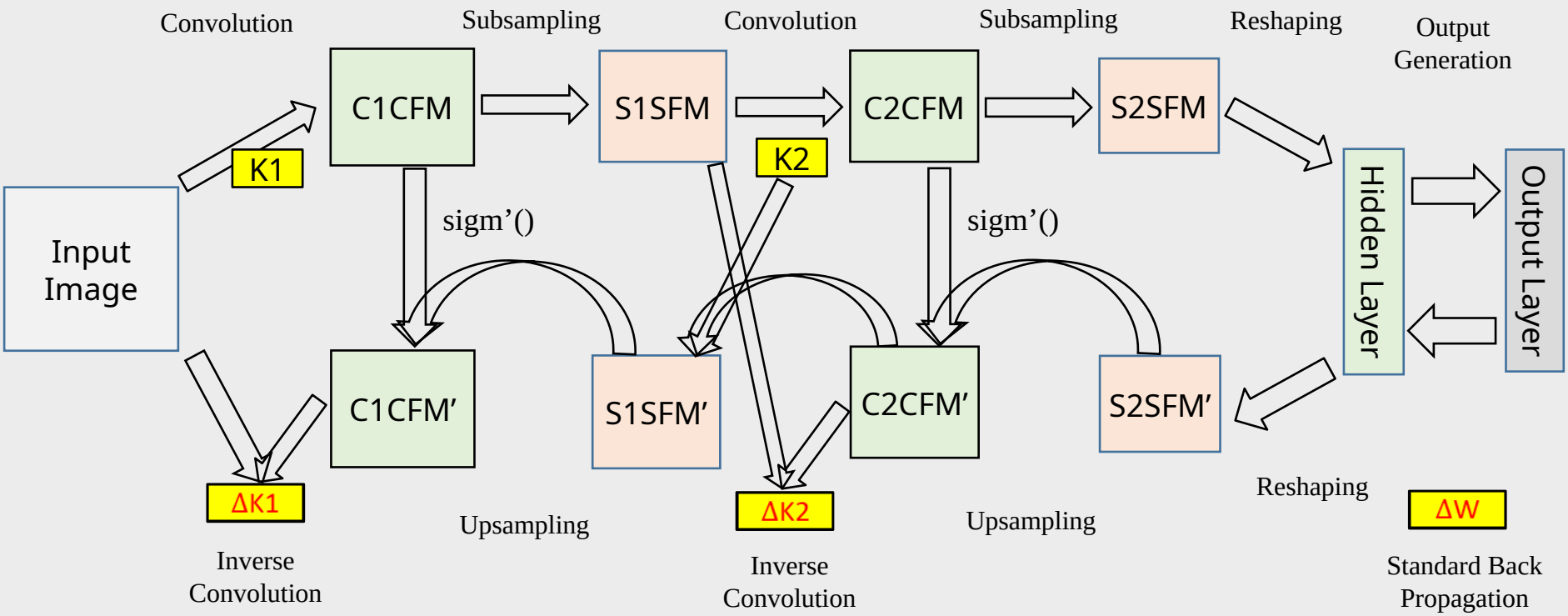


Figure 4.7: Operational flowchart of the CNN of Fig. 4.6.

Details of a Sample CNN Structure and Training

$$CFM_{x,y} = f\left(b + \sum_{h=1}^{H_K}\sum_{w=1}^{W_K} K_{h,w} * IFM_{x+h,y+w}\right) \quad (4.1)$$

$$SFM_{x,y} = down\left(\sum_{r=0}^{R-1}\sum_{c=0}^{C-1} CFM_{xR-1+r,yC-1+c}\right), \quad (4.2)$$

$$y_o = f\left(\sum_{h=1}^{H}(w_{oh} * h_o) + b\right), \quad (4.3)$$

$$E = \frac{1}{2}\frac{1}{PO}\sum_{p=1}^{P}\sum_{o=1}^{O}\left(d_o(p) - y_o(p)\right)^2, \quad (4.4)$$

Details of a Sample CNN Structure and Training

$$\Delta w = -\eta\, h_o \delta_o \tag{4.5}$$

$$\delta_o = f'(y_o) \circ (y_o - d_o), \tag{4.6}$$

where "$\circ$" denotes element wise operation.

$$\delta_h = f'(h_o) \sum_o w_{oh} * \delta_o \tag{4.7}$$

$$C2CFM' = f'(\mathrm{C2CFM}) \circ up(\mathrm{S2SFM}'), \tag{4.8}$$

$$S1SFM' = conv^{-1}(C2CFM', K2) \tag{4.9}$$

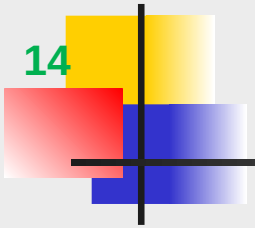
$$\Delta K2 = conv^{-1}(C2CFM', S1SFM) \tag{4.10}$$

$$\Delta bK2 = \sum C2CFM' \tag{4.11}$$

$$C1CFM' = f'(\mathrm{C1CFM}) \circ up(\mathrm{S1SFM}') \tag{4.12}$$

$$\Delta K1 = conv^{-1}(C1CFM', IFM) \tag{4.13}$$

$$\Delta bK1 = \sum C1CFM' \tag{4.14}$$



Thanks for your attention

Question and Answer